Soft Actor-Critic and Complex Transformer Layers

Table of Contents

Soft Actor-Critic

Table of Contents

Background

(Previously: Background for TD3)

Soft Actor Critic (SAC) is an algorithm that optimizes a stochastic policy in an off-policy way, forming a bridge between stochastic policy optimization and DDPG-style approaches. It isn't a direct successor to TD3 (having been published roughly concurrently), but it incorporates the clipped double-Q trick, and due to the inherent stochasticity of the policy in SAC, it also winds up benefiting from something like target policy smoothing.

A central feature of SAC is **entropy regularization.** The policy is trained to maximize a trade-off between expected return and entropy, a measure of randomness in the policy. This has a close connection to the exploration-exploitation trade-off: increasing entropy results in more exploration, which can accelerate learning later on. It can also prevent the policy from prematurely converging to a bad local optimum.

Quick Facts

- SAC is an off-policy algorithm.
- The version of SAC implemented here can only be used for environments with continuous action spaces.
- An alternate version of SAC, which slightly changes the policy update rule, can be implemented to handle discrete action spaces.
- The Spinning Up implementation of SAC does not support parallelization.

Key Equations

To explain Soft Actor Critic, we first have to introduce the entropy-regularized reinforcement learning setting. In entropy-regularized RL, there are slightly-different equations for value functions.

Entropy-Regularized Reinforcement Learning

Entropy is a quantity which, roughly speaking, says how random a random variable is. If a coin is weighted so that it almost always comes up heads, it has low entropy; if it's evenly weighted and has a half chance of either outcome, it has high entropy.

Let x be a random variable with probability mass or density function P. The entropy H of x is computed from its distribution P according to

$$H(P) = \underset{x\sim P}{\mathrm{E}}\left[-\log P(x)\right].$$

In entropy-regularized reinforcement learning, the agent gets a bonus reward at each time step proportional to the entropy of the policy at that timestep. This changes the RL problem to:

$$\pi^* = \arg\max_{\pi} \underset{\tau\sim\pi}{\mathrm{E}}\left[\sum_{t=0}^{\infty} \gamma^t \Big(R(s_t, a_t, s_{t+1}) + \alpha H\left(\pi(\cdot|s_t)\right) \Big)\right],$$

where $\alpha > 0$ is the trade-off coefficient. (Note: we're assuming an infinite-horizon discounted setting here, and we'll do the same for the rest of this page.) We can now define the slightly-different value functions in this setting. V^π is changed to include the entropy bonuses from every timestep:

$$V^\pi(s) = \underset{\tau\sim\pi}{\mathrm{E}}\left[\sum_{t=0}^{\infty} \gamma^t \Big(R(s_t, a_t, s_{t+1}) + \alpha H\left(\pi(\cdot|s_t)\right) \Big) \,\middle|\, s_0 = s\right]$$

Q^π is changed to include the entropy bonuses from every timestep *except the first*:

$$Q^\pi(s,a) = \underset{\tau\sim\pi}{\mathrm{E}}\left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t, s_{t+1}) + \alpha \sum_{t=1}^{\infty} \gamma^t H\left(\pi(\cdot|s_t)\right) \,\middle|\, s_0 = s, a\right]$$

With these definitions, V^π and Q^π are connected by:

$$V^\pi(s) = \underset{a\sim\pi}{\mathrm{E}}\left[Q^\pi(s,a)\right] + \alpha H\left(\pi(\cdot|s)\right)$$

and the Bellman equation for Q^π is

$$\begin{aligned} Q^\pi(s,a) &= \underset{\substack{s'\sim P\\ a'\sim\pi}}{\mathrm{E}}\left[R(s,a,s') + \gamma\left(Q^\pi(s',a') + \alpha H\left(\pi(\cdot|s')\right)\right)\right] \\ &= \underset{s'\sim P}{\mathrm{E}}\left[R(s,a,s') + \gamma V^\pi(s')\right]. \end{aligned}$$

You Should Know

The way we've set up the value functions in the entropy-regularized setting is a little bit arbitrary, and actually we could have done it differently (eg make Q^π include the entropy bonus at the first timestep). The choice of definition may vary slightly across papers on the subject.

Soft Actor-Critic

SAC concurrently learns a policy π_θ and two Q-functions Q_{ϕ_1}, Q_{ϕ_2}. There are two variants of SAC that are currently standard: one that uses a fixed entropy regularization coefficient α, and another that enforces an entropy constraint by varying α over the course of training. For simplicity, Spinning Up makes use of the version with a fixed entropy regularization coefficient, but the entropy-constrained variant is generally preferred by practitioners.

You Should Know

The SAC algorithm has changed a little bit over time. An older version of SAC also learns a value function V_ψ in addition to the Q-functions; this page will focus on the modern version that omits the extra value function.

Learning Q. The Q-functions are learned in a similar way to TD3, but with a few key differences.

First, what's similar?

1. Like in TD3, both Q-functions are learned with MSBE minimization, by regressing to a single shared target.
2. Like in TD3, the shared target is computed using target Q-networks, and the target Q-networks are obtained by polyak averaging the Q-network parameters over the course of training.
3. Like in TD3, the shared target makes use of the **clipped double-Q** trick.

What's different?

1. Unlike in TD3, the target also includes a term that comes from SAC's use of entropy

regularization.
2. Unlike in TD3, the next-state actions used in the target come from the **current policy** instead of a target policy.
3. Unlike in TD3, there is no explicit target policy smoothing. TD3 trains a deterministic policy, and so it accomplishes smoothing by adding random noise to the next-state actions. SAC trains a stochastic policy, and so the noise from that stochasticity is sufficient to get a similar effect.

Before we give the final form of the Q-loss, let's take a moment to discuss how the contribution from entropy regularization comes in. We'll start by taking our recursive Bellman equation for the entropy-regularized Q^π from earlier, and rewriting it a little bit by using the definition of entropy:

$$\begin{aligned} Q^\pi(s,a) &= \mathop{\mathrm{E}}_{\substack{s' \sim P \\ a' \sim \pi}} \left[R(s,a,s') + \gamma \left(Q^\pi(s',a') + \alpha H\left(\pi(\cdot|s')\right) \right) \right] \\ &= \mathop{\mathrm{E}}_{\substack{s' \sim P \\ a' \sim \pi}} \left[R(s,a,s') + \gamma \left(Q^\pi(s',a') - \alpha \log \pi(a'|s') \right) \right] \end{aligned}$$

The RHS is an expectation over next states (which come from the replay buffer) and next actions (which come from the current policy, and **not** the replay buffer). Since it's an expectation, we can approximate it with samples:

$$Q^\pi(s,a) \approx r + \gamma \left(Q^\pi(s', \tilde{a}') - \alpha \log \pi(\tilde{a}'|s') \right), \qquad \tilde{a}' \sim \pi(\cdot|s').$$

You Should Know

We switch next action notation to $\tilde{a}'$, instead of a', to highlight that the next actions have to be sampled fresh from the policy (whereas by contrast, r and s' should come from the replay buffer).

SAC sets up the MSBE loss for each Q-function using this kind of sample approximation for the target. The only thing still undetermined here is which Q-function gets used to compute the sample backup: like TD3, SAC uses the clipped double-Q trick, and takes the minimum Q-value between the two Q approximators.

Putting it all together, the loss functions for the Q-networks in SAC are:

$$L(\phi_i, \mathcal{D}) = \mathop{\mathrm{E}}_{(s,a,r,s',d) \sim \mathcal{D}} \left[\left(Q_{\phi_i}(s,a) - y(r,s',d) \right)^2 \right],$$

where the target is given by

$$y(r,s',d) = r + \gamma(1-d) \left(\min_{j=1,2} Q_{\phi_{\text{targ},j}}(s', \tilde{a}') - \alpha \log \pi_\theta(\tilde{a}'|s') \right), \qquad \tilde{a}'$$

Learning the Policy. The policy should, in each state, act to maximize the expected future return plus expected future entropy. That is, it should maximize $V^{\pi}(s)$, which we expand out into

$$\begin{aligned} V^{\pi}(s) &= \underset{a\sim\pi}{\mathrm{E}}\left[Q^{\pi}(s,a)\right] + \alpha H\left(\pi(\cdot|s)\right) \\ &= \underset{a\sim\pi}{\mathrm{E}}\left[Q^{\pi}(s,a) - \alpha \log \pi(a|s)\right]. \end{aligned}$$

The way we optimize the policy makes use of the **reparameterization trick**, in which a sample from $\pi_{\theta}(\cdot|s)$ is drawn by computing a deterministic function of state, policy parameters, and independent noise. To illustrate: following the authors of the SAC paper, we use a squashed Gaussian policy, which means that samples are obtained according to

$$\tilde{a}_{\theta}(s,\xi) = \tanh\left(\mu_{\theta}(s) + \sigma_{\theta}(s) \odot \xi\right), \qquad \xi \sim \mathcal{N}(0, I).$$

You Should Know

This policy has two key differences from the policies we use in the other policy optimization algorithms:

1. The squashing function. The $\tanh$ in the SAC policy ensures that actions are bounded to a finite range. This is absent in the VPG, TRPO, and PPO policies. It also changes the distribution: before the $\tanh$ the SAC policy is a factored Gaussian like the other algorithms' policies, but after the $\tanh$ it is not. (You can still compute the log-probabilities of actions in closed form, though: see the paper appendix for details.)

2. The way standard deviations are parameterized. In VPG, TRPO, and PPO, we represent the log std devs with state-independent parameter vectors. In SAC, we represent the log std devs as outputs from the neural network, meaning that they depend on state in a complex way. SAC with state-independent log std devs, in our experience, did not work. (Can you think of why? Or better yet: run an experiment to verify?)

The reparameterization trick allows us to rewrite the expectation over actions (which contains a pain point: the distribution depends on the policy parameters) into an expectation over noise (which removes the pain point: the distribution now has no dependence on parameters):

$$\underset{a\sim\pi_{\theta}}{\mathrm{E}}\left[Q^{\pi_{\theta}}(s,a) - \alpha \log \pi_{\theta}(a|s)\right] = \underset{\xi\sim\mathcal{N}}{\mathrm{E}}\left[Q^{\pi_{\theta}}(s,\tilde{a}_{\theta}(s,\xi)) - \alpha \log \pi_{\theta}(\tilde{a}_{\theta}(s,\xi)|s)\right]$$

To get the policy loss, the final step is that we need to substitute $Q^{\pi_{\theta}}$ with one of our function approximators. Unlike in TD3, which uses Q_{ϕ_1} (just the first Q approximator), SAC uses $\min_{j=1,2} Q_{\phi_j}$ (the minimum of the two Q approximators). The policy is thus optimized according to

$$\max_{\theta} \underset{\substack{s\sim\mathcal{D} \\ \xi\sim\mathcal{N}}}{\mathrm{E}}\left[\min_{j=1,2} Q_{\phi_j}(s,\tilde{a}_{\theta}(s,\xi)) - \alpha \log \pi_{\theta}(\tilde{a}_{\theta}(s,\xi)|s)\right],$$

which is almost the same as the DDPG and TD3 policy optimization, except for the min-double-Q trick, the stochasticity, and the entropy term.

Exploration vs. Exploitation

SAC trains a stochastic policy with entropy regularization, and explores in an on-policy way. The entropy regularization coefficient α explicitly controls the explore-exploit tradeoff, with higher α corresponding to more exploration, and lower α corresponding to more exploitation. The right coefficient (the one which leads to the stablest / highest-reward learning) may vary from environment to environment, and could require careful tuning.

At test time, to see how well the policy exploits what it has learned, we remove stochasticity and use the mean action instead of a sample from the distribution. This tends to improve performance over the original stochastic policy.

You Should Know

Our SAC implementation uses a trick to improve exploration at the start of training. For a fixed number of steps at the beginning (set with the `start_steps` keyword argument), the agent takes actions which are sampled from a uniform random distribution over valid actions. After that, it returns to normal SAC exploration.

Pseudocode

Algorithm 1 Soft Actor-Critic

Input: initial policy parameters θ, Q-function parameters ϕ_1, ϕ_2, empty replay buffer $\mathcal{D}$
Set target parameters equal to main parameters $\phi_{\text{targ},1} \leftarrow \phi_1$, $\phi_{\text{targ},2} \leftarrow \phi_2$
repeat
  Observe state s and select action $a \sim \pi_\theta(\cdot|s)$
  Execute a in the environment
  Observe next state s', reward r, and done signal d to indicate whether s' is terminal
  Store (s, a, r, s', d) in replay buffer $\mathcal{D}$
  If s' is terminal, reset environment state.
  if it's time to update **then**
    for j in range(however many updates) **do**
      Randomly sample a batch of transitions, $B = \{(s, a, r, s', d)\}$ from $\mathcal{D}$
      Compute targets for the Q functions:

$$y(r, s', d) = r + \gamma(1 - d)\left(\min_{i=1,2} Q_{\phi_{\text{targ},i}}(s', \tilde{a}') - \alpha \log \pi_\theta(\tilde{a}'|s')\right), \quad \tilde{a}' \sim \pi_\theta(\cdot|s')$$

      Update Q-functions by one step of gradient descent using

$$\nabla_{\phi_i} \frac{1}{|B|} \sum_{(s,a,r,s',d) \in B} \left(Q_{\phi_i}(s, a) - y(r, s', d)\right)^2 \qquad \text{for } i = 1, 2$$

      Update policy by one step of gradient ascent using

$$\nabla_\theta \frac{1}{|B|} \sum_{s \in B} \left(\min_{i=1,2} Q_{\phi_i}(s, \tilde{a}_\theta(s)) - \alpha \log \pi_\theta\left(\tilde{a}_\theta(s)|\, s\right)\right),$$

      where $\tilde{a}_\theta(s)$ is a sample from $\pi_\theta(\cdot|s)$ which is differentiable wrt θ via the reparametrization trick.
      Update target networks with

$$\phi_{\text{targ},i} \leftarrow \rho \phi_{\text{targ},i} + (1 - \rho)\phi_i \qquad \text{for } i = 1, 2$$

    end for
  end if
until convergence

Documentation

You Should Know

In what follows, we give documentation for the PyTorch and Tensorflow implementations of SAC in Spinning Up. They have nearly identical function calls and docstrings, except for details relating to model construction. However, we include both full docstrings for completeness.

Documentation: PyTorch Version

spinup.sac_pytorch(*env_fn, actor_critic=<MagicMock spec='str' id='140554319922904'>, ac_kwargs={}, seed=0, steps_per_epoch=4000, epochs=100, replay_size=1000000, gamma=0.99, polyak=0.995, alpha=0.2, batch_size=100, start_steps=10000, update_after=1000, update_every=50, num_test_episodes=10, max_ep_len=1000, logger_kwargs={}, save_freq=1*)

latest

Soft Actor-Critic (SAC)

Parameters:

- **env_fn** – A function which creates a copy of the environment. The environment must satisfy the OpenAI Gym API.
- **actor_critic** –
  The constructor method for a PyTorch Module with an `act` method, a `pi` module, a `q1` module, and a `q2` module. The `act` method and `pi` module should accept batches of observations as inputs, and `q1` and `q2` should accept a batch of observations and a batch of actions as inputs. When called, `act`, `q1`, and `q2` should return:

Call	Output Shape	Description
`act`	(batch, act_dim)	Numpy array of actions for each observation.
`q1`	(batch,)	Tensor containing one current estimate of Q* for the provided observations and actions. (Critical: make sure to flatten this!)
`q2`	(batch,)	Tensor containing the other current estimate of Q* for the provided observations and actions. (Critical: make sure to flatten this!)

  Calling `pi` should return:

Symbol	Shape	Description
`a`	(batch, act_dim)	Tensor containing actions from policy given observations.
`logp_pi`	(batch,)	Tensor containing log probabilities of actions in `a`. Importantly: gradients should be able to flow back into `a`.

- **ac_kwargs** (*dict*) – Any kwargs appropriate for the ActorCritic object you provided to SAC.
- **seed** (*int*) – Seed for random number generators.
- **steps_per_epoch** (*int*) – Number of steps of interaction (state-action pairs) for the agent and the environment in each epoch.
- **epochs** (*int*) – Number of epochs to run and train agent.
- **replay_size** (*int*) – Maximum length of replay buffer.
- **gamma** (*float*) – Discount factor. (Always between 0 and 1.)
- **polyak** (*float*) –
  Interpolation factor in polyak averaging for target networks. Target

networks are updated towards main networks according to:

$$\theta_{\text{targ}} \leftarrow \rho\theta_{\text{targ}} + (1-\rho)\theta$$

where ρ is polyak. (Always between 0 and 1, usually close to 1.)

- **lr** (*float*) – Learning rate (used for both policy and value learning).
- **alpha** (*float*) – Entropy regularization coefficient. (Equivalent to inverse of reward scale in the original SAC paper.)
- **batch_size** (*int*) – Minibatch size for SGD.
- **start_steps** (*int*) – Number of steps for uniform-random action selection, before running real policy. Helps exploration.
- **update_after** (*int*) – Number of env interactions to collect before starting to do gradient descent updates. Ensures replay buffer is full enough for useful updates.
- **update_every** (*int*) – Number of env interactions that should elapse between gradient descent updates. Note: Regardless of how long you wait between updates, the ratio of env steps to gradient steps is locked to 1.
- **num_test_episodes** (*int*) – Number of episodes to test the deterministic policy at the end of each epoch.
- **max_ep_len** (*int*) – Maximum length of trajectory / episode / rollout.
- **logger_kwargs** (*dict*) – Keyword args for EpochLogger.
- **save_freq** (*int*) – How often (in terms of gap between epochs) to save the current policy and value function.

Saved Model Contents: PyTorch Version

The PyTorch saved model can be loaded with `ac = torch.load('path/to/model.pt')`, yielding an actor-critic object (`ac`) that has the properties described in the docstring for `sac_pytorch`.

You can get actions from this model with

```
actions = ac.act(torch.as_tensor(obs, dtype=torch.float32))
```

Documentation: Tensorflow Version

spinup.sac_tf1(*env_fn, actor_critic=<function mlp_actor_critic>, ac_kwargs={}, seed=0, steps_per_epoch=4000, epochs=100, replay_size=1000000, gamma=0.99, polyak=0.995, lr=0.001, alpha=0.2, batch_size=100, start_steps=10000, update_after=1000, update_every=50, num_test_episodes=10, max_ep_len=1000, logger_kwargs={}, save_freq=1*)

Soft Actor-Critic (SAC)

Parameters:

- **env_fn** – A function which creates a copy of the environment. The environment must satisfy the OpenAI Gym API.
- **actor_critic** –
  A function which takes in placeholder symbols for state, `x_ph`, and action, `a_ph`, and returns the main outputs from the agent's Tensorflow computation graph:

Symbol	Shape	Description
`mu`	(batch, act_dim)	Computes mean actions from policy given states.
`pi`	(batch, act_dim)	Samples actions from policy given states.
`logp_pi`	(batch,)	Gives log probability, according to the policy, of the action sampled by `pi`. Critical: must be differentiable with respect to policy parameters all the way through action sampling.
`q1`	(batch,)	Gives one estimate of Q* for states in `x_ph` and actions in `a_ph`.
`q2`	(batch,)	Gives another estimate of Q* for states in `x_ph` and actions in `a_ph`.

- **ac_kwargs** (*dict*) – Any kwargs appropriate for the actor_critic function you provided to SAC.
- **seed** (*int*) – Seed for random number generators.
- **steps_per_epoch** (*int*) – Number of steps of interaction (state-action pairs) for the agent and the environment in each epoch.
- **epochs** (*int*) – Number of epochs to run and train agent.
- **replay_size** (*int*) – Maximum length of replay buffer.
- **gamma** (*float*) – Discount factor. (Always between 0 and 1.)
- **polyak** (*float*) –
  Interpolation factor in polyak averaging for target networks. Target networks are updated towards main networks according to:

$$\theta_{\text{targ}} \leftarrow \rho \theta_{\text{targ}} + (1-\rho)\theta$$

  where ρ is polyak. (Always between 0 and 1, usually close t

- **lr** (*float*) – Learning rate (used for both policy and value learning).
- **alpha** (*float*) – Entropy regularization coefficient. (Equivalent to inverse of reward

scale in the original SAC paper.)

- **batch_size** (*int*) – Minibatch size for SGD.
- **start_steps** (*int*) – Number of steps for uniform-random action selection, before running real policy. Helps exploration.
- **update_after** (*int*) – Number of env interactions to collect before starting to do gradient descent updates. Ensures replay buffer is full enough for useful updates.
- **update_every** (*int*) – Number of env interactions that should elapse between gradient descent updates. Note: Regardless of how long you wait between updates, the ratio of env steps to gradient steps is locked to 1.
- **num_test_episodes** (*int*) – Number of episodes to test the deterministic policy at the end of each epoch.
- **max_ep_len** (*int*) – Maximum length of trajectory / episode / rollout.
- **logger_kwargs** (*dict*) – Keyword args for EpochLogger.
- **save_freq** (*int*) – How often (in terms of gap between epochs) to save the current policy and value function.

Saved Model Contents: Tensorflow Version

The computation graph saved by the logger includes:

Key	Value
`x`	Tensorflow placeholder for state input.
`a`	Tensorflow placeholder for action input.
`mu`	Deterministically computes mean action from the agent, given states in `x`.
`pi`	Samples an action from the agent, conditioned on states in `x`.
`q1`	Gives one action-value estimate for states in `x` and actions in `a`.
`q2`	Gives the other action-value estimate for states in `x` and actions in `a`.
`v`	Gives the value estimate for states in `x`.

This saved model can be accessed either by

- running the trained policy with the test_policy.py tool,
- or loading the whole saved graph into a program with restore_tf_graph.

Note: for SAC, the correct evaluation policy is given by `mu` and not by `pi`. The policy `pi` may be thought of as the exploration policy, while `mu` is the exploitation policy.

References

Soft Actor-Critic:
Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor

Tuomas Haarnoja [1] **Aurick Zhou** [1] **Pieter Abbeel** [1] **Sergey Levine** [1]

Abstract

Model-free deep reinforcement learning (RL) algorithms have been demonstrated on a range of challenging decision making and control tasks. However, these methods typically suffer from two major challenges: very high sample complexity and brittle convergence properties, which necessitate meticulous hyperparameter tuning. Both of these challenges severely limit the applicability of such methods to complex, real-world domains. In this paper, we propose soft actor-critic, an off-policy actor-critic deep RL algorithm based on the maximum entropy reinforcement learning framework. In this framework, the actor aims to maximize expected reward while also maximizing entropy. That is, to succeed at the task while acting as randomly as possible. Prior deep RL methods based on this framework have been formulated as Q-learning methods. By combining off-policy updates with a stable stochastic actor-critic formulation, our method achieves state-of-the-art performance on a range of continuous control benchmark tasks, outperforming prior on-policy and off-policy methods. Furthermore, we demonstrate that, in contrast to other off-policy algorithms, our approach is very stable, achieving very similar performance across different random seeds.

1. Introduction

Model-free deep reinforcement learning (RL) algorithms have been applied in a range of challenging domains, from games (Mnih et al., 2013; Silver et al., 2016) to robotic control (Schulman et al., 2015). The combination of RL and high-capacity function approximators such as neural networks holds the promise of automating a wide range of decision making and control tasks, but widespread adoption of these methods in real-world domains has been hampered by two major challenges. First, model-free deep RL methods are notoriously expensive in terms of their sample complexity. Even relatively simple tasks can require millions of steps of data collection, and complex behaviors with high-dimensional observations might need substantially more. Second, these methods are often brittle with respect to their hyperparameters: learning rates, exploration constants, and other settings must be set carefully for different problem settings to achieve good results. Both of these challenges severely limit the applicability of model-free deep RL to real-world tasks.

One cause for the poor sample efficiency of deep RL methods is on-policy learning: some of the most commonly used deep RL algorithms, such as TRPO (Schulman et al., 2015), PPO (Schulman et al., 2017b) or A3C (Mnih et al., 2016), require new samples to be collected for each gradient step. This quickly becomes extravagantly expensive, as the number of gradient steps and samples per step needed to learn an effective policy increases with task complexity. Off-policy algorithms aim to reuse past experience. This is not directly feasible with conventional policy gradient formulations, but is relatively straightforward for Q-learning based methods (Mnih et al., 2015). Unfortunately, the combination of off-policy learning and high-dimensional, nonlinear function approximation with neural networks presents a major challenge for stability and convergence (Bhatnagar et al., 2009). This challenge is further exacerbated in continuous state and action spaces, where a separate actor network is often used to perform the maximization in Q-learning. A commonly used algorithm in such settings, deep deterministic policy gradient (DDPG) (Lillicrap et al., 2015), provides for sample-efficient learning but is notoriously challenging to use due to its extreme brittleness and hyperparameter sensitivity (Duan et al., 2016; Henderson et al., 2017).

We explore how to design an efficient and stable model-free deep RL algorithm for continuous state and action spaces. To that end, we draw on the maximum entropy framework, which augments the standard maximum reward reinforcement learning objective with an entropy maximization term (Ziebart et al., 2008; Toussaint, 2009; Rawlik et al.,

[1]Berkeley Artificial Intelligence Research, University of California, Berkeley, USA. Correspondence to: Tuomas Haarnoja <haarnoja@berkeley.edu>.

2012; Fox et al., 2016; Haarnoja et al., 2017). Maximum entropy reinforcement learning alters the RL objective, though the original objective can be recovered using a temperature parameter (Haarnoja et al., 2017). More importantly, the maximum entropy formulation provides a substantial improvement in exploration and robustness: as discussed by Ziebart (2010), maximum entropy policies are robust in the face of model and estimation errors, and as demonstrated by (Haarnoja et al., 2017), they improve exploration by acquiring diverse behaviors. Prior work has proposed model-free deep RL algorithms that perform on-policy learning with entropy maximization (O'Donoghue et al., 2016), as well as off-policy methods based on soft Q-learning and its variants (Schulman et al., 2017a; Nachum et al., 2017a; Haarnoja et al., 2017). However, the on-policy variants suffer from poor sample complexity for the reasons discussed above, while the off-policy variants require complex approximate inference procedures in continuous action spaces.

In this paper, we demonstrate that we can devise an off-policy maximum entropy actor-critic algorithm, which we call soft actor-critic (SAC), which provides for both sample-efficient learning and stability. This algorithm extends readily to very complex, high-dimensional tasks, such as the Humanoid benchmark (Duan et al., 2016) with 21 action dimensions, where off-policy methods such as DDPG typically struggle to obtain good results (Gu et al., 2016). SAC also avoids the complexity and potential instability associated with approximate inference in prior off-policy maximum entropy algorithms based on soft Q-learning (Haarnoja et al., 2017). We present a convergence proof for policy iteration in the maximum entropy framework, and then introduce a new algorithm based on an approximation to this procedure that can be practically implemented with deep neural networks, which we call soft actor-critic. We present empirical results that show that soft actor-critic attains a substantial improvement in both performance and sample efficiency over both off-policy and on-policy prior methods. We also compare to twin delayed deep deterministic (TD3) policy gradient algorithm (Fujimoto et al., 2018), which is a concurrent work that proposes a deterministic algorithm that substantially improves on DDPG.

2. Related Work

Our soft actor-critic algorithm incorporates three key ingredients: an actor-critic architecture with separate policy and value function networks, an off-policy formulation that enables reuse of previously collected data for efficiency, and entropy maximization to enable stability and exploration. We review prior works that draw on some of these ideas in this section. Actor-critic algorithms are typically derived starting from policy iteration, which alternates between *policy evaluation*—computing the value function for a policy—and *policy improvement*—using the value function to obtain a better policy (Barto et al., 1983; Sutton & Barto, 1998). In large-scale reinforcement learning problems, it is typically impractical to run either of these steps to convergence, and instead the value function and policy are optimized jointly. In this case, the policy is referred to as the actor, and the value function as the critic. Many actor-critic algorithms build on the standard, on-policy policy gradient formulation to update the actor (Peters & Schaal, 2008), and many of them also consider the entropy of the policy, but instead of maximizing the entropy, they use it as an regularizer (Schulman et al., 2017b; 2015; Mnih et al., 2016; Gruslys et al., 2017). On-policy training tends to improve stability but results in poor sample complexity.

There have been efforts to increase the sample efficiency while retaining robustness by incorporating off-policy samples and by using higher order variance reduction techniques (O'Donoghue et al., 2016; Gu et al., 2016). However, fully off-policy algorithms still attain better efficiency. A particularly popular off-policy actor-critic method, DDPG (Lillicrap et al., 2015), which is a deep variant of the deterministic policy gradient (Silver et al., 2014) algorithm, uses a Q-function estimator to enable off-policy learning, and a deterministic actor that maximizes this Q-function. As such, this method can be viewed both as a deterministic actor-critic algorithm and an approximate Q-learning algorithm. Unfortunately, the interplay between the deterministic actor network and the Q-function typically makes DDPG extremely difficult to stabilize and brittle to hyperparameter settings (Duan et al., 2016; Henderson et al., 2017). As a consequence, it is difficult to extend DDPG to complex, high-dimensional tasks, and on-policy policy gradient methods still tend to produce the best results in such settings (Gu et al., 2016). Our method instead combines off-policy actor-critic training with a *stochastic* actor, and further aims to maximize the entropy of this actor with an entropy maximization objective. We find that this actually results in a considerably more stable and scalable algorithm that, in practice, exceeds both the efficiency and final performance of DDPG. A similar method can be derived as a zero-step special case of stochastic value gradients (SVG(0)) (Heess et al., 2015). However, SVG(0) differs from our method in that it optimizes the standard maximum expected return objective, and it does not make use of a separate value network, which we found to make training more stable.

Maximum entropy reinforcement learning optimizes policies to maximize both the expected return and the expected entropy of the policy. This framework has been used in many contexts, from inverse reinforcement learning (Ziebart et al., 2008) to optimal control (Todorov, 2008; Toussaint, 2009; Rawlik et al., 2012). In guided policy search (Levine & Koltun, 2013; Levine et al., 2016), the maximum entropy distribution is used to guide policy learn-

ing towards high-reward regions. More recently, several papers have noted the connection between Q-learning and policy gradient methods in the framework of maximum entropy learning (O'Donoghue et al., 2016; Haarnoja et al., 2017; Nachum et al., 2017a; Schulman et al., 2017a). While most of the prior model-free works assume a discrete action space, Nachum et al. (2017b) approximate the maximum entropy distribution with a Gaussian and Haarnoja et al. (2017) with a sampling network trained to draw samples from the optimal policy. Although the soft Q-learning algorithm proposed by Haarnoja et al. (2017) has a value function and actor network, it is not a true actor-critic algorithm: the Q-function is estimating the optimal Q-function, and the actor does not directly affect the Q-function except through the data distribution. Hence, Haarnoja et al. (2017) motivates the actor network as an approximate sampler, rather than the actor in an actor-critic algorithm. Crucially, the convergence of this method hinges on how well this sampler approximates the true posterior. In contrast, we prove that our method converges to the optimal policy from a given policy class, regardless of the policy parameterization. Furthermore, these prior maximum entropy methods generally do not exceed the performance of state-of-the-art off-policy algorithms, such as DDPG, when learning from scratch, though they may have other benefits, such as improved exploration and ease of fine-tuning. In our experiments, we demonstrate that our soft actor-critic algorithm does in fact exceed the performance of prior state-of-the-art off-policy deep RL methods by a wide margin.

3. Preliminaries

We first introduce notation and summarize the standard and maximum entropy reinforcement learning frameworks.

3.1. Notation

We address policy learning in continuous action spaces. We consider an infinite-horizon Markov decision process (MDP), defined by the tuple $(\mathcal{S}, \mathcal{A}, p, r)$, where the state space $\mathcal{S}$ and the action space $\mathcal{A}$ are continuous, and the unknown state transition probability $p : \mathcal{S} \times \mathcal{S} \times \mathcal{A} \to [0, \infty)$ represents the probability density of the next state $\mathbf{s}_{t+1} \in \mathcal{S}$ given the current state $\mathbf{s}_t \in \mathcal{S}$ and action $\mathbf{a}_t \in \mathcal{A}$. The environment emits a bounded reward $r : \mathcal{S} \times \mathcal{A} \to [r_{\min}, r_{\max}]$ on each transition. We will use $\rho_\pi(\mathbf{s}_t)$ and $\rho_\pi(\mathbf{s}_t, \mathbf{a}_t)$ to denote the state and state-action marginals of the trajectory distribution induced by a policy $\pi(\mathbf{a}_t|\mathbf{s}_t)$.

3.2. Maximum Entropy Reinforcement Learning

Standard RL maximizes the expected sum of rewards $\sum_t \mathbb{E}_{(\mathbf{s}_t, \mathbf{a}_t) \sim \rho_\pi} [r(\mathbf{s}_t, \mathbf{a}_t)]$. We will consider a more general maximum entropy objective (see e.g. Ziebart (2010)), which favors stochastic policies by augmenting the objective with the expected entropy of the policy over $\rho_\pi(\mathbf{s}_t)$:

$$J(\pi) = \sum_{t=0}^{T} \mathbb{E}_{(\mathbf{s}_t, \mathbf{a}_t) \sim \rho_\pi} \left[r(\mathbf{s}_t, \mathbf{a}_t) + \alpha \mathcal{H}(\pi(\,\cdot\,|\mathbf{s}_t)) \right]. \quad (1)$$

The temperature parameter α determines the relative importance of the entropy term against the reward, and thus controls the stochasticity of the optimal policy. The maximum entropy objective differs from the standard maximum expected reward objective used in conventional reinforcement learning, though the conventional objective can be recovered in the limit as $\alpha \to 0$. For the rest of this paper, we will omit writing the temperature explicitly, as it can always be subsumed into the reward by scaling it by α^{-1}.

This objective has a number of conceptual and practical advantages. First, the policy is incentivized to explore more widely, while giving up on clearly unpromising avenues. Second, the policy can capture multiple modes of near-optimal behavior. In problem settings where multiple actions seem equally attractive, the policy will commit equal probability mass to those actions. Lastly, prior work has observed improved exploration with this objective (Haarnoja et al., 2017; Schulman et al., 2017a), and in our experiments, we observe that it considerably improves learning speed over state-of-art methods that optimize the conventional RL objective function. We can extend the objective to infinite horizon problems by introducing a discount factor γ to ensure that the sum of expected rewards and entropies is finite. Writing down the maximum entropy objective for the infinite horizon discounted case is more involved (Thomas, 2014) and is deferred to Appendix A.

Prior methods have proposed directly solving for the optimal Q-function, from which the optimal policy can be recovered (Ziebart et al., 2008; Fox et al., 2016; Haarnoja et al., 2017). We will discuss how we can devise a soft actor-critic algorithm through a policy iteration formulation, where we instead evaluate the Q-function of the current policy and update the policy through an *off-policy* gradient update. Though such algorithms have previously been proposed for conventional reinforcement learning, our method is, to our knowledge, the first off-policy actor-critic method in the maximum entropy reinforcement learning framework.

4. From Soft Policy Iteration to Soft Actor-Critic

Our off-policy soft actor-critic algorithm can be derived starting from a maximum entropy variant of the policy iteration method. We will first present this derivation, verify that the corresponding algorithm converges to the optimal policy from its density class, and then present a practical deep reinforcement learning algorithm based on this theory.

4.1. Derivation of Soft Policy Iteration

We will begin by deriving soft policy iteration, a general algorithm for learning optimal maximum entropy policies that alternates between policy evaluation and policy improvement in the maximum entropy framework. Our derivation is based on a tabular setting, to enable theoretical analysis and convergence guarantees, and we extend this method into the general continuous setting in the next section. We will show that soft policy iteration converges to the optimal policy within a set of policies which might correspond, for instance, to a set of parameterized densities.

In the policy evaluation step of soft policy iteration, we wish to compute the value of a policy π according to the maximum entropy objective in Equation 1. For a fixed policy, the soft Q-value can be computed iteratively, starting from any function $Q : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ and repeatedly applying a modified Bellman backup operator $\mathcal{T}^\pi$ given by

$$\mathcal{T}^\pi Q(\mathbf{s}_t, \mathbf{a}_t) \triangleq r(\mathbf{s}_t, \mathbf{a}_t) + \gamma \, \mathbb{E}_{\mathbf{s}_{t+1} \sim p}\left[V(\mathbf{s}_{t+1})\right], \quad (2)$$

where

$$V(\mathbf{s}_t) = \mathbb{E}_{\mathbf{a}_t \sim \pi}\left[Q(\mathbf{s}_t, \mathbf{a}_t) - \log \pi(\mathbf{a}_t|\mathbf{s}_t)\right] \quad (3)$$

is the soft state value function. We can obtain the soft value function for any policy π by repeatedly applying $\mathcal{T}^\pi$ as formalized below.

Lemma 1 (Soft Policy Evaluation)**.** *Consider the soft Bellman backup operator $\mathcal{T}^\pi$ in Equation 2 and a mapping $Q^0 : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ with $|\mathcal{A}| < \infty$, and define $Q^{k+1} = \mathcal{T}^\pi Q^k$. Then the sequence Q^k will converge to the soft Q-value of π as $k \rightarrow \infty$.*

Proof. See Appendix B.1. □

In the policy improvement step, we update the policy towards the exponential of the new Q-function. This particular choice of update can be guaranteed to result in an improved policy in terms of its soft value. Since in practice we prefer policies that are tractable, we will additionally restrict the policy to some set of policies Π, which can correspond, for example, to a parameterized family of distributions such as Gaussians. To account for the constraint that $\pi \in \Pi$, we project the improved policy into the desired set of policies. While in principle we could choose any projection, it will turn out to be convenient to use the information projection defined in terms of the Kullback-Leibler divergence. In the other words, in the policy improvement step, for each state, we update the policy according to

$$\pi_{\text{new}} = \arg\min_{\pi' \in \Pi} \mathrm{D}_{\mathrm{KL}}\left(\pi'(\cdot|\mathbf{s}_t) \,\middle\|\, \frac{\exp\left(Q^{\pi_{\text{old}}}(\mathbf{s}_t, \cdot)\right)}{Z^{\pi_{\text{old}}}(\mathbf{s}_t)}\right). \quad (4)$$

The partition function $Z^{\pi_{\text{old}}}(\mathbf{s}_t)$ normalizes the distribution, and while it is intractable in general, it does not contribute to the gradient with respect to the new policy and can thus be ignored, as noted in the next section. For this projection, we can show that the new, projected policy has a higher value than the old policy with respect to the objective in Equation 1. We formalize this result in Lemma 2.

Lemma 2 (Soft Policy Improvement)**.** *Let $\pi_{\text{old}} \in \Pi$ and let π_{new} be the optimizer of the minimization problem defined in Equation 4. Then $Q^{\pi_{\text{new}}}(\mathbf{s}_t, \mathbf{a}_t) \geq Q^{\pi_{\text{old}}}(\mathbf{s}_t, \mathbf{a}_t)$ for all $(\mathbf{s}_t, \mathbf{a}_t) \in \mathcal{S} \times \mathcal{A}$ with $|\mathcal{A}| < \infty$.*

Proof. See Appendix B.2. □

The full soft policy iteration algorithm alternates between the soft policy evaluation and the soft policy improvement steps, and it will provably converge to the optimal maximum entropy policy among the policies in Π (Theorem 1). Although this algorithm will provably find the optimal solution, we can perform it in its exact form only in the tabular case. Therefore, we will next approximate the algorithm for continuous domains, where we need to rely on a function approximator to represent the Q-values, and running the two steps until convergence would be computationally too expensive. The approximation gives rise to a new practical algorithm, called soft actor-critic.

Theorem 1 (Soft Policy Iteration)**.** *Repeated application of soft policy evaluation and soft policy improvement from any $\pi \in \Pi$ converges to a policy π^* such that $Q^{\pi^*}(\mathbf{s}_t, \mathbf{a}_t) \geq Q^{\pi}(\mathbf{s}_t, \mathbf{a}_t)$ for all $\pi \in \Pi$ and $(\mathbf{s}_t, \mathbf{a}_t) \in \mathcal{S} \times \mathcal{A}$, assuming $|\mathcal{A}| < \infty$.*

Proof. See Appendix B.3. □

4.2. Soft Actor-Critic

As discussed above, large continuous domains require us to derive a practical approximation to soft policy iteration. To that end, we will use function approximators for both the Q-function and the policy, and instead of running evaluation and improvement to convergence, alternate between optimizing both networks with stochastic gradient descent. We will consider a parameterized state value function $V_\psi(\mathbf{s}_t)$, soft Q-function $Q_\theta(\mathbf{s}_t, \mathbf{a}_t)$, and a tractable policy $\pi_\phi(\mathbf{a}_t|\mathbf{s}_t)$. The parameters of these networks are ψ, θ, and ϕ. For example, the value functions can be modeled as expressive neural networks, and the policy as a Gaussian with mean and covariance given by neural networks. We will next derive update rules for these parameter vectors.

The state value function approximates the soft value. There is no need in principle to include a separate function approximator for the state value, since it is related to the Q-function and policy according to Equation 3. This quantity can be

estimated from a single action sample from the current policy without introducing a bias, but in practice, including a separate function approximator for the soft value can stabilize training and is convenient to train simultaneously with the other networks. The soft value function is trained to minimize the squared residual error

$$J_V(\psi) = \mathbb{E}_{\mathbf{s}_t \sim \mathcal{D}} \left[\frac{1}{2} \left(V_\psi(\mathbf{s}_t) - \mathbb{E}_{\mathbf{a}_t \sim \pi_\phi} \left[Q_\theta(\mathbf{s}_t, \mathbf{a}_t) - \log \pi_\phi(\mathbf{a}_t | \mathbf{s}_t) \right] \right)^2 \right] \tag{5}$$

where $\mathcal{D}$ is the distribution of previously sampled states and actions, or a replay buffer. The gradient of Equation 5 can be estimated with an unbiased estimator

$$\hat{\nabla}_\psi J_V(\psi) = \nabla_\psi V_\psi(\mathbf{s}_t) \left(V_\psi(\mathbf{s}_t) - Q_\theta(\mathbf{s}_t, \mathbf{a}_t) + \log \pi_\phi(\mathbf{a}_t | \mathbf{s}_t) \right), \tag{6}$$

where the actions are sampled according to the current policy, instead of the replay buffer. The soft Q-function parameters can be trained to minimize the soft Bellman residual

$$J_Q(\theta) = \mathbb{E}_{(\mathbf{s}_t, \mathbf{a}_t) \sim \mathcal{D}} \left[\frac{1}{2} \left(Q_\theta(\mathbf{s}_t, \mathbf{a}_t) - \hat{Q}(\mathbf{s}_t, \mathbf{a}_t) \right)^2 \right], \tag{7}$$

with

$$\hat{Q}(\mathbf{s}_t, \mathbf{a}_t) = r(\mathbf{s}_t, \mathbf{a}_t) + \gamma \, \mathbb{E}_{\mathbf{s}_{t+1} \sim p} \left[V_{\bar{\psi}}(\mathbf{s}_{t+1}) \right], \tag{8}$$

which again can be optimized with stochastic gradients

$$\hat{\nabla}_\theta J_Q(\theta) = \nabla_\theta Q_\theta(\mathbf{a}_t, \mathbf{s}_t) \left(Q_\theta(\mathbf{s}_t, \mathbf{a}_t) - r(\mathbf{s}_t, \mathbf{a}_t) - \gamma V_{\bar{\psi}}(\mathbf{s}_{t+1}) \right). \tag{9}$$

The update makes use of a target value network $V_{\bar{\psi}}$, where $\bar{\psi}$ can be an exponentially moving average of the value network weights, which has been shown to stabilize training (Mnih et al., 2015). Alternatively, we can update the target weights to match the current value function weights periodically (see Appendix E). Finally, the policy parameters can be learned by directly minimizing the expected KL-divergence in Equation 4:

$$J_\pi(\phi) = \mathbb{E}_{\mathbf{s}_t \sim \mathcal{D}} \left[\mathrm{D}_{\mathrm{KL}} \left(\pi_\phi(\cdot | \mathbf{s}_t) \,\middle\|\, \frac{\exp\left(Q_\theta(\mathbf{s}_t, \cdot) \right)}{Z_\theta(\mathbf{s}_t)} \right) \right]. \tag{10}$$

There are several options for minimizing J_π. A typical solution for policy gradient methods is to use the likelihood ratio gradient estimator (Williams, 1992), which does not require backpropagating the gradient through the policy and the target density networks. However, in our case, the target density is the Q-function, which is represented by a neural network an can be differentiated, and it is thus convenient to apply the reparameterization trick instead, resulting in a lower variance estimator. To that end, we reparameterize the policy using a neural network transformation

$$\mathbf{a}_t = f_\phi(\epsilon_t; \mathbf{s}_t), \tag{11}$$

Algorithm 1 Soft Actor-Critic

Initialize parameter vectors ψ, $\bar{\psi}$, θ, ϕ.
for each iteration **do**
  for each environment step **do**
    $\mathbf{a}_t \sim \pi_\phi(\mathbf{a}_t | \mathbf{s}_t)$
    $\mathbf{s}_{t+1} \sim p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)$
    $\mathcal{D} \leftarrow \mathcal{D} \cup \{(\mathbf{s}_t, \mathbf{a}_t, r(\mathbf{s}_t, \mathbf{a}_t), \mathbf{s}_{t+1})\}$
  end for
  for each gradient step **do**
    $\psi \leftarrow \psi - \lambda_V \hat{\nabla}_\psi J_V(\psi)$
    $\theta_i \leftarrow \theta_i - \lambda_Q \hat{\nabla}_{\theta_i} J_Q(\theta_i)$ for $i \in \{1, 2\}$
    $\phi \leftarrow \phi - \lambda_\pi \hat{\nabla}_\phi J_\pi(\phi)$
    $\bar{\psi} \leftarrow \tau \psi + (1 - \tau) \bar{\psi}$
  end for
end for

where ϵ_t is an input noise vector, sampled from some fixed distribution, such as a spherical Gaussian. We can now rewrite the objective in Equation 10 as

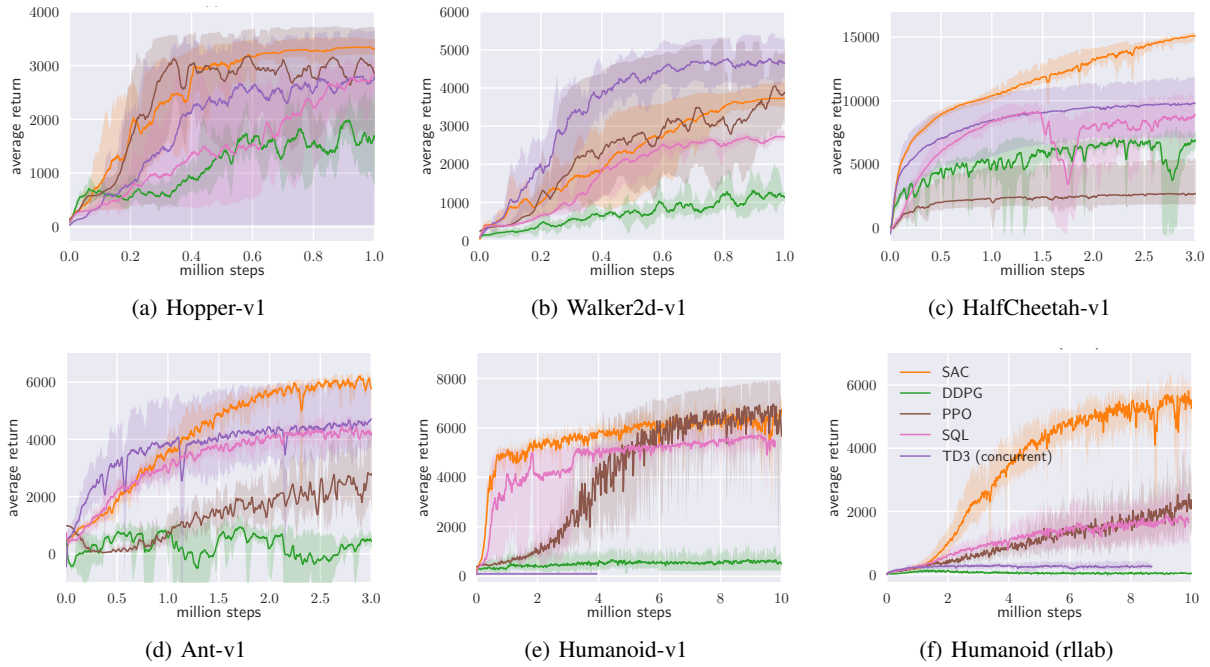
$$J_\pi(\phi) = \mathbb{E}_{\mathbf{s}_t \sim \mathcal{D}, \epsilon_t \sim \mathcal{N}} \left[\log \pi_\phi(f_\phi(\epsilon_t; \mathbf{s}_t) | \mathbf{s}_t) - Q_\theta(\mathbf{s}_t, f_\phi(\epsilon_t; \mathbf{s}_t)) \right], \tag{12}$$

where π_ϕ is defined implicitly in terms of f_ϕ, and we have noted that the partition function is independent of ϕ and can thus be omitted. We can approximate the gradient of Equation 12 with

$$\begin{aligned} \hat{\nabla}_\phi J_\pi(\phi) = {} & \nabla_\phi \log \pi_\phi(\mathbf{a}_t | \mathbf{s}_t) \\ & + \left(\nabla_{\mathbf{a}_t} \log \pi_\phi(\mathbf{a}_t | \mathbf{s}_t) - \nabla_{\mathbf{a}_t} Q(\mathbf{s}_t, \mathbf{a}_t) \right) \nabla_\phi f_\phi(\epsilon_t; \mathbf{s}_t), \end{aligned} \tag{13}$$

where $\mathbf{a}_t$ is evaluated at $f_\phi(\epsilon_t; \mathbf{s}_t)$. This unbiased gradient estimator extends the DDPG style policy gradients (Lillicrap et al., 2015) to any tractable stochastic policy.

Our algorithm also makes use of two Q-functions to mitigate positive bias in the policy improvement step that is known to degrade performance of value based methods (Hasselt, 2010; Fujimoto et al., 2018). In particular, we parameterize two Q-functions, with parameters θ_i, and train them independently to optimize $J_Q(\theta_i)$. We then use the minimum of the Q-functions for the value gradient in Equation 6 and policy gradient in Equation 13, as proposed by Fujimoto et al. (2018). Although our algorithm can learn challenging tasks, including a 21-dimensional Humanoid, using just a single Q-function, we found two Q-functions significantly speed up training, especially on harder tasks. The complete algorithm is described in Algorithm 1. The method alternates between collecting experience from the environment with the current policy and updating the function approximators using the stochastic gradients from batches sampled from a replay buffer. In practice, we take a single environment step followed by one or several gradient steps (see Appendix D



(a) Hopper-v1 (b) Walker2d-v1 (c) HalfCheetah-v1

(d) Ant-v1 (e) Humanoid-v1 (f) Humanoid (rllab)

Figure 1. Training curves on continuous control benchmarks. Soft actor-critic (yellow) performs consistently across all tasks and outperforming both on-policy and off-policy methods in the most challenging tasks.

for all hyperparameter). Using off-policy data from a replay buffer is feasible because both value estimators and the policy can be trained entirely on off-policy data. The algorithm is agnostic to the parameterization of the policy, as long as it can be evaluated for any arbitrary state-action tuple.

5. Experiments

The goal of our experimental evaluation is to understand how the sample complexity and stability of our method compares with prior off-policy and on-policy deep reinforcement learning algorithms. We compare our method to prior techniques on a range of challenging continuous control tasks from the OpenAI gym benchmark suite (Brockman et al., 2016) and also on the rllab implementation of the Humanoid task (Duan et al., 2016). Although the easier tasks can be solved by a wide range of different algorithms, the more complex benchmarks, such as the 21-dimensional Humanoid (rllab), are exceptionally difficult to solve with off-policy algorithms (Duan et al., 2016). The stability of the algorithm also plays a large role in performance: easier tasks make it more practical to tune hyperparameters to achieve good results, while the already narrow basins of effective hyperparameters become prohibitively small for the more sensitive algorithms on the hardest benchmarks, leading to poor performance (Gu et al., 2016).

We compare our method to deep deterministic policy gradient (DDPG) (Lillicrap et al., 2015), an algorithm that is regarded as one of the more efficient off-policy deep RL methods (Duan et al., 2016); proximal policy optimization (PPO) (Schulman et al., 2017b), a stable and effective on-policy policy gradient algorithm; and soft Q-learning (SQL) (Haarnoja et al., 2017), a recent off-policy algorithm for learning maximum entropy policies. Our SQL implementation also includes two Q-functions, which we found to improve its performance in most environments. We additionally compare to twin delayed deep deterministic policy gradient algorithm (TD3) (Fujimoto et al., 2018), using the author-provided implementation. This is an extension to DDPG, proposed concurrently to our method, that first applied the double Q-learning trick to continuous control along with other improvements. We have included trust region path consistency learning (Trust-PCL) (Nachum et al., 2017b) and two other variants of SAC in Appendix E. We turned off the exploration noise for evaluation for DDPG and PPO. For maximum entropy algorithms, which do not explicitly inject exploration noise, we either evaluated with the exploration noise (SQL) or use the mean action (SAC). The source code of our SAC implementation[1] and videos[2] are available online.

[1] github.com/haarnoja/sac

[2] sites.google.com/view/soft-actor-critic

5.1. Comparative Evaluation

Figure 1 shows the total average return of evaluation rollouts during training for DDPG, PPO, and TD3. We train five different instances of each algorithm with different random seeds, with each performing one evaluation rollout every 1000 environment steps. The solid curves corresponds to the mean and the shaded region to the minimum and maximum returns over the five trials.

The results show that, overall, SAC performs comparably to the baseline methods on the easier tasks and outperforms them on the harder tasks with a large margin, both in terms of learning speed and the final performance. For example, DDPG fails to make any progress on Ant-v1, Humanoid-v1, and Humanoid (rllab), a result that is corroborated by prior work (Gu et al., 2016; Duan et al., 2016). SAC also learns considerably faster than PPO as a consequence of the large batch sizes PPO needs to learn stably on more high-dimensional and complex tasks. Another maximum entropy RL algorithm, SQL, can also learn all tasks, but it is slower than SAC and has worse asymptotic performance. The quantitative results attained by SAC in our experiments also compare very favorably to results reported by other methods in prior work (Duan et al., 2016; Gu et al., 2016; Henderson et al., 2017), indicating that both the sample efficiency and final performance of SAC on these benchmark tasks exceeds the state of the art. All hyperparameters used in this experiment for SAC are listed in Appendix D.

5.2. Ablation Study

The results in the previous section suggest that algorithms based on the maximum entropy principle can outperform conventional RL methods on challenging tasks such as the humanoid tasks. In this section, we further examine which particular components of SAC are important for good performance. We also examine how sensitive SAC is to some of the most important hyperparameters, namely reward scaling and target value update smoothing constant.

Stochastic vs. deterministic policy. Soft actor-critic learns stochastic policies via a maximum entropy objective. The entropy appears in both the policy and value function. In the policy, it prevents premature convergence of the policy variance (Equation 10). In the value function, it encourages exploration by increasing the value of regions of state space that lead to high-entropy behavior (Equation 5). To compare how the stochasticity of the policy and entropy maximization affects the performance, we compare to a deterministic variant of SAC that does not maximize the entropy and that closely resembles DDPG, with the exception of having two Q-functions, using hard target updates, not having a separate target actor, and using fixed rather than learned exploration noise. Figure 2 compares five individual runs with both variants, initialized with different random seeds. Soft actor-critic performs much more consistently, while the deterministic variant exhibits very high variability across seeds, indicating substantially worse stability. As evident from the figure, learning a stochastic policy with entropy maximization can drastically stabilize training. This becomes especially important with harder tasks, where tuning hyperparameters is challenging. In this comparison, we updated the target value network weights with hard updates, by periodically overwriting the target network parameters to match the current value network (see Appendix E for a comparison of average performance on all benchmark tasks).

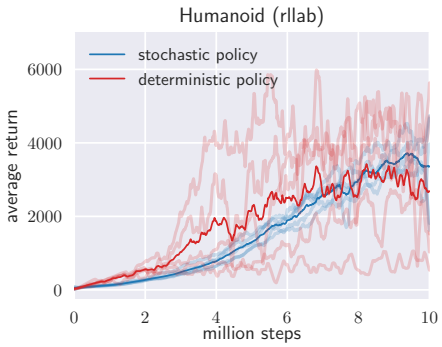
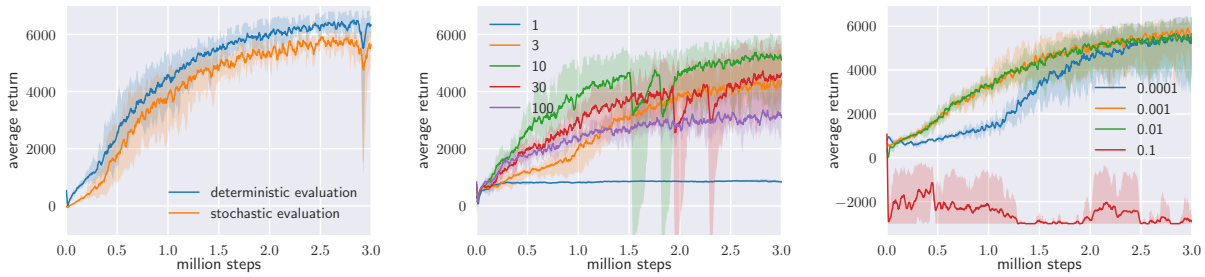


Figure 2. Comparison of SAC (blue) and a deterministic variant of SAC (red) in terms of the stability of individual random seeds on the Humanoid (rllab) benchmark. The comparison indicates that stochasticity can stabilize training as the variability between the seeds becomes much higher with a deterministic policy.

Policy evaluation. Since SAC converges to stochastic policies, it is often beneficial to make the final policy deterministic at the end for best performance. For evaluation, we approximate the maximum a posteriori action by choosing the mean of the policy distribution. Figure 3(a) compares training returns to evaluation returns obtained with this strategy indicating that deterministic evaluation can yield better performance. It should be noted that all of the training curves depict the sum of rewards, which is different from the objective optimized by SAC and other maximum entropy RL algorithms, including SQL and Trust-PCL, which maximize also the entropy of the policy.

Reward scale. Soft actor-critic is particularly sensitive to the scaling of the reward signal, because it serves the role of the temperature of the energy-based optimal policy and thus controls its stochasticity. Larger reward magnitudes correspond to lower entries. Figure 3(b) shows how learning performance changes when the reward scale is varied: For small reward magnitudes, the policy becomes nearly uniform, and consequently fails to exploit the reward signal, resulting in substantial degradation of performance. For large reward magnitudes, the model learns quickly at first,



(a) Evaluation (b) Reward Scale (c) Target Smoothing Coefficient (τ)

Figure 3. Sensitivity of soft actor-critic to selected hyperparameters on Ant-v1 task. (a) Evaluating the policy using the mean action generally results in a higher return. Note that the policy is trained to maximize also the entropy, and the mean action does not, in general, correspond the optimal action for the maximum return objective. (b) Soft actor-critic is sensitive to reward scaling since it is related to the temperature of the optimal policy. The optimal reward scale varies between environments, and should be tuned for each task separately. (c) Target value smoothing coefficient τ is used to stabilize training. Fast moving target (large τ) can result in instabilities (red), whereas slow moving target (small τ) makes training slower (blue).

but the policy then becomes nearly deterministic, leading to poor local minima due to lack of adequate exploration. With the right reward scaling, the model balances exploration and exploitation, leading to faster learning and better asymptotic performance. In practice, we found reward scale to be the only hyperparameter that requires tuning, and its natural interpretation as the inverse of the temperature in the maximum entropy framework provides good intuition for how to adjust this parameter.

Target network update. It is common to use a separate target value network that slowly tracks the actual value function to improve stability. We use an exponentially moving average, with a smoothing constant τ, to update the target value network weights as common in the prior work (Lillicrap et al., 2015; Mnih et al., 2015). A value of one corresponds to a hard update where the weights are copied directly at every iteration and zero to not updating the target at all. In Figure 3(c), we compare the performance of SAC when τ varies. Large τ can lead to instabilities while small τ can make training slower. However, we found the range of suitable values of τ to be relatively wide and we used the same value (0.005) across all of the tasks. In Figure 4 (Appendix E) we also compare to another variant of SAC, where instead of using exponentially moving average, we copy over the current network weights directly into the target network every 1000 gradient steps. We found this variant to benefit from taking more than one gradient step between the environment steps, which can improve performance but also increases the computational cost.

6. Conclusion

We present soft actor-critic (SAC), an off-policy maximum entropy deep reinforcement learning algorithm that provides sample-efficient learning while retaining the benefits of entropy maximization and stability. Our theoretical results derive soft policy iteration, which we show to converge to the optimal policy. From this result, we can formulate a soft actor-critic algorithm, and we empirically show that it outperforms state-of-the-art model-free deep RL methods, including the off-policy DDPG algorithm and the on-policy PPO algorithm. In fact, the sample efficiency of this approach actually exceeds that of DDPG by a substantial margin. Our results suggest that stochastic, entropy maximizing reinforcement learning algorithms can provide a promising avenue for improved robustness and stability, and further exploration of maximum entropy methods, including methods that incorporate second order information (e.g., trust regions (Schulman et al., 2015)) or more expressive policy classes is an exciting avenue for future work.

Acknowledgments

We would like to thank Vitchyr Pong for insightful discussions and help in implementing our algorithm as well as providing the DDPG baseline code; Ofir Nachum for offering support in running Trust-PCL experiments; and George Tucker for his valuable feedback on an early version of this paper. This work was supported by Siemens and Berkeley DeepDrive.

A. Maximum Entropy Objective

The exact definition of the discounted maximum entropy objective is complicated by the fact that, when using a discount factor for policy gradient methods, we typically do not discount the state distribution, only the rewards. In that sense, discounted policy gradients typically do not optimize the true discounted objective. Instead, they optimize average reward, with the discount serving to reduce variance, as discussed by Thomas (2014). However, we can define the objective that *is* optimized under a discount factor as

$$J(\pi) = \sum_{t=0}^{\infty} \mathbb{E}_{(\mathbf{s}_t, \mathbf{a}_t) \sim \rho_\pi} \left[\sum_{l=t}^{\infty} \gamma^{l-t} \, \mathbb{E}_{\mathbf{s}_l \sim p, \mathbf{a}_l \sim \pi} \left[r(\mathbf{s}_t, \mathbf{a}_t) + \alpha \mathcal{H}(\pi(\,\cdot\,|\mathbf{s}_t)) | \mathbf{s}_t, \mathbf{a}_t \right] \right]. \tag{14}$$

This objective corresponds to maximizing the discounted expected reward and entropy for future states originating from every state-action tuple $(\mathbf{s}_t, \mathbf{a}_t)$ weighted by its probability ρ_π under the current policy.

B. Proofs

B.1. Lemma 1

Lemma 1 (Soft Policy Evaluation). *Consider the soft Bellman backup operator $\mathcal{T}^\pi$ in Equation 2 and a mapping $Q^0 : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ with $|\mathcal{A}| < \infty$, and define $Q^{k+1} = \mathcal{T}^\pi Q^k$. Then the sequence Q^k will converge to the soft Q-value of π as $k \rightarrow \infty$.*

Proof. Define the entropy augmented reward as $r_\pi(\mathbf{s}_t, \mathbf{a}_t) \triangleq r(\mathbf{s}_t, \mathbf{a}_t) + \mathbb{E}_{\mathbf{s}_{t+1} \sim p} \left[\mathcal{H} \left(\pi(\,\cdot\,|\mathbf{s}_{t+1}) \right) \right]$ and rewrite the update rule as

$$Q(\mathbf{s}_t, \mathbf{a}_t) \leftarrow r_\pi(\mathbf{s}_t, \mathbf{a}_t) + \gamma \, \mathbb{E}_{\mathbf{s}_{t+1} \sim p, \mathbf{a}_{t+1} \sim \pi} \left[Q(\mathbf{s}_{t+1}, \mathbf{a}_{t+1}) \right] \tag{15}$$

and apply the standard convergence results for policy evaluation (Sutton & Barto, 1998). The assumption $|\mathcal{A}| < \infty$ is required to guarantee that the entropy augmented reward is bounded. □

B.2. Lemma 2

Lemma 2 (Soft Policy Improvement). *Let $\pi_{\text{old}} \in \Pi$ and let π_{new} be the optimizer of the minimization problem defined in Equation 4. Then $Q^{\pi_{\text{new}}}(\mathbf{s}_t, \mathbf{a}_t) \geq Q^{\pi_{\text{old}}}(\mathbf{s}_t, \mathbf{a}_t)$ for all $(\mathbf{s}_t, \mathbf{a}_t) \in \mathcal{S} \times \mathcal{A}$ with $|\mathcal{A}| < \infty$.*

Proof. Let $\pi_{\text{old}} \in \Pi$ and let $Q^{\pi_{\text{old}}}$ and $V^{\pi_{\text{old}}}$ be the corresponding soft state-action value and soft state value, and let π_{new} be defined as

$$\begin{aligned} \pi_{\text{new}}(\,\cdot\,|\mathbf{s}_t) &= \arg\min_{\pi' \in \Pi} \mathrm{D}_{\mathrm{KL}} \left(\pi'(\,\cdot\,|\mathbf{s}_t) \parallel \exp\left(Q^{\pi_{\text{old}}}(\mathbf{s}_t, \,\cdot\,) - \log Z^{\pi_{\text{old}}}(\mathbf{s}_t) \right) \right) \\ &= \arg\min_{\pi' \in \Pi} J_{\pi_{\text{old}}}(\pi'(\,\cdot\,|\mathbf{s}_t)). \end{aligned} \tag{16}$$

It must be the case that $J_{\pi_{\text{old}}}(\pi_{\text{new}}(\,\cdot\,|\mathbf{s}_t)) \leq J_{\pi_{\text{old}}}(\pi_{\text{old}}(\,\cdot\,|\mathbf{s}_t))$, since we can always choose $\pi_{\text{new}} = \pi_{\text{old}} \in \Pi$. Hence

$$\mathbb{E}_{\mathbf{a}_t \sim \pi_{\text{new}}} \left[\log \pi_{\text{new}}(\mathbf{a}_t|\mathbf{s}_t) - Q^{\pi_{\text{old}}}(\mathbf{s}_t, \mathbf{a}_t) + \log Z^{\pi_{\text{old}}}(\mathbf{s}_t) \right] \leq \mathbb{E}_{\mathbf{a}_t \sim \pi_{\text{old}}} \left[\log \pi_{\text{old}}(\mathbf{a}_t|\mathbf{s}_t) - Q^{\pi_{\text{old}}}(\mathbf{s}_t, \mathbf{a}_t) + \log Z^{\pi_{\text{old}}}(\mathbf{s}_t) \right], \tag{17}$$

and since partition function $Z^{\pi_{\text{old}}}$ depends only on the state, the inequality reduces to

$$\mathbb{E}_{\mathbf{a}_t \sim \pi_{\text{new}}} \left[Q^{\pi_{\text{old}}}(\mathbf{s}_t, \mathbf{a}_t) - \log \pi_{\text{new}}(\mathbf{a}_t|\mathbf{s}_t) \right] \geq V^{\pi_{\text{old}}}(\mathbf{s}_t). \tag{18}$$

Next, consider the soft Bellman equation:

$$\begin{aligned} Q^{\pi_{\text{old}}}(\mathbf{s}_t, \mathbf{a}_t) &= r(\mathbf{s}_t, \mathbf{a}_t) + \gamma \, \mathbb{E}_{\mathbf{s}_{t+1} \sim p} \left[V^{\pi_{\text{old}}}(\mathbf{s}_{t+1}) \right] \\ &\leq r(\mathbf{s}_t, \mathbf{a}_t) + \gamma \, \mathbb{E}_{\mathbf{s}_{t+1} \sim p} \left[\mathbb{E}_{\mathbf{a}_{t+1} \sim \pi_{\text{new}}} \left[Q^{\pi_{\text{old}}}(\mathbf{s}_{t+1}, \mathbf{a}_{t+1}) - \log \pi_{\text{new}}(\mathbf{a}_{t+1}|\mathbf{s}_{t+1}) \right] \right] \\ &\vdots \\ &\leq Q^{\pi_{\text{new}}}(\mathbf{s}_t, \mathbf{a}_t), \end{aligned} \tag{19}$$

where we have repeatedly expanded $Q^{\pi_{\text{old}}}$ on the RHS by applying the soft Bellman equation and the bound in Equation 18. Convergence to $Q^{\pi_{\text{new}}}$ follows from Lemma 1. □

B.3. Theorem 1

Theorem 1 (Soft Policy Iteration). *Repeated application of soft policy evaluation and soft policy improvement to any $\pi \in \Pi$ converges to a policy π^* such that $Q^{\pi^*}(\mathbf{s}_t, \mathbf{a}_t) \geq Q^{\pi}(\mathbf{s}_t, \mathbf{a}_t)$ for all $\pi \in \Pi$ and $(\mathbf{s}_t, \mathbf{a}_t) \in \mathcal{S} \times \mathcal{A}$, assuming $|\mathcal{A}| < \infty$.*

Proof. Let π_i be the policy at iteration i. By Lemma 2, the sequence Q^{π_i} is monotonically increasing. Since Q^{π} is bounded above for $\pi \in \Pi$ (both the reward and entropy are bounded), the sequence converges to some π^*. We will still need to show that π^* is indeed optimal. At convergence, it must be case that $J_{\pi^*}(\pi^*(\,\cdot\,|\mathbf{s}_t)) < J_{\pi^*}(\pi(\,\cdot\,|\mathbf{s}_t))$ for all $\pi \in \Pi$, $\pi \neq \pi^*$. Using the same iterative argument as in the proof of Lemma 2, we get $Q^{\pi^*}(\mathbf{s}_t, \mathbf{a}_t) > Q^{\pi}(\mathbf{s}_t, \mathbf{a}_t)$ for all $(\mathbf{s}_t, \mathbf{a}_t) \in \mathcal{S} \times \mathcal{A}$, that is, the soft value of any other policy in Π is lower than that of the converged policy. Hence π^* is optimal in Π. □

C. Enforcing Action Bounds

We use an unbounded Gaussian as the action distribution. However, in practice, the actions needs to be bounded to a finite interval. To that end, we apply an invertible squashing function ($\tanh$) to the Gaussian samples, and employ the change of variables formula to compute the likelihoods of the bounded actions. In the other words, let $\mathbf{u} \in \mathbb{R}^D$ be a random variable and $\mu(\mathbf{u}|\mathbf{s})$ the corresponding density with infinite support. Then $\mathbf{a} = \tanh(\mathbf{u})$, where $\tanh$ is applied elementwise, is a random variable with support in $(-1, 1)$ with a density given by

$$\pi(\mathbf{a}|\mathbf{s}) = \mu(\mathbf{u}|\mathbf{s}) \left| \det \left(\frac{\mathrm{d}\mathbf{a}}{\mathrm{d}\mathbf{u}} \right) \right|^{-1}. \tag{20}$$

Since the Jacobian $\mathrm{d}\mathbf{a}/\mathrm{d}\mathbf{u} = \mathrm{diag}(1 - \tanh^2(\mathbf{u}))$ is diagonal, the log-likelihood has a simple form

$$\log \pi(\mathbf{a}|\mathbf{s}) = \log \mu(\mathbf{u}|\mathbf{s}) - \sum_{i=1}^{D} \log \left(1 - \tanh^2(u_i)\right), \tag{21}$$

where u_i is the i^{th} element of $\mathbf{u}$.

D. Hyperparameters

Table 1 lists the common SAC parameters used in the comparative evaluation in Figure 1 and Figure 4. Table 2 lists the reward scale parameter that was tuned for each environment.

Table 1. SAC Hyperparameters

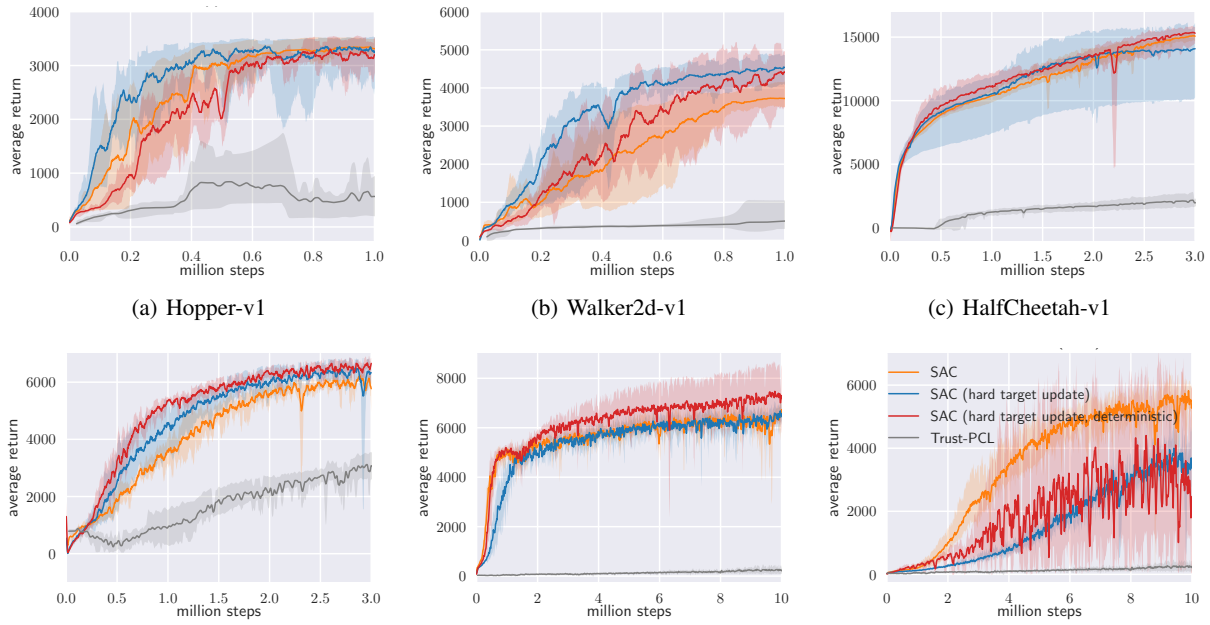
Parameter	Value
Shared	
optimizer	Adam (Kingma & Ba, 2015)
learning rate	$3 \cdot 10^{-4}$
discount (γ)	0.99
replay buffer size	10^6
number of hidden layers (all networks)	2
number of hidden units per layer	256
number of samples per minibatch	256
nonlinearity	ReLU
SAC	
target smoothing coefficient (τ)	0.005
target update interval	1
gradient steps	1
SAC (hard target update)	
target smoothing coefficient (τ)	1
target update interval	1000
gradient steps (except humanoids)	4
gradient steps (humanoids)	1

Table 2. SAC Environment Specific Parameters

Environment	Action Dimensions	Reward Scale
Hopper-v1	3	5
Walker2d-v1	6	5
HalfCheetah-v1	6	5
Ant-v1	8	5
Humanoid-v1	17	20
Humanoid (rllab)	21	10

E. Additional Baseline Results

Figure 4 compares SAC to Trust-PCL (Figure 4. Trust-PC fails to solve most of the task within the given number of environment steps, although it can eventually solve the easier tasks (Nachum et al., 2017b) if ran longer. The figure also includes two variants of SAC: a variant that periodically copies the target value network weights directly instead of using exponentially moving average, and a deterministic ablation which assumes a deterministic policy in the value update (Equation 6) and the policy update (Equation 13), and thus strongly resembles DDPG with the exception of having two Q-functions, using hard target updates, not having a separate target actor, and using fixed exploration noise rather than learned. Both of these methods can learn all of the tasks and they perform comparably to SAC on all but Humanoid (rllab) task, on which SAC is the fastest.



(a) Hopper-v1 (b) Walker2d-v1 (c) HalfCheetah-v1

(d) Ant-v1 (e) Humanoid-v1 (f) Humanoid (rllab)

Figure 4. Training curves for additional baseline (Trust-PCL) and for two SAC variants. Soft actor-critic with hard target update (blue) differs from standard SAC in that it copies the value function network weights directly every 1000 iterations, instead of using exponentially smoothed average of the weights. The deterministic ablation (red) uses a deterministic policy with fixed Gaussian exploration noise, does not use a value function, drops the entropy terms in the actor and critic function updates, and uses hard target updates for the target Q-functions. It is equivalent to DDPG that uses two Q-functions, hard target updates, and removes the target actor.

Soft Actor-Critic Algorithms and Applications

Tuomas Haarnoja[*†‡] **Aurick Zhou**[*†] **Kristian Hartikainen**[*†] **George Tucker**[‡]

Sehoon Ha[‡] **Jie Tan**[‡] **Vikash Kumar**[‡] **Henry Zhu**[†] **Abhishek Gupta**[†]

Pieter Abbeel[†] **Sergey Levine**[†‡]

Abstract

Model-free deep reinforcement learning (RL) algorithms have been successfully applied to a range of challenging sequential decision making and control tasks. However, these methods typically suffer from two major challenges: high sample complexity and brittleness to hyperparameters. Both of these challenges limit the applicability of such methods to real-world domains. In this paper, we describe Soft Actor-Critic (SAC), our recently introduced off-policy actor-critic algorithm based on the maximum entropy RL framework. In this framework, the actor aims to simultaneously maximize expected return and entropy; that is, to succeed at the task while acting as randomly as possible. We extend SAC to incorporate a number of modifications that accelerate training and improve stability with respect to the hyperparameters, including a constrained formulation that automatically tunes the temperature hyperparameter. We systematically evaluate SAC on a range of benchmark tasks, as well as challenging real-world tasks such as locomotion for a quadrupedal robot and robotic manipulation with a dexterous hand. With these improvements, SAC achieves state-of-the-art performance, outperforming prior on-policy and off-policy methods in sample-efficiency and asymptotic performance. Furthermore, we demonstrate that, in contrast to other off-policy algorithms, our approach is very stable, achieving similar performance across different random seeds. These results suggest that SAC is a promising candidate for learning in real-world robotics tasks.

1 Introduction

Model-free deep reinforcement learning (RL) algorithms have been applied in a range of challenging domains, from games (Mnih et al., 2013; Silver et al., 2016) to robotic control (Gu et al., 2017; Haarnoja et al., 2018b). The combination of RL and high-capacity function approximators such as neural networks holds the promise of automating a wide range of decision making and control tasks, but widespread adoption of these methods in real-world domains has been hampered by two major challenges. First, model-free deep RL methods are notoriously expensive in terms of their sample complexity. Even relatively simple tasks can require millions of steps of data collection, and complex behaviors with high-dimensional observations might need substantially more. Second, these methods are often brittle with respect to their hyperparameters: learning rates, exploration constants, and other settings must be set carefully for different problem settings to achieve good results. Both of these challenges severely limit the applicability of model-free deep RL to real-world tasks.

One cause for the poor sample efficiency of deep RL methods is on-policy learning: some of the most commonly used deep RL algorithms, such as TRPO (Schulman et al., 2015), PPO (Schulman et al.,

[†]UC Berkeley, [‡]Google Brain, [*]Contributed equally

2017b) or A3C (Mnih et al., 2016), require new samples to be collected for (nearly) every update to the policy. This quickly becomes extravagantly expensive, as the number of gradient steps and samples per step needed to learn an effective policy increases with task complexity. Off-policy algorithms aim to reuse past experience. This is not directly feasible with conventional policy gradient formulations, but is relatively straightforward for Q-learning based methods (Mnih et al., 2015). Unfortunately, the combination of off-policy learning and high-dimensional, nonlinear function approximation with neural networks presents a major challenge for stability and convergence (Bhatnagar et al., 2009). This challenge is further exacerbated in continuous state and action spaces, where a separate actor network is often used to perform the maximization in Q-learning.

In (Haarnoja et al., 2018c), we introduced the Soft Actor-Critic (SAC) algorithm based on the maximum entropy framework (Ziebart et al., 2008; Toussaint, 2009; Rawlik et al., 2012; Fox et al., 2016; Haarnoja et al., 2017). In the first sections of this paper, we summarize the SAC algorithm, describe the reasoning behind the design choices, and present key theoretical results from (Haarnoja et al., 2018c). Unfortunately, SAC as presented in (Haarnoja et al., 2018c) can suffer from brittleness to the temperature hyperparameter. Unlike in conventional reinforcement learning, where the optimal policy is independent of scaling of the reward function, in maximum entropy reinforcement learning the scaling factor has to be compensated by the choice a of suitable temperature, and a sub-optimal temperature can drastically degrade performance (Haarnoja et al., 2018c). To resolve this issue, we devise an automatic gradient-based temperature tuning method that adjusts the expected entropy over the visited states to match a target value. Although this modification is technically simple, we find that in practice it largely eliminates the need for per-task hyperparameter tuning. Finally, we present empirical results that show that Soft Actor-Critic attains a substantial improvement in both performance and sample efficiency over prior off-policy and on-policy methods including the recently introduced twin delayed deep deterministic (TD3) policy gradient algorithm (Fujimoto et al., 2018). We also evaluate our method on real-world challenging tasks such as locomotion for a quadrupedal robot and robotic manipulation with a dexterous hand from image observations.

2 Related Work

Maximum entropy reinforcement learning generalizes the expected return RL objective, although the original objective can be recovered in the zero temperature limit (Haarnoja et al., 2017). More importantly, the maximum entropy formulation provides a substantial improvement in exploration and robustness: as discussed by Ziebart (2010), maximum entropy policies are robust in the face of model and estimation errors, and as demonstrated by (Haarnoja et al., 2017), they improve exploration by acquiring diverse behaviors. Prior work has proposed model-free deep RL algorithms that perform on-policy learning with entropy maximization (O'Donoghue et al., 2016), as well as off-policy methods based on soft Q-learning and its variants (Schulman et al., 2017a; Nachum et al., 2017a; Haarnoja et al., 2017). However, the on-policy variants suffer from poor sample complexity for the reasons discussed above, while the off-policy variants require complex approximate inference procedures in continuous action spaces.

Our soft actor-critic algorithm incorporates three key ingredients: an actor-critic architecture with separate policy and value function networks, an off-policy formulation that enables reuse of previously collected data for efficiency, and entropy maximization to encourage stability and exploration. We review prior work that draw on some of these ideas in this section. Actor-critic algorithms are typically derived starting from policy iteration, which alternates between *policy evaluation*—computing the value function for a policy—and *policy improvement*—using the value function to obtain a better policy (Barto et al., 1983; Sutton & Barto, 1998). In large-scale reinforcement learning problems, it is typically impractical to run either of these steps to convergence, and instead the value function and policy are optimized jointly. In this case, the policy is referred to as the actor, and the value function as the critic. Many actor-critic algorithms build on the standard, on-policy policy gradient formulation to update the actor (Peters & Schaal, 2008), and many of them also consider the entropy of the policy, but instead of maximizing the entropy, they use it as an regularizer (Schulman et al., 2017b, 2015; Mnih et al., 2016; Gruslys et al., 2017). On-policy training tends to improve stability but results in poor sample complexity.

There have been efforts to increase the sample efficiency while retaining robustness by incorporating off-policy samples and by using higher order variance reduction techniques (O'Donoghue et al., 2016; Gu et al., 2016). However, fully off-policy algorithms still attain better efficiency. A particularly

popular off-policy actor-critic method, DDPG (Lillicrap et al., 2015), which is a deep variant of the deterministic policy gradient (Silver et al., 2014) algorithm, uses a Q-function estimator to enable off-policy learning, and a deterministic actor that maximizes this Q-function. As such, this method can be viewed both as a deterministic actor-critic algorithm and an approximate Q-learning algorithm. Unfortunately, the interplay between the deterministic actor network and the Q-function typically makes DDPG extremely difficult to stabilize and brittle to hyperparameter settings (Duan et al., 2016; Henderson et al., 2017). As a consequence, it is difficult to extend DDPG to complex, high-dimensional tasks, and on-policy policy gradient methods still tend to produce the best results in such settings (Gu et al., 2016). Our method instead combines off-policy actor-critic training with a *stochastic* actor, and further aims to maximize the entropy of this actor with an entropy maximization objective. We find that this actually results in a considerably more stable and scalable algorithm that, in practice, exceeds both the efficiency and final performance of DDPG. A similar method can be derived as a zero-step special case of stochastic value gradients (SVG(0)) (Heess et al., 2015). However, SVG(0) differs from our method in that it optimizes the standard maximum expected return objective.

Maximum entropy reinforcement learning optimizes policies to maximize both the expected return and the expected entropy of the policy. This framework has been used in many contexts, from inverse reinforcement learning (Ziebart et al., 2008) to optimal control (Todorov, 2008; Toussaint, 2009; Rawlik et al., 2012). Maximum a posteriori policy optimization (MPO) makes use of the probablistic view and optimizes the standard RL objective via expectation maximization (Abdolmaleki et al., 2018). In guided policy search (Levine & Koltun, 2013; Levine et al., 2016), the maximum entropy distribution is used to guide policy learning towards high-reward regions. More recently, several papers have noted the connection between Q-learning and policy gradient methods in the framework of maximum entropy learning (O'Donoghue et al., 2016; Haarnoja et al., 2017; Nachum et al., 2017a; Schulman et al., 2017a). While most of the prior model-free works assume a discrete action space, Nachum et al. (2017b) approximate the maximum entropy distribution with a Gaussian, and Haarnoja et al. (2017) with a sampling network trained to draw samples from the optimal policy. Although the soft Q-learning algorithm proposed by Haarnoja et al. (2017) has a value function and actor network, it is not a true actor-critic algorithm: the Q-function is estimating the optimal Q-function, and the actor does not directly affect the Q-function except through the data distribution. Hence, Haarnoja et al. (2017) motivates the actor network as an approximate sampler, rather than the actor in an actor-critic algorithm. Crucially, the convergence of this method hinges on how well this sampler approximates the true posterior. In contrast, we prove that our method converges to the optimal policy from a given policy class, regardless of the policy parameterization. Furthermore, these prior maximum entropy methods generally do not exceed the performance of state-of-the-art off-policy algorithms, such as TD3 (Fujimoto et al., 2018) or MPO (Abdolmaleki et al., 2018), when learning from scratch, though they may have other benefits, such as improved exploration and ease of fine-tuning.

3 Preliminaries

We first introduce notation and summarize the standard and maximum entropy reinforcement learning frameworks.

3.1 Notation

We will address learning of maximum entropy policies in continuous action spaces. Our reinforcement learning problem can be defined as policy search in an a Markov decision process (MDP), defined by a tuple $(\mathcal{S}, \mathcal{A}, p, r)$. The state space $\mathcal{S}$ and action space $\mathcal{A}$ are assumed to be continuous, and the state transition probability $p : \mathcal{S} \times \mathcal{S} \times \mathcal{A} \to [0, \infty)$ represents the probability density of the next state $\mathbf{s}_{t+1} \in \mathcal{S}$ given the current state $\mathbf{s}_t \in \mathcal{S}$ and action $\mathbf{a}_t \in \mathcal{A}$. The environment emits a reward $r : \mathcal{S} \times \mathcal{A} \to [r_{\min}, r_{\max}]$ on each transition. We will also use $\rho_\pi(\mathbf{s}_t)$ and $\rho_\pi(\mathbf{s}_t, \mathbf{a}_t)$ to denote the state and state-action marginals of the trajectory distribution induced by a policy $\pi(\mathbf{a}_t|\mathbf{s}_t)$.

3.2 Maximum Entropy Reinforcement Learning

The standard reinforcement learning objective is the expected sum of rewards $\sum_t \mathbb{E}_{(\mathbf{s}_t, \mathbf{a}_t) \sim \rho_\pi} [r(\mathbf{s}_t, \mathbf{a}_t)]$ and our goal is to learn a policy $\pi(\mathbf{a}_t|\mathbf{s}_t)$ that maximizes that ob-

jective. The maximum entropy objective (see e.g. (Ziebart, 2010) generalizes the standard objective by augmenting it with an entropy term, such that the optimal policy additionally aims to maximize its entropy at each visited state:

$$\pi^* = \arg\max_{\pi} \sum_t \mathbb{E}_{(\mathbf{s}_t, \mathbf{a}_t) \sim \rho_\pi} \left[r(\mathbf{s}_t, \mathbf{a}_t) + \alpha \mathcal{H}(\pi(\cdot\,|\mathbf{s}_t)) \right], \tag{1}$$

where α is the temperature parameter that determines the relative importance of the entropy term versus the reward, and thus controls the stochasticity of the optimal policy. Although the maximum entropy objective differs from the standard maximum expected return objective used in conventional reinforcement learning, the conventional objective can be recovered in the limit as $\alpha \to 0$. If we wish to extend either the conventional or the maximum entropy RL objective to infinite horizon problems, it is convenient to also introduce a discount factor γ to ensure that the sum of expected rewards (and entropies) is finite. In the context of policy search algorithms, the use of a discount factor is actually a somewhat nuanced choice, and writing down the precise objective that is optimized when using the discount factor is non-trivial (Thomas, 2014). We include the discounted, infinite-horizon objective in Appendix A, but we will use the discount γ in the following derivations and in our final algorithm.

The maximum entropy objective has a number of conceptual and practical advantages. First, the policy is incentivized to explore more widely, while giving up on clearly unpromising avenues. Second, the policy can capture multiple modes of near-optimal behavior. In problem settings where multiple actions seem equally attractive, the policy will commit equal probability mass to those actions. In practice, we observe improved exploration with this objective, as also has been reported in the prior work (Schulman et al., 2017a), and we observe that it considerably improves learning speed over state-of-art methods that optimize the conventional RL objective function.

Optimization problems of this type have been explored in a number of prior works (Kappen, 2005; Todorov, 2007; Ziebart et al., 2008). These prior methods have proposed directly solving for the optimal Q-function, from which the optimal policy can be recovered (Ziebart et al., 2008; Fox et al., 2016; Haarnoja et al., 2017). In the following section, we discuss how we can devise a soft actor-critic algorithm through a policy iteration formulation, where we instead evaluate the Q-function of the current policy and update the policy through an *off-policy* gradient update. Though such algorithms have previously been proposed for conventional reinforcement learning, our method is, to our knowledge, the first off-policy actor-critic method in the maximum entropy reinforcement learning framework.

4 From Soft Policy Iteration to Soft Actor-Critic

Our off-policy soft actor-critic algorithm can be derived starting from a maximum entropy variant of the policy iteration method. We will first present this derivation, verify that the corresponding algorithm converges to the optimal policy from its density class, and then present a practical deep reinforcement learning algorithm based on this theory. In this section, we treat the temperature as a constant, and later in Section 5 propose an extension to SAC that adjusts the temperature automatically to match an entropy target in expectation.

4.1 Soft Policy Iteration

We will begin by deriving soft policy iteration, a general algorithm for learning optimal maximum entropy policies that alternates between policy evaluation and policy improvement in the maximum entropy framework. Our derivation is based on a tabular setting, to enable theoretical analysis and convergence guarantees, and we extend this method into the general continuous setting in the next section. We will show that soft policy iteration converges to the optimal policy within a set of policies which might correspond, for instance, to a set of parameterized densities.

In the policy evaluation step of soft policy iteration, we wish to compute the value of a policy π according to the maximum entropy objective. For a fixed policy, the soft Q-value can be computed iteratively, starting from any function $Q : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ and repeatedly applying a modified Bellman backup operator $\mathcal{T}^\pi$ given by

$$\mathcal{T}^\pi Q(\mathbf{s}_t, \mathbf{a}_t) \triangleq r(\mathbf{s}_t, \mathbf{a}_t) + \gamma\, \mathbb{E}_{\mathbf{s}_{t+1} \sim p} \left[V(\mathbf{s}_{t+1}) \right], \tag{2}$$

where

$$V(\mathbf{s}_t) = \mathbb{E}_{\mathbf{a}_t \sim \pi}\left[Q(\mathbf{s}_t, \mathbf{a}_t) - \alpha \log \pi(\mathbf{a}_t|\mathbf{s}_t)\right] \tag{3}$$

is the soft state value function. We can obtain the soft Q-function for any policy π by repeatedly applying $\mathcal{T}^\pi$ as formalized below.

Lemma 1 (Soft Policy Evaluation)**.** *Consider the soft Bellman backup operator $\mathcal{T}^\pi$ in Equation 2 and a mapping $Q^0 : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ with $|\mathcal{A}| < \infty$, and define $Q^{k+1} = \mathcal{T}^\pi Q^k$. Then the sequence Q^k will converge to the soft Q-function of π as $k \rightarrow \infty$.*

Proof. See Appendix B.1. □

In the policy improvement step, we update the policy towards the exponential of the new soft Q-function. This particular choice of update can be guaranteed to result in an improved policy in terms of its soft value. Since in practice we prefer policies that are tractable, we will additionally restrict the policy to some set of policies Π, which can correspond, for example, to a parameterized family of distributions such as Gaussians. To account for the constraint that $\pi \in \Pi$, we project the improved policy into the desired set of policies. While in principle we could choose any projection, it will turn out to be convenient to use the information projection defined in terms of the Kullback-Leibler divergence. In the other words, in the policy improvement step, for each state, we update the policy according to

$$\pi_{\text{new}} = \arg\min_{\pi' \in \Pi} \mathrm{D}_{\mathrm{KL}}\left(\pi'(\,\cdot\,|\mathbf{s}_t) \,\middle\|\, \frac{\exp\left(\frac{1}{\alpha} Q^{\pi_{\text{old}}}(\mathbf{s}_t, \,\cdot\,)\right)}{Z^{\pi_{\text{old}}}(\mathbf{s}_t)}\right). \tag{4}$$

The partition function $Z^{\pi_{\text{old}}}(\mathbf{s}_t)$ normalizes the distribution, and while it is intractable in general, it does not contribute to the gradient with respect to the new policy and can thus be ignored. For this projection, we can show that the new, projected policy has a higher value than the old policy with respect to the maximum entropy objective. We formalize this result in Lemma 2.

Lemma 2 (Soft Policy Improvement)**.** *Let $\pi_{\text{old}} \in \Pi$ and let π_{new} be the optimizer of the minimization problem defined in Equation 4. Then $Q^{\pi_{\text{new}}}(\mathbf{s}_t, \mathbf{a}_t) \geq Q^{\pi_{\text{old}}}(\mathbf{s}_t, \mathbf{a}_t)$ for all $(\mathbf{s}_t, \mathbf{a}_t) \in \mathcal{S} \times \mathcal{A}$ with $|\mathcal{A}| < \infty$.*

Proof. See Appendix B.2. □

The full soft policy iteration algorithm alternates between the soft policy evaluation and the soft policy improvement steps, and it will provably converge to the optimal maximum entropy policy among the policies in Π (Theorem 1). Although this algorithm will provably find the optimal solution, we can perform it in its exact form only in the tabular case. Therefore, we will next approximate the algorithm for continuous domains, where we need to rely on a function approximator to represent the Q-values, and running the two steps until convergence would be computationally too expensive. The approximation gives rise to a new practical algorithm, called soft actor-critic.

Theorem 1 (Soft Policy Iteration)**.** *Repeated application of soft policy evaluation and soft policy improvement from any $\pi \in \Pi$ converges to a policy π^* such that $Q^{\pi^*}(\mathbf{s}_t, \mathbf{a}_t) \geq Q^{\pi}(\mathbf{s}_t, \mathbf{a}_t)$ for all $\pi \in \Pi$ and $(\mathbf{s}_t, \mathbf{a}_t) \in \mathcal{S} \times \mathcal{A}$, assuming $|\mathcal{A}| < \infty$.*

Proof. See Appendix B.3. □

4.2 Soft Actor-Critic

As discussed above, large continuous domains require us to derive a practical approximation to soft policy iteration. To that end, we will use function approximators for both the soft Q-function and the policy, and instead of running evaluation and improvement to convergence, alternate between optimizing both networks with stochastic gradient descent. We will consider a parameterized soft Q-function $Q_\theta(\mathbf{s}_t, \mathbf{a}_t)$ and a tractable policy $\pi_\phi(\mathbf{a}_t|\mathbf{s}_t)$. The parameters of these networks are θ and ϕ. For example, the soft Q-function can be modeled as expressive neural networks, and the policy as a Gaussian with mean and covariance given by neural networks. We will next derive update rules for these parameter vectors.

The soft Q-function parameters can be trained to minimize the soft Bellman residual

$$J_Q(\theta) = \mathbb{E}_{(\mathbf{s}_t,\mathbf{a}_t)\sim\mathcal{D}}\left[\frac{1}{2}\left(Q_\theta(\mathbf{s}_t,\mathbf{a}_t) - \left(r(\mathbf{s}_t,\mathbf{a}_t) + \gamma\,\mathbb{E}_{\mathbf{s}_{t+1}\sim p}\left[V_{\bar{\theta}}(\mathbf{s}_{t+1})\right]\right)\right)^2\right], \tag{5}$$

where the value function is implicitly parameterized through the soft Q-function parameters via Equation 3, and it can be optimized with stochastic gradients[1]

$$\hat{\nabla}_\theta J_Q(\theta) = \nabla_\theta Q_\theta(\mathbf{a}_t,\mathbf{s}_t)\left(Q_\theta(\mathbf{s}_t,\mathbf{a}_t) - \left(r(\mathbf{s}_t,\mathbf{a}_t) + \gamma\left(Q_{\bar{\theta}}(\mathbf{s}_{t+1},\mathbf{a}_{t+1}) - \alpha\log\left(\pi_\phi(\mathbf{a}_{t+1}|\mathbf{s}_{t+1})\right)\right)\right)\right). \tag{6}$$

The update makes use of a target soft Q-function with parameters $\bar{\theta}$ that are obtained as an exponentially moving average of the soft Q-function weights, which has been shown to stabilize training (Mnih et al., 2015). Finally, the policy parameters can be learned by directly minimizing the expected KL-divergence in Equation 4 (multiplied by α and ignoring the constant log-partition function and by α):

$$J_\pi(\phi) = \mathbb{E}_{\mathbf{s}_t\sim\mathcal{D}}\left[\mathbb{E}_{\mathbf{a}_t\sim\pi_\phi}\left[\alpha\log\left(\pi_\phi(\mathbf{a}_t|\mathbf{s}_t)\right) - Q_\theta(\mathbf{s}_t,\mathbf{s}_t)\right]\right] \tag{7}$$

There are several options for minimizing J_π. A typical solution for policy gradient methods is to use the likelihood ratio gradient estimator (Williams, 1992), which does not require backpropagating the gradient through the policy and the target density networks. However, in our case, the target density is the Q-function, which is represented by a neural network an can be differentiated, and it is thus convenient to apply the reparameterization trick instead, resulting in a lower variance estimator. To that end, we reparameterize the policy using a neural network transformation

$$\mathbf{a}_t = f_\phi(\epsilon_t;\mathbf{s}_t), \tag{8}$$

where ϵ_t is an input noise vector, sampled from some fixed distribution, such as a spherical Gaussian. We can now rewrite the objective in Equation 7 as

$$J_\pi(\phi) = \mathbb{E}_{\mathbf{s}_t\sim\mathcal{D},\epsilon_t\sim\mathcal{N}}\left[\alpha\log\pi_\phi(f_\phi(\epsilon_t;\mathbf{s}_t)|\mathbf{s}_t) - Q_\theta(\mathbf{s}_t, f_\phi(\epsilon_t;\mathbf{s}_t))\right], \tag{9}$$

where π_ϕ is defined implicitly in terms of f_ϕ. We can approximate the gradient of Equation 9 with

$$\hat{\nabla}_\phi J_\pi(\phi) = \nabla_\phi\alpha\log\left(\pi_\phi(\mathbf{a}_t|\mathbf{s}_t)\right) + \left(\nabla_{\mathbf{a}_t}\alpha\log\left(\pi_\phi(\mathbf{a}_t|\mathbf{s}_t)\right) - \nabla_{\mathbf{a}_t}Q(\mathbf{s}_t,\mathbf{a}_t)\right)\nabla_\phi f_\phi(\epsilon_t;\mathbf{s}_t), \tag{10}$$

where $\mathbf{a}_t$ is evaluated at $f_\phi(\epsilon_t;\mathbf{s}_t)$. This unbiased gradient estimator extends the DDPG style policy gradients (Lillicrap et al., 2015) to any tractable stochastic policy.

5 Automating Entropy Adjustment for Maximum Entropy RL

In the previous section, we derived a practical off-policy algorithm for learning maximum entropy policies of a given temperature. Unfortunately, choosing the optimal temperature is non-trivial, and the temperature needs to be tuned for each task. Instead of requiring the user to set the temperature manually, we can automate this process by formulating a different maximum entropy reinforcement learning objective, where the entropy is treated as a constraint. The magnitude of the reward differs not only across tasks, but it also depends on the policy, which improves over time during training. Since the optimal entropy depends on this magnitude, this makes the temperature adjustment particularly difficult: the entropy can vary unpredictably both across tasks and during training as the policy becomes better. Simply forcing the entropy to a fixed value is a poor solution, since the policy should be free to explore more in regions where the optimal action is uncertain, but remain more deterministic in states with a clear distinction between good and bad actions. Instead, we formulate a constrained optimization problem where the average entropy of the policy is constrained, while the entropy at different states can vary. Similar approach was taken in (Abdolmaleki et al., 2018), where the policy was constrained to remain close to the previous policy. We show that the dual to this constrained optimization leads to the soft actor-critic updates, along with an additional update for the dual variable, which plays the role of the temperature. Our formulation also makes it possible to learn the entropy with more expressive policies that can model multi-modal distributions, such as policies based on normalizing flows (Haarnoja et al., 2018a) for which no closed form expression for

[1] In (Haarnoja et al., 2018c) we introduced an additional function approximator for the value function, but later found it to be unnecessary.

the entropy exists. We will derive the update for finite horizon case, and then derive an approximation for stationary policies by dropping the time dependencies from the policy, soft Q-function, and the temperature.

Our aim is to find a stochastic policy with maximal expected return that satisfies a minimum expected entropy constraint. Formally, we want to solve the constrained optimization problem

$$\max_{\pi_{0:T}} \mathbb{E}_{\rho_\pi}\left[\sum_{t=0}^{T} r(\mathbf{s}_t, \mathbf{a}_t)\right] \text{ s.t. } \mathbb{E}_{(\mathbf{s}_t,\mathbf{a}_t)\sim\rho_\pi}\left[-\log(\pi_t(\mathbf{a}_t|\mathbf{s}_t))\right] \geq \mathcal{H} \;\; \forall t \tag{11}$$

where $\mathcal{H}$ is a desired minimum expected entropy. Note that, for fully observed MDPs, the policy that optimizes the expected return is deterministic, so we expect this constraint to usually be tight and do not need to impose an upper bound on the entropy.

Since the policy at time t can only affect the future objective value, we can employ an (approximate) dynamic programming approach, solving for the policy backward through time. We rewrite the objective as an iterated maximization

$$\max_{\pi_0}\left(\mathbb{E}\left[r(\mathbf{s}_0, \mathbf{a}_0)\right] + \max_{\pi_1}\left(\mathbb{E}\left[\ldots\right] + \max_{\pi_T}\mathbb{E}\left[r(\mathbf{s}_T, \mathbf{a}_T)\right]\right)\right), \tag{12}$$

subject to the constraint on entropy. Starting from the last time step, we change the constrained maximization to the dual problem. Subject to $\mathbb{E}_{(\mathbf{s}_T,\mathbf{a}_T)\sim\rho_\pi}\left[-\log(\pi_T(\mathbf{s}_T|\mathbf{s}_T))\right] \geq \mathcal{H}$,

$$\max_{\pi_T} \mathbb{E}_{(\mathbf{s}_t,\mathbf{a}_t)\sim\rho_\pi}\left[r(\mathbf{s}_T, \mathbf{a}_T)\right] = \min_{\alpha_T\geq 0}\max_{\pi_T}\mathbb{E}\left[r(\mathbf{s}_T, \mathbf{a}_T) - \alpha_T \log\pi(\mathbf{a}_T|\mathbf{s}_T)\right] - \alpha_T\mathcal{H}, \tag{13}$$

where α_T is the dual variable. We have also used strong duality, which holds since the objective is linear and the constraint (entropy) is convex function in π_T. This dual objective is closely related to the maximum entropy objective with respect to the policy, and the optimal policy is the maximum entropy policy corresponding to temperature α_T: $\pi^*_T(\mathbf{a}_T|\mathbf{s}_T;\alpha_T)$. We can solve for the optimal dual variable α^*_T as

$$\arg\min_{\alpha_T}\mathbb{E}_{\mathbf{s}_t,\mathbf{a}_t\sim\pi^*_t}\left[-\alpha_T\log\pi^*_T(\mathbf{a}_T|\mathbf{s}_T;\alpha_T) - \alpha_T\mathcal{H}\right]. \tag{14}$$

To simplify notation, we make use of the recursive definition of the soft Q-function

$$Q^*_t(\mathbf{s}_t, \mathbf{a}_t; \pi^*_{t+1:T}, \alpha^*_{t+1:T}) = \mathbb{E}\left[r(\mathbf{s}_t, \mathbf{a}_t)\right] + \mathbb{E}_{\rho_\pi}\left[Q^*_{t+1}(\mathbf{s}_{t+1}, \mathbf{a}_{t+1}) - \alpha^*_{t+1}\log\pi^*_{t+1}(\mathbf{a}_{t+1}|\mathbf{s}_{t+1})\right], \tag{15}$$

with $Q^*_T(\mathbf{s}_T, \mathbf{a}_T) = \mathbb{E}\left[r(\mathbf{s}_T, \mathbf{a}_T)\right]$. Now, subject to the entropy constraints and again using the dual problem, we have

$$\begin{aligned}
&\max_{\pi_{T-1}}\left(\mathbb{E}\left[r(\mathbf{s}_{T-1}, \mathbf{a}_{T-1})\right] + \max_{\pi_T}\mathbb{E}\left[r(\mathbf{s}_T, \mathbf{a}_T)\right]\right) \\
&\quad= \max_{\pi_{T-1}}\left(Q^*_{T-1}(\mathbf{s}_{T-1}, \mathbf{a}_{T-1}) - \alpha_T\mathcal{H}\right) \\
&\quad= \min_{\alpha_{T-1}\geq 0}\max_{\pi_{T-1}}\left(\mathbb{E}\left[Q^*_{T-1}(\mathbf{s}_{T-1}, \mathbf{a}_{T-1})\right] - \mathbb{E}\left[\alpha_{T-1}\log\pi(\mathbf{a}_{T-1}|\mathbf{s}_{T-1})\right] - \alpha_{T-1}\mathcal{H}\right) + \alpha^*_T\mathcal{H}.
\end{aligned} \tag{16}$$

In this way, we can proceed backwards in time and recursively optimize Equation 11. Note that the optimal policy at time t is a function of the dual variable α_t. Similarly, we can solve the optimal dual variable α^*_t after solving for Q^*_t and π^*_t:

$$\alpha^*_t = \arg\min_{\alpha_t}\mathbb{E}_{\mathbf{a}_t\sim\pi^*_t}\left[-\alpha_t\log\pi^*_t(\mathbf{a}_t|\mathbf{s}_t;\alpha_t) - \alpha_t\bar{\mathcal{H}}\right]. \tag{17}$$

The solution in Equation 17 along with the policy and soft Q-function updates described in Section 4 constitute the core of our algorithm, and in theory, exactly solving them recursively optimize the optimal entropy-constrained maximum expected return objective in Equation 11, but in practice, we will need to resort to function approximators and stochastic gradient descent.

Algorithm 1 Soft Actor-Critic

Input: θ_1, θ_2, ϕ ▷ Initial parameters
$\bar{\theta}_1 \leftarrow \theta_1, \bar{\theta}_2 \leftarrow \theta_2$ ▷ Initialize target network weights
$\mathcal{D} \leftarrow \emptyset$ ▷ Initialize an empty replay pool
for each iteration **do**
  for each environment step **do**
    $\mathbf{a}_t \sim \pi_\phi(\mathbf{a}_t|\mathbf{s}_t)$ ▷ Sample action from the policy
    $\mathbf{s}_{t+1} \sim p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t)$ ▷ Sample transition from the environment
    $\mathcal{D} \leftarrow \mathcal{D} \cup \{(\mathbf{s}_t, \mathbf{a}_t, r(\mathbf{s}_t, \mathbf{a}_t), \mathbf{s}_{t+1})\}$ ▷ Store the transition in the replay pool
  end for
  for each gradient step **do**
    $\theta_i \leftarrow \theta_i - \lambda_Q \hat{\nabla}_{\theta_i} J_Q(\theta_i)$ for $i \in \{1, 2\}$ ▷ Update the Q-function parameters
    $\phi \leftarrow \phi - \lambda_\pi \hat{\nabla}_\phi J_\pi(\phi)$ ▷ Update policy weights
    $\alpha \leftarrow \alpha - \lambda \hat{\nabla}_\alpha J(\alpha)$ ▷ Adjust temperature
    $\bar{\theta}_i \leftarrow \tau\theta_i + (1-\tau)\bar{\theta}_i$ for $i \in \{1, 2\}$ ▷ Update target network weights
  end for
end for
Output: θ_1, θ_2, ϕ ▷ Optimized parameters

6 Practical Algorithm

Our algorithm makes use of two soft Q-functions to mitigate positive bias in the policy improvement step that is known to degrade performance of value based methods (Hasselt, 2010; Fujimoto et al., 2018). In particular, we parameterize two soft Q-functions, with parameters θ_i, and train them independently to optimize $J_Q(\theta_i)$. We then use the minimum of the the soft Q-functions for the stochastic gradient in Equation 6 and policy gradient in Equation 10, as proposed by Fujimoto et al. (2018). Although our algorithm can learn challenging tasks, including a 21-dimensional Humanoid, using just a single Q-function, we found two soft Q-functions significantly speed up training, especially on harder tasks.

In addition to the soft Q-function and the policy, we also learn α by minimizing the dual objective in Equation 17. This can be done by approximating dual gradient descent (Boyd & Vandenberghe, 2004). Dual gradient descent alternates between optimizing the Lagrangian with respect to the primal variables to convergence, and then taking a gradient step on the dual variables. While optimizing with respect to the primal variables fully is impractical, a truncated version that performs incomplete optimization (even for a single gradient step) can be shown to converge under convexity assumptions (Boyd & Vandenberghe, 2004). While such assumptions do not apply to the case of nonlinear function approximators such as neural networks, we found this approach to still work in practice. Thus, we compute gradients for α with the following objective:

$$J(\alpha) = \mathbb{E}_{\mathbf{a}_t \sim \pi_t}\left[-\alpha \log \pi_t(\mathbf{a}_t|\mathbf{s}_t) - \alpha\bar{\mathcal{H}}\right]. \tag{18}$$

The final algorithm is listed in Algorithm 1. The method alternates between collecting experience from the environment with the current policy and updating the function approximators using the stochastic gradients from batches sampled from a replay pool. Using off-policy data from a replay pool is feasible because both value estimators and the policy can be trained entirely on off-policy data. The algorithm is agnostic to the parameterization of the policy, as long as it can be evaluated for any arbitrary state-action tuple.

7 Experiments

The goal of our experimental evaluation is to understand how the sample complexity and stability of our method compares with prior off-policy and on-policy deep reinforcement learning algorithms. We compare our method to prior techniques on a range of challenging continuous control tasks from the OpenAI gym benchmark suite (Brockman et al., 2016) and also on the rllab implementation of the Humanoid task (Duan et al., 2016). Although the easier tasks can be solved by a wide range of different algorithms, the more complex benchmarks, such as the 21-dimensional Humanoid (rllab),

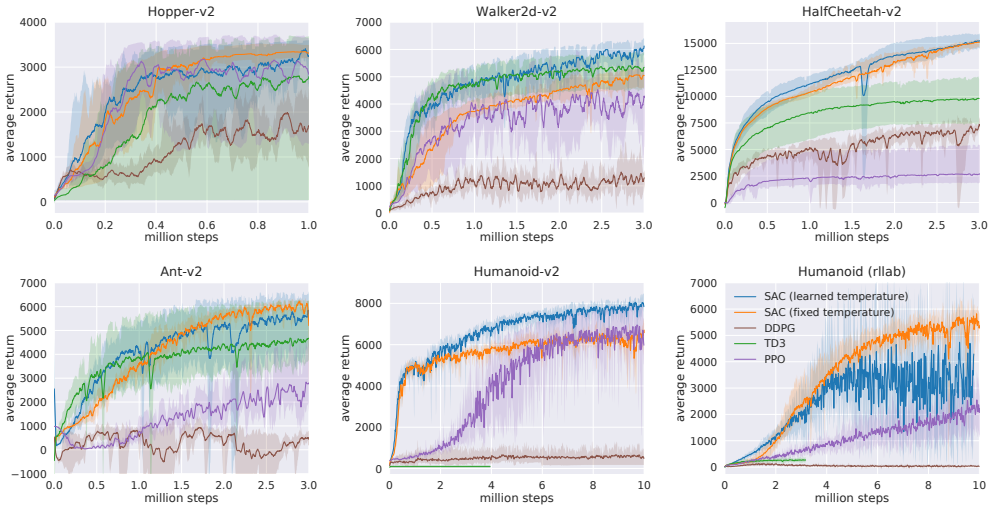


Figure 1: Training curves on continuous control benchmarks. Soft actor-critic (blue and yellow) performs consistently across all tasks and outperforming both on-policy and off-policy methods in the most challenging tasks.

are exceptionally difficult to solve with off-policy algorithms (Duan et al., 2016). The stability of the algorithm also plays a large role in performance: easier tasks make it more practical to tune hyperparameters to achieve good results, while the already narrow basins of effective hyperparameters become prohibitively small for the more sensitive algorithms on the hardest benchmarks, leading to poor performance (Gu et al., 2016).

7.1 Simulated Benchmarks

We compare our method to deep deterministic policy gradient (DDPG) (Lillicrap et al., 2015), an algorithm that is regarded as one of the more efficient off-policy deep RL methods (Duan et al., 2016); proximal policy optimization (PPO) (Schulman et al., 2017b), a stable and effective on-policy policy gradient algorithm; and soft Q-learning (SQL) (Haarnoja et al., 2017), a recent off-policy algorithm for learning maximum entropy policies. Our SQL implementation also includes two Q-functions, which we found to improve its performance in most environments. We additionally compare to twin delayed deep deterministic policy gradient algorithm (TD3) (Fujimoto et al., 2018), using the author-provided implementation. This is an extension to DDPG, proposed concurrently to our method, that first applied the double Q-learning trick to continuous control along with other improvements. We turned off the exploration noise for evaluation for DDPG and PPO. For maximum entropy algorithms, which do not explicitly inject exploration noise, we either evaluated with the exploration noise (SQL) or use the mean action (SAC). The source code of our SAC implementation[2] is available online.

Figure 1 shows the total average return of evaluation rollouts during training for DDPG, PPO, and TD3. We train five different instances of each algorithm with different random seeds, with each performing one evaluation rollout every 1000 environment steps. The solid curves corresponds to the mean and the shaded region to the minimum and maximum returns over the five trials. For SAC, we include both a version, where the temperature parameter is fixed and treated as a hyperparameter and tuned for each environment separately (orange), and a version where the temperature is adjusted automatically (blue). The results show that, overall, SAC performs comparably to the baseline methods on the easier tasks and outperforms them on the harder tasks with a large margin, both in terms of learning speed and the final performance. For example, DDPG fails to make any progress on Ant-v1, Humanoid-v1, and Humanoid (rllab), a result that is corroborated by prior work (Gu et al., 2016; Duan et al., 2016). SAC also learns considerably faster than PPO as a consequence of the large batch sizes PPO needs to learn stably on more high-dimensional and complex tasks. Another

[2] http://github.com/rail-berkeley/softlearning/

maximum entropy RL algorithm, SQL, can also learn all tasks, but it is slower than SAC and has worse asymptotic performance. The quantitative results attained by SAC in our experiments also compare very favorably to results reported by other methods in prior work (Duan et al., 2016; Gu et al., 2016; Henderson et al., 2017), indicating that both the sample efficiency and final performance of SAC on these benchmark tasks exceeds the state of the art. The results also indicate that the automatic temperature tuning scheme works well across all the environments, and thus effectively eliminates the need for tuning the temperature. All hyperparameters used in this experiment for SAC are listed in Appendix D.

7.2 Quadrupedal Locomotion in the Real World

In this section, we describe an application of our method to learn walking gaits directly in the real world. We use the Minitaur robot, a small-scale quadruped with eight direct-drive actuators (Kenneally et al., 2016). Each leg is controlled by two actuators that allow it to move in the sagittal plane. The Minitaur is equipped with motor encoders that measure the motor angles and an IMU that measures orientation and angular velocity of Minitaur's base. The action space are the swing angle and the extension of each leg, which are then mapped to desired motor positions and tracked with a PD controller (Tan et al., 2018). The observations include the motor angles as well as roll and pitch angles and angular velocities of the base. We exclude yaw since it is unreliable due to drift and irrelevant for the walking task. Note that latencies and contacts in the system make the dynamics non-Markovian, which can significantly degrade learning performance. We therefore construct the state out of the current and past five observations and actions. The reward function rewards large forward velocity, which is estimated using a motion capture system, and penalizes large angular accelerations, computed via finite differentiation from the last three actions. We also found it necessary to penalize for large pitch angles and for extending the front legs under the robot, which we found to be the most common failure cases that would require manual reset. We parameterize the policy and the value functions with feed-forward neural networks with two hidden-layers and 256 neurons per layer.

We have developed a semi-automatic robot training pipeline that consists of two components parallel jobs: training and data collection. These jobs run asynchronously on two different computers. The training process runs on a workstation, which updates the neural networks and periodically downloads the latest data from the robot and uploads the latest policy to the robot. On the robot, the on-board Nvidia Jetson TX2 runs the data collection job, which executes the policy, collects the trajectory and uploads these data to the workstation through Ethernet. Once the training is started, minimal human intervention is needed, except for the need to reset the robot state if it falls or drifts far from the initial state.

This learning task presents substantially challenges for real-world reinforcement learning. The robot is underactuated, and must therefore delicately balance contact forces on the legs to make forward progress. An untrained policy can lose balance and fall, and too many falls will eventually damage the robot, making sample-efficient learning essentially. Our method successfully learns to walk from 160k environment steps, or approximately 400 episodes with maximum length of 500 steps, which amount to approximately 2 hours of real-world training time.

However, in the real world, the utility of a locomotion policy hinges critically on its ability to generalize to different terrains and obstacles. Although we trained our policy only on flat terrain, as illustrated in Figure 2 (first row), we then tested it on varied terrains and obstacles (other rows). Because soft actor-critic learns robust policies, due to entropy maximization at training time, the policy can readily generalize to these perturbations without any additional learning. The robot is able to walk up and down a slope (first row), ram through an obstacle made of wooden blocks (second row), and step down stairs (third row) without difficulty, despite not being trained in these settings. To our knowledge, this experiment is the first example of a deep reinforcement learning algorithm learning underactuated quadrupedal locomotion directly in the real world without any simulation or pretraining. We have included videos of the the training process and evaluation on our project website[3].

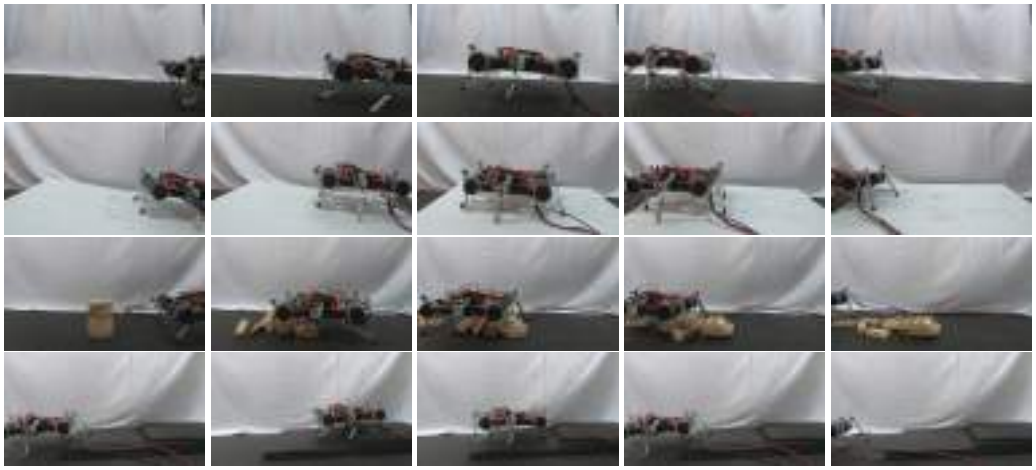


Figure 2: We trained the Minitaur robot to walk on flat terrain (first row) and tested how the learned gait generalizes to unseen situations (other rows)

7.3 Dexterous Hand Manipulation

Our second real-world robotic task involves training a 3-finger dexterous robotic hand to manipulate an object. The hand is based on the "dclaw" hand, discussed by (Zhu et al., 2018). This hand has 9 DoFs, each controlled by a Dynamixel servo-motor. The policy controls the hand by sending target joint angle positions for the on-board PID controller. The manipulation task requires the hand to rotate a "valve"-like object (resembling a sink faucet), as shown in Figure 3. In order to perceive the valve, the robot must use raw RGB images, which are illustrated in the second row of Figure 3. The images are processed in a neural network, consisting of two convolutional (four 3x3 filters) and max pool (3x3) layers, followed by two fully connected layers (256 units). The robot must rotate the valve into the correct position, with the colored part of the valve facing directly to the right, from any random starting position. The initial position of the valve is reset uniformly at random for each episode, forcing the policy to learn to use the raw RGB images to perceive the current valve orientation. A small motor is attached to the valve to automate resets and to provide the ground truth position for the determination of the reward function. The position of this motor is not provided to the policy.

This task is exceptionally challenging due to both the perception challenges and the physical difficulty of rotating the valve with such a complex robotic hand. As can be seen in the accompanying video on the project website[3], rotating the valve requires a complex finger gait where the robot moves the fingers over the valve in a coordinated pattern, and stops precisely at the desired position.

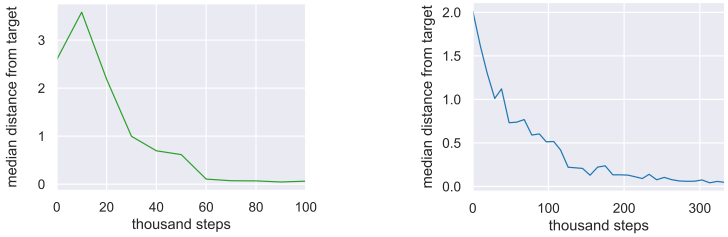
Learning this task directly from raw RGB images requires 300k environment interaction steps, which is the equivalent of 20 hours of training, including all resets and neural network training time (Figure 4). To our knowledge, this task represents one of the most complex robotic manipulation tasks learned directly end-to-end from raw images in the real world with deep reinforcement learning, without any simulation or pretraining. We also learned the same task without images by feeding the valve position directly to the neural networks. In that case, learning takes 3 hours, which is substantially faster than what has been reported earlier on the same task using PPO (7.4 hours) (Zhu et al., 2018).

8 Conclusion

In this article, we presented soft actor-critic (SAC), an off-policy maximum entropy deep reinforcement learning algorithm that provides sample-efficient learning while retaining the benefits of entropy maximization and stability. Our theoretical results derive soft policy iteration, which we show to converge to the optimal policy. From this result, we can formulate a practical soft actor-critic algorithm

[3] https://sites.google.com/view/sac-and-applications/

Figure 3: Valve rotation task with Dynamixel Claw. Top row shows a high resolution image sequence of a rollout with the policy. Bottom row shows the 32x32 pixel image observations of the same situations. The policy is also provided the joint positions and velocities of the fingers, but it has to infer the valve position from the image.



(a) Learning from valve joint angle.

(b) Learning from images.

Figure 4: Learning curves for the valve rotation task. The curves correspond to the median distance (measured in radians) of the valve from the target angle during a rollout. (a) Learning without images takes about 3 hours. In this case, the valve is reset to point away from the target initially. (b) Learning from images takes about 20 hours. The initial valve position is sampled uniformly at random.

that can be used to train deep neural network policies, and we empirically show that it matches or exceeds the performance of state-of-the-art model-free deep RL methods, including the off-policy TD3 algorithm and the on-policy PPO algorithm without any environment specific hyperparameter tuning. Our real-world experiments indicate that soft actor-critic is robust and sample efficient enough for robotic tasks learned directly in the real world, such as locomotion and dexterous manipulation. To our knowledge, these results represent the first evaluation of deep reinforcement learning for real-world training of underactuated walking skills with a quadrupedal robot, as well as one of the most complex dexterous manipulation behaviors learned with deep reinforcement learning end-to-end from raw image observations.

Acknowledgments

We would like to thank Vitchyr Pong and Haoran Tang for constructive discussions during the development of soft actor-critic, Vincent Vanhoucke for his support towards the project at Google, and Amazon for providing computing support.

References

Abdolmaleki, A., Springenberg, J. T., Tassa, Y., Munos, R., Heess, N., and Riedmiller, M. Maximum a posteriori policy optimisation. *arXiv preprint arXiv:1806.06920*, 2018.

Barto, A. G., Sutton, R. S., and Anderson, C. W. Neuronlike adaptive elements that can solve difficult learning control problems. *IEEE transactions on systems, man, and cybernetics*, pp. 834–846, 1983.

Todorov, E. General duality between optimal control and estimation. In *IEEE Conference on Decision and Control (CDC)*, pp. 4286–4292. IEEE, 2008.

Toussaint, M. Robot trajectory optimization using approximate inference. In *International Conference on Machine Learning (ICML)*, pp. 1049–1056. ACM, 2009.

Williams, R. J. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine learning*, 8(3-4):229–256, 1992.

Zhu, H., Gupta, A., Rajeswaran, A., Levine, S., and Kumar, V. Dexterous manipulation with deep reinforcement learning: Efficient, general, and low-cost. *arXiv preprint arXiv:1810.06045*, 2018.

Ziebart, B. D. *Modeling purposeful adaptive behavior with the principle of maximum causal entropy*. Carnegie Mellon University, 2010.

Ziebart, B. D., Maas, A. L., Bagnell, J. A., and Dey, A. K. Maximum entropy inverse reinforcement learning. In *AAAI Conference on Artificial Intelligence (AAAI)*, pp. 1433–1438, 2008.

Appendix

A Infinite Horizon Discounted Maximum Entropy Objective

The exact definition of the discounted maximum entropy objective is complicated by the fact that, when using a discount factor for policy gradient methods, we typically do not discount the state distribution, only the rewards. In that sense, discounted policy gradients typically do not optimize the true discounted objective. Instead, they optimize average reward, with the discount serving to reduce variance, as discussed by Thomas (2014). However, we can define the objective that *is* optimized under a discount factor as

$$J(\pi) = \sum_{t=0}^{\infty} \mathbb{E}_{(\mathbf{s}_t,\mathbf{a}_t)\sim\rho_\pi}\left[\sum_{l=t}^{\infty} \gamma^{l-t}\, \mathbb{E}_{\mathbf{s}_l\sim p,\mathbf{a}_l\sim\pi}\left[r(\mathbf{s}_t,\mathbf{a}_t) + \alpha\mathcal{H}(\pi(\,\cdot\,|\mathbf{s}_t))|\mathbf{s}_t,\mathbf{a}_t\right]\right]. \tag{19}$$

This objective corresponds to maximizing the discounted expected reward and entropy for future states originating from every state-action tuple $(\mathbf{s}_t, \mathbf{a}_t)$ weighted by its probability ρ_π under the current policy.

B Proofs

B.1 Lemma 1

Lemma 1 (Soft Policy Evaluation). *Consider the soft Bellman backup operator $\mathcal{T}^\pi$ in Equation 2 and a mapping $Q^0 : \mathcal{S}\times\mathcal{A}\rightarrow\mathbb{R}$ with $|\mathcal{A}|<\infty$, and define $Q^{k+1} = \mathcal{T}^\pi Q^k$. Then the sequence Q^k will converge to the soft Q-value of π as $k\rightarrow\infty$.*

Proof. Define the entropy augmented reward as $r_\pi(\mathbf{s}_t,\mathbf{a}_t) \triangleq r(\mathbf{s}_t,\mathbf{a}_t) + \mathbb{E}_{\mathbf{s}_{t+1}\sim p}\left[\mathcal{H}\left(\pi(\,\cdot\,|\mathbf{s}_{t+1})\right)\right]$ and rewrite the update rule as

$$Q(\mathbf{s}_t,\mathbf{a}_t) \leftarrow r_\pi(\mathbf{s}_t,\mathbf{a}_t) + \gamma\,\mathbb{E}_{\mathbf{s}_{t+1}\sim p,\mathbf{a}_{t+1}\sim\pi}\left[Q(\mathbf{s}_{t+1},\mathbf{a}_{t+1})\right] \tag{20}$$

and apply the standard convergence results for policy evaluation (Sutton & Barto, 1998). The assumption $|\mathcal{A}|<\infty$ is required to guarantee that the entropy augmented reward is bounded. □

B.2 Lemma 2

Lemma 2 (Soft Policy Improvement). *Let $\pi_{\text{old}}\in\Pi$ and let π_{new} be the optimizer of the minimization problem defined in Equation 4. Then $Q^{\pi_{\text{new}}}(\mathbf{s}_t,\mathbf{a}_t)\geq Q^{\pi_{\text{old}}}(\mathbf{s}_t,\mathbf{a}_t)$ for all $(\mathbf{s}_t,\mathbf{a}_t)\in\mathcal{S}\times\mathcal{A}$ with $|\mathcal{A}|<\infty$.*

Proof. Let $\pi_{\text{old}} \in \Pi$ and let $Q^{\pi_{\text{old}}}$ and $V^{\pi_{\text{old}}}$ be the corresponding soft state-action value and soft state value, and let π_{new} be defined as

$$\begin{aligned}\pi_{\text{new}}(\,\cdot\,|\mathbf{s}_t) &= \arg\min_{\pi' \in \Pi} \mathrm{D}_{\mathrm{KL}}\left(\pi'(\,\cdot\,|\mathbf{s}_t) \parallel \exp\left(Q^{\pi_{\text{old}}}(\mathbf{s}_t, \,\cdot\,) - \log Z^{\pi_{\text{old}}}(\mathbf{s}_t)\right)\right) \\ &= \arg\min_{\pi' \in \Pi} J_{\pi_{\text{old}}}(\pi'(\,\cdot\,|\mathbf{s}_t)). \end{aligned} \tag{21}$$

It must be the case that $J_{\pi_{\text{old}}}(\pi_{\text{new}}(\,\cdot\,|\mathbf{s}_t)) \leq J_{\pi_{\text{old}}}(\pi_{\text{old}}(\,\cdot\,|\mathbf{s}_t))$, since we can always choose $\pi_{\text{new}} = \pi_{\text{old}} \in \Pi$. Hence

$$\mathbb{E}_{\mathbf{a}_t \sim \pi_{\text{new}}}\left[\log \pi_{\text{new}}(\mathbf{a}_t|\mathbf{s}_t) - Q^{\pi_{\text{old}}}(\mathbf{s}_t, \mathbf{a}_t) + \log Z^{\pi_{\text{old}}}(\mathbf{s}_t)\right] \leq \mathbb{E}_{\mathbf{a}_t \sim \pi_{\text{old}}}\left[\log \pi_{\text{old}}(\mathbf{a}_t|\mathbf{s}_t) - Q^{\pi_{\text{old}}}(\mathbf{s}_t, \mathbf{a}_t) + \log Z^{\pi_{\text{old}}}(\mathbf{s}_t)\right], \tag{22}$$

and since partition function $Z^{\pi_{\text{old}}}$ depends only on the state, the inequality reduces to

$$\mathbb{E}_{\mathbf{a}_t \sim \pi_{\text{new}}}\left[Q^{\pi_{\text{old}}}(\mathbf{s}_t, \mathbf{a}_t) - \log \pi_{\text{new}}(\mathbf{a}_t|\mathbf{s}_t)\right] \geq V^{\pi_{\text{old}}}(\mathbf{s}_t). \tag{23}$$

Next, consider the soft Bellman equation:

$$\begin{aligned} Q^{\pi_{\text{old}}}(\mathbf{s}_t, \mathbf{a}_t) &= r(\mathbf{s}_t, \mathbf{a}_t) + \gamma\, \mathbb{E}_{\mathbf{s}_{t+1} \sim p}\left[V^{\pi_{\text{old}}}(\mathbf{s}_{t+1})\right] \\ &\leq r(\mathbf{s}_t, \mathbf{a}_t) + \gamma\, \mathbb{E}_{\mathbf{s}_{t+1} \sim p}\left[\mathbb{E}_{\mathbf{a}_{t+1} \sim \pi_{\text{new}}}\left[Q^{\pi_{\text{old}}}(\mathbf{s}_{t+1}, \mathbf{a}_{t+1}) - \log \pi_{\text{new}}(\mathbf{a}_{t+1}|\mathbf{s}_{t+1})\right]\right] \\ &\;\;\vdots \\ &\leq Q^{\pi_{\text{new}}}(\mathbf{s}_t, \mathbf{a}_t), \end{aligned} \tag{24}$$

where we have repeatedly expanded $Q^{\pi_{\text{old}}}$ on the RHS by applying the soft Bellman equation and the bound in Equation 23. Convergence to $Q^{\pi_{\text{new}}}$ follows from Lemma 1. □

B.3 Theorem 1

Theorem 1 (~~Soft~~ Policy Iteration). *Repeated application of soft policy evaluation and soft policy improvement to any $\pi \in \Pi$ converges to a policy π^* such that $Q^{\pi^*}(\mathbf{s}_t, \mathbf{a}_t) \geq Q^{\pi}(\mathbf{s}_t, \mathbf{a}_t)$ for all ~~$\pi \in \Pi$ and~~ $(\mathbf{s}_t, \mathbf{a}_t) \in \mathcal{S} \times \mathcal{A}$, assuming $|\mathcal{A}| < \infty$.*

Proof. Let π_i be the policy at iteration i. By Lemma 2, the sequence Q^{π_i} is monotonically increasing. Since Q^{π} is bounded above for $\pi \in \Pi$ (both the reward and entropy are bounded), the sequence converges to some π^*. We will still need to ~~show that~~ π^* is indeed optimal. At convergence, it must be case that $J_{\pi^*}(\pi^*(\,\cdot\,|\mathbf{s}_t)) < J_{\pi^*}(\pi(\,\cdot\,|\mathbf{s}_t))$ ~~for all~~ $\pi \in \Pi$, $\pi \neq \pi^*$. Using the same iterative argument as in the proof of Lemma 2, we get $Q^{\pi^*}(\mathbf{s}_t, \mathbf{a}_t) > Q^{\pi}(\mathbf{s}_t, \mathbf{a}_t)$ for all $(\mathbf{s}_t, \mathbf{a}_t) \in \mathcal{S} \times \mathcal{A}$, that is, the soft value of any other policy in Π is lower than that of the converged policy. Hence π^* is optimal in Π. □

C Enforcing Action Bounds

We use an unbounded Gaussian as the action distribution. However, in practice, the actions needs to be bounded to a finite interval. To that end, we apply an invertible squashing function ($\tanh$) to the Gaussian samples, and employ the change of variables formula to compute the likelihoods of the bounded actions. In the other words, let $\mathbf{u} \in \mathbb{R}^D$ be a random variable and $\mu(\mathbf{u}|\mathbf{s})$ the corresponding density with infinite support. Then $\mathbf{a} = \tanh(\mathbf{u})$, where $\tanh$ is applied elementwise, is a random variable with support in $(-1, 1)$ with a density given by

$$\pi(\mathbf{a}|\mathbf{s}) = \mu(\mathbf{u}|\mathbf{s}) \left| \det\left(\frac{\mathrm{d}\mathbf{a}}{\mathrm{d}\mathbf{u}}\right) \right|^{-1}. \tag{25}$$

Since the Jacobian $\mathrm{d}\mathbf{a}/\mathrm{d}\mathbf{u} = \operatorname{diag}(1 - \tanh^2(\mathbf{u}))$ is diagonal, the log-likelihood has a simple form

$$\log \pi(\mathbf{a}|\mathbf{s}) = \log \mu(\mathbf{u}|\mathbf{s}) - \sum_{i=1}^{D} \log\left(1 - \tanh^2(u_i)\right), \tag{26}$$

where u_i is the i^{th} element of $\mathbf{u}$.

D Hyperparameters

Table 1 lists the common SAC parameters used in the comparative evaluation in Figure 1.

Table 1: SAC Hyperparameters

Parameter	Value
optimizer	Adam (Kingma & Ba, 2015)
learning rate	$3 \cdot 10^{-4}$
discount (γ)	0.99
replay buffer size	10^6
number of hidden layers (all networks)	2
number of hidden units per layer	256
number of samples per minibatch	256
entropy target	$-\dim(\mathcal{A})$ (e.g. , -6 for HalfCheetah-v1)
nonlinearity	ReLU
target smoothing coefficient (τ)	0.005
target update interval	1
gradient steps	1

CROSSQ: BATCH NORMALIZATION IN DEEP REINFORCEMENT LEARNING FOR GREATER SAMPLE EFFICIENCY AND SIMPLICITY

Aditya Bhatt [*1,4] **Daniel Palenicek** [*1,2] **Boris Belousov** [1,4] **Max Argus** [3]
Artemij Amiranashvili [3] **Thomas Brox** [3] **Jan Peters** [1,2,4,5]
[*]Equal contribution [1]Intelligent Autonomous Systems, TU Darmstadt [2]Hessian.AI [3]University of Freiburg
[4]German Research Center for AI (DFKI) [5]Centre for Cognitive Science, TU Darmstadt
aditya.bhatt@dfki.de, daniel.palenicek@tu-darmstadt.de

ABSTRACT

Sample efficiency is a crucial problem in deep reinforcement learning. Recent algorithms, such as REDQ and DroQ, found a way to improve the sample efficiency by increasing the update-to-data (UTD) ratio to 20 gradient update steps on the critic per environment sample. However, this comes at the expense of a greatly increased computational cost. To reduce this computational burden, we introduce CrossQ: A lightweight algorithm for continuous control tasks that makes careful use of Batch Normalization and removes target networks to surpass the current state-of-the-art in sample efficiency while maintaining a low UTD ratio of 1. Notably, CrossQ does not rely on advanced bias-reduction schemes used in current methods. CrossQ's contributions are threefold: (1) it matches or surpasses current state-of-the-art methods in terms of sample efficiency, (2) it substantially reduces the computational cost compared to REDQ and DroQ, (3) it is easy to implement, requiring just a few lines of code on top of SAC.

1 INTRODUCTION

Sample efficiency is a crucial concern when applying Deep Reinforcement Learning (Deep RL) methods on real physical systems. One of the first successful applications of Deep RL to a challenging problem of quadruped locomotion was achieved using Soft Actor-Critic (SAC, Haarnoja et al. (2018a)), allowing a robot dog to learn to walk within 2h of experience (Haarnoja et al., 2018b). Subsequently, it was noted that the critic in SAC may be underfitted, as only a single gradient update step on the network parameters is performed for each environment step. Therefore, Randomized Ensembled Double Q-Learning (REDQ, Chen et al. (2021)) was proposed, which increased this number of gradient steps, termed update-to-data (UTD) ratio. In addition, Dropout Q functions (DroQ, Hiraoka et al. (2021)) improved the computational efficiency of REDQ while maintaining the same sample efficiency by replacing its ensemble of critics with dropout. This enabled learning quadruped locomotion in a mere 20min (Smith et al., 2022). Thus, REDQ and DroQ represent the state-of-the-art in terms of sample efficiency in Deep RL for continuous control.

Importantly, both REDQ and DroQ showed that naively increasing the UTD ratio of SAC does not perform well due to the critic networks' Q value estimation bias. Therefore, ensembling techniques were introduced for bias reduction (explicit ensemble in REDQ and implicit

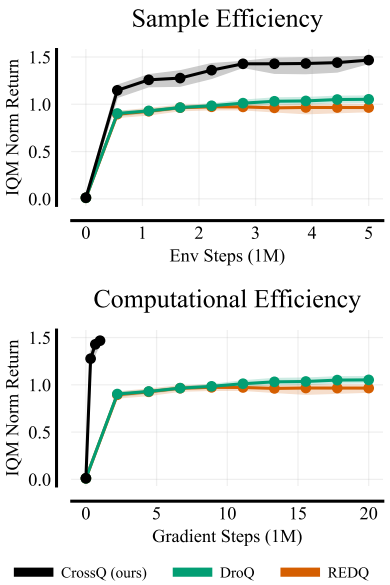


Figure 1: **CrossQ training performance aggregated over environments.** CrossQ is more sample efficient (top) while being significantly more computationally efficient (bottom) in terms of the gradient steps, thanks to a low UTD = 1. Following Agarwal et al. (2021), we normalize performance by the maximum of REDQ in each environment.

ensemble via dropout in DroQ), which allowed increasing the UTD to 20 critic updates per environment step. Higher UTD ratios improve sample efficiency by paying the price of increased computational cost, which manifests in higher wallclock time and energy consumption. It is, therefore, desirable to seek alternative methods that achieve the same or better sample efficiency at a lower computational cost, e.g., by using lower UTDs.

It turns out that even UTD $= 1$ can perform surprisingly well if other algorithmic components are adjusted appropriately. In this paper, we introduce **Cross$Q$**, a lightweight algorithm that achieves superior performance by removing much of the algorithmic design complexity that was added over the years, culminating in the current state-of-the-art methods. First, it *removes target networks*, an ingredient widely believed to slow down training in exchange for stability (Mnih et al., 2015; Lillicrap et al., 2016; Kim et al., 2019; Fan et al., 2020). Second, we find that *Batch Normalization* variants (Ioffe & Szegedy (2015); Ioffe (2017)), when applied in a particular manner, effectively stabilize training and significantly improve sample efficiency. This contradicts others' observations that it hurts the learning performance in Deep RL, e.g. Hiraoka et al. (2021). Third, Cross$Q$ uses *wider critic layers*, motivated by prior research on the ease of optimization of wider networks (Ota et al., 2021). In addition to the first two improvements, wider networks enable even higher returns.

Contributions. (1) We present the Cross$Q$ algorithm, which matches or surpasses the current state-of-the-art for model-free off-policy RL for continuous control environments with state observations in sample efficiency while being multiple times more computationally efficient; (2) By removing target networks, we are able to successfully accelerate off-policy Deep RL with BatchNorm; (3) We provide empirical investigations and hypotheses for Cross$Q$'s success. Cross$Q$'s changes mainly pertain to the deep network architecture of SAC; therefore, our study is chiefly empirical: through a series of ablations, we isolate and study the contributions of each part. We find that Cross$Q$ matches or surpasses the state-of-the-art algorithms in sample efficiency while being up to $4\times$ faster in terms of wallclock time without requiring critic ensembles, target networks, or high UTD ratios. We provide the Cross$Q$ source code at github.com/adityab/CrossQ.

2 Background

2.1 Off-policy Reinforcement Learning and Soft Actor-Critic

We consider a discrete-time Markov Decision Process (MDP, Puterman (2014)), defined by the tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \rho, \gamma \rangle$ with state space $\mathcal{S}$, action space $\mathcal{A}$, transition probability $\boldsymbol{s}_{t+1} \sim \mathcal{P}(\cdot|\boldsymbol{s}_t, \boldsymbol{a}_t)$, reward function $r_t = \mathcal{R}(\boldsymbol{s}_t, \boldsymbol{a}_t)$, initial state distribution $\boldsymbol{s}_0 \sim \rho$ and discount factor $\gamma \in [0, 1)$. RL describes the problem of an agent learning an optimal policy π for a given MDP. At each time step t, the agent receives a state $\boldsymbol{s}_t$ and interacts with the environment according to its policy π. We focus on the Maximum Entropy RL setting (Ziebart et al., 2008), where the agent's objective is to find the optimal policy π^*, which maximizes the expected cumulative reward while keeping the entropy $\mathcal{H}$ high; $\arg\max_{\pi^*} \mathbb{E}_{\boldsymbol{s}_0 \sim \rho} \left[\sum_{t=0}^{\infty} \gamma^t (r_t - \alpha \mathcal{H}(\pi(\cdot\,|\boldsymbol{s}_t)))\right]$. The action-value function is defined by $Q(\boldsymbol{s}, \boldsymbol{a}) = \mathbb{E}_{\pi, \mathcal{P}} \left[\sum_{t=0}^{\infty} \gamma^t (r_t - \alpha \log \pi(\boldsymbol{a}_t|\boldsymbol{s}_t))|\boldsymbol{s}_0 = \boldsymbol{s}, \boldsymbol{a}_0 = \boldsymbol{a}\right]$ and describes the expected reward when taking action $\boldsymbol{a}$ in state $\boldsymbol{s}$. Soft Actor-Critic (SAC, (Haarnoja et al., 2018a)) is a popular algorithm that solves the MaxEnt RL problem. SAC parametrizes the Q function and policy as neural networks and trains two independent versions of the Q function, using the minimum of their estimates to compute the regression targets for Temporal Difference (TD) learning. This *clipped double-Q* trick, originally proposed by Fujimoto et al. (2018) in TD3, helps in reducing the potentially destabilizing overestimation bias inherent in approximate Q-learning (Hasselt, 2010).

2.2 High update-to-data Ratios, REDQ, and DroQ

Despite its popularity among practitioners and as a foundation for other more complex algorithms, SAC leaves much room for improvement in terms of sample efficiency. Notably, SAC performs exactly one gradient-based optimization step per environment interaction. SAC's UTD $= 1$ setting is analogous to simply training for fewer epochs in supervised learning. Therefore, in recent years, gains in sample efficiency within RL have been achieved through increasing the UTD ratio (Janner et al., 2019; Chen et al., 2021; Hiraoka et al., 2021; Nikishin et al., 2022). Different algorithms, however, substantially vary in their approaches to achieving high UTD ratios. Janner et al. (2019)

```python
def critic_loss(Q_params, policy_params, obs, acts, rews, next_obs):
    next_acts, next_logpi = policy.apply(policy_params, obs)

    # Concatenated forward pass
    all_q, new_Q_params = Q.apply(Q_params,
        jnp.concatenate([obs, next_obs]),
        jnp.concatenate([acts, next_acts])
    )
    # Split all_q predictions and stop gradient on next_q
    q, next_q = jnp.split(all_q, 2)
    next_q = jnp.min(next_q, axis=0)    # min over double Q function
    next_q = jax.lax.stop_gradient(next_q - alpha * next_logpi)
    return jnp.mean((q - (rews + gamma * next_q))**2), new_Q_params
```

Figure 2: **Cross$Q$ critic loss in JAX.** The Cross$Q$ critic loss is easy to implement on top of an existing SAC implementation. One just adds the batch normalization layers into the critic network and removes the target network. As we are now left with only the critic network, one can simply concatenate observations and next observations, as well as actions and next actions along the batch dimension, perform a joint forward pass, and split up the batches afterward. Combining two forward passes into one grants a small speed-up thanks to requiring only one CUDA call instead of two.

uses a model to generate synthetic data, which allows for more overall gradient steps. Nikishin et al. (2022) adopt a simpler approach: they increase the number of gradient steps while periodically resetting the policy and critic networks to fight premature convergence to local minima. We now briefly outline the two high-UTD methods to which we compare Cross$Q$.

REDQ. Chen et al. (2021) find that merely raising SAC's UTD ratio hurts performance. They attribute this to the accumulation of the learned Q functions' estimation bias over multiple update steps—despite the clipped double-Q trick—which destabilizes learning. To remedy this bias more strongly, they increase the number of Q networks from two to an ensemble of 10. Their method, called REDQ, permits stable training at high UTD ratios up to 20.

DroQ. Hiraoka et al. (2021) note that REDQ's ensemble size, along with its high UTD ratio, makes training computationally expensive. They instead propose using a smaller ensemble of Q functions equipped with Dropout (Srivastava et al., 2014), along with Layer Normalization (Ba et al., 2016) to stabilize training in response to the noise introduced by Dropout. Called DroQ, their method is computationally cheaper than REDQ, yet still expensive due to its UTD ratio of 20.

3 The Cross$Q$ Algorithm

In this paper, we challenge this current trend of high UTD ratios and demonstrate that we can achieve competitive sample efficiency at a much lower computational cost with a UTD $= 1$ method. Cross$Q$ is our new state-of-the-art off-policy actor-critic algorithm. Based on SAC, it uses purely network-architectural engineering insights from deep learning to accelerate training. As a result, it ~~crosses out~~ much of the algorithmic design complexity that was added over the years and which led to the current state-of-the-art methods. In doing so, we present a much simpler yet more efficient algorithm. In the following paragraphs, we introduce the three design choices that constitute Cross$Q$.

3.1 Design Choice 1: Removing Target Networks

Mnih et al. (2015) originally introduced target networks to stabilize the training of value-based off-policy RL methods, and today, most algorithms require them (Lillicrap et al., 2016; Fujimoto et al., 2018; Haarnoja et al., 2018a). SAC updates the critics' target networks with Polyak Averaging

$$\boldsymbol{\theta}^{\circ} \leftarrow (1-\tau)\boldsymbol{\theta}^{\circ} + \tau\boldsymbol{\theta}, \tag{1}$$

where $\boldsymbol{\theta}^{\circ}$ are the target network parameters, and $\boldsymbol{\theta}$ are those of the trained critic. Here τ is the *target network smoothing coefficient*; with a high $\tau = 1$ (equivalent to cutting out the target network), SAC training can diverge, leading to explosive growth in $\boldsymbol{\theta}$ and the Q predictions. Target networks stabilize training by explicitly delaying value function updates, arguably slowing down online learning (Plappert et al., 2018; Kim et al., 2019; Morales, 2020).

SAC:	CrossQ (Ours):
$Q_{\boldsymbol{\theta}}(\boldsymbol{S}_t, \boldsymbol{A}_t) = \boldsymbol{q}_t$	$Q_{\boldsymbol{\theta}}\left(\begin{bmatrix}\boldsymbol{S}_t \\ \boldsymbol{S}_{t+1}\end{bmatrix}, \begin{bmatrix}\boldsymbol{A}_t \\ \boldsymbol{A}_{t+1}\end{bmatrix}\right) = \begin{bmatrix}\boldsymbol{q}_t \\ \boldsymbol{q}_{t+1}\end{bmatrix}$
$Q_{\boldsymbol{\theta}^\circ}(\boldsymbol{S}_{t+1}, \boldsymbol{A}_{t+1}) = \boldsymbol{q}^\circ_{t+1}$	
$\mathcal{L}_{\boldsymbol{\theta}} = (\boldsymbol{q}_t - \boldsymbol{r}_t - \gamma\, \boldsymbol{q}^\circ_{t+1})^2$	$\mathcal{L}_{\boldsymbol{\theta}} = (\boldsymbol{q}_t - \boldsymbol{r}_t - \gamma\, \lvert\boldsymbol{q}_{t+1}\rvert_{\text{sg}})^2$

Figure 3: SAC without BatchNorm in the critic $Q_{\boldsymbol{\theta}}$ (left) requires target Q values $\boldsymbol{q}^\circ_{t+1}$ to stabilize learning. CrossQ with BatchNorm in the critic $Q_{\boldsymbol{\theta}}$ (right) removes the need for target networks and allows for a joint forward pass of both current and future values. Batches are sampled from the replay buffer $\mathcal{B}$: $\boldsymbol{S}_t, \boldsymbol{A}_t, \boldsymbol{r}_t, \boldsymbol{S}_{t+1} \sim \mathcal{B}$ and $\boldsymbol{A}_{t+1} \sim \pi_\phi(\boldsymbol{S}_{t+1})$ from the current policy. $|\cdot|_{\text{sg}}$ denotes the `stop-gradient` operation.

Recently, Yang et al. (2021) found that critics with Random Fourier Features can be trained without target networks, suggesting that the choice of layer activations affects the stability of training. Our experiments in Section 4.4 uncover an even simpler possibility: using bounded activation functions or feature normalizers is sufficient to prevent critic divergence in the absence of target networks, whereas the common choice of `relu` without normalization diverges. While others have used normalizers in Deep RL before, we are the first to identify that they make target networks redundant. Our next design choice exploits this insight to obtain an even greater boost.

3.2 Design Choice 2: Using Batch Normalization

BatchNorm has not yet seen wide adoption in value-based off-policy RL methods, despite its success and widespread use in supervised learning (He et al., 2016; Santurkar et al., 2018), attempts at doing so have fared poorly. Lillicrap et al. (2016) use BatchNorm layers on the state-only representation layers in the DDPG critic but find that it does not help significantly. Others use BatchNorm in decoupled feature extractors for Deep RL networks (Ota et al., 2020; 2021), but not in critic networks. Hiraoka et al. (2021) report that using BatchNorm in critics causes training to fail in DroQ.

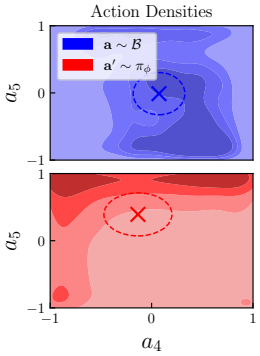


Figure 4: **Replay buffer and current policy actions are distributed differently.** Darker colors denote higher density. Estimated from a batch of 10^4 transitions $(\boldsymbol{a}, \boldsymbol{s}') \sim \mathcal{B}$; $\boldsymbol{a}' \sim \pi_\phi(\boldsymbol{s}')$, after 3×10^5 training steps on `Walker2d`; a_4 and a_5 are random action dimensions.

We find using BatchNorm *carefully*, when *additionally* removing target networks, performs surprisingly well, trains stably, and is, in fact, algorithmically simpler than current methods.

First, we explain why BatchNorm needs to be used *carefully*. Within the critic loss $[Q_{\boldsymbol{\theta}}(\boldsymbol{S}, \boldsymbol{A}) - (\boldsymbol{r} + \gamma Q_{\boldsymbol{\theta}^\circ}(\boldsymbol{S}', \boldsymbol{A}'))]^2$, predictions are made for two differently distributed batches of state-action pairs; $(\boldsymbol{S}, \boldsymbol{A})$ and $(\boldsymbol{S}', \boldsymbol{A}')$, where $\boldsymbol{A}' \sim \pi_\phi(\boldsymbol{S}')$ is sampled from the *current policy*, while $\boldsymbol{A}$ originates from old behavior policies.

Just like the target network, the BatchNorm parameters are updated by Polyak Averaging from the live network (Equation 1). The BatchNorm running statistics of the live network, which were estimated from batches of $(\boldsymbol{s}, \boldsymbol{a})$ pairs, will clearly not have *seen* samples $(\boldsymbol{s}', \pi_\phi(\boldsymbol{s}'))$ and will further not match their statistics. In other words, the state-action inputs evaluated by the target network will be out-of-distribution, given its mismatched BatchNorm running statistics. It is well known that the prediction quality of BatchNorm-equipped networks degrades in the face of such test-time distribution shifts (Pham et al., 2022; Lim et al., 2023).

Removing the target network provides an *elegant* solution. With the target network removed, we can concatenate both batches and feed them through the Q network in a single forward pass, as illustrated in Figure 3 and shown in code in Figure 2. This simple trick ensures that BatchNorm's normalization moments arise from the union of both batches, corresponding to a $50/50$ mixture of their respective distributions. Such normalization layers *do not* perceive the $(\boldsymbol{s}', \pi_\phi(\boldsymbol{s}'))$ batch as being out-of-distribution. This small change to SAC allows the safe use of BatchNorm and greatly accelerates training. We are not the only ones to identify this way of using BatchNorm to tackle the distribution mismatch; other works in supervised learning, e.g., Test-Time Adaptation (Lim et al.,

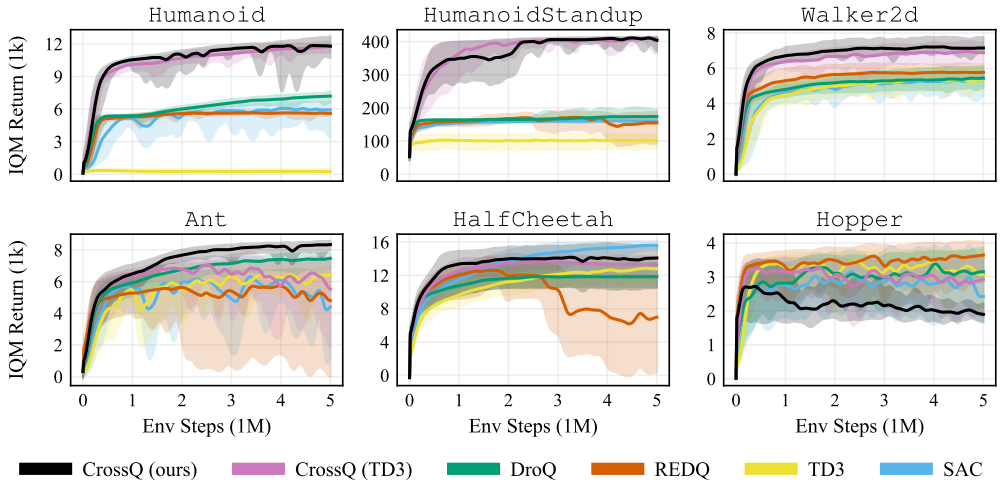


Figure 5: **Cross$Q$ sample efficiency.** Compared to REDQ and DroQ (UTD $=$ 20) Cross$Q$ (UTD $=$ 1) performs either comparably, better, or—for the more challenging `Humanoid` tasks—substantially better. These results directly transfer to TD3 as the base algorithm in Cross$Q$ (TD3). We plot *interquartile mean* (IQM) and 70% quantile interval of the episodic returns over 10 seeds.

2023), EvalNorm (Singh & Shrivastava, 2019), and *Four Things Everyone Should Know to Improve Batch Normalization* (Summers & Dinneen, 2020) also use mixed moments to bridge this gap.

In practice, Cross$Q$'s actor and critic networks use Batch Renormalization (`BRN`, Ioffe (2017)), an improved version of the original `BN` (Ioffe & Szegedy, 2015) that is robust to long-term training instabilities originating from minibatch noise. `BRN` performs batch normalization using the less noisy *running statistics* after a warm-up period, instead of noisy minibatch estimates as in `BN`. In the rest of this paper, all discussions with "BatchNorm" apply equally to both versions unless explicitly disambiguated by `BN` or `BRN`.

3.3 Design Choice 3: Wider Critic Networks

Following Ota et al. (2021), we find that wider critic network layers in Cross$Q$ lead to even faster learning. As we show in our ablations in Section 4.4, most performance gains originate from the first two design choices; however, wider critic networks further boost the performance, helping to match or outperform REDQ and DroQ sample efficiency.

We want to stress again that **Cross$Q$**, a UTD $=$ 1 method, ***does not use bias-reducing ensembles, high UTD ratios or target networks***. Despite this, it achieves its competitive sample efficiency at a fraction of the compute cost of REDQ and DroQ (see Figures 5 and 6). Note that our proposed changes can just as well be combined with other off-policy TD-learning methods, such as TD3, as shown in our experiments in Section 4.1.

4 Experiments and Analysis

We conduct experiments to provide empirical evidence for Cross$Q$'s performance, and investigate:

1. Sample efficiency of Cross$Q$ compared to REDQ and DroQ;
2. Computational efficiency in terms of wallclock time and performed gradient step;
3. Effects of the proposed design choices on the performance via Q function bias evaluations;

And conduct further ablation studies for the above design choices. We evaluate across a wide range of continuous-control `MuJoCo` (Todorov et al., 2012) environments, with 10 random seeds each. Following Janner et al. (2019); Chen et al. (2021) and Hiraoka et al. (2021), we evaluate on the same four `Hopper`, `Walker2d`, `Ant`, and `Humanoid` tasks, as well as two additional tasks: `HalfCheetah` and the more challenging `HumanoidStandup` from Gymnasium (Towers et al., 2023). We adapted the JAX version of stable-baselines (Raffin et al., 2021) for our experiments.

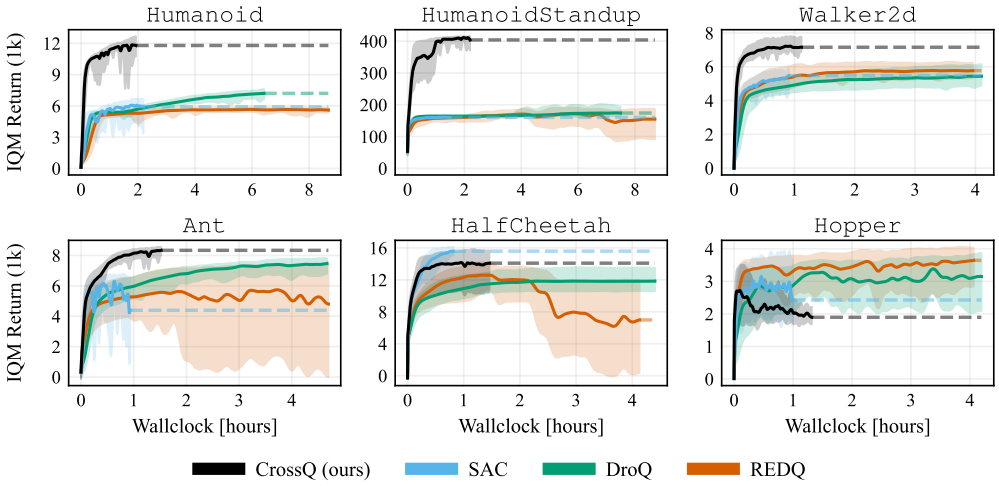


Figure 6: **Computational efficiency.** Cross$Q$ trains an order of magnitude faster, taking only 5% of the gradient steps, substantially saving on wallclock time. The dashed horizontal lines are visual aids to better compare the final performance after training for 5×10^6 environment steps. We plot IQM and 70% quantile interval over 10 seeds. Appendix A.3 provides a table of wallclock times.

4.1 Sample Efficiency of Cross$Q$

Figure 5 compares our proposed Cross$Q$ algorithm with REDQ, DroQ, SAC and TD3 in terms of their sample efficiency, i.e., average episode return at a given number of environment interactions. As a proof of concept, we also present Cross$Q$ (TD3), a version of Cross$Q$ which uses TD3 instead of SAC as the base algorithm. We perform periodic evaluations during training to obtain the episodic reward. From these, we report the mean and standard deviations over 10 random seeds. All subsequent experiments in this paper follow the same protocol.

This experiment shows that Cross$Q$ matches or outperforms the best baseline in all the presented environments except on `Ant`, where REDQ performs better in the early training stage, but Cross$Q$ eventually matches it. On `Hopper`, `Walker`, and `HalfCheetah`, the learning curves of Cross$Q$ and REDQ overlap, and there is no significant difference. On the harder `Humanoid` and `HumanoidStandup` tasks, Cross$Q$ and Cross$Q$ (TD3) both substantially surpass all baselines.

4.2 Computational Efficiency of Cross$Q$

Figure 6 compares the computational efficiency of Cross$Q$ to the baselines. This metric is where Cross$Q$ makes the biggest leap forward. Cross$Q$ requires $20\times$ fewer gradient steps than REDQ and DroQ, which results in roughly $4\times$ faster wallclock speeds (Table 2). Especially on the more challenging `Humanoid` and `HumanoidStandup` tasks the speedup is the most pronounced. In our view, this is a noteworthy feature. On the one hand, it opens the possibility of training agents in a truly online and data-efficient manner, such as in real-time robot learning. On the other hand, with large computing budgets Cross$Q$ can allow the training of even larger models for longer than what is currently feasible, because of its computational efficiency stemming from its low UTD $= 1$.

4.3 Evaluating Q Function Estimation Bias

All methods we consider in this paper are based on SAC and, thus, include the clipped double-Q trick to reduce Q function overestimation bias (Fujimoto et al., 2018). Chen et al. (2021) and Hiraoka et al. (2021) stress the importance of keeping this bias even lower to achieve their high performances and intentionally design REDQ and DroQ to additionally reduce bias with explicit and implicit ensembling. In contrast, Cross$Q$ outperforms both baselines without any ensembling. Could Cross$Q$'s high performance be attributed to implicitly reducing the bias as a side effect of our design choices? Using the same evaluation protocol as Chen et al. (2021), we compare the

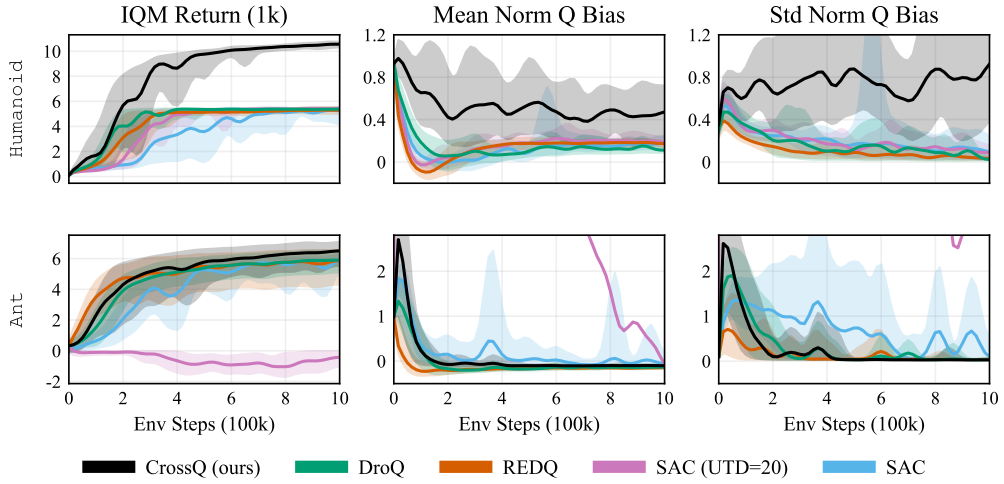


Figure 7: ***Q* estimation bias does not reliably influence learning performance**. Following the analysis of Chen et al. (2021), we plot the IQM and 70% quantile interval of the normalized Q function bias. REDQ generally has the least bias over 10 seeds. Cross$Q$ matches or outperforms DroQ, REDQ and SAC while showing more Q function bias in all environments. The full set of environments is shown in Fig. 17 in the Appendix.

normalized Q prediction biases in Figure 4.3. Due to space constraints, here we show `Hopper` and `Ant` and place the rest of the environments in Figure 17 in the Appendix.

We find that REDQ and DroQ indeed have lower bias than SAC and significantly lower bias than SAC with UTD $= 20$. The results for Cross$Q$ are mixed: while its bias trend exhibits a lower mean and variance than SAC, in some environments, its bias is higher than DroQ, and in others, it is lower or comparable. REDQ achieves comparable or worse returns than CrossQ while maintaining the least bias. As Cross$Q$ performs better *despite* having—perhaps paradoxically—generally higher Q estimation bias, we conclude that the relationship between performance and estimation bias is complex, and one does not seem to have clear implications on the other.

4.4 Ablations

We conduct ablation studies to better understand the impact of different design choices in Cross$Q$.

4.4.1 Disentangling the Effects of Target Networks and BatchNorm

Cross$Q$ changes SAC in three ways; of these, two explicitly aim to accelerate optimization: the removal of target networks, and the introduction of BatchNorm. Unfortunately, SAC without target networks diverges; therefore, to study the contribution of the first change, we need a way to compare SAC—divergence-free—*with and without target networks*. Fortunately, we find that such a way exists: according to our supplementary experiments in Appendix A.6, simply using bounded activation functions in the critic appears to prevent divergence. This is a purely empirical observation and an in-depth study regarding the influence of activations and normalizers on the stability of Deep RL is beyond the scope of this paper. In this specific ablation, we use `tanh` activations instead of `relu`, solely as a tool to make the intended comparison possible.

Figure 8 shows the results of our experiment. The performance of SAC without target networks supports the common intuition that target networks indeed slow down learning to a small extent. We find that the combination of BatchNorm and Target Networks performs inconsistently, failing to learn anything in half of the environments. Lastly, the configuration of BatchNorm without target networks—and the closest to Cross$Q$—achieves the best aggregate performance, with the boost being significantly bigger than that from removing target networks alone. In summary, even though removing target networks may slightly improve performance in some environments, it is the combination of removing target networks and adding BatchNorm that accelerates learning the most.

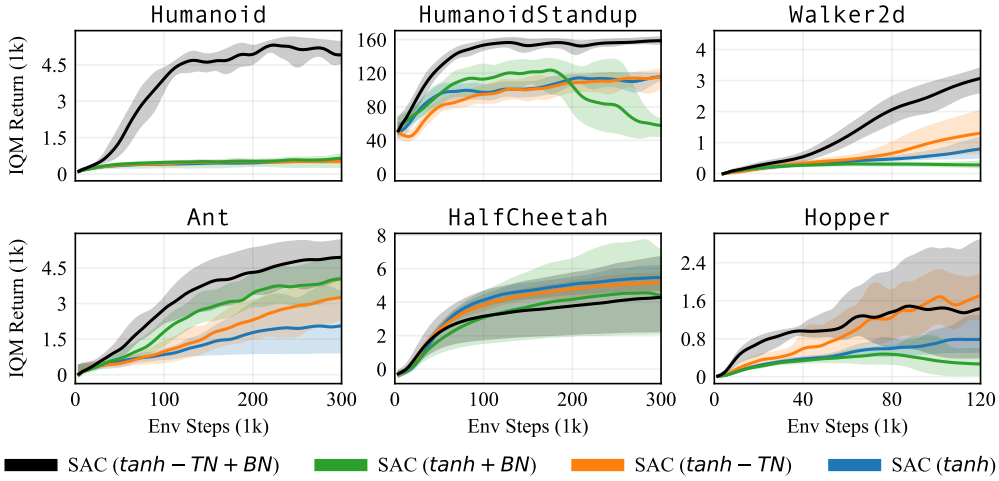


Figure 8: **The effects of target networks and BatchNorm on sample efficiency.** All SAC variants in this experiment use critics with tanh activations, since they allow divergence-free training without target networks, enabling this comparison. This ablation uses the original BatchNorm (BN, Ioffe & Szegedy (2015)). Removing target networks (−TN) provides only small improvements over the SAC baseline with target nets. BatchNorm with target nets (+BN, green) is unstable. Using BatchNorm after removing target nets (−TN+BN)—the configuration most similar to Cross$Q$—performs best. We plots IQM return and 70% quantile intervals over 10 seeds.

4.4.2 Ablating the Different Design Choices and Hyperparameters

In this subsection, we examine the contributions of the different Cross$Q$ design choices to show their importance. Figure 9 shows aggregated ablations of these components and various hyperparameters, while Figure 10 ablates the BatchNorm layer itself.

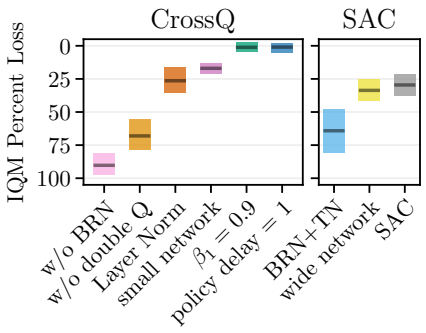


Figure 9: **Ablations on Cross$Q$ and SAC.** Loss in IQM return in percent—relative to Cross$Q$—at 1M environment interactions. Aggregated over all environments and six seeds each, with 95% bootstrapped confidence intervals (Agarwal et al., 2021). Left shows Cross$Q$ ablations; Right shows effects of adding parts on top of SAC. Figure 13 in Appendix shows individual training curves.

Hyperparameters. Cross$Q$ uses the best hyperparameters obtained from a series of grid searches. Of these, only three are different from SAC's default values. First, we find that reducing the β_1 momentum for the Adam optimizer (Kingma & Ba, 2015) from 0.9 to 0.5 as well the *policy delay* of 3 have the smallest impact on the performance. However, since fewer actor gradient steps reduce compute, this setting is favorable. Second, reducing the critic network's width to 256—the same small size as SAC—reduces performance and yet still significantly outperforms SAC. This suggests that practitioners may be able to make use of a larger compute budget, i.e., train efficiently across a range of different network sizes, by scaling up layer widths according to the available hardware resources. Third, as expected, removing the BRN layers proves to be detrimental and results in the worst overall performance. A natural question that comes to mind is whether other normalization strategies in the critic, such as Layer Normalization (LayerNorm, Ba et al. (2016)), would also give the same results. However, in our ablation, we find that replacing BatchNorm with LayerNorm degrades Cross$Q$'s performance significantly, roughly to the level of the SAC baseline. Lastly, SAC does not benefit from simply widening critic layers to 2048. And naively adding BRN to SAC while keeping the target networks proves detrimental. This finding is in line with our diagnosis of mismatched statistics being detrimental to the training.

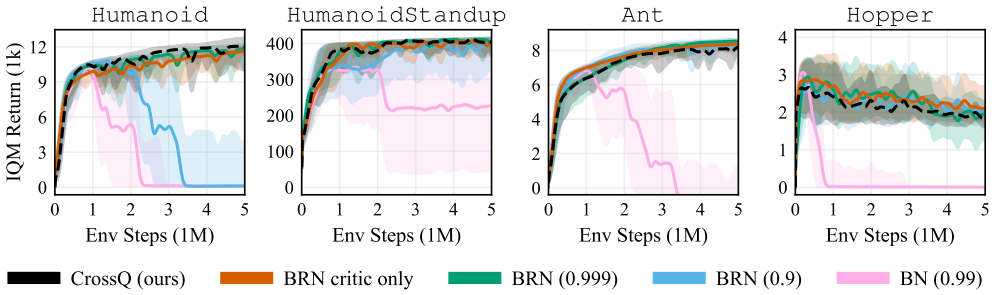


Figure 10: **Comparing BatchNorm hyperparameters.** All variants have comparably strong and stable curves early in the training. Omitting normalization in the actor (`BRN` critic only) does not significantly affect Cross$Q$. Using the original Batch Normalization (`BN`, with moving-average momentum 0.99) is prone to sudden performance collapses during longer training runs. Using `BRN` permits stabler training, which improves with higher momentums; Cross$Q$'s default 0.99 (black) and higher show no collapses. We plot IQM return and 70% quantile intervals over five seeds.

Batch Normalization Layers. In Figure 10, we ablate the BatchNorm versions (`BN` (Ioffe & Szegedy, 2015) and `BRN` (Ioffe, 2017)) and their internal moving-average momentums. Compared to Cross$Q$'s optimal combination—`BRN` with momentum 0.99—all variants have similar sample efficiency in the early stages of training (1M steps). When using `BN`, we sometimes observe sudden performance collapses later in training; we attribute these to `BN`'s unique approach of using noisy *minibatch estimates* of normalization moments. `BRN`'s improved approach of using the less noisy *moving-averages* makes these collapses less likely; further noise-reduction via higher momentums eliminates these collapses entirely. Additionally, we find that using BatchNorm only in the critic (instead of both the actor and the critic) is sufficient to drive the strong performance of Cross$Q$; however, including it in both networks performs slightly better.

5 Conclusion & Future Work

We introduced Cross$Q$, a new off-policy RL algorithm that matches or exceeds the performance of REDQ and DroQ—the current state-of-the-art on continuous control environments with state observations—in terms of sample efficiency while being multiple times more computationally efficient. To the best of our knowledge, Cross$Q$ is the first method to successfully use BatchNorm to greatly accelerate off-policy actor-critic RL. Through benchmarks and ablations, we confirmed that target networks do indeed slow down training and showed a way to remove them without sacrificing training stability. We also showed that BatchNorm has the same accelerating effect on training in Deep RL as it does in supervised deep learning. The combined effect of removing target networks and adding BatchNorm is what makes Cross$Q$ so efficient. We investigated the relationship between the Q estimation bias and the learning performance of Cross$Q$, but did not identify a straightforward dependence. This indicates that the relationship between the Q estimation bias and the agent performance is more complex than previously thought.

In future work, it would be interesting to analyze the Q estimation bias more extensively, similar to Li et al. (2022). Furthermore, a deeper theoretical analysis of the used BatchNorm approach in the context of RL would be valuable, akin to the works in supervised learning, e.g., Summers & Dinneen (2020). Although the wider critic networks do provide an additional performance boost, they increase the computation cost, which could potentially be reduced. Finally, while our work focuses on the standard continuous control benchmarking environments, a logical extension would be applying Cross$Q$ to a real robot system and using visual observations in addition to the robot state. Techniques from image-based RL, such as state augmentation (Laskin et al., 2020; Yarats et al., 2021) and auxiliary losses (Schwarzer et al., 2021; He et al., 2022), also aim to learn efficiently from limited data. We believe some of these ideas could potentially be applied to CrossQ.

A Appendix

A.1 DeepMind Control Suite Experiments

Figure 11 presents an additional set of experiments performed on the DeepMind Control Suite (Tassa et al., 2018). The experiments shown here are an extension to the experiments shown in Figure 5 in the main paper and have been moved to the Appendix due to space constraints. For the presented tasks, we lowered the learning rate to 8×10^{-4} for all algorithms, and set the Cross$Q$ policy delay to 1. All other hyperparameters remained the same as for the main paper.

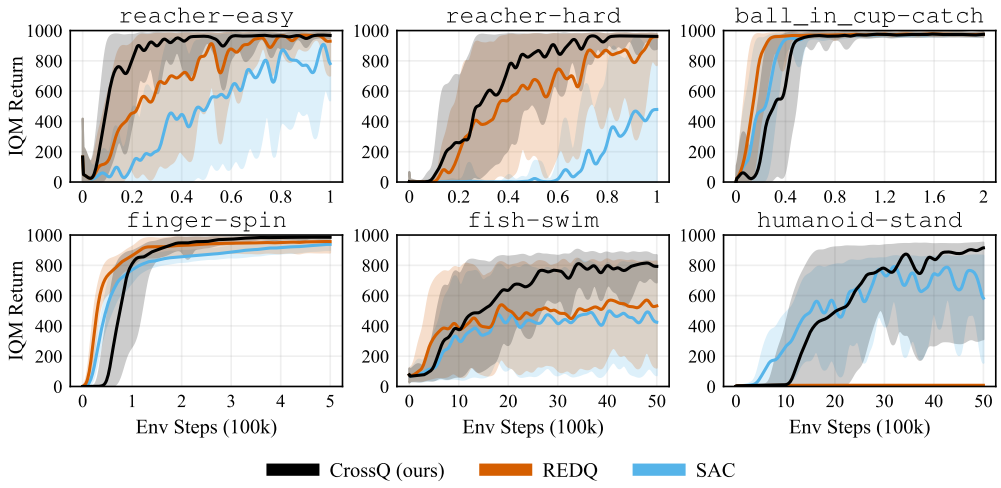


Figure 11: **Sample efficiency of Cross$Q$ on DeepMind Control.** The experiments here were each performed on 5 different random seeds. Cross$Q$'s good sample efficiency transfers well to the presented tasks from the DeepMind Control Suite.

A.2 HYPERPARAMETERS

Experiment hyperparameters, used in the main paper. We adapted most hyperparameters that are commonly used in other works (Haarnoja et al., 2018b; Chen et al., 2021; Hiraoka et al., 2021). The Moving-Average *Momentum* corresponds to 1 minus the Moving-Average *Update Rate* as defined in both BatchNorm papers (Ioffe & Szegedy, 2015; Ioffe, 2017).

Table 1: Learning Hyperparameters

Parameter	SAC	REDQ	DroQ	Cross$Q$ (ours)
Discount Factor (γ)	0.99			
Learning Rate (Actor & Critic)	0.001			
Replay Buffer Size	10^6			
Batch Size	256			
Activation Function	`relu`			
Layer Normalization	No		Yes	No
Dropout Rate	`N/A`		0.01	`N/A`
BatchNorm / Version	`N/A`			`BRN`
BatchNorm / Moving-Average Momentum	`N/A`			0.99
BatchNorm / `BRN` Warm-up Steps	`N/A`			10^5
Critic Width	256			2048
Target Update Rate (τ)	0.005			`N/A`
Adam β_1	0.9			0.5
Update-To-Data ratio (UTD)	1	20		1
Policy Delay	1	20		3
Number of Critics	2	10		2

A.3 WALLCLOCK TIME MEASUREMENT

Wallclock times were measured by timing and averaging over four seeds each and represent *pure training times*, without the overhead of synchronous evaluation and logging, until reaching 5×10^6 environment steps. The times are recorded on an `Nvidia RTX 3090 Turbo` with an `AMD EPYC 7453` CPU.

Table 2: **Wallclock times.** Evaluated for Cross$Q$ and baselines across environments in hours and recorded on an `RTX 3090`, the details of the measurement procedure are described in Appendix 4.2. Comparing Cross$Q$ with Cross$Q$ (Small) and SAC, it is apparent that using wider critic networks does come with a performance penalty. However, compared to REDQ and DroQ, one clearly sees the substantial improvement in Wallclock time of Cross$Q$ over those baselines.

	Wallclock Time [hours]				
	SAC	Cross$Q$ (small)	**Cross$Q$ (ours)**	REDQ	DroQ
`HumanoidStandup-v4`	1.5	2.1	2.2	8.7	7.5
`Walker2d-v4`	0.9	0.9	1.1	4.0	4.1
`Ant-v4`	0.9	1.2	1.5	4.7	4.7
`HalfCheetah-v4`	0.8	1.2	1.5	4.1	4.4
`Hopper-v4`	1.0	1.1	1.3	4.1	4.2

A.4 EVOLVING ACTION DISTRIBUTIONS

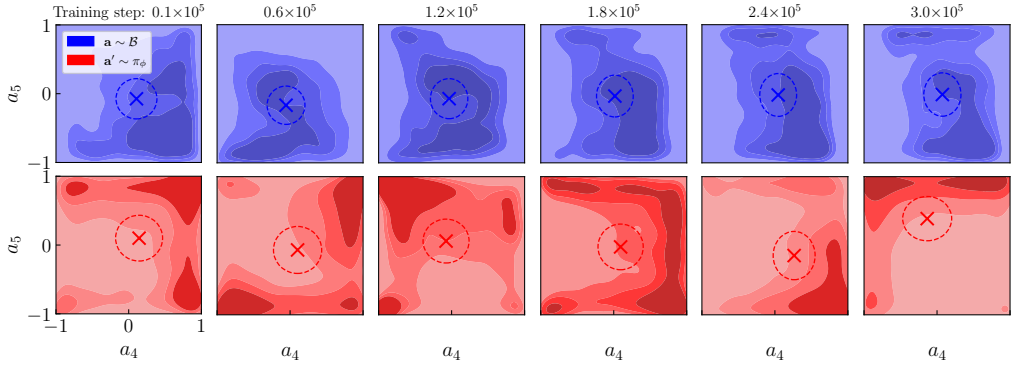


Figure 12: **Replay and policy action distributions are different, and evolve during training.** We train an agent for $300,000$ steps on `Walker2d`. We take snapshots of the replay buffer $\mathcal{B}$ and policy π_ϕ every $60,000$ steps. For each snapshot (one column), we sample a large batch of $10,000$ transitions $(\boldsymbol{s}, \boldsymbol{a}, \boldsymbol{s}', \boldsymbol{a}' = \pi_\phi(\boldsymbol{s}'))$ and use this to compute a visually interpretable 2D kernel density estimate of the distributions of $\boldsymbol{a}$ (blue) and $\boldsymbol{a}'$ (red), as seen through the action-space dimensions 4 and 5. The cross denotes the mean, and the dashed ellipse is one standard deviation wide for each of the two dimensions. We observe that the distributions as well as the means and standard deviations of the off-policy and on-policy actions are visibly and persistently different throughout the training run, and keep drifting as the training progresses. This discrepancy implies that BatchNorm must be used with care in off-policy TD learning.

A.4.1 Ablating the Different Design Choices and Hyperparameters

Figure 13 depicts in detail the Cross$Q$ and SAC ablations, previously shown in aggregate form by Figure 9.

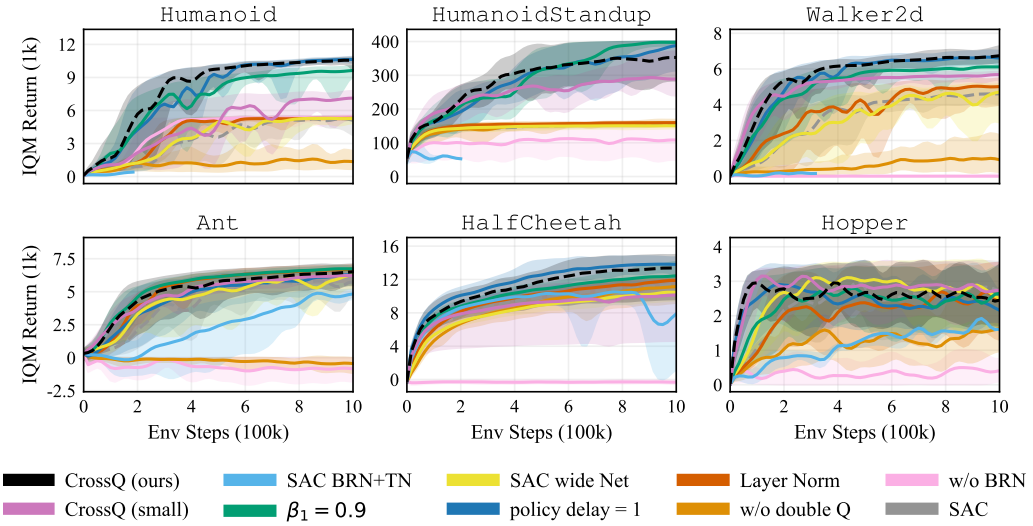


Figure 13: **Cross$Q$ ablation study.** We ablate across different hyperparameter settings and architectural configurations. Using the same network width as SAC, Cross$Q$ (small) shows weaker performance, yet is still competitive with Cross$Q$ in four out of six environments. At the same time, SAC with a wider critic does not work better. Using the default Adam momentum $\beta_1 = 0.9$ instead of 0.5 degrades performance in some environments. Using a policy delay of 1 instead of 3 has a very small effect, except on `Ant`. Using LayerNorm instead of BatchNorm results in slower learning; it also trains stably without target networks. Removing BatchNorm results in failure of training due to divergence. Adding BatchNorm to SAC and reusing the live critic's normalization moments in the target network fails to train. Training without double Q networks (single critic) harms performance.

A.5 REDQ AND DROQ ABLATIONS

Figures 14 and 15 show REDQ and DroQ ablations on 5 seeds each. They show both baselines with the Cross$Q$ hyperparameters: wider critic networks as well as $\beta_1 = 0.5$. Neither baseline benefits from the added changes. In most cases, the performance is unchanged, while in some cases, it deteriorates. The dashed black line shows Cross$Q$ as a reference.

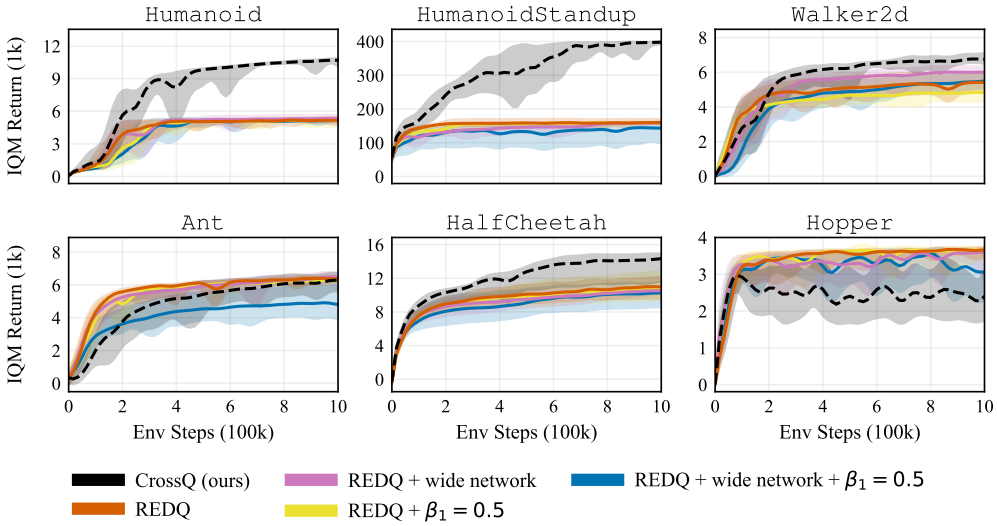


Figure 14: **REDQ ablation.** Showing performance for different combinations of the Cross$Q$ hyperparameters. The changes in hyperparameters do not help REDQ to get better performance. In fact, in some cases, they even hurt the performance.

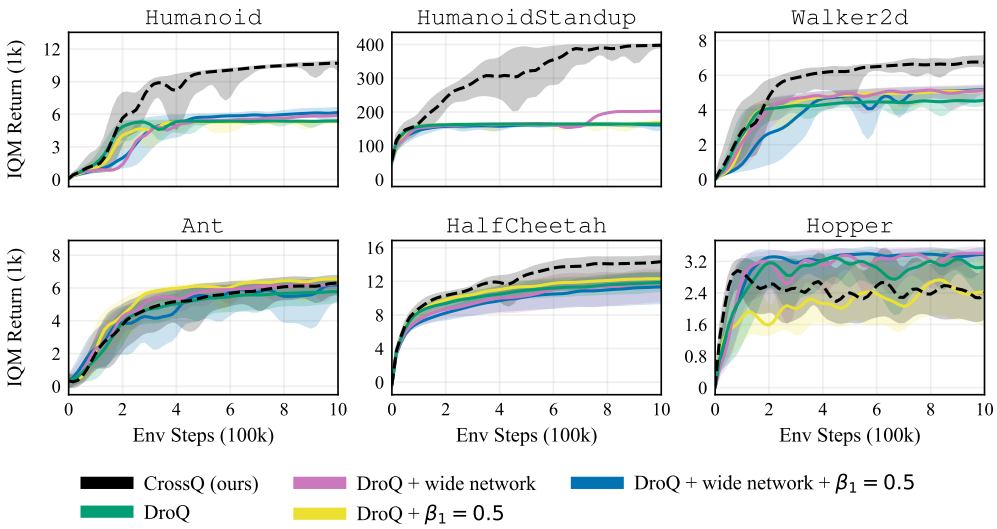


Figure 15: **DroQ ablation.** The changes in hyperparameters do not help DroQ to get better performance overall. In `Hopper` and `Ant`, performance rises to the Cross$Q$ performance, however, on the `Humanoid`, it hurts performance.

A.6 Effect of Activations and Normalizers on Learning Stability

Figure 8 depicts a small exploratory experiment in which we remove target networks from SAC, and train it with different activation functions and feature normalizers. We do this only to explore whether the boundedness of activations has an influence on training stability. We learn from this experiment that SAC with $\tanh$ activations trains without divergence, allowing us to conduct the study in Section 4.4.1. We also observe that at least two feature normalization schemes (on top of the unbounded relu activations) permit divergence-free optimization.

For vectors $\boldsymbol{x}$, $\text{relu\_over\_max}(\boldsymbol{x})$ denotes a simple normalization scheme using an underlying unbounded activation: $\text{relu}(\boldsymbol{x})/\max(\boldsymbol{x})$, with the maximum computed over the entire feature vector. layernormed_relu simply denotes LayerNorm applied *after* the relu activations. Both of these schemes prevent divergence. Using LayerNorm *before* the relu activations also prevent divergence, and is already explored in the ablations in Figure 13. None of these normalizers perform as strongly as BatchNorm.

A thorough theoretical or experimental study of how activations and normalizers affect the stability of Deep RL is beyond the scope of this paper. We hope, however, that our observations help inform future research directions for those interested in this topic.

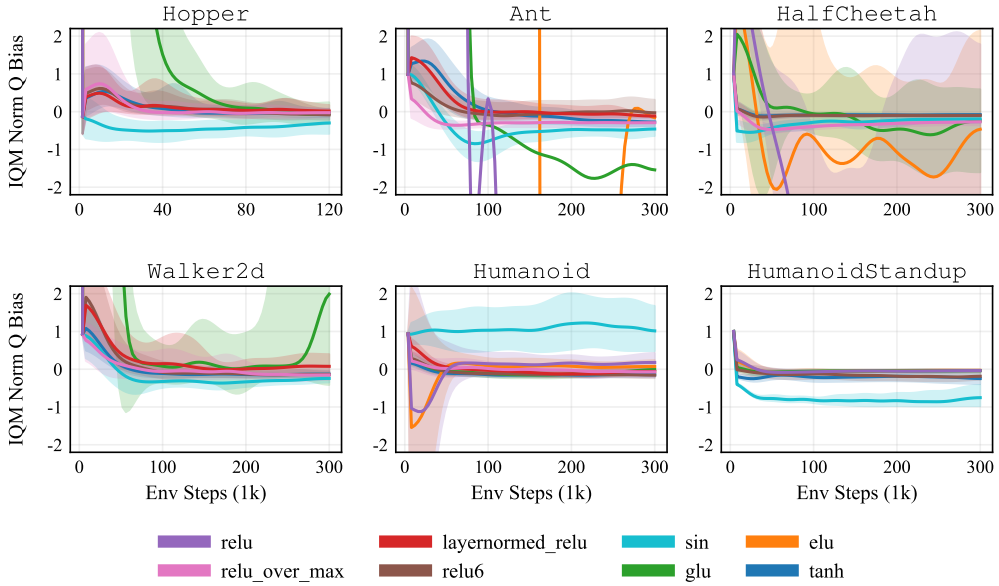


Figure 16: **(In)stability of SAC without target networks.** Observed through the Q estimation bias. In this small-scale experiment, we run SAC with unbounded $(\text{relu}, \text{glu}, \text{elu})$ and bounded $(\tanh, \text{relu6}, \sin)$ activation functions, as well as "indirectly" bounded activations through the use of two custom normalizers other than BatchNorm $(\text{relu\_over\_max}, \text{layernormed\_relu})$. SAC variants with unbounded activations appear highly unstable in most environments, whereas the variants with bounded activations (as well as the normalizers) do not diverge, maintaining relatively low bias.

A.7 NORMALIZED Q BIAS PLOTS

Figure 17 shows the results of the Q function bias analysis for all environments.

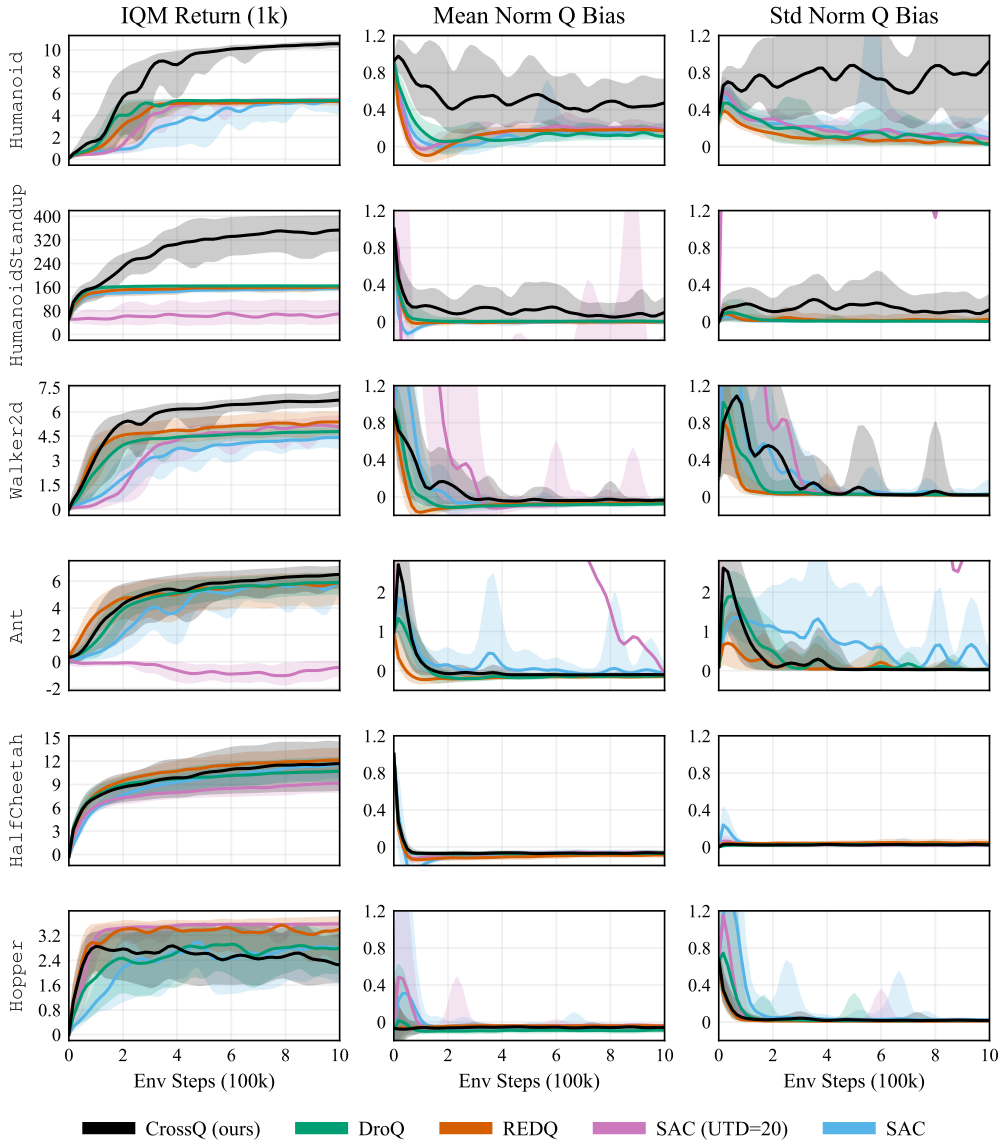


Figure 17: **Q estimation bias.** Mean and standard deviation of the normalized Q function bias, computed as described by Chen et al. (2021). As in the main paper, we do not find a straightforward connection between normalized Q function bias and learning performance. Cross$Q$ generally shows the same or larger Q estimation bias compared to REDQ but matches or outperforms REDQ in learning speed, especially on the challenging `Humanoid` tasks.

Normalization and effective learning rates in reinforcement learning

Clare Lyle[†] **Zeyu Zheng**[†] **Khimya Khetarpal**[†] **James Martens**[†] **Hado van Hasselt**[†]

Razvan Pascanu[†] **Will Dabney**[†]

Abstract

Normalization layers have recently experienced a renaissance in the deep reinforcement learning and continual learning literature, with several works highlighting diverse benefits such as improving loss landscape conditioning and combatting overestimation bias. However, normalization brings with it a subtle but important side effect: an equivalence between growth in the norm of the network parameters and decay in the effective learning rate. This becomes problematic in continual learning settings, where the resulting effective learning rate schedule may decay to near zero too quickly relative to the timescale of the learning problem. We propose to make the learning rate schedule explicit with a simple re-parameterization which we call Normalize-and-Project (NaP), which couples the insertion of normalization layers with weight projection, ensuring that the effective learning rate remains constant throughout training. This technique reveals itself as a powerful analytical tool to better understand learning rate schedules in deep reinforcement learning, and as a means of improving robustness to nonstationarity in synthetic plasticity loss benchmarks along with both the single-task and sequential variants of the Arcade Learning Environment. We also show that our approach can be easily applied to popular architectures such as ResNets and transformers while recovering and in some cases even slightly improving the performance of the base model in common stationary benchmarks.

1 Introduction

Many of the most promising application areas of deep learning, in particular reinforcement learning (RL), require training on a problem which is in some way nonstationary. In order for this type of training to be effective, the neural network must maintain its ability to adapt to new information as it becomes available, i.e. it must remain *plastic*. Several recent works have shown that loss of plasticity can present a major barrier to performance improvement in RL and in continual learning [Dohare et al., 2021, Lyle et al., 2021, Nikishin et al., 2022]. These works have proposed a variety of explanations for plasticity loss such as the accumulation of saturated ReLU unit and increased sharpness of the loss landscape [Lyle et al., 2023], along with mitigation strategies, such as resetting dead units [Sokar et al., 2023] and regularizing the parameters towards their initial values [Kumar et al., 2023]. Many of these explanations and their corresponding mitigation strategies center around reducing drift in the distribution of pre-activations [Lyle et al., 2024], a problem which has historically been resolved in the supervised learning setting by incorporating normalization layers into the network architecture. Indeed, normalization layers have been shown to be highly effective at stabilizing optimization in both continual learning and RL [Hussing et al., 2024, Ball et al., 2023].

[†]Google DeepMind. Correspondence to clarelyle@google.com

Preprint. Under review.

While effective, normalization on its own is insufficient to avoid loss of plasticity [Lee et al., 2023]. Part of the reason for this lies in a subtle property of normalization: a normalization layer causes the subnetwork preceding it to become scale-invariant, which means that the layer's *effective learning rate* (ELR) now depends on the norm of its parameters [Van Laarhoven, 2017]. In particular, when the norm of the parameters grows, as it typically does in neural networks trained without regularization, the effective learning rate shrinks. While this property has been studied extensively in idealized supervised learning settings [Arora et al., 2018], its practical implications on optimization in nonstationary problems have previously only been noted in the form of 'saturation' of normalization layers [Lyle et al., 2024].

This work begins with an analysis of the effect of layer normalization on a network's plasticity, revealing two surprising benefits. We first show that when coupled with adaptive optimizers such as Adam and RMSProp whose step size is independent of gradient norm, saturated units still receive sufficient gradient signal to make non-trivial changes to their parameters, providing the opportunity for them to recover as a natural byproduct of optimization. We go on to study the implicit learning rate schedule that layer normalization induces, arriving at the striking observation that even without an explicit learning rate schedule, the implicit learning rate decay induced by parameter norm growth in value-based deep RL algorithms is in fact critical to the optimization process. This observation yields an explanation for the dramatic performance degradations often induced by weight decay in value-based deep RL agents: certain components of the value function require a sufficiently small ELR in order to be learned, and an optimization process which does not reach this value will therefore underfit the value function in ways that can inhibit performance improvement. We further show that, while often beneficial in single-task settings, this implicit schedule can harm performance in nonstationary regimes, where the natural effective learning rate schedule drives the learning rate to trivial values too quickly relative to the task duration.

Leveraging this insight, we propose Normalize-and-Project (NaP), a simple protocol which ensures that the learning rate used by the optimizer reflects the true effective learning rate on the network. NaP avoids implicit learning rate decay, allowing us to attain impressive performance in a variety of nonstationary learning problems even without explicit regularization intended to prevent loss of plasticity. We conduct an empirical evaluation of NaP, confirming that it can be applied to a variety of architectures and datasets without interfering with (indeed, in some cases even improving) performance, including 400M transformer models trained on the C4 dataset and vision models trained on CIFAR-10 and ImageNet. We conclude with a study on the Sequential Arcade Learning Environment, where NaP demonstrates remarkable robustness to task changes and outperforms a baseline Rainbow agent with freshly initialized parameters after 400M training frames (100M optimizer steps).

2 Background and related work

We begin by providing background on trainability and its loss in nonstationary learning problems. We additionally give an overview of neural network training dynamics and effective learning rates.

2.1 Training dynamics and plasticity in neural networks

Early work on neural network initialization centered around the idea of controlling the norm of the activation vectors [LeCun et al., 2002, Glorot and Bengio, 2010, He et al., 2015] using informal arguments. More recently, this perspective has been formalized and expanded [Poole et al., 2016, Daniely et al., 2016, Martens et al., 2021] to include the inner-products between pairs of activation vectors (for different inputs to the network). The function that describes the evolution of these inner-products determines the network's gradients at initialization up to rotation, and this in turn determines trainability (which was shown formally in the Neural Tangent Kernel regime by Xiao et al. [2020] and Martens et al. [2021]). A variety of initialization methods have been developed to ensure the network avoids "shattering" [Balduzzi et al., 2017] or collapsing gradients [Poole et al., 2016, Martens et al., 2021, Zhang et al., 2021b].

Once training begins, learning dynamics can be well-characterized in the infinite-width limit by the neural tangent kernel and related quantities [Jacot et al., 2018, Yang, 2019], although in practice optimization dynamics diverge significantly from the infinite-width limit [Fort et al., 2020]. A complementary line of work has empirically and theoretically characterized self-stabilization properties

[Lewkowycz et al., 2020, Cohen et al., 2021, Agarwala et al., 2022], along with implicit regularization effects [Barrett and Dherin, 2020, Smith et al., 2020] from using non-infinitesimal learning rate. A variety of architectural choices can accelerate the training of extremely deep networks, including residual connections [He et al., 2016] and normalization layers [Ioffe and Szegedy, 2015, Ba et al., 2016]. Some additional works have aimed to replicate the benefits of normalization layers via normalization of the network *parameters* [Salimans and Kingma, 2016, Arpit et al., 2016], though layer normalization remains standard practice.

Maintaining trainability not only at initialization, but also, over the course of optimization is of critical importance in reinforcement and continual learning, and failure to do so has been extensively documented throughout the literature under the term *loss of plasticity* [Abbas et al., 2023, Lyle et al., 2021, Dohare et al., 2021, Sodhani et al., 2020, Nikishin et al., 2022]. This phenomenon has been shown to present a limiting factor to performance in a number of RL tasks [Nikishin et al., 2023], along with continual learning and warm-starting neural network training [Berariu et al., 2021, Ash and Adams, 2020]. Plasticity loss can be further decomposed into two distinct components [Lee et al., 2023]: loss of trainability and reduced generalization performance, of which we focus on the former.

2.2 Effective learning rates

As noted by several prior works [Van Laarhoven, 2017, Li and Arora, 2020, Li et al., 2020b], normalization introduces scale-invariance into the layers to which it is applied, where by a scale-invariant function f we mean $f(c\theta, \mathbf{x}) = f(\theta, \mathbf{x})$ for any positive scalar $c > 0$. This leads to the gradient scaling inversely with the parameter norm, whereby $\nabla f(c\theta) = \frac{1}{c}\nabla f(\theta)$. The intuition behind this property is simple: changing the direction of a large vector requires a greater perturbation than changing the direction of a small vector. This motivates the concept of an 'effective learning rate', which provides a scale-invariant notion of optimizer step size. In the following definition, we take the approach of Kodryan et al. [2022] and assume an implicit 'reference norm' of size 1 for the parameters.

Definition 1 (Effective learning rate)**.** *Consider a scale-invariant function f, parameters θ and update function $\theta_{t+1} \leftarrow \theta_t + \eta g(\theta_t)$. Letting $\rho = \frac{1}{\|\theta\|}$, we then define the* effective learning rate $\tilde{\eta}$ *as follows:*

$$\tilde{\eta} = \begin{cases} \eta\rho^2, & \text{if } g(\theta_t) = \nabla_\theta f(\theta_t) \\ \eta\rho, & \text{if } g(\theta_t) = \frac{\nabla_\theta f(\theta_t)}{\|\nabla_\theta f(\theta_t)\|} \end{cases} \tag{1}$$

where, letting $\tilde{\theta} = \theta\frac{1}{\|\theta\|}$ we then have $f(\tilde{\theta} + \tilde{\eta} g(\tilde{\theta})) = f(\theta + \eta g(\theta))$

This suggests that regularization of the network norm can have the dual effect of increasing the effective learning rate, as noted by prior work [Van Laarhoven, 2017, Hoffer et al., 2018]. Several works have since offered more fine-grained theoretical analyses of such implicit learning rate schedules [Arora et al., 2018], and suggested means of translating these implicit schedules into explicit ones [Li and Arora, 2020, Li et al., 2020a]. More recently, the work of Lobacheva et al. [2021] and Kodryan et al. [2022] has studied the training properties of scale-invariant networks trained with parameters constrained to the unit sphere, a training regime we expand upon in this work.

3 Analysis of normalization layers and plasticity

Although widely used and studied, the precise reasons behind the effectiveness of layer normalization remain mysterious. In this section, we provide some new insights into how normalization can help neural networks to maintain plasticity by facilitating the recovery of saturated nonlinearities, and highlight the importance of controlling the parameter norm in networks which incorporate normalization layers. We leverage these insights to propose Normalize-and-Project, a simple training protocol to maintain important statistics of the layers and gradients throughout training.

3.1 Layer normalization

It is widely accepted that achieving approximately mean-zero, unit-variance pre-activations (assuming suitable choices of activation functions) is useful to ensure a network is trainable at initialization [e.g Martens et al., 2021], and many neural network initialization schemes aim to maintain this property

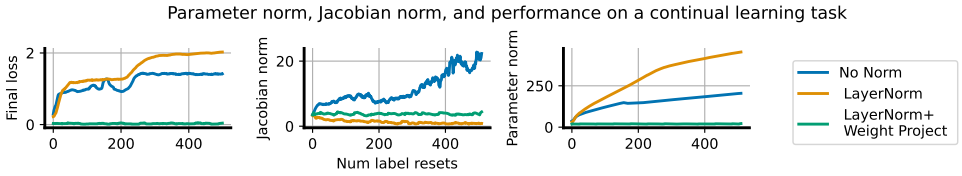


Figure 1: Continual random-labels CIFAR training: simple feedforward network architecture (No Norm) exhibits rapid growth in its parameter norm and the norm of its gradients, whereas the otherwise-identical network with layer normalization sees parameter norm growth coupled with a *reduction* in the norm of its gradients and reduced performance on later tasks. Constraining the parameter norm of this network maintains the performance of a random initialization.

as the depth of the network grows. Indeed, it is easy to show that in extreme cases large deviations of these statistics from their initial values can lead to a variety of network pathologies including saturated units and low numerical rank of the empirical neural tangent kernel [Xiao et al., 2020]. Layer normalization not only guarantees that activations are unit-norm, mean-zero at initialization, but also that they stay that way over the course of training even if the data distribution changes, assuming no scale or offset parameters. This property not only improves robustness to a variety of pathologies that cause loss of plasticity [Lyle et al., 2024], but also helps to improve the conditioning of the network's gradients in RL [Ball et al., 2023].

Beyond re-normalizing the pre-activation statistics, layer normalization also introduces a dependency between units in a given layer via the mean subtraction and division by standard deviation transformations, which translates to correlations in the gradients of their corresponding weights. This mixing step allows gradients to propagate through a pre-activation even if the unit is saturated, provided layer normalization is applied prior to the nonlinearity, a property we highlight in Proposition 1. We will use the notation f_{RMS} to refer to the transform $h \mapsto \frac{h}{\|h\|}$, and $f'(x) \equiv \frac{\partial}{\partial x} f(x)$ to refer to the scalar derivative of any f at scalar x.

Proposition 1. *Consider two indices i and j of a feature embedding $\phi(f_{\text{RMS}}(h))$ such that $\phi'(f_{\text{RMS}}(h)_j) \neq 0$, and $h_i, h_j \neq 0$. Then we have*

$$\frac{d}{dh_i}\phi(f_{\text{RMS}}(h))_j = -\phi'(f_{\text{RMS}}(h)_j)\frac{1}{\|h\|^3}h_i h_j \neq 0\ .$$

In contrast, for post-activation normalization the gradient is zero whenever $\phi'(h_i) = 0$, taking the form $\partial_{h_i} f_{\text{RMS}}(\phi(h))_j = -\phi'(h_i)\frac{1}{\|\phi(h)\|^3}\phi(h_i)\phi(h_j)$.

In other words, normalization effectively gives dead ReLU units a second chance at life – rather than immediately decaying to zero, the gradients flowing through a dead ReLU unit will instead take small, non-zero values, which depends on the gradients of the mean and variance of that particular layer. These gradients will be much smaller than those that would typically backpropagate to the unit, but if an optimizer such as Adam or RMSProp is used to correct for the gradient norm, then the unit may still be able to take nontrivial steps, which have a chance at propelling it back into the activated regime. We include the full derivation in Appendix A.2, and we demonstrate the effect this can have on both a theoretical model and empirical neural network training in Figure 10 in Appendix C.4. This property is also naturally inherited by layer normalization, which can be viewed as the composition of RMSNorm with a centering transform. While layer normalization can help the network to recover from saturated nonlinearities, it introduces a new source of potential saturation which must be carefully considered, which is something we will do in the next section.

3.2 Parameter norm and effective learning rate decay

While the *output* of a scale-invariant function is insensitive to scalar multiplication of the parameters, its *gradient* magnitude scales inversely with the parameter norm. This results in the opposite behaviour of what we would expect in an unnormalized network: in networks with layer normalization, growth in the norm of the parameters corresponds to a decline in the network's sensitivity to changes in these parameters. In a sense this is preferable, as the glacially slow but stable regime of vanishing gradients is easier to recover from than the unstable exploding gradient regime. However, if the parameter

Algorithm 1 NaP: Normalize-and-Project

Input: network $\mathcal{N}$, input x
for nonlinearity ϕ_l in network **do**
  if ϕ_l not already normalized **then**
    $\phi_l \leftarrow \phi_l \circ \text{LayerNorm}$
  end if
end for
compute $\theta' = \texttt{update}(\theta)$
for parameter W_l in network **do**
  compute $W_l \leftarrow \text{WeightProject}(W_l)$
end for

$\text{WeightProject}(W_l, \rho_l)$:
if W_l is a weight parameter **then**
  $W_l \leftarrow \frac{\rho_l W_l}{\|W_l\|}$
else
  $\sigma_l, \mu_l \leftarrow W_l, d \leftarrow \text{len}(\sigma_l)$
  $\sigma_l \leftarrow \sigma_l \sqrt{\frac{d}{\|\sigma_l\|^2 + \|\mu_l\|^2}}$
  $\mu_l \leftarrow \mu_l \sqrt{\frac{d}{\|\sigma_l\|^2 + \|\mu_l\|^2}}$
end if

norm grows indefinitely then the corresponding reduction in the effective learning rate will eventually cause noticeable slowdowns in learning – indeed, it is quite easy to induce this type of situation as we show in Figure 1. To do so, we take a small base convolutional neural network architecture (detailed in Appendix B.4) and train it on random labels of the CIFAR-10 dataset, akin to the classic setting of Zhang et al. [2021a]. We then re-randomize these labels and continue training, repeating this process 500 times. When we apply this process to a network without normalization layers, the Jacobian norm grows to unstable values as the parameter norm increases; in contrast, an equivalent architecture with normalization layers sees a sharp decline in the Jacobian norm as the parameter norm increases. In both cases, the end result is similar: increased parameter norm accompanies reduced performance.

While this particular problem is artificial, it is a real and widely observed underlying phenomenon that the magnitudes of neural network parameters tend to increase over the course of training [Nikishin et al., 2022, Abbas et al., 2023]. In a supervised learning problem, where one is using a fixed training budget, the ELR decay induced by growing parameter norms might be desirable and help to protect against too-large learning rates [Arora et al., 2018, Salimans and Kingma, 2016]. Allowed to continue to extremes, however, ELR decay becomes problematic [Lyle et al., 2024]. Instead, if we re-normalize the parameters after every task (which does not affect the output of our scale-invariant model) so they recover the norm they had at initialization (but do not change their direction), we see in Figure 1 that we can maintain the performance of a random initialization even after hundreds of training iterations.

3.3 Normalize-and-Project

We conclude from the above investigation that normalizing a network's pre-activations and fixing the parameter norm presents a simple but effective defense against loss of plasticity. In this section, we propose a principled approach to combine these two steps which we call NaP. Our goal for NaP is to provide a flexible recipe which can be applied to essentially any architecture, and which improves the stability of training, motivated by but not limited to non-stationary problems. Our approach can be decomposed into two steps: the **insertion of normalization layers** prior to nonlinearities in the network architecture, and the **periodic projection** of the network's weights onto a fixed-norm radius throughout training, along with a corresponding update to the per-layer learning rates into the optimization process. Algorithm 1 provides an overview of NaP. While some design choices, such as the use of scale and offset terms, can be tailored to a particular problem setting, we will aim to provide principled guidance on how to reason about the effects of these choices.

Layer normalization: Introducing layer normalization allows us to benefit from the properties discussed in Section 3.1, though fully benefiting from these properties depend on normalization being applied each time a linear transform precedes a nonlinearity. While it might seem extreme, this proposal is in line with most popular language and vision architectures. For example, Vaswani et al. [2017] apply normalization after every two fully-connected layers, and recent results suggesting that adding normalization to the key and query matrices in attention heads [Henry et al., 2020] can provide further benefits.

Weight projection: As discussed in Section 3.2, we must then take care in controlling the network's effective learning rate. We propose disentangling the parameter norm from the effective learning rate by enforcing a constant norm on the weight parameters of the network, allowing scaling of the layer outputs to depend only on the learnable scale and offset parameters. This approach is similar to that proposed by Kodryan et al. [2022], but importantly takes care to treat the scale and offset parameters

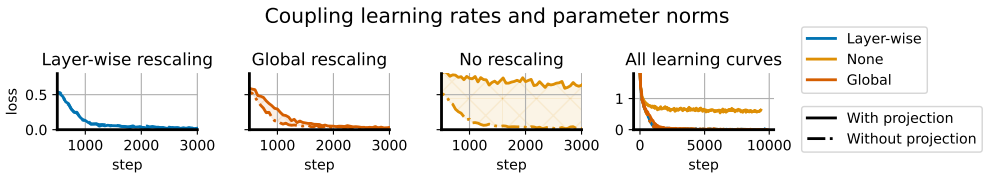


Figure 2: We run a 'coupled networks' experiment as described in the text. All networks exhibit similar learning curves, as seen by the rightmost subplot, however there is small but visible gap between the learning curves obtained by NaP and an unconstrained network with fixed learning rates. Using a global learning rate schedule almost entirely closes this gap, but does not induce a precise equivalence in the dynamics as obtained by layer-wise rescaling (leftmost).

separately from weights. For simplicity, we remove bias terms as these are made redundant by the learnable offset parameters. In order to maintain constant parameter norm, we rescale the parameters of each layer to match their initial norm periodically throughout training – the precise frequency is not important as long as the parameter norm does not meaningfully grow to a point of slowing optimization between projections. For example, we find that in Rainbow agents an interval of 1000 steps and 1 step produce nearly identical empirical results.

We find it is absolutely critical to normalize the weight parameters, as these represent the bulk of trainable parameters in the network. The learnable scale and offset parameters, assuming they are included in the network[3], permit more flexibility and can be dealt with in one of three ways. The strictly correct version in the case of homogeneous activations such as ReLUs is the approach we outline in Appendix D, which projects the concatenated scale-offset vector. This can require some effort to implement due to the dependency between the scale and offset parameters and may not be worthwhile for small training runs – indeed, most of our empirical results did not require this step, though we include it in our analysis of smaller networks in Figure 2. A softer version of this normalization which requires less implementation overhead and which generalizes to non-homogeneous activations is to regularize the scale and offset parameters to their initial values, a strategy which we employ in our continual learning evaluations. Finally, they can be allowed to drift unconstrained from their initialization, a choice we find unproblematic in most shorter, stationary learning problems. For a more detailed discussion on this choice, we refer to Appendix D.

4 Lessons on the effective learning rate

NaP constrains the network's effective learning rate to follow an explicit rather than implicit schedule. In this section, we explore how this property affects network training dynamics, demonstrating how implicit learning rate schedules due to parameter norm growth can be made explicit and be leveraged to improve the performance of NaP in deep RL domains.

4.1 Replicating the dynamics of parameter norm growth in NaP

We begin our study of effective learning rates by illustrating how the implicit learning rate schedule induced by the evolution of the parameter norm can be translated to an explicit schedule in NaP. We study a small CNN described in Appendix B.4 with layer normalization prior to each nonlinearity trained on CIFAR-10 with the usual label set. We train two 'twin' scale-invariant networks with the Adam optimizer in tandem: both networks see the exact same data stream and start from the same initialization, but the per-layer weights of one are projected after every gradient step to have constant norm, while the other is allowed to vary the norms of the weights. We then consider three experimental settings: in the first, we re-scale the per-layer learning rates of the projected network so that the explicit learning rate is equal to the effective learning rate of its twin. In the second, we re-scale the global learning rate based on the ratio of parameter norms between the projected and unprojected network, but do not tune per-layer. In the third, we do no learning rate re-scaling. We see in Figure 2 that the shapes of the learning curves for all networks except for the constant-ELR

[3]We observed in many of our experiments that removing trainable scale and offset parameters often has little effect on network performance. In Rainbow agents, for example, removing the trainable offset parameter even improves performance in several environments.

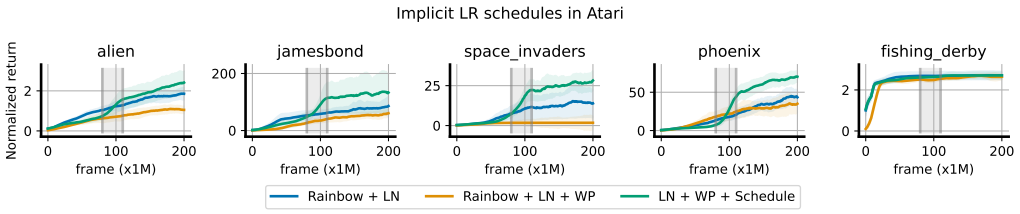


Figure 3: Without an explicit learning rate schedule, a Rainbow trained with NaP may fail to make any performance improvement; while the implicit schedule induced by the parameter norm is clearly important to performance, in several games this is significantly outperformed by a simple linear schedule terminating halfway through training. Intriguingly, we see a characteristic sharp improvement near the end of the decay schedule in several (though not all, e.g. fishing derby) games.

variant are quite similar, with the global learning rate scaling strategy producing a smaller gap than the no-rescaling strategy. By construction, the dynamics of the per-layer rescaling network and its twin are identical. Because global learning rate schedules are standard practice and induce dynamics that are quite close to those obtained by parameter norm growth in Figure 2, we take this approach in the remainder of the paper, leaving layer-specific learning rates and schedules for future work.

4.2 Implicit learning rate schedules in deep RL

When taken to extremes, learning rate decay will eventually prevent the network from making nontrivial learning progress. However, learning rate decay plays an integral role in the training of many modern architectures, and is required to achieve convergence for stochastic training objectives (unless the interpolation applies or Polyak averaging is employed). In this section we will show that, perhaps unsurprisingly, naive application of NaP with a constant effective learning rate can sometimes harm performance in settings where the implicit learning rate schedule induced by parameter norm growth was in fact critical to the optimization process. More surprising is that the domain where this phenomenon is most apparent is one where common wisdom would suggest learning rate decay would be undesirable: deep RL.

RL involves a high degree of nonstationarity. As a result, deep RL algorithms such as DQN and Rainbow often use a constant learning rate schedule. Given that layer normalization has been widely observed to improve performance in value-based RL agents on the arcade learning environment, and that parameter norm tends to increase significantly in these agents, one might at first believe that the performance improvement offered by layer normalization is happening in *spite* of the resulting implicit learning rate decay. A closer look at the literature, however, reveals that several well-known algorithms such as AlphaZero [Schrittwieser et al., 2020], along with many implementations of popular methods such as Proximal Policy Optimization [Schulman et al., 2017], incorporate some form of learning rate decay, suggesting that a constant learning rate is not always desirable. Indeed, Figure 3 shows that constraining the parameter norm to induce a fixed ELR in the Rainbow agent frequently results in *worse* performance compared to unconstrained parameters. This is particularly striking given that many of the benefits supposedly provided by layer normalization, such as better conditioning of the loss landscape [Lyle et al., 2023] and mitigation of overestimation bias [Ball et al., 2023], should be independent of the effective learning rate. Instead, these properties appear to either be irrelevant for optimization or dependent on reductions in the effective learning rate.

We can close this gap by introducing a learning rate schedule (linear decay from the default $6.25 \cdot 10^{-5}$ to 10^{-6}, roughly proportional to the average parameter norm growth across games). We further observe in Appendix C.1 that when we vary the endpoint of the learning rate schedule, we often obtain a corresponding x-axis shift in the learning curves, suggesting that reaching a particular learning rate was necessary to master some aspect of the game. We conclude that, while beneficial, the implicit schedule induced by the parameter norm is not necessarily *optimal* for deep RL agents, and it is possible that a more principled adaptive approach could provide still further improvements.

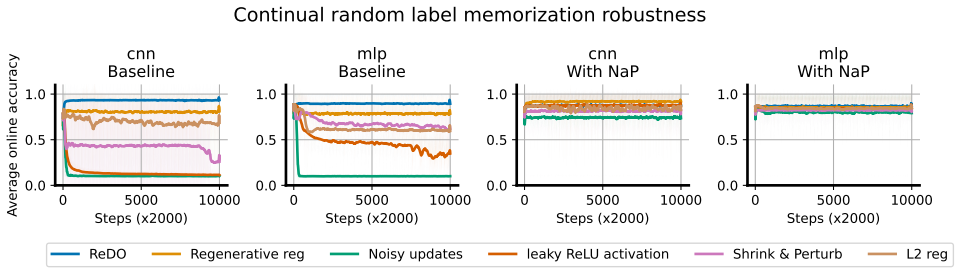


Figure 4: Robustness to nonstationarity: we see that without NaP, there is a wide spread in the effectiveness of various plasticity-preserving methods across two architectures. Once we incorporate NaP, however, the gaps between these methods shrink significantly and almost uniformly improves over the unconstrained baseline.

	CIFAR-10	ImageNet-1k
NaP	**94.64**	77.26
Baseline	**94.65**	77.08
Norm only	94.47	**77.45**

C4	Pile	WikiText	Lambada	SIQA	PIQA
45.7	**47.9**	**45.4**	**56.6**	**44.2**	**68.8**
44.8	47.4	44.2	54.1	43.5	67.3
44.9	47.6	44.3	53.6	43.8	67.1

Table 1: Left: Top-1 prediction accuracy on the test sets of CIFAR-10 and ImageNet-1k. **Right:** per-token accuracy of a 400M transformer model pretrained on the C4 dataset, evaluated on a variety of language benchmarks. See Appendix C.5 for more results with variation measures.

5 Experiments

We now validate the utility of NaP empirically. Our goal in this section is to validate two key properties: first, that NaP does not hurt performance on stationary tasks; second, that NaP can mitigate plasticity loss under a variety of both synthetic and natural nonstationarities.

5.1 Robustness to nonstationarity

We begin with the continual classification problem described in Appendix B.4. We evaluate our approach on a variety of sources of nonstationarity, using two architectures: a small CNN, and a fully-connected MLP (see Appendix B.4. for details). We first evaluate a number of methods designed to maintain plasticity including Regenerative regularization [Kumar et al., 2023], Shrink and Perturb [Ash and Adams, 2020], ReDo [Sokar et al., 2023], along with leaky ReLU units (inspired by the CReLU trick of Abbas et al. [2023]), L2 regularization, and random Gaussian perturbations to the optimizer update, a heuristic form of Langevin Dynamics. We track the average online accuracy over the course of training for 20M steps, equivalent to 200 data relabelings, using a constant learning rate. We find varying degrees of efficacy in these approaches in the base network architectures, with regenerative regularization and ReDO tending to perform the best. When we apply the same suite of methods to networks with NaP, in Figure 4, we observe near-monotonic improvements (with the exception of ReDO, which we conjecture is because the reset method designed for unnormalized networks) in performance and a significant reduction in the gaps between methods, with the performance curves of the different methods nearly indistinguishable in the MLP. Further, we observe constant or increasing slopes in the online accuracy, suggesting that the difference between methods has more to do with their effect on within-task performance than on plasticity loss once the parameter and layer norms have been constrained.

5.2 Stationary supervised benchmarks

Having observed remarkable improvements in synthetic tasks, we now confirm that NaP does not interfere with learning on more widely-studied, natural datasets.

Large-scale image classification. We begin by studying the effect of NaP on two well-established benchmarks: a VGG16-like network [Simonyan and Zisserman, 2014] on CIFAR-10, and a ResNet-50 [He et al., 2016] on the ImageNet-1k dataset. We provide full details in Appendix B.4. In Table 1 we obtain comparable performance in both cases using the same learning rate schedule as the baseline.

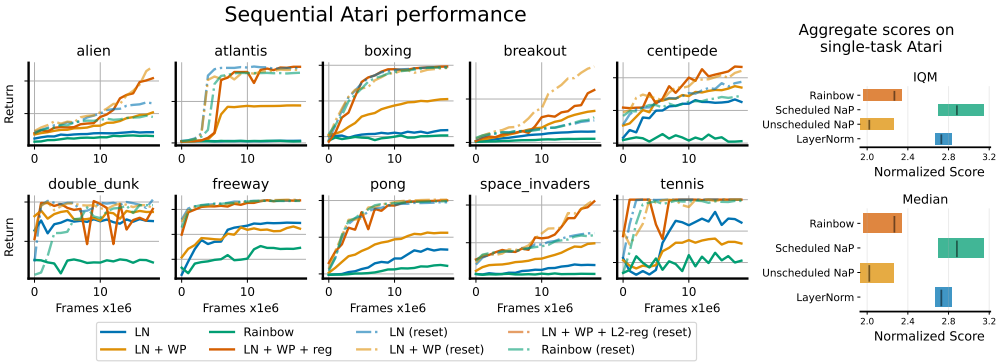


Figure 5: Left: We visualize the learning curves of continual atari agents on sequential ALE training (i.e. 200M frames). Each game is played for 20M frames, and agents pass sequentially from one to another, repeating all ten games twice for a total of 400M training frames. Solid lines indicate performance on the second visit to each game, and dotted lines indicate performance of a randomly initialized network on the game. Even in its second visit to each game, NaP performs comparably the randomly initialized networks, whereas the standard rainbow agent exhibits poor performance on all games in the sequential training regime. **Right:** aggregate effects of normalization on single-task atari, computed via the approach of Agarwal et al. [2021]. Bars indicate 95% confidence intervals over 4 seeds and 57 environments.

Natural language: we now turn our attention to language tasks, training a 400M-parameter transformer architecture (details in Appendix B.3) on the C4 benchmark [Raffel et al., 2020]. Table 1 shows that our approach does not interfere with performance on this task, where we match final performance in terms of training accuracy. When evaluating the pre-trained network on a variety of other datasets, we find that NaP slightly outperforms baselines in terms of performance on a variety of benchmarks, including WikiText-103, Lambada [Paperno et al., 2016], piqa [Bisk et al., 2020], SocialIQA [Sap et al., 2019], and Pile [Gao et al., 2020].

5.3 Deep reinforcement learning

Finally, we evaluate our approach on a setting where maintaining plasticity is critical to performance: RL on the Arcade Learning Environment. We conduct a full sweep over 57 Atari 2600 games comparing the effects of normalization, weight projection, and learning rate schedules on a Rainbow agent [Hessel et al., 2018]. In the RHS of Figure 5 we plot the spread of scores, along with estimates of the Mean and IQM of four agents: standard Rainbow, Rainbow + LayerNorm, Rainbow + NaP without an explicit LR schedule, and Rainbow + NaP with the LR schedule described in Section 4.2. We find that NaP with a linear schedule outperforms the other methods.

We also consider the sequential setting of Abbas et al. [2023]. In this case, we consider an idealized setup where we reset the optimizer state and schedule every time the environment changes, using a cosine schedule with warmup described in Appendix B.2. To evaluate NaP on this regime, we train on each of 10 games for 20M frames, going through this cycle twice. We do not reset parameters of the continual agents between games, but do reset the optimizer. We plot learning curves for the second round of games in the LHS of Figure 5, finding that NaP significantly outperforms a baseline Rainbow agent with and without layer normalization. Indeed, even after 200M steps the networks trained with NaP make similar learning progress to a random initialization.

6 Discussion

This paper has shown that loss of plasticity can be substantially mitigated by constraining the norms of the network's layers and parameters. This finding was made possible by two key insights: first, that in addition to the obvious benefits relating to maintaining constant (pre-)activation norms, layer normalization can also protect against saturated units, and second, that it introduces a correspondence between the parameter norm and the effective learning rate which has significant consequences on performance. We proposed NaP as a means of making the effective learning rate explicit in the optimization process, and leveraged this to gain new insights into the importance of learning rate decay schedules in deep reinforcement learning. In particular, we showed that Rainbow agents trained

on the Arcade Learning Environment in fact under-perform their unconstrained counterparts when a constant effective learning rate is enforced. Provided a suitable decay schedule is used, however, a network's performance and robustness to nonstationarity can both be improved by using NaP. Our analysis suggests a number of exciting directions for future work; in particular, the use of adaptive learning rate schedules in reinforcement learning, and the possibility of tuning not only the global learning rate but also per-layer learning rates in networks which use normalization layers.

Broader Impact

This work concerns basic properties of optimization of neural networks. We do not anticipate any broader societal impacts as a direct consequence of our findings.

References

Zaheer Abbas, Rosie Zhao, Joseph Modayil, Adam White, and Marlos C Machado. Loss of plasticity in continual deep reinforcement learning. *ICML*, 2023.

Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron C Courville, and Marc Bellemare. Deep reinforcement learning at the edge of the statistical precipice. *Advances in neural information processing systems*, 34:29304–29320, 2021.

Atish Agarwala, Fabian Pedregosa, and Jeffrey Pennington. Second-order regression models exhibit progressive sharpening to the edge of stability. *arXiv preprint arXiv:2210.04860*, 2022.

Sanjeev Arora, Zhiyuan Li, and Kaifeng Lyu. Theoretical analysis of auto rate-tuning by batch normalization. In *International Conference on Learning Representations*, 2018.

Devansh Arpit, Yingbo Zhou, Bhargava Kota, and Venu Govindaraju. Normalization propagation: A parametric technique for removing internal covariate shift in deep networks. In Maria Florina Balcan and Kilian Q. Weinberger, editors, *Proceedings of The 33rd International Conference on Machine Learning*, volume 48 of *Proceedings of Machine Learning Research*, pages 1168–1176, New York, New York, USA, 20–22 Jun 2016. PMLR.

Jordan Ash and Ryan P Adams. On warm-starting neural network training. *Advances in Neural Information Processing Systems*, 33:3884–3894, 2020.

Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.

David Balduzzi, Marcus Frean, Lennox Leary, JP Lewis, Kurt Wan-Duo Ma, and Brian McWilliams. The shattered gradients problem: If resnets are the answer, then what is the question? In *International Conference on Machine Learning*, pages 342–350. PMLR, 2017.

Philip J. Ball, Laura Smith, Ilya Kostrikov, and Sergey Levine. Efficient online reinforcement learning with offline data. In Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett, editors, *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 1577–1594. PMLR, 23–29 Jul 2023.

David GT Barrett and Benoit Dherin. Implicit gradient regularization. *arXiv preprint arXiv:2009.11162*, 2020.

Marc G Bellemare, Yavar Naddaf, Joel Veness, and Michael Bowling. The arcade learning environment: An evaluation platform for general agents. *Journal of Artificial Intelligence Research*, 47: 253–279, 2013.

Tudor Berariu, Wojciech Czarnecki, Soham De, Jorg Bornschein, Samuel Smith, Razvan Pascanu, and Claudia Clopath. A study on the plasticity of neural networks. *arXiv preprint arXiv:2106.00042*, 2021.

Yonatan Bisk, Rowan Zellers, Jianfeng Gao, Yejin Choi, et al. Piqa: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI conference on artificial intelligence*, volume 34, pages 7432–7439, 2020.

A Derivations

A.1 Notation

Analysis of a network's training dynamics depends on characterizing the evolution of (pre-)activations and gradients in the forward and backward passes respectively. We lay out basic notation for fully-connected layers first, and then note additional details which must be considered for convolutional, skip connection, and attention layers. We will write the parameters of a layer as θ_l, and use f to denote a neural network.

Fully-connected layers: We write the forward pass through a network $f : \mathbb{R}^{d_0} \to \mathbb{R}^{d_L}$ as a composition of layer-wise computations $f^l : \mathbb{R}^{d_{l-1}} \to \mathbb{R}^{d_l}$ of the form:

$$a^l = f^l(a^{l-1}) = \phi(\sigma^l f_{\text{LN}}(h^l) + \mu^l), \quad h^l = W^l a^{l-1} \; W^l = \theta_l \tag{2}$$

where ϕ denotes a nonlinearity and f_{LN} is a (possibly absent) normalization operator. We let a^0 denote the network inputs. We will refer to a forward pass through a subset of a network with the notation $f^{l_1:l_2} = f^{l_2} \circ \cdots \circ f^{l_1}$, and use interchangeably $f = f^{1:L}$. We will refer to the full set of network parameters by $\theta = \text{vec}(\{\theta^l\}_{l=1}^L)$.

Convolutional layers: since a convolution can be viewed as a parameterization of a matrix with a particular symmetry, we express these layers identically to the fully-connected layers, with a change in semantics such that W^l is the matrix representation of the convolutional parameters θ_l, where we write the embedding of θ_l into a matrix as $W_{\text{conv}}(\theta_l)$. For simplicity, we ignore the choice of padding.

$$\phi(\sigma^l f_{\text{LN}}(h^l) + \mu^l), \quad h^l = W^l a^{l-1} \text{ and } W^l = W_{\text{conv}}(\theta_l) \tag{3}$$

Skip-connect layers: in some network architectures, a nonlinearity is placed on the outputs of two subnetworks to produce a function of the form

$$f^l = \phi\left(\sum_{l_i \in L} a_{l_i}\right) \quad \text{or} \quad f^l = \phi\left(f_{\text{LN}}\left(\sum_{l_i \in L} a_{l_i}\right)\right) \tag{4}$$

for some index set L. The choice of whether to apply normalization to the sum of the subnetwork outputs or to each output individually depends on the desired signal propagation properties of the network [De and Smith, 2020].

A.2 Derivation of Proposition 1

The result described in proposition 1 follows straightforwardly from the chain rule. For pre-activation RMSNorm we have

$$\frac{d}{dh_i}\phi(f_{\text{RMS}}(h))_j = \phi'(f_{\text{RMS}}(h))_j \frac{d}{dh_i} f_{\text{RMS}}(h)_j \tag{5}$$

$$\frac{d}{dh_i} f_{\text{RMS}}(h)_j = \frac{d}{dh_i} \frac{h_j}{\left(\sum h_k^2\right)^{1/2}} \tag{6}$$

$$= -\frac{1}{2} \frac{h_j}{\left(\sum h_k^2\right)^{3/2}} \frac{d}{dh_i} \sum h_k^2 \tag{7}$$

$$= -\frac{h_j}{\left(\sum h_k^2\right)^{3/2}} h_i \tag{8}$$

$$\frac{d}{dh_i}\phi(f_{\text{RMS}}(h))_j = -\frac{\phi'(f_{\text{RMS}}(h))_j h_j h_i}{\|h\|^3} \tag{9}$$

A.3 Gradients and signal propagation

One perspective which can provide some additional insight into our approach is to consider the following decomposition of the gradient being backpropagated through a layer.

$$\nabla_{W_k}\mathcal{L}(\theta) = \partial_{a_k}\mathcal{L}(\theta) \cdot \partial_{W_k}\phi(f_{\text{LN}}(W_k a_{k-1})) \tag{10}$$

$$= \partial_{a_k}\mathcal{L}(\theta) \cdot D_{\dot{\phi}} \nabla_{h_k} f_{\text{LN}}(h_k)) a_{k-1}^{\top} \tag{11}$$

With this decomposition, we obtain the following interpretation of some common pathologies.

Saturated nonlinearities: a saturated nonlinearity implies that $D_{\dot{\phi}}$, and the gradient is exactly (in the case of ReLU) or very close to (e.g. tanh) zero. As a result, u_t will be zero for the affected coordinates and the corresponding parameters will remain frozen. NaP addresses this problem by using Layer or RMSNorm prior to any nonlinearity in the network.

Saturated normalization layers: the normalization transformation $x \mapsto \frac{x}{\|x\|}$ is vulnerable to saturation as $\|x\|$ grows, in the sense that for a fixed-norm update u we will have

$$\lim_{\|x\|\to\infty} \frac{x+u}{\|x+u\|} - \frac{x}{\|x\|} = 0 \tag{12}$$

and so the term $\nabla_{h_k} f_{\text{LN}}(h_k))$ will vanish. In this case, while the optimizer will be able to update the parameters, these updates will have a diminishing effect on the network's output.

Vanishing and exploding gradients: divergence and disappearance of the activations a_{k-1} and backpropagated gradients $\partial_{a_k}\mathcal{L}(\theta)$ are well-known pathologies which can make networks untrainable. However, even networks which start training from a well-tuned initialization may still encounter exploding gradients due to parameter norm growth over time [Dohare et al., 2021, Wortsman et al., 2023], or vanishing gradients and activations, such as in the case of saturated (i.e. dead/dormant) ReLU units [Sokar et al., 2023].

A.4 Details of NaP

Guiding principles: in general, the goal of NaP is to avoid dramatic distribution shifts in the pre-activation and parameter norms, and to ensure that the network can perform updates to parameters even if a nonlinearity is saturated. With these in mind, there are two key properties that a network designer should aim to maintain:

1. All **parametric** functions entering a nonlinearity should have a normalization layer that at ensures the gradients of all units' parameters are correlated. If there are no parameters between nonlinearities (as is sometimes the case in e.g. resnets) normalization is not essential.
2. Based on our investigations in Appendix C.4, *L2 normalization* of the pre-activations is crucial to obtain the positive benefits of layer normalization, while centering does not have noticeable effects on the network's robustness to unit saturation. As a result, applying at least RMSNorm is crucial prior to nonlinearities, but the choice of whether or not to incorporate centering is up to the designer's discretion.

Batch normalization layers: by default, we put layer normalization prior to batch normalization if an architecture already incorporates batch normalization prior to a nonlinearity. This preserves the property of batch norm that individual units have mean zero across the batch, which may not be the case if layer normalization is applied after. We also always omit offset parameters if layernorm is succeeded by batchnorm, as these offset parameters will be zeroed out by batchnorm.

Skip-connect layers: provided that layer normalization is applied to the outputs of a linear transformation prior to a nonlinearity, NaP is agnostic to whether normalization is applied prior to or after a residual connection's outputs are added to the output of a layer. In particular, if we have a layer of the form $\phi(a_1 + a_2)$ and a_1 and a_2 are the outputs of some subnetwork of the form $\phi_1(f_{\text{LN}}(h_1))$ and $\phi_2(f_{\text{LN}}(h_2))$ where ϕ_1 and ϕ_2 are (possibly trivial) activation functions, then the relevant parameters will already benefit from Proposition 1 and it is not necessary to add an additional normalization layer prior to the activation ϕ.

Attention layers: unlike linear layers, attention layers do not typically include bias terms. To analogize this common practice in NaP, we omit the offset and mean subtraction components of the LayerNorm transform, obtaining

$$\mathrm{MHA}(W_Q X, W_K X) \mapsto \mathrm{MHA}(\sigma_Q f_{\mathrm{RMS}}(W_Q X), \sigma_K f_{\mathrm{RMS}}(W_K X)) \tag{13}$$

where crucially f_{RMS} is not applied along the token axis as this can lead to leakage of information during training on next-token-prediction objectives. A similar problem also prevents us from using normalization directly on the QK product matrix, along with the observation that the intuition of normalizing vectors so that their dot product is equal to the cosine similarity is lost once the dot products have already occurred [Henry et al., 2020]. Empirically, we find that the scale parameters σ_Q and σ_K don't seem to be strictly necessary for expressivity, and that networks can even form selective attention masks for in-context learning without using these parameters to further saturate the softmax.

A.5 Dynamics of NaP

Weight projection non-interference: NaP incorporates a projection onto the ball of constant norm after each update step. A natural question is whether this projection step might simply be the inverse of the update step, leaving the parameters of the network constant. Fortunately, we note that the normalization layers have the effect of projecting gradients onto a subspace which is orthogonal to the current parameter values, i.e.

$$\left(\nabla_{\mathbf{x}} f_{\mathrm{RMS}}(\mathbf{x})\right)(\mathbf{x}) = \mathbf{0}\,. \tag{14}$$

We also note that except for extreme situations such as Neural Collapse [Papyan et al., 2020], real-world gradient updates are almost never colinear with the parameters, meaning that even without normalization layers this problem would be unlikely.

Another concern that arises from the constraints we place on the weights and features is the possibility that these constraints will limit the network's expressivity. Normalization does remove the ability to distinguish colinear inputs of differing norms, meaning that the inputs $\mathbf{x}$ and $\alpha\mathbf{x}$ will map to the same output for all α; however, since many data preprocessing pipelines already normalize inputs, we argue this is not a significant limitation. Indeed, under a more widely-used notion of expressivity, the number of *activation patterns* [Raghu et al., 2017], NaP does not limit expressivity at all. While straightforward, we provide a formal statement and proof of this claim in Appendix A.7.

Layer normalization and parameter growth: In fact, if we incorporate normalization layers into the network we might expect an even more aggressive decay schedule. Recall that in a scale invariant network, we have $\langle \nabla_\theta f(\theta), \theta \rangle = 0$. Thus we know that the gradient at each time step will be orthogonal to the current parameters. In an idealized setting where we use the update rule $\theta_{t+1} \leftarrow \theta_t + \alpha \frac{\nabla_\theta \ell(\theta_t)}{\|\nabla_\theta \ell(\theta_t)\|}$, this would result in the parameter norm growing at a rate $\Theta(t)$, corresponding to an effectively linear learning rate decay.

A.6 Scale-invariance and layer-wise gradient norms

One benefit of NaP is that, because we normalize layer outputs, we limit the extent to which divergence in the norm of one layer's parameters can propagate to the gradients of other layers. For e.g. linear homogeneous activations such as ReLUs, the gradient of some objective function for some input with respect to the parameters of a particular layer contains a sum of matrix products whose norm will depend multilinearly on the norm of each matrix. In particular, in the simplified setting of a deep linear network where $f(\theta, \mathbf{x}) = \prod W^l \mathbf{x}$, we recall Saxe et al. [2013]

$$\nabla_{W^l} f(\theta; \mathbf{x}) = \left[\prod_{k>l} W^k\right]^\top \mathbf{x}^\top \left[\prod_{k<l} W^k\right]^\top \tag{15}$$

In particular, with $\theta' = W^1, \ldots, cW^k, \ldots, W^L$, for $k \neq l$ we would have

$$\implies \nabla_{W^l} f(\theta'; \mathbf{x}) = c \nabla_{W^l} f(\theta; \mathbf{x}) \tag{16}$$

The situation changes little if we add ReLU nonlinearities to the network. In this case, we use the notation $\mathbf{D}_{\phi_l}(\mathbf{x})$ to denote the diagonal matrix indicating whether $a^l[i] > 0$

$$\nabla_{W^l} f(\theta; \mathbf{x}) = \left[\prod_{k>l} \mathbf{D}_{\phi_k}(\mathbf{x}) W^k\right]^\top \mathbf{x}^\top \left[\prod_{k<l} \mathbf{D}_{\phi_k}(\mathbf{x}) W^k\right]^\top \tag{17}$$

$$\implies \nabla_{W^l} f(\theta'; \mathbf{x}) = c \nabla_{W^l} f(\theta; \mathbf{x}) \tag{18}$$

If we incorporate a normalization layer at the end of the network (for simplicity we consider RMSNorm here, but a similar argument applies to standard LayerNorm), the scale-invariance of the resulting output means that the norms of each layer's gradients are independent of the norms of the other layers' parameters, i.e.

$$\nabla_{W^l} f_{\text{RMS}} \circ f(\theta; \mathbf{x}) = \nabla_{W^l} f_{\text{RMS}} \circ f(\theta'; \mathbf{x}) \;\text{ whenever } \theta' = (W_1, \ldots, cW_k, \ldots, W_L), k \neq l \tag{19}$$

This property is appealing as it means that growth or decay of the norm of a single layer will not interfere with the dynamics of the others. However, it does mean that a layer's effective learning rate will still be sensitive to scaling, which motivates our use of renormalization. It also does not help to avoid saturated units, motivating our use of layer normalization prior to nonlinearities.

A.7 Expressivity of NaP

Finally, we discuss the effect of normalization and weight projection on a notion of expressivity known as the number of activation patterns [Raghu et al., 2017] exhibited by a neural network. This quantity relates to the complexity of the function class a network can compute, giving the following result the corollary that NaP doesn't interfere with this notion of expressivity.

Proposition 2. *Let f be a fully-connected network with ReLU nonlinearities. Let $\tilde{f}$ be the function computed by f after applying NaP. Then the activation pattern of a particular architecture f and parameter θ be $\mathcal{A}_\theta$, we have*

$$\mathcal{A}_f(\theta, \mathbf{x}) = \mathcal{A}_{\tilde{f}}(N(\theta), \mathbf{x}) \,. \tag{20}$$

Further, the decision boundary $\max_{i \in d_{out}} f_i(\mathbf{x})$ is preserved under NaP.

Proof. We apply an inductive argument on each layer. In particular, when ϕ is a ReLU nonlinearity we have

$$\begin{aligned}
\mathcal{A}(\phi(\mathbf{h})) &= \mathcal{A}(\phi(f_{\text{RMS}}(\mathbf{h}))) \\
\phi(f_{\text{RMS}}(\mathbf{h})) &= \frac{\phi(\mathbf{h})}{\|\mathbf{h}\|} \\
\tilde{f}(\mathbf{x}) &= \frac{1}{\Pi_{l=1}^L \|\mathbf{h}_l(\mathbf{x})\|} f(\mathbf{x})
\end{aligned}$$

which trivially results in identical activation patterns in the normalized and unnormalized networks. It is worth noting that one distinguishing factor from a standard ReLU network is that the resulting scaling factor will be different for each $\mathbf{x}$. Thus while the activation patterns will be the same, the two different inputs $\mathbf{x}$ $\mathbf{y}$ might have different scaling factors, which will be a nonlinear function of the input. NaP networks, even with ReLU activations, thus do not have the property of being piecewise linear. □

A.8 Rescaling scale/offset parameters (linear homogeneous networks)

We observe that for any $c > 0$, letting $\phi(x) = \max(x, 0)$ we have:

$$\begin{aligned}
f_{\text{LN}}(\phi(W\sigma\mathbf{x} + \mu)) &= f_{\text{LN}}(c\phi(W\sigma\mathbf{x} + \mu)) && (21)\\
&= f_{\text{LN}}(\phi(cW(\sigma\mathbf{x} + \mu))) \text{ by homogeneity of ReLU} && (22)\\
&= f_{\text{LN}}(\phi(W(c\sigma\mathbf{x} + c\mu))) && (23)
\end{aligned}$$

Then we obtain analogous effective learning rates, letting

$$g_{c\mu} = \nabla_\mu f(c\sigma, c\mu; \mathbf{x}) \quad g_{c\sigma} = \nabla_\sigma f(c\sigma, c\mu; \mathbf{x}) \tag{24}$$

we then have equivalent updates for $\|\sigma^2 + \mu^2\| = 1$ and $\eta_c = c^2$

$$f_{\text{LN}}((c\sigma + \eta_c g_{c\sigma})\mathbf{x} + c\mu + \eta_c g_{c\mu}) = f_{\text{LN}}((c\sigma + c^2 g_{c\sigma})\mathbf{x} + c\mu + c^2 g_{c\mu}) \tag{25}$$

$$= f_{\text{LN}}((c\sigma + c^2 \frac{1}{c} g_\sigma + c\mu + c^2 \frac{1}{c} g_\mu) \tag{26}$$

$$= f_{\text{LN}}(c((\sigma + g_\sigma)\mathbf{x} + \mu + g_\mu) = f_{\text{LN}}((\sigma + g_\sigma)\mathbf{x} + \mu + g_\mu) \tag{27}$$

Finally, we note that the above also holds if we apply a linear transformation W to the output of the scale-offset transform, since

$$f_{\text{LN}}(W(c\sigma\mathbf{x} + c\mu)) = f_{\text{LN}}(cW(\sigma\mathbf{x} + \mu)) = f_{\text{LN}}(W(\sigma\mathbf{x} + \mu)) \tag{28}$$

and so the effective learning rate scales precisely as we had previously for linear layers, but now with respect to the joint norm of the scale and offset parameters μ, σ.

B Experiment details

B.1 Toy experiment details

We conduct a variety of illustrative experiments on toy problem settings and small networks.

Network: in Figures 1 and 2, we use a DQN-style network which consists of two sets of two convolutional layers with 32 and 64 channels respectively. We then apply max pooling and flatten the output, feeding through a 512-unit hidden linear layer before applying a final linear transform to obtain the output logits. The network uses ReLU nonlinearities. When NaP is applied, we add layer normalization prior to each nonlinearity.

B.2 RL details

Single-task atari: We base our RL experiments off of the publicly available implementation of the Rainbow agent [Hessel et al., 2018] in DQN Zoo [Quan and Ostrovski, 2020]. We follow the default hyperparameters detailed in this codebase. In our implementation, we add normalization layers prior to each nonlinearity except for the final softmax. We train for 200M frames on the Atari 57 suite [Bellemare et al., 2013]. We also allow for a learning rate schedule, which we explicitly detail in cases where non-constant learning rates are used.

Sequential ALE: we use the same rainbow implementation as for the single-task results, using a cosine decay learning rate for all variants. We restart the cosine decay schedule at every task change for all agents. Our cosine decay schedule uses an init value of 10^{-8}, a peak value of the default LR for Rainbow (0.000625), 1000 warmup steps after the optimizer is reset, and end-value equal to 10^{-6} as in the single-task settings. We choose cosine decay due to its popularity in supervised learning, and to highlight the versatility of NaP to different LR schedules. We follow the game sequence used by Abbas et al. [2023], training for 20M frames per game.

B.3 Language details

Sequence memorization: we set a dataset size of 1024 and a sequence length of 512. We use a vocabulary size of 256, equivalent to ASCII tokenization. We use the adam optimizer, and train all networks for a minimum of 10 000 steps. We reset the dataset every 1000 optimizer steps, generating a new set of 1024 random strings of length 512. We use as a baseline a transformer architecture [Vaswani et al., 2017, Raffel et al., 2020] consisting of 4 attention blocks, with 8 heads and d_{model} equal to 256. We use a batch size of 128.

In-context learning: our in-context learning experiments use the same overall setup as the sequence memorization experiments, with identical architectures and baseline optimization algorithm. In this

case we train on a dataset consisting of 4096 randomly generated strings, in which the final 100 tokens are a contiguous subsequence of the first 412, selected uniformly at random from indices in [1, 312].

Natural language: we run our natural language experiments on a 400M parameter transformer architecture based on the same backbone as the previous two tasks, this time consisting of 12 blocks with 12 heads and model dimension 1536. We use the standard practice of learning rate warmup followed by cosine decay, setting a peak learning rate of $2 \cdot 10^{-4}$ which is reached after a linear warmup of 1000 steps. We use a batch size of 128 and train for 30,000 steps. We use a weight decay parameter of 0.1 with the adamw optimizer.

B.4 Vision details

CIFAR-10 Memorization: We consider three classes of network architecture: a fully-connected multilayer perceptron (MLP), for which we default to a width of 512 and depth of 4 in our evaluations; a convolutional network with k convolutional layers followed by two fully connected layers, for which we default to depth four, 32 channels, and fully-connected hidden layer width of 256. In all networks, we apply layer normalization before batch normalization if both are used at the same time. By default, we typically do not use batch normalization. We use ReLU activations

Our continual supervised learning domain is constructed from the CIFAR-10 dataset, from which we construct a family of continual classification problems. Each continual classification problem is characterized by a transformation function on the labels. For label transformations, we permute classes (for example, all images with the label 5 will be re-assigned the label 2), and random label assignment, where each input is uniformly at random assigned a new label independent of its class in the underlying classification dataset. Figures in the main paper concern random label assignments, as this is a more challenging task which produces more pronounced effects.

In our figures in the main paper, we run a total of 20M steps and a total of 200 random target resets. All networks are trained using the Adam optimizer. We conducted a sweep over learning rates for the different architectures, settling on 10^{-4} as this provided a reasonable balance between convergence speed and stability in all architectures, to ensure that all networks could at least solve the single-task version of each label and target transformation.

VGGNet and ResNet-50 baselines: we use the standard data augmentation policies for the CIFAR-10 and ImageNet-1k experiments. In our ImageNet-1k experiments, we use the ResNetv2 architecture variant, with a label smoothing parameter of 0.1, weight decay 10^{-4}, and as an optimizer we use SGD with a cosine annealing learning rate schedule, and Nesterov momentum with decay rate 0.9. We use a batch size of 256. Our CIFAR-10 experiments use a VGG-Net architecture. We use the sgd optimizer with a batch size of 32, and set a learning rate schedule which starts at 0.025 and decays by a factor of 0.1 iteratively through training. We use Nesterov momentum with decay rate 0.9. We train for a total of 400 000 steps.

C Additional experimental results

C.1 Arcade learning environment

Our choice of learning rate schedule in the paper is motivated by an attempt to roughly approximate the shape of the implicit schedule obtained by parameter growth on average across games in the suite, with a slightly smaller terminal point than would typically be achieved by the parameter norm alone. We consider learning rate schedules which linearly interpolate between the initial learning rate of 0.000625 and a learning rate of 10^{-6}, which is roughly equivalent to increasing the parameter norm by a factor of 60. We explore the importance of the duration of this decay in Figure 6, where we conduct a sweep over a subset of the full Atari 57 suite (selected by sorting alphabetically and selecting every third environment). We observe that faster decay schedules result in better performance initially, but often plateau at a lower value. Decaying linearly over the entire course of training, in contrast, exhibits slow initial progress but often picks up significantly towards the end of training. We conclude that in many games, reducing the learning rate is necessary for performance improvements in this agent, and the linear decay over the entire training period doesn't give the agent sufficien time to take advantage of the finer-grained updates to its predictions that a lower learning rate affords.

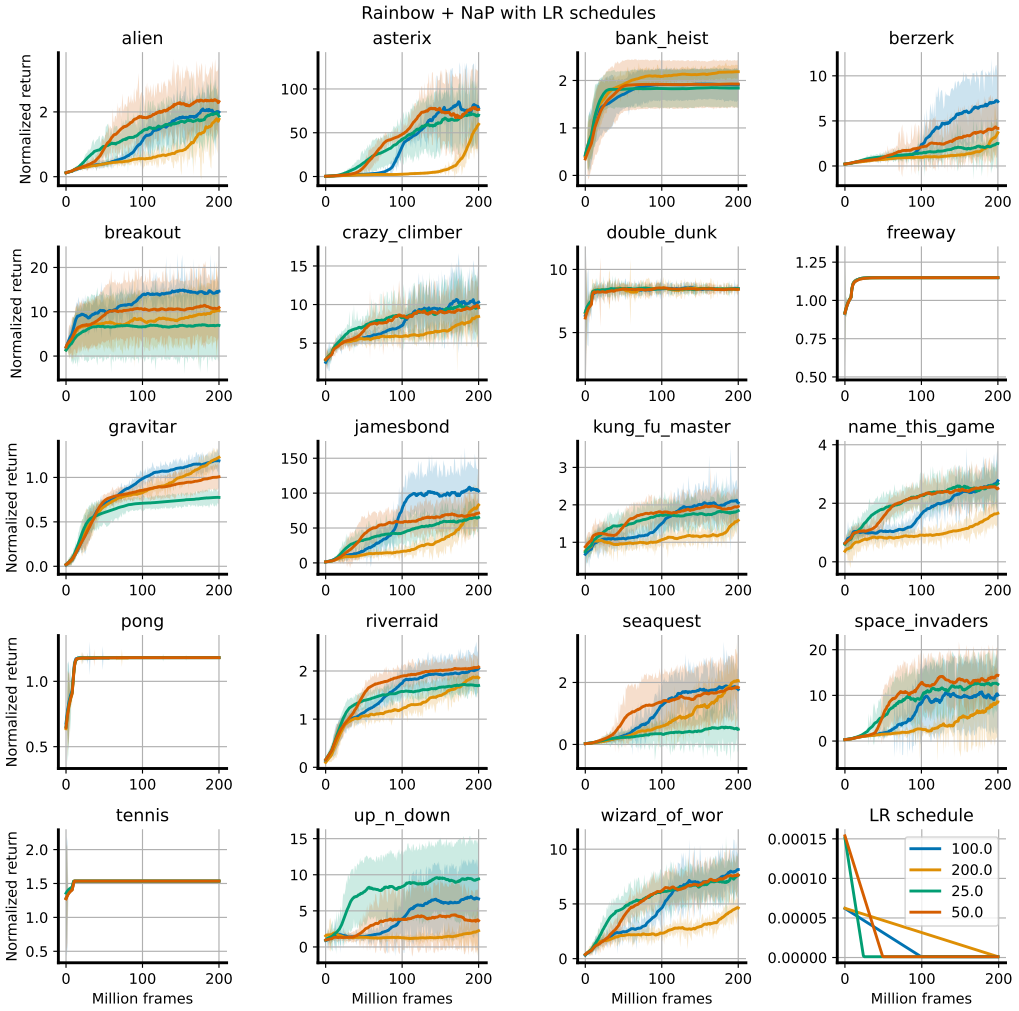


Figure 6: We see that faster LR decays typically accompany fast initial progress followed by plateaus. Terminating the linear schedule halfway through training strikes the best balance of the four settings considered for overall progress.

C.2 Non-stationary MNIST

We include additional network statistics from the experiments shown in Figure 4 in Figure 7.

Linearized units: given a large batch, what fraction of the ReLU units in the penultimate layer are either 0 for all units or nonzero for all units. This gives a slightly more nuanced take on the amount of computation performed in the penultimate layer than the feature rank.

Feature rank: we compute the numerical rank of the penultimate layer features by sampling a batch of data and computing the singular values of the $b \times d$ matrix of $d-$dimensional feature vectors. $\sigma_1, \ldots, \sigma_d$. We then compute the numerical rank as $\sum \mathbb{1}(\sigma_i/\sigma_1 > 0.01)$.

Parameter and gradient norm: these are both computed in the standard way by flattening out the set of parameters / per-parameter gradients and computing the 2-norm of this vector.

C.3 Non-stationary sequence modeling

We find additionally that NaP is capable of improving the robustness of sequence models to nonstationarities, while also not interfering with the formation of in-context learning circuits. We demonstrate

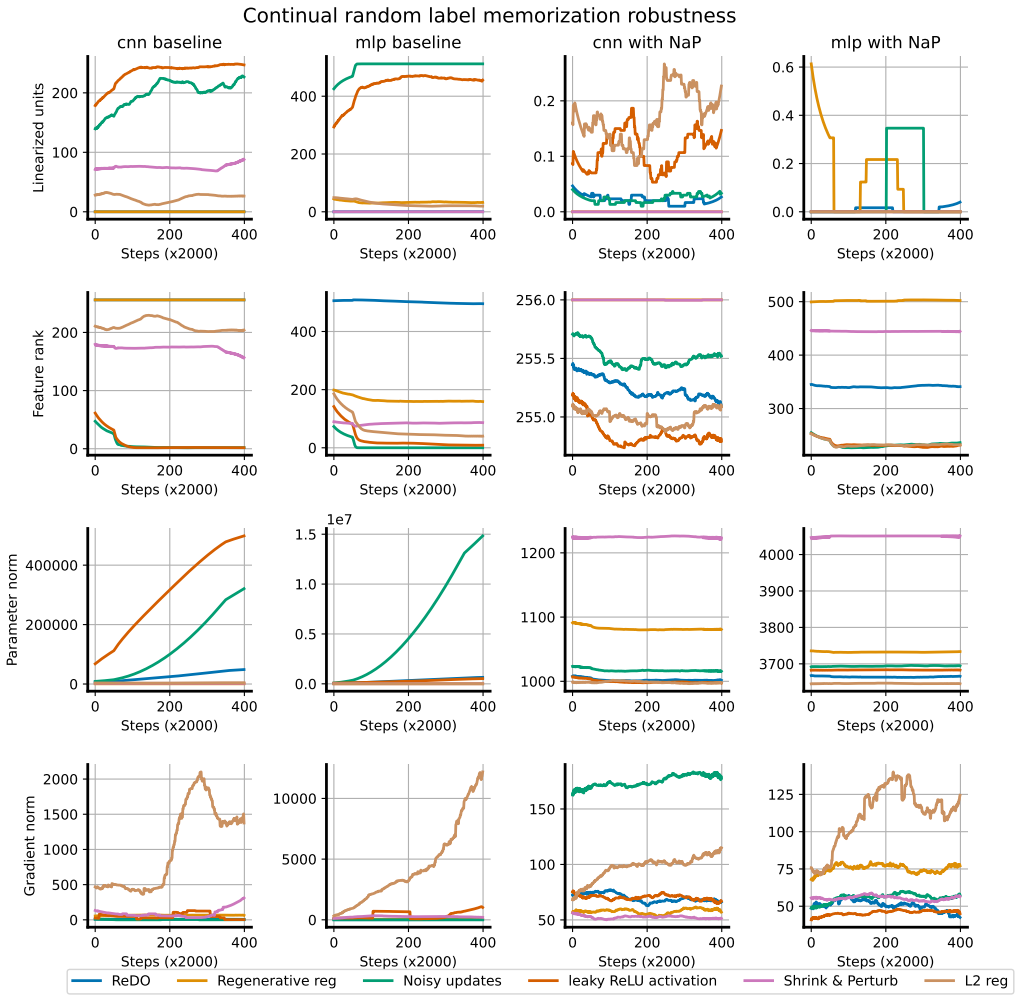


Figure 7: We plot a variety of additional network statistics in the continual CIFAR-10 experiment shown in Figure 4.

the latter point in Figure 8, where we train a small transformer model on a dataset of the form $s_p \oplus s_s$, where s_p is a prefix string of length 100 and s_s is a suffix string of length 50 which is a contiguous subset of the prefix string. We use a fixed dataset, such that in theory the network could memorize all strings, though we use a sufficiently large dataset that this is not possible to achieve within the training budget. We train four protocols using next-token prediction: with and without weight projection, and with and without normalization (the networks are small enough that normalization is not critical for training stability). We evaluate accuracy on the final 48 tokens of s_s as training progresses, and observe that all networks very quickly learn to identify the starting point of the suffix string and copy the relevant subset of the prefix. We observe that normalization accelerates learning of both the retrieval component of the accuracy and the memorization component of the accuracy, and that the weight projection step in fact also accelerates this process.

In Figure 9, we see that normalization and weight projection can also help to improve robustness in a nonstationary random string memorization task, a sequence-modelling analogue of random label memorization in CIFAR-10. We observe that in this case, allowing a learnable scale to evolve unregularized can somewhat slow down learning compared to omitting this parameter, and that weight projection recovers similar dynamics to learning with a large weight decay factor.

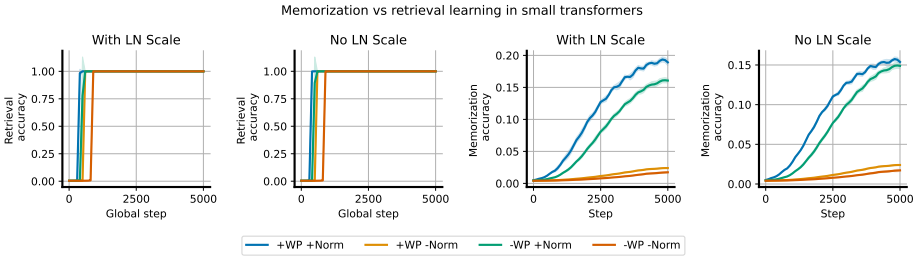


Figure 8: Demonstration that applying normalization and removing the learnable scale parameter does not prevent the network from learning to copy previously-observed subsequences.

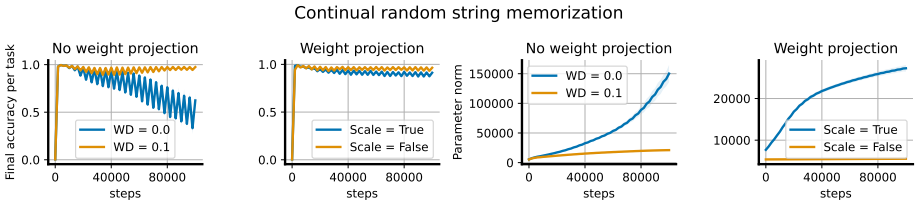


Figure 9: Random string memorization: unregularized networks exhibit plasticity loss when trained to memorize a sequence of random strings, while weight projection and weight decay improve robustness to this nonstationarity.

C.4 ReLU revival experiments

Many previous works have noted that adaptive optimizers are particularly damaging to network plasticity [Dohare et al., 2021, Lyle et al., 2023, Dohare et al., 2023]. The primary mechanism underlying this is due to the sudden distribution shift in gradient moments due to changes in the learning objective – when the gradient norm increases, adaptive optimizers are slow to catch up and can take enormous update steps when this occurs. Layer normalization mitigates this effect due to two facts: first, the gradients of negative preactivations are nonzero, and second, all nonzero gradients are treated essentially the same by adaptive optimizers (up to ϵ). As a result, networks with layer norm can still perform significant updates to parameters feeding into 'dead' units, meaning that these networks have a good chance of turning on again later.

We illustrate this with a simple experimental setting, where we model optimizer updates with isotropic Gaussian-distributed gradient signals and perform a (truncated at zero) random walk. Formally we look at the time evolution of the system:

$$\mathbf{v}_t = \mathbf{v}_{t-1} + \max(\mathbf{v}, \mathbf{0}) \odot \mathbf{z}_t, \quad \mathbf{z}_t \sim \mathcal{N}(0, I) \tag{29}$$

to model the evolution of features under a gradient descent trajectory. To simulate the steps taken by an adaptive optimizer like RMSProp or Adam, where updates to each parameter have fixed norm, we modify this process slightly as follows:

$$\mathbf{v}_t = \mathbf{v}_{t-1} + \text{sign}\left(\max(\mathbf{v}, \mathbf{0}) \odot \mathbf{z}_t\right), \quad \mathbf{z}_t \sim \mathcal{N}(0, I)\ . \tag{30}$$

To simulate layer normalization, we compute the dot product between $\mathbf{z}_t$ and the Jacobian $\nabla_{\mathbf{v}} \frac{\mathbf{v}}{\|\mathbf{v}\|}$ to simulate gradient descent:

$$\mathbf{v}_t = \mathbf{v}_{t-1} + \mathbf{z}_t^\top \nabla_{\mathbf{v}} \max\left(\frac{\mathbf{v}}{\|\mathbf{v}\|}, \mathbf{0}\right), \quad \mathbf{z}_t \sim \mathcal{N}(0, I) \tag{31}$$

and analogously compute the sign of this update to model RMSProp-style optimizers:

$$\mathbf{v}_t = \mathbf{v}_{t-1} + \text{sign}\left(\mathbf{z}_t^\top \nabla_{\mathbf{v}} \max\left(\frac{\mathbf{v}}{\|\mathbf{v}\|}, \mathbf{0}\right)\right), \quad \mathbf{z}_t \sim \mathcal{N}(0, I) \tag{32}$$

In Figure 10 we simulate each of these processes for 1000 steps, and track the number of negative (i.e. 'dead') indices. We observe that layer normalization doesn't avoid dead units in GD, but does reduce

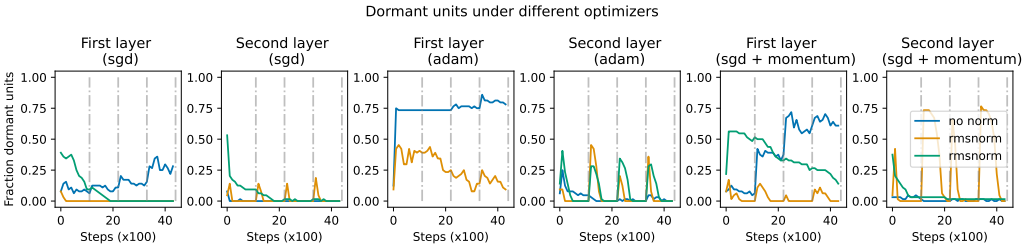


Figure 10: Simple MLP model with dead unit recovery after sudden changes in the classification task. We

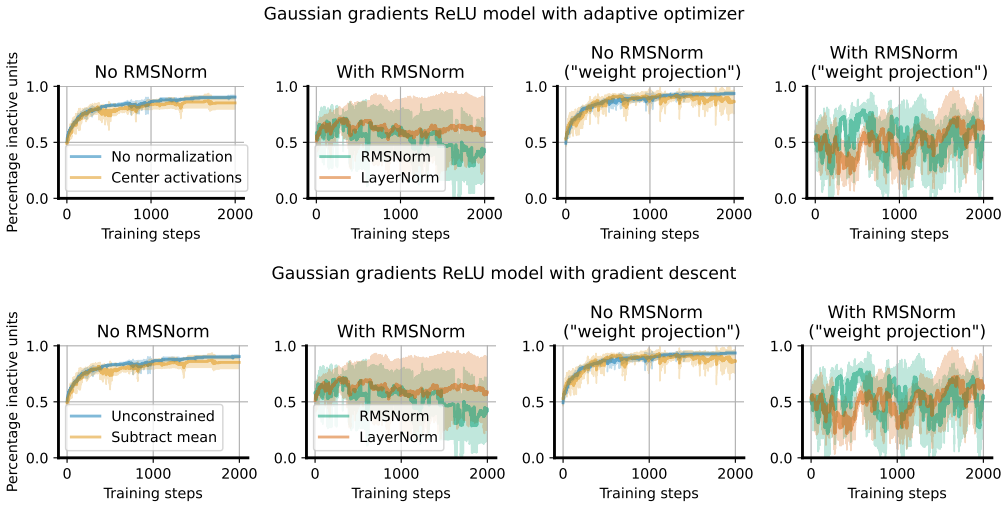


Figure 11: Accumulation of gradients in a random walk model: backpropagated 'gradients' are isotropic random Gaussian vectors and updates are computed by taking the product of these vectors with the layer jacobian. We see that the centering transform actually does relatively little to reduce the risk of dead units, and can in fact produce a 'winner-take-all' effect wherein one large

the rate at which they accumulate. We also observe that layer normalization does avoid monotonic increases in the number of dead units in a model of RMSProp. Intuitively, this makes sense: rather than freezing once they reach a negative value, parameters continue updating once they become negative with equally large steps as they did before, making escape from the dead zone more probable. One visualization of a few trajectories in each setting is shown in Figure 11.

C.5 Stationary supervised benchmarks

We provide the results with standard deviations in Table 2 and Table 3.

	CIFAR-10	**ImageNet-1k**
NaP	**94.64 (0.12)**	77.26 (0.04)
Baseline	**94.65 (0.08)**	77.08 (0.11)
Norm only	94.47 (0.18)	**77.45 (0.08)**

Table 2: Top-1 prediction accuracy on the test sets of CIFAR-10 and ImageNet-1k. Numbers in parentheses are standard deviations.

	C4	Pile	WikiText	Lambada	SIQA	PIQA
NaP	**45.7 (0.0)**	**47.9 (0.1)**	**45.4 (0.1)**	**56.6 (0.4)**	**44.2 (0.2)**	**68.8 (0.7)**
Baseline	44.8 (0.0)	47.4 (0.1)	44.2 (0.0)	54.1 (0.2)	43.5 (0.6)	67.3 (0.2)
Norm only	44.9 (0.0)	47.6 (0.1)	44.3 (0.0)	53.6 (0.3)	43.8 (0.6)	67.1 (0.4)

Table 3: Per-token accuracy of a 400M transformer model pretrained on the C4 dataset, evaluated on a variety of language benchmarks. Numbers in parentheses are standard deviations.

D A note on scale and offset parameters

One loose end from our presentation of NaP is what to do with the learnable scale and offset terms, which are not by default projected and so may drift from their initial values. Most supervised tasks are too short for this drift to present problems, and adding layer-specific regularization or normalization adds additional engineering overhead to an experiment. However, in deep RL or in the synthetic continual tasks we present in Figure 4, this is a real concern. In the case of homogeneous activations such as ReLU, the scale and offset parameters can be viewed identically to the weight and bias terms and normalized accordingly, noting that now all that matters is the relative ratio of the scale and the offset. To account for this, we propose to treat the joint set σ, μ as a single parameter to be normalized. This resolves the issues involved with normalizing a parameter to an initial value of zero, and can be shown not to change the network output (see Appendix A.8 for a derivation of this fact). With non-homogeneous nonlinearities, however, this property will not hold, and we suggest in the general case to use mild weight decay towards a the initial values of 1 and 0 for the scale and offset terms respectively. These two approaches can be summarized in the following two update rules:

$$\mathcal{U}_{\text{norm}}(\sigma, \mu) = \frac{(\sigma, \mu)}{\sqrt{\|\sigma\|^2 + \|\mu\|^2}} \quad \text{and} \quad \mathcal{U}^{\alpha}_{\text{decay}}(\sigma, \mu) = (\alpha\sigma + (1-\alpha)\mathbb{1}, \alpha\mu)\,. \tag{33}$$

Most of our evaluations on single tasks do not use any regularization or projection of the scale and offset parameters, but we do include a regularization-based approach in our evaluations on the sequential ALE in Section 5. In general, if it did not appear that the scale/offset drift was causing problems, we did not introduce additional complexity by adding regularization or normalization. Indeed, in many cases a simpler solution was to omit these parameters entirely; for example we observed that removing offsets was beneficial in several, though not all, games in the Arcade Learning Environment.

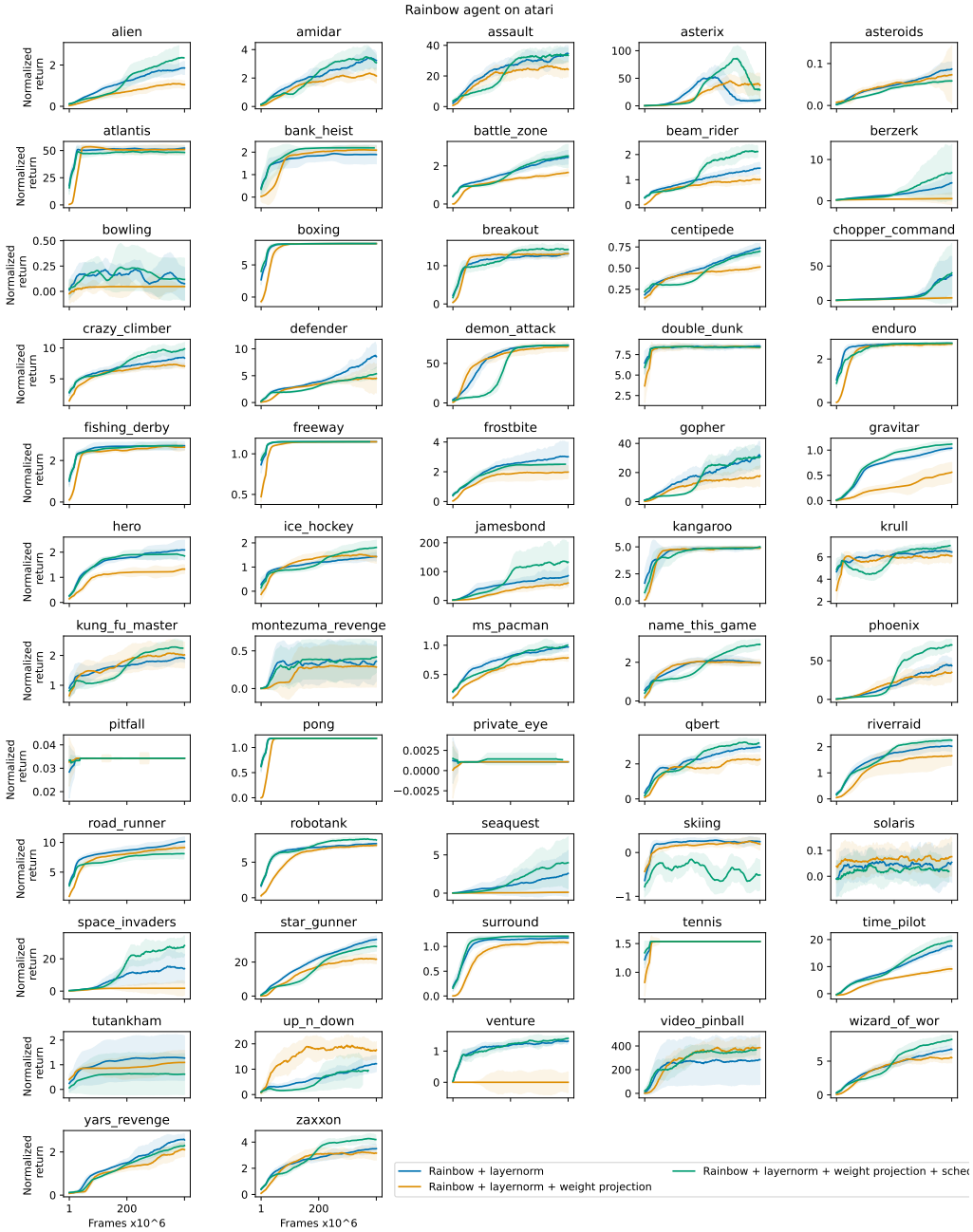


Figure 12: Rainbow agents on atari.

Scaling Off-Policy Reinforcement Learning with Batch and Weight Normalization

Daniel Palenicek [1 2] **Florian Vogt** [3] **Jan Peters** [1 2 4 5]

Abstract

Reinforcement learning has achieved significant milestones, but sample efficiency remains a bottleneck for real-world applications. Recently, CrossQ has demonstrated state-of-the-art sample efficiency with a low update-to-data (UTD) ratio of 1. In this work, we explore CrossQ's scaling behavior with higher UTD ratios. We identify challenges in the training dynamics, which are emphasized by higher UTD ratios. To address these, we integrate weight normalization into the CrossQ framework, a solution that stabilizes training, has been shown to prevent potential loss of plasticity and keeps the effective learning rate constant. Our proposed approach reliably scales with increasing UTD ratios, achieving competitive performance across 25 challenging continuous control tasks on the DeepMind Control Suite and Myosuite benchmarks, notably the complex `dog` and `humanoid` environments. This work eliminates the need for drastic interventions, such as network resets, and offers a simple yet robust pathway for improving sample efficiency and scalability in model-free reinforcement learning.

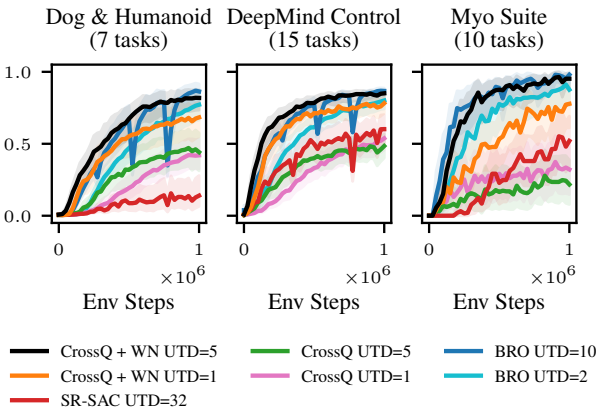


Figure 1. CrossQ + WN UTD=5 is competitive to BRO UTD=10. In comparison, our proposed CrossQ + WN is a simple algorithm that does not require extra exploration policies or full parameter resets. We present results for 25 complex continuous control tasks from the DMC and MyoSuite benchmarking suites. 1.0 marks the maximum score achievable on the respective benchmarks (DMC *return* up to 1000 / Myosuite up to 100% *success rate*).

1. Introduction

Reinforcement Learning (RL) has shown great successes in recent years, achieving breakthroughs in diverse areas. Despite these advancements, a fundamental challenge that remains in RL is enhancing the sample efficiency of algorithms. Indeed, in real-world applications, such as robotics, collecting large amounts of data can be time-consuming, costly, and sometimes impractical due to physical constraints or safety concerns. Thus, addressing this is crucial to make RL methods more accessible and scalable.

Different approaches have been explored to address the problem of low sample efficiency in RL. Model-based RL, on the one hand, attempts to increase sample efficiency by learning dynamic models that reduce the need for collecting real data, a process often expensive and time-consuming (Sutton, 1990; Janner et al., 2019; Feinberg et al., 2018; Heess et al., 2015). Model-free RL approaches, on the other hand, have explored increasing the number of gradient updates on the available data, referred to as the update-to-data (UTD) ratio (Nikishin et al., 2022; D'Oro et al., 2022), modifying network architectures (Bhatt et al., 2024), or both (Chen et al., 2021; Hiraoka et al., 2021; Hussing et al., 2024; Nauman et al., 2024).

In this work, we build upon CrossQ (Bhatt et al., 2024), a recent model-free RL algorithm that recently showed state-of-the-art sample efficiency on the `MuJoCo` (Todorov et al., 2012) continuous control benchmarking tasks. Notably, the authors achieved this by carefully utilizing Batch Normal-

[1]Intelligent Autonomous Systems Group, Technical University of Darmstadt, Germany [2]hessian.AI, Darmstadt, Germany [3]Department of Computer Science, University of Freiburg, Germany [4]German Research Center for Artificial Intelligence (DFKI) [5]Robotics Institute Germany (RIG). Correspondence to: Daniel Palenicek <daniel.palenicek@tu-darmstadt.de>.

Preprint. Under review.

ization (BN, Ioffe (2015)) within the actor-critic architecture. A technique previously thought not to work in RL, as famously reported by Hiraoka et al. (2021) and others. The insight that Bhatt et al. (2024) offered is that one needs to carefully consider the different state-action distributions within the Bellman equation and handle them correctly to succeed. This novelty allowed CrossQ at a low UTD of 1 to outperform the then state-of-the-art algorithms that scaled their UTD ratios up to 20. Even though higher UTD ratios are more computationally expensive, they allow for larger policy improvements using the same amount of data.

This naturally raises the question: *How can we extend the sample efficiency benefits of CrossQ and* BN *to the high* UTD *training regime?* Which we address in this manuscript.

Contributions. In this work, we show that the vanilla CrossQ algorithm is brittle to tune on DeepMind Control (DMC) and Myosuite environments and can fail to scale reliably with increased compute. To address these limitations, we propose the addition of weight normalization (WN), which we show to be a simple yet effective enhancement that stabilizes CrossQ. We motivate the combined use of WN and BN on insights from the continual learning and loss of plasticity literature and connections to the effective learning rate. Our experiments show that incorporating WN not only improves the stability of CrossQ but also allows us to scale its UTD, thereby significantly enhancing sample efficiency.

2. Preliminaries

This section briefly outlines the required background knowledge for this paper.

Reinforcement learning. A Markov Decision Process (MDP) (Puterman, 2014) is a tuple $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, \mathcal{P}, r, \mu_0, \gamma \rangle$, with state space $\mathcal{S} \subseteq \mathbb{R}^n$, action space $\mathcal{A} \subseteq \mathbb{R}^m$, transition probability $\mathcal{P} : \mathcal{S} \times \mathcal{A} \to \Delta(\mathcal{S})$, the reward function $r : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$, initial state distribution μ_0 and discount factor γ. We define the RL problem according to Sutton & Barto (2018). A policy $\pi : \mathcal{S} \to \Delta(\mathcal{A})$ is a behavior plan, which maps a state $\boldsymbol{s}$ to a distribution over actions $\boldsymbol{a}$. The discounted cumulative return is defined as

$$\mathcal{R}(\boldsymbol{s}, \boldsymbol{a}) = \textstyle\sum_{t=0}^{\infty} \gamma^t r(\boldsymbol{s}_t, \boldsymbol{a}_t),$$

where $\boldsymbol{s}_0 = \boldsymbol{s}$ and $\boldsymbol{a}_0 = \boldsymbol{a}$. Further, $\boldsymbol{s}_{t+1} \sim \mathcal{P}(\,\cdot\,|\boldsymbol{s}_t, \boldsymbol{a}_t)$ and $\boldsymbol{a}_t \sim \pi(\,\cdot\,|\boldsymbol{s}_t)$. The Q-function of a policy π is the expected discounted return $Q^\pi(\boldsymbol{s}, \boldsymbol{a}) = \mathbb{E}_{\pi,\mathcal{P}}[\mathcal{R}(\boldsymbol{s}, \boldsymbol{a})]$.

The goal of an RL agent is to find an optimal policy π^* that maximizes the expected return from the initial state distribution

$$\pi^* = \arg\max_\pi \mathbb{E}_{\boldsymbol{s}\sim\mu_0}\left[Q^\pi(\boldsymbol{s}, \boldsymbol{a})\right].$$

Soft Actor-Critic (SAC, Haarnoja et al. (2018)) addresses this optimization problem by jointly learning neural network representations for the Q-function and the policy. The policy network is optimized to maximize the Q-values, while the Q-function is optimized to minimize the Bellmann error

$$\mathcal{L} = \mathop{\mathbb{E}}_{\mathcal{D}}\left[\frac{1}{2}\left(Q_\theta(\boldsymbol{s}_t, \boldsymbol{a}_t) - \left(r(\boldsymbol{s}_t, \boldsymbol{a}_t) + \gamma \mathop{\mathbb{E}}_{\mathcal{P}}[V(\boldsymbol{s}_{t+1})]\right)\right)^2\right],$$

where the value function is computed by taking an expectation over the learned Q function

$$V(\boldsymbol{s}_{t+1}) = \mathbb{E}_{\mathcal{P},\pi_\theta}\left[Q_{\bar{\theta}}(\boldsymbol{s}_{t+1}, \boldsymbol{a}_{t+1})\right]. \tag{1}$$

To stabilize the Q-function learning, Haarnoja et al. (2018) found it necessary to use a target Q-network in the computation of the value function instead of the regular Q-network. The target Q-network is structurally equal to the regular Q-network, and its parameters $\bar{\theta}$ are obtained via Polyak Averaging over the learned parameters θ. While this scheme ensures stability during training by explicitly delaying value function updates, it also arguably slows down online learning (Plappert et al., 2018; Kim et al., 2019; Morales, 2020).

Instead of relying on target networks, CrossQ (Bhatt et al., 2024) addresses training stability issues by introducing Batch Normalization (BN, Ioffe (2015)) in its Q-function and achieves substantial improvements in sample and computational efficiency over SAC. A central challenge when using BN in Q networks is distribution mismatch: during training, the Q-function is optimized with samples $\boldsymbol{s}_t, \boldsymbol{a}_t$ from the replay buffer. However, when the Q-function is evaluated to compute the target values (Equation (1)), it receives actions sampled from the current policy $\boldsymbol{a}_{t+1} \sim \pi_\theta(\,\cdot\,|\boldsymbol{s}_{t+1})$. Those samples have no guarantee of lying within the training distribution of the Q-function. BN is known to struggle with out-of-distribution samples, as such, training can become unstable if the distribution mismatch is not correctly accounted for (Bhatt et al., 2024). To deal with this issue, CrossQ removes the separate target Q-function and evaluates both Q values during the critic update in a single forward pass, which causes the BN layers to compute shared statistics over the samples from the replay buffer and the current policy. This scheme effectively tackles distribution mismatch problems, ensuring that all inputs and intermediate activations are effectively forced to lie within the training distribution.

Normalization techniques in RL. Normalization techniques are widely recognized for improving the training of neural networks, as they generally accelerate training and improve generalization (Huang et al., 2023). There are many ways of introducing different types of normalizations into the RL framework. Most commonly, authors have used Layer Normalization (LN) within the network architectures to stabilize training (Hiraoka et al., 2021; Lyle et al., 2024). Recently, CrossQ has been the first algorithm to successfully

use BN layers in RL (Bhatt et al., 2024). The addition of BN leads to substantial gains in sample efficiency. In contrast to LN, however, one needs to carefully consider the different state-action distributions within the critic loss when integrating BN. In a different line of work, Hussing et al. (2024) proposed the integration of unit ball normalization and projected the output features of the penultimate layer onto the unit ball in order to reduce Q-function overestimation.

Increasing update-to-data ratios. Although scaling up the UTD ratio is an intuitive approach to increase the sample efficiency, in practice, it comes with several challenges. Nikishin et al. (2022) demonstrated that overfitting on early training data can inhibit the agent from learning anything later in the training. The authors dub this phenomenon the primacy bias. To address the primacy bias, they suggest to periodically reset the network parameters while retraining the replay buffer. Many works that followed have adapted this intervention (D'Oro et al., 2022; Nauman et al., 2024). While often effective, regularly resetting is a very drastic intervention and by design induces regular drops in performance. Since the agent has to start learning from scratch repeatedly, it is also not very computing efficient. Finally, the exact reasons why parameter resets work well in practice are not yet well understood (Li et al., 2023). Instead of resetting there have also been other types of regularization that allowed practitioners to train stably with high UTD ratios. Janner et al. (2019) generate additional modeled data, by virtually increasing the UTD. In REDQ, Chen et al. (2021) leverage ensembles of Q-functions, while Hiraoka et al. (2021) use dropout and LN to effectively scale to higher UTD ratios.

3. CrossQ fails to scale up stably

Bhatt et al. (2024) demonstrated CrossQ's state-of-the-art sample efficiency on the `MuJoCo` task suite (Todorov et al., 2012), while at the same time also being very computationally efficient. However, on the more extensive DMC and Myosuite task suites, we find that CrossQ requires tuning. We further find that it works on some, but not all, environments stably and reliably.

Mixed performance of CrossQ. Figure 2 shows CrossQ training performance on a subset of DMC tasks. Namely, the `dog-stand`, `dog-trot` and `humanoid-walk`, selected for their varying difficulty levels to demonstrate a wide range of behaviors, including both successful learning and challenges encountered during training. The figure compares a SAC baseline with standard hyperparameters against tuned CrossQ agents with UTD ratios of 1 and 5, where the hyperparameters were identified through a grid search over learning rates and network sizes, as detailed in Table 1. The first row of the figure shows the IQM training

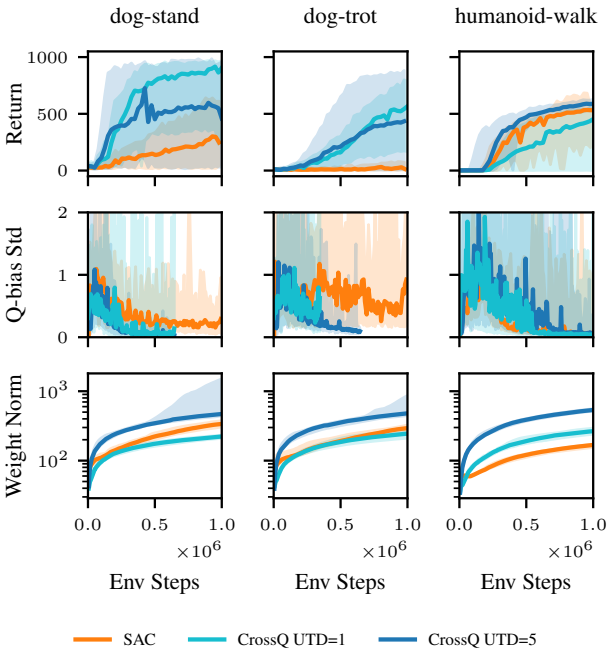


Figure 2. Q-bias and weight norms. CrossQ critic weight norms increase significantly with increasing UTD ratios.

performance and 95% confidence intervals for each agent across 10 seeds. Here, we identify three different training behaviors. On `dog-stand` CrossQ trains stably at UTD= 1, but increasing the UTD to 5 introduces instabilities and decreases performance. On `dog-trot`, both UTD ratios perform very similarly. Finally, on `humanoid-walk`, UTD=5 outperforms UTD=1, although it merely manages to catch up to the SAC baseline in this case. Overall, for all CrossQ runs we notice very large confidence intervals.

Q-function bias analysis. The second row in Figure 2 shows the standard deviation of the normalized Q-function bias. This bias measures how much the Q-function is overestimating or underestimating the true expected return of the current policy.

To compute the normalized Q-function bias, we follow the protocol of Chen et al. (2021). We gather 5 trajectories in the environment using the current policy and use each trajectory's first 350 state-action pairs to calculate the bias. For this, we compare the cumulative discounted rewards for each state-action pair with its Q-value predicted by the Q-function. This bias is then normalized using the cumulative discounted rewards. The mean over these normalized Q-function biases measures the expected bias. Even if the mean is high, as long as the bias is consistent, the selected actions of the policy do not change and, therefore, it is not a problem (Van Hasselt et al., 2016). If the standard deviation

is high, the change in bias is high, which might hinder learning. Thus, following the work of Chen et al. (2021); Bhatt et al. (2024), we focus on the standard deviation.

We find large fluctuations for the Q-function bias standard deviation with all three agents across environments. And even on `dog-stand`, where CrossQ UTD=5 does not learn reliably, it maintains a small Q-function bias standard deviation. From these mixed results, we conclude that we cannot directly link the standard deviation of the Q-function bias to the learning performance. While this does not mean that for larger UTD ratios, the Q-bias would not explode. Rather, it means that, in our case, the Q-bias is not at fault, and the root cause for unstable training lies elsewhere.

Growing network parameter norms. The third row of Figure 2 displays the sum over the L2 norms of the dense layers in the critic network. This includes three dense layers, each with a hidden dimension of 512.

All three baselines exhibit growing network weights over the course of training. We find that the effect is particularly pronounced for CrossQ with increasing UTD ratios. We further find that in the second training phase, the spread of network weight norms increases for the `dog` runs with large confidence intervals. This is visualized by the large spread of the shaded areas, which show the 95% inter percentile ranges for the weight norms.

Growing network weights have been linked to a loss of plasticity, a phenomenon where networks become increasingly resistant to parameter update, which can lead to premature convergence (Elsayed et al., 2024). Additionally, the growing magnitudes pose a challenge for optimization, connected to the issue of growing activations, which has recently been analyzed by Hussing et al. (2024). Further, growing network weights decrease the effective learning rate when the networks contain normalization layers (Van Hasselt et al., 2019; Lyle et al., 2024).

In summary, the scaling results for vanilla CrossQ are mixed. While increasing UTD ratios is known to yield increased sample efficiency, if careful regularization is used (Janner et al., 2019; Chen et al., 2021; Nikishin et al., 2022), CrossQ alone with BN cannot benefit from it. We notice that with increasing UTD ratios, CrossQ's weight layer norms grow significantly faster and overall larger. This observation motivates us to further study the weight norms in CrossQ to increase UTD ratios.

4. Combining batch normalization and weight normalization for scaling up

Inspired by the combined insights of Van Hasselt et al. (2019) and Lyle et al. (2024), we propose to integrate CrossQ with Weight Normalization (WN) as a means of counteracting the rapid growth of weight norms we observe with increasing update-to-data (UTD) ratios.

Our approach is based on the following reasoning: Due to the use of BN in CrossQ, the critic network exhibits scale invariance, as previously noted by Van Laarhoven (2017).

Theorem 4.1 (Van Laarhoven (2017))**.** *Let $f(\boldsymbol{X}; \boldsymbol{w}, b, \gamma, \beta)$ be a function, with inputs $\boldsymbol{X}$ and parameters $\boldsymbol{w}$ and $\boldsymbol{b}$ and γ and β batch normalization parameters. When f is normalized with batch normalization, f becomes scale-invariant with respect to its parameters, i.e.,*

$$f(\boldsymbol{X}; c\boldsymbol{w}, cb, \gamma, \beta) = f(\boldsymbol{X}; \boldsymbol{w}, b, \gamma, \beta),$$

with scaling factor $c > 0$.

Proof. Appendix A

This property allows us to introduce WN as a mechanism to regulate the growth of weight norms in CrossQ without affecting the critic's outputs. Further, it can be shown, that for such a scale invariant function, the gradient scales inversely proportionally to the scaling factor $c > 0$.

Theorem 4.2 (Van Laarhoven (2017))**.** *Let $f(\boldsymbol{X}; \boldsymbol{w}, b, \gamma, \beta)$ be a scale-invariant function. Then, the gradients of f scale inversely proportional to the scaling factor $c \in \mathbb{R}$ of its parameters $\boldsymbol{w}$,*

$$\nabla f(\boldsymbol{X}; c\boldsymbol{w}, cb, \gamma, \beta) = \nabla f(\boldsymbol{X}; \boldsymbol{w}, b, \gamma, \beta)/c.$$

Proof. Appendix B

Recently, Lyle et al. (2024) demonstrated that the combination of LN and WN can help mitigate loss of plasticity. Since the gradient scale is inversely proportional to c, keeping norms constant helps to maintain a stable effective learning rate (ELR,Van Hasselt et al. (2019)), further enhancing training stability.

We conjecture that maintaining a stable ELR could also be beneficial when increasing the UTD ratios in continuous control RL. As the UTD ratio increases, the networks are updated more frequently with each environment interaction. Empirically, we find that the network norms tend to grow quicker with increased UTD ratios (Figure 2), which in turn decreases the ELR even quicker and could be the case for instabilities and low training performance. As such, we empirically investigate the effectiveness of combining CrossQ with WN with increasing UTD ratios.

Implementation details. We apply WN to the first two linear layers, ensuring that their weights remain unit norm after each gradient step by projecting them onto the unit ball, similar to Lyle et al. (2024). Since this constraint effectively stabilizes the ELR in these layers, we found it beneficial to slightly reduce the learning rate from $1e{-}3$ to $1e{-}4$. While

we could employ a learning rate schedule (Lyle et al., 2024) we did not investigate this here. Additionally, we impose weight decay on all parameters that remain unbounded—specifically the final dense output layer. In practice, we use AdamW (Loshchilov, 2017) with a decay of 0 (which falls back to vanilla Adam (Kingma, 2014)) for the normalized intermediate dense layers and $1e{-}2$ otherwise. We chose a maximum UTD of 5 for our experiments due to our computational budget and good strong in sample efficiency.

Target networks. CrossQ famously removed the target networks from the actor-critic framework and showed that using BN training remains stable even without them (Bhatt et al., 2024). While we find this to be true in many cases, we find that especially in DMC, the re-integration of target networks can help stabilize training overall (see Section 5.4). However, not surprisingly, we find that the integration of target networks with BN requires careful consideration of the different state-action distributions between the $\boldsymbol{s}, \boldsymbol{a}$ and $\boldsymbol{s}', \boldsymbol{a}' \sim \pi(\boldsymbol{s}')$ exactly as proposed by Bhatt et al. (2024). To satisfy this, we keep the joined forward pass through both the critic network as well as the target critic network. We evaluate both networks in *training mode*, i.e., they calculate the joined state-action batch statistics on the current batches. As is common, we use Polyak-averaging with a $\tau = 0.005$ from the critic network to the target network.

5. Experiments

To evaluate the effectiveness of our proposed CrossQ + WN method, we conduct a comprehensive set of experiments on the DeepMind Control Suite (Tassa et al., 2018) and MyoSuite (Caggiano et al., 2022) benchmarks. Our primary goal is to investigate the scalability of CrossQ + WN with increasing UTD ratios and to assess the stabilizing effects of combining CrossQ with WN. We compare our approach to several baselines, including the recent BRO (Nauman et al., 2024), CrossQ (Bhatt et al., 2024) and SR-SAC (D'Oro et al., 2022) a version of SAC (Haarnoja et al., 2018) with high UTD ratios and network resets.

5.1. Experimental setup

Our implementation is based on the SAC implementation of `jaxrl` codebase (Kostrikov, 2021). We implement CrossQ following the author's original codebase and add the architectural modifications introduced by (Bhatt et al., 2024), incorporating batch normalization in the actor and critic networks. We extend this approach by introducing WN to regulate the growth of weight norms and prevent loss of plasticity and add target networks. We perform a grid search to focus on learning rate selection and layer width.

We evaluate 25 diverse continuous control tasks, 15 from DMC and 10 from MyoSuite. These tasks vary significantly in complexity, requiring different levels of fine motor control and policy adaptation with high dimensional state spaces up to $\mathbb{R}^{223}$.

Each experiment is run for 1 million environment steps and across 10 random seeds to ensure statistical robustness. We evaluate agents every $25,000$ environment steps for 5 trajectories. As proposed by Agarwal et al. (2021), we report the interquartile mean (IQM) and 95% stratified bootstrap confidence intervals (CIs) of the return, if not otherwise stated.

For the BRO baseline results, for computational reasons, we take the official evaluation data the authors provide. The official BRO codebase is also based on `jaxrl`, and the authors followed the same evaluation protocol, making it a fair comparison.

5.2. Weight normalization allows CrossQ to scale effectively

We provide empirical evidence for our hypothesis that controlling the weight norm and, thereby, the ELR can stabilize training. We show that through the addition of WN, CrossQ + WN shows stable training and can stably scale with increasing UTD ratios.

Figure 3 shows per environment results of our experiments encompassing all 25 tasks evaluated across 10 seeds each. Based on that, Figure 1 shows aggregated performance over all environments from Figure 3 per task suite, with a separate aggregation for the most complex `dog` and `humanoid` environments.

These results show that CrossQ + WN UTD=5 is competitive to the BRO baseline on both DMC and Myosuite, especially on the more complex `dog` and `humanoid` tasks. Notably, CrossQ + WN UTD=5 uses only half the UTD of BRO and does not require any parameter resets and no additional exploration policy. Further, it uses $\sim 90\%$ fewer network parameters—BRO reports $\sim 5M$, while our proposed CrossQ + WN uses only $\sim 600k$ (these numbers vary slightly per environment, depending on the state and action dimensionalities).

In contrast, vanilla CrossQ UTD=1 exhibits much slower learning on most tasks and, in some environments, fails to learn performant policies. Moreover, the instability of vanilla CrossQ at UTD=5 is particularly notable, as it does not reliably converge across environments.

These findings highlight the necessity of incorporating additional normalization techniques to sustain effective training at higher UTD ratios. This leads us to conclude that CrossQ benefits from the addition of WN, which results in stable training and scales well with higher UTD ratios. The resulting algorithm can match or outperform state-of-the-art

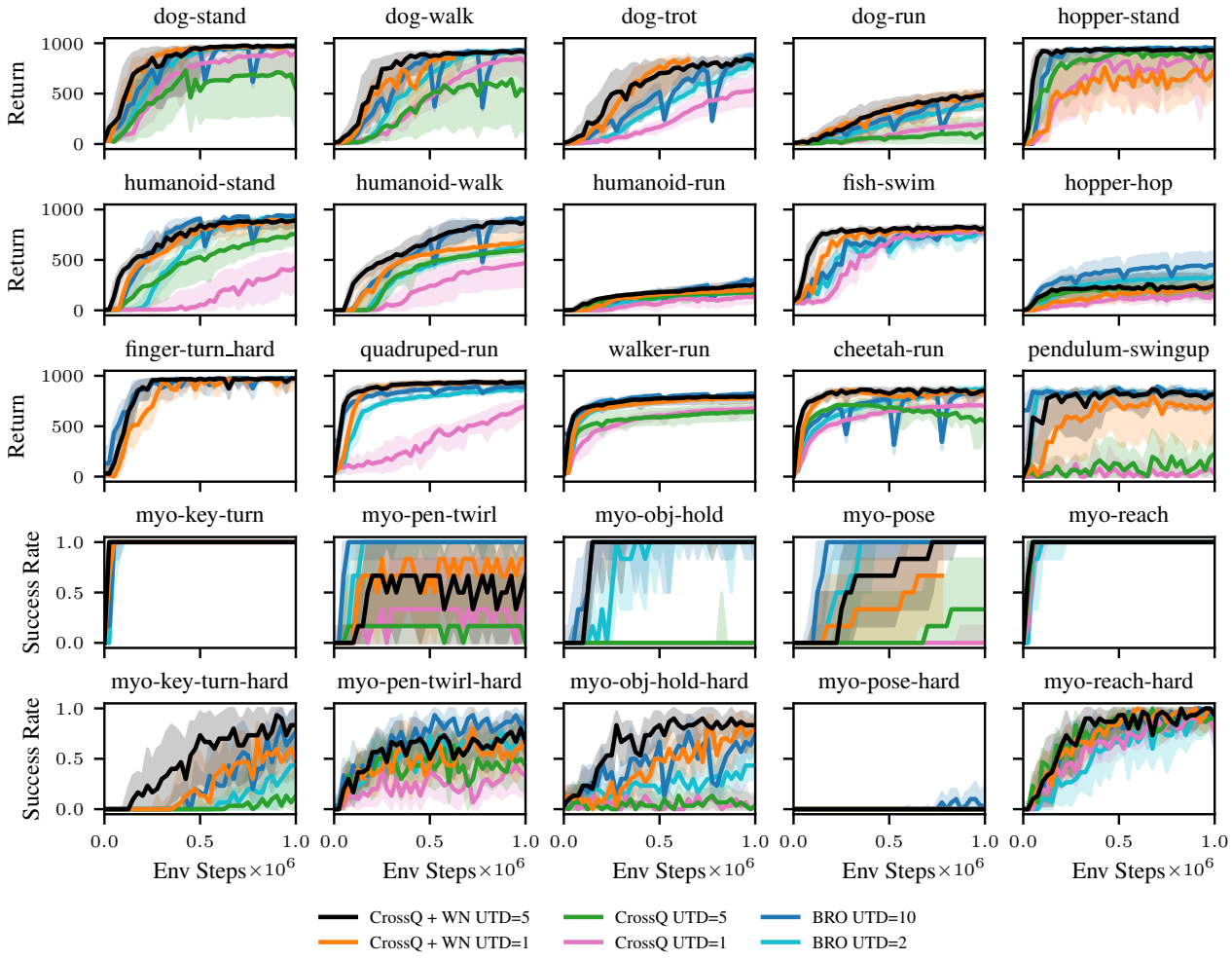


Figure 3. CrossQ WN + UTD=5 *against baselines.* We compare our proposed CrossQ + WN UTD=5 against two baselines, BRO (Nauman et al., 2024) and SR-SAC UTD=32. Results are reported on all 15 DMC and 10 Myosuite tasks. We plot the IQM and 95% CIs over 10 random random seeds. Our proposed approach proves competitive to BRO and outperforms the CrossQ baseline. We want to note that our approach achieves this performance without requiring any parameter resetting or additional exploration policies.

baselines on the continuous control DMC and Myosuite benchmarks while being much simpler algorithmically.

5.3. Stable scaling of CrossQ + WN with UTD ratios

To visualize the stable scaling behavior of CrossQ + WN we ablate across three different UTD ratios $\in \{1, 2, 5\}$. Figure 4 shows training curves aggregated over all 15 DMC tasks. As expected, CrossQ + WN shows reliable scaling behavior, with the learning curves ordered in increasing order accordance to their respective UTD ratio.

5.4. Hyperparameter ablation studies

We also ablate the different hyperparameters of CrossQ + WN UTD=5, by changing each one at a time. Figure 5 shows aggregated results of the final performances of each ablation. We will briefly discuss each ablation individually.

Removing weight normalization. Not performing weight normalization results in the biggest drop in performance across all our ablations. This loss is most drastic on the Myosuite tasks and often results in no meaningful learning. Showing that, as hypothesized, the inclusion of WN into the CrossQ framework yields great improvements in terms of sample efficiency and training stability, especially for larger UTD ratios.

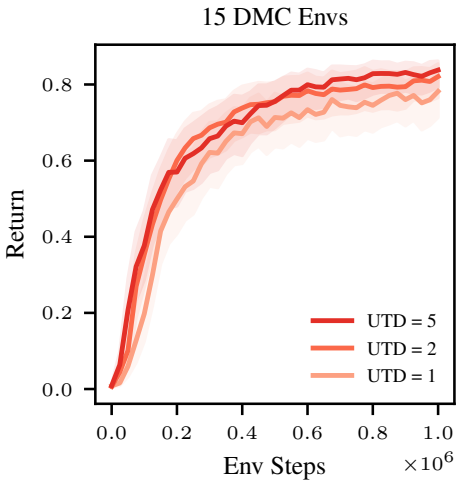


Figure 4. CrossQ WN UTD *scaling behavior.* We plot the IQM return and 95% confidence intervals for different UTD ratios $\in \{1, 2, 5\}$. The results are aggregated over 15 DMC environments and 10 random seeds each according to Agarwal et al. (2021). The sample efficiency scales reliably with increasing UTD ratios.

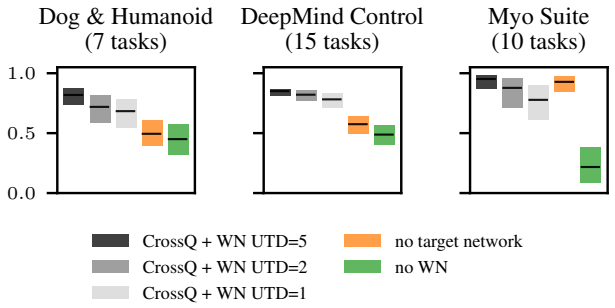


Figure 5. Hyperparameter ablations. We ablate CrossQ + WN with respect to the WN, target networks and different UTD.

Update-to-data ratio. As expected, decreasing the UTD ratio decreases the performance of CrossQ + WN, as demonstrated in the previous section. Aggregated over all DMC environments, this effect is the smallest, since this aggregation includes easier environments as well. Looking at the harder `dog` and `humanoid` tasks, as well as Myosuite, the effect is more pronounced. However, lower UTDs are already reasonably competitive in overall performance.

Target networks. Ablating the target networks shows that on Myosuite, there is no significant difference between using a target network and or no target network. Results on DMC differ significantly. There, removing target networks leads to a significant drop in performance, nearly as large as removing weight normalization. This finding is interesting, as it suggests that CrossQ + WN without target networks is not inherently unstable. But there are situations where the inclusion of target networks is required. Further investigating the role and necessity of target networks in RL is an interesting direction for future research.

6. Related work

RL has demonstrated remarkable success across various domains, yet sample efficiency remains a significant challenge, especially in real-world applications where data collection is expensive or impractical. Various approaches have been explored to address this issue, including model-based RL, UTD ratio scaling, and architectural modifications.

Model-based RL methods enhance sample efficiency by constructing predictive models of the environment to reduce reliance on real data collection (Sutton, 1990; Janner et al., 2019; Feinberg et al., 2018; Heess et al., 2015). However, such methods introduce additional complexity, computational overhead, and potential biases due to model inaccuracies.

Update-to-data ratio scaling. Model-free RL methods, including those utilizing higher UTD ratios, focus on increasing the number of gradient updates per collected sample to maximize learning from available data. High UTD training introduces several challenges, such as overfitting to early training data, a phenomenon known as primacy bias (Nikishin et al., 2022). This can be counteracted by periodically resetting the network parameters (Nikishin et al., 2022; D'Oro et al., 2022). However, network resets introduce abrupt performance drops. Alternative approaches use techniques such as Q-function ensembles (Chen et al., 2021; Hiraoka et al., 2021) and architectural changes (Nauman et al., 2024).

Normalization techniques in RL. Normalization techniques have long been recognized for their impact on neural network training. LN (Ba et al., 2016) and other architectural modifications have been used to stabilize learning in RL (Hiraoka et al., 2021; Nauman et al., 2024). Yet BN has only recently been successfully applied in this context (Bhatt et al., 2024), challenging previous findings, where BN in critics caused training to fail (Hiraoka et al., 2021). WN has been shown to keep ELRs stable and prevent loss of plasticity (Lyle et al., 2024), when combined with LN, making it a promising candidate for integration into existing RL frameworks.

7. Limitations & future work

In this work, we only consider continuous state-action benchmarking tasks. While our proposed CrossQ + WN performs competitively on these tasks, its performance on discrete state-action spaces or vision-based tasks remains unexplored. We plan to investigate this in future work.

8. Conclusion

In this work, we have addressed the instability and scalability limitations of CrossQ in RL by integrating WN. Our empirical results demonstrate that WN effectively stabilizes training and allows CrossQ to scale reliably with higher UTD ratios. The proposed CrossQ + WN approach achieves competitive or superior performance compared to state-of-the-art baselines across a diverse set of 25 complex continuous control tasks from the DMC and Myosuite benchmarks. These tasks include complex and high-dimensional humanoid and dog environments. This extension preserves simplicity while enhancing robustness and scalability by eliminating the need for drastic interventions such as network resets.

Acknowledgments

We wish to thank Joe Watson for helpful discussions and advice during the project. We would also like to thank Tim Schneider, Cristiana de Farias, João Carvalho and Theo Gruner for proofreading and constructive criticism on the manuscript. This research was funded by the research cluster "Third Wave of AI", funded by the excellence program of the Hessian Ministry of Higher Education, Science, Research and the Arts, hessian.AI.

References

Agarwal, R., Schwarzer, M., Castro, P. S., Courville, A. C., and Bellemare, M. Deep reinforcement learning at the edge of the statistical precipice. *Advances in neural information processing systems*, 2021.

Ba, J. L., Kiros, J. R., and Hinton, G. E. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.

Bhatt, A., Palenicek, D., Belousov, B., Argus, M., Amiranashvili, A., Brox, T., and Peters, J. CrossQ: Batch normalization in deep reinforcement learning for greater sample efficiency and simplicity. In *International conference on learning representations*, 2024.

Caggiano, V., Wang, H., Durandau, G., Sartori, M., and Kumar, V. Myosuite–a contact-rich simulation suite for musculoskeletal motor control. *arXiv preprint arXiv:2205.13600*, 2022.

Chen, X., Wang, C., Zhou, Z., and Ross, K. Randomized ensembled double Q-learning: Learning fast without a model. In *International conference on learning representations*, 2021.

D'Oro, P., Schwarzer, M., Nikishin, E., Bacon, P.-L., Bellemare, M. G., and Courville, A. Sample-efficient reinforcement learning by breaking the replay ratio barrier. In *International conference on learning representations*, 2022.

Elsayed, M., Lan, Q., Lyle, C., and Mahmood, A. R. Weight clipping for deep continual and reinforcement learning. In *Reinforcement Learning Conference*, 2024.

Feinberg, V., Wan, A., Stoica, I., Jordan, M. I., Gonzalez, J. E., and Levine, S. Model-based value estimation for efficient model-free reinforcement learning. In *International Conference on Machine Learning*, 2018.

Haarnoja, T., Zhou, A., Hartikainen, K., Tucker, G., Ha, S., Tan, J., Kumar, V., Zhu, H., Gupta, A., Abbeel, P., and Levine, S. Soft actor-critic algorithms and applications. *arXiv preprint arXiv:1812.05905*, 2018.

Heess, N., Wayne, G., Silver, D., Lillicrap, T., Erez, T., and Tassa, Y. Learning continuous control policies by stochastic value gradients. In *Advances in neural information processing systems*, 2015.

Hiraoka, T., Imagawa, T., Hashimoto, T., Onishi, T., and Tsuruoka, Y. Dropout q-functions for doubly efficient reinforcement learning. In *International conference on learning representations*, 2021.

Huang, L., Qin, J., Zhou, Y., Zhu, F., Liu, L., and Shao, L. Normalization techniques in training dnns: Methodology, analysis and application. *IEEE transactions on pattern analysis and machine intelligence*, 2023.

Hussing, M., Voelcker, C., Gilitschenski, I., Farahmand, A.-m., and Eaton, E. Dissecting deep rl with high update ratios: Combatting value overestimation and divergence. *arXiv preprint arXiv:2403.05996*, 2024.

Ioffe, S. Batch normalization: Accelerating deep network training by reducing internal covariate shift. *arXiv preprint arXiv:1502.03167*, 2015.

Janner, M., Fu, J., Zhang, M., and Levine, S. When to trust your model: Model-based policy optimization. In *Advances in neural information processing systems*, 2019.

Kim, S., Asadi, K., Littman, M., and Konidaris, G. Deepmellow: Removing the need for a target network in deep q-learning. In *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence,*

A. Proof Scale Invariance

Proof of Theorem 4.1.

$$\begin{aligned} f(\boldsymbol{X}; c\boldsymbol{w}, cb, \gamma, \beta) &= \frac{g(c\boldsymbol{X}\boldsymbol{w} + cb) - \mu(g(c\boldsymbol{X}\boldsymbol{w} + cb))}{\sigma(g(c\boldsymbol{X}\boldsymbol{w} + cb))}\gamma + \beta \\ &= \frac{cg(\boldsymbol{X}\boldsymbol{w} + b) - c\mu(g(\boldsymbol{X}\boldsymbol{w} + b))}{|c|\sigma(g(\boldsymbol{X}\boldsymbol{w} + b))}\gamma + \beta \\ &= \frac{g(\boldsymbol{X}\boldsymbol{w} + b) - \mu(g(\boldsymbol{X}\boldsymbol{w} + b))}{\sigma(g(\boldsymbol{X}\boldsymbol{w} + b))}\gamma + \beta = f(\boldsymbol{X}; \boldsymbol{w}, b, \gamma, \beta) \end{aligned}$$

B. Proof Inverse Proportional Gradients

To show that the gradients scale inversely proportional to the parameter norm, we can first write

$$\begin{aligned} f(\boldsymbol{X}; c\boldsymbol{w}, cb, \gamma, \beta) &= \frac{g(c\boldsymbol{X}\boldsymbol{w} + cb) - \mu(g(c\boldsymbol{X}\boldsymbol{w} + cb))}{\sigma(g(c\boldsymbol{X}\boldsymbol{w} + cb))}\gamma + \beta \\ &= \frac{g(c\boldsymbol{X}\boldsymbol{w} + cb)}{\sigma(g(c\boldsymbol{X}\boldsymbol{w} + cb))}\gamma - \frac{\mu(g(c\boldsymbol{X}\boldsymbol{w} + cb))}{\sigma(g(c\boldsymbol{X}\boldsymbol{w} + cb))}\gamma + \beta. \end{aligned}$$

As the gradient of the weights is not backpropagated through the mean and standard deviation, we have

$$\nabla_w f(\boldsymbol{X}; c\boldsymbol{w}, cb, \gamma, \beta) = \frac{g'(c\boldsymbol{X}\boldsymbol{w} + cb)X}{|c|\sigma(g(\boldsymbol{X}\boldsymbol{w} + b))}\gamma.$$

The gradient of the bias can be computed analogously

$$\nabla_b f(\boldsymbol{X}; c\boldsymbol{w}, cb, \gamma, \beta) = \frac{g'(c\boldsymbol{X}\boldsymbol{w} + cb)}{|c|\sigma(g(\boldsymbol{X}\boldsymbol{w} + b))}\gamma.$$

C. Hyperparameters

Table 1 gives an overview of the hyperparameters that were used for each algorithm that was considered in this work.

Table 1. Hyperparameters

Hyperparameter	**CrossQ**	**CrossQ + WN**	**SAC**	**SR-SAC**	**BRO**
Critic learning rate	0.0001	0.0001	0.0003	0.0003	0.0003
Critic hidden dim	512	512	256	256	512
Actor learning rate	0.0001	0.0001	0.0003	0.0003	0.0003
Actor hidden dim	256	256	256	256	256
Initial temperature	1.0	1.0	1.0	1.0	1.0
Temperature learning rate	0.0001	0.0001	0.0003	0.0003	0.0003
Target entropy	$\|A\|$	$\|A\|$	$\|A\|$	$\|A\|$	$\|A\|$
Target network momentum	0.005	0.005	0.005	0.005	0.005
UTD	1,5	1,5	1	32	10
Number of critics	2	2	2	2	1
Action repeat	2	2	2	2	1
Discount	0.99 (DMC) 0.95 (Myo)	0.99 (DMC) 0.95 (Myo)	0.99 (DMC) 0.95 (Myo)	0.99 (DMC) 0.95 (Myo)	0.99 (DMC) 0.99 (Myo)
Optimizer	Adam	AdamW	Adam	Adam	AdamW
Optimizer momentum (β_1, β_2)	(0.9, 0.999)	(0.9, 0.999)	(0.9, 0.999)	(0.9, 0.999)	(0.9, 0.999)
Policy delay	3	3	1	1	1
Warmup transitions	5000	5000	10000	10000	10000
AdamW weight decay critic	0.0	0.01	0.0	0.0	0.0001
AdamW weight decay actor	0.0	0.01	0.0	0.0	0.0001
AdamW weight decay temperature	0.0	0.0	0.0	0.0	0.0
Batch Normalization momentum	0.99	0.99	N/A	N/A	N/A
Reset Interval of networks	N/A	N/A	N/A	every 80k steps	*at 15k, 50k, 250k, 500k and 750k steps*
Batch Size	256	256	256	256	128

D. Individual training curves for ablation results

Here, we provide detailed individual training curves for each ablation experiment conducted in our study. Figure 6 shows experiments with no WN, no target network, CrossQ with WN and UTD=1, and CrossQ with WN and UTD=5.

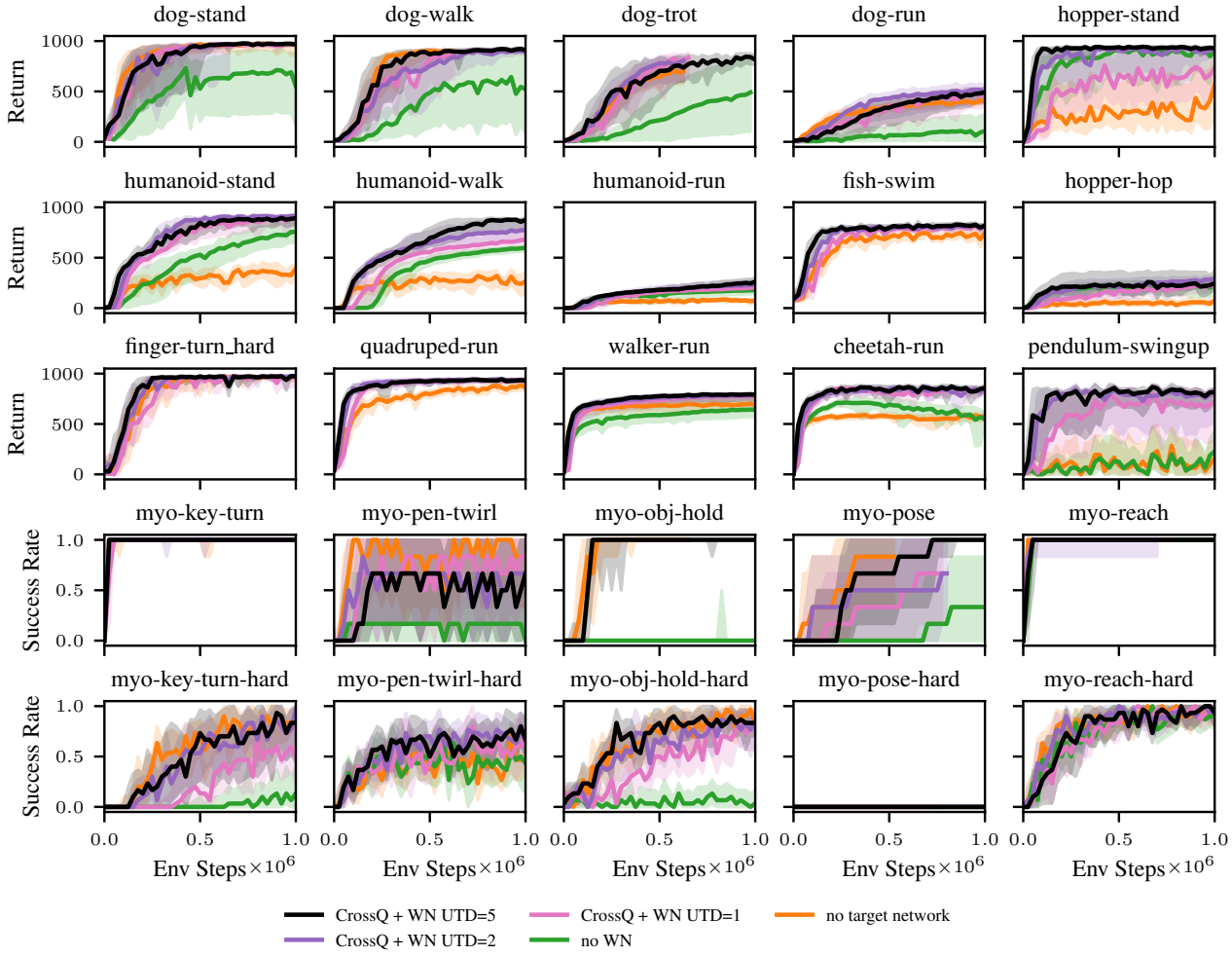


Figure 6. Individual training curves for ablations

FOUR THINGS EVERYONE SHOULD KNOW TO IMPROVE BATCH NORMALIZATION

Cecilia Summers
Department of Computer Science
University of Auckland
cecilia.summers.07@gmail.com

Michael J. Dinneen
Department of Computer Science
University of Auckland
mjd@cs.auckland.ac.nz

ABSTRACT

A key component of most neural network architectures is the use of normalization layers, such as Batch Normalization. Despite its common use and large utility in optimizing deep architectures, it has been challenging both to generically improve upon Batch Normalization and to understand the circumstances that lend themselves to other enhancements. In this paper, we identify four improvements to the generic form of Batch Normalization and the circumstances under which they work, yielding performance gains across all batch sizes while requiring no additional computation during training. These contributions include proposing a method for reasoning about the current example in inference normalization statistics, fixing a training vs. inference discrepancy; recognizing and validating the powerful regularization effect of Ghost Batch Normalization for small and medium batch sizes; examining the effect of weight decay regularization on the scaling and shifting parameters γ and β; and identifying a new normalization algorithm for very small batch sizes by combining the strengths of Batch and Group Normalization. We validate our results empirically on six datasets: CIFAR-100, SVHN, Caltech-256, Oxford Flowers-102, CUB-2011, and ImageNet.

1 INTRODUCTION

Neural networks have transformed machine learning, forming the backbone of models for tasks in computer vision, natural language processing, and robotics, among many other domains (Krizhevsky & Hinton, 2009; He et al., 2017; Levine et al., 2016; Sutskever et al., 2014; Graves et al., 2013). A key component of many neural networks is the use of normalization layers such as Batch Normalization (Ioffe & Szegedy, 2015), Group Normalization (Wu & He, 2018), or Layer Normalization (Ba et al., 2016), with Batch Normalization the most commonly used for vision-based tasks. While the true reason why these methods work is still an active area of research (Santurkar et al., 2018), normalization techniques typically serve the purpose of making neural networks more amenable to optimization, allowing the training of very deep networks without the use of careful initialization schemes (Simonyan & Zisserman, 2015; Zhang et al., 2019), custom nonlinearities (Klambauer et al., 2017), or other more complicated techniques (Xiao et al., 2018). Even in situations where training without normalization layers is possible, their usage can still aid generalization (Zhang et al., 2019). In short, normalization layers make neural networks train faster and generalize better.

Despite this, it has been challenging to improve normalization layers. In the general case, a new approach would need to be uniformly better than existing normalization methods, which has proven difficult. It has even been difficult to tackle a simpler task: characterizing when specific changes to common normalization approaches might yield benefits. In all, this has created an environment where approaches such as Batch Normalization are still used as-is, unchanged since their creation.

In this work we identify four techniques that everyone should know to improve their usage of Batch Normalization, arguably the most common method for normalization in neural networks. Taken together, these techniques apply in all circumstances in which Batch Normalization is currently used, ranging from large to very small batch sizes, including one method which is even useful when the batch size $B = 1$, and for each technique we identify the circumstances under which it is expected to be of use. In summary, our contributions are:

1. A way to more effectively use the current example during inference, fixing a discrepancy between training and inference that had been previously overlooked,
2. Identifying Ghost Batch Normalization, a technique designed for very large-batch multi-GPU training (Hoffer et al., 2017), as surprisingly effective even in the medium-batch, single-GPU regime,
3. Recognizing weight decay of the scaling and centering variables γ and β as a valuable source of regularization, an unstudied detail typically neglected, and
4. Proposing a generalization of Batch and Group Normalization in the small-batch setting, effectively making use of cross-example information present in the minibatch even when such information is not enough for effective normalization on its own.

Experimentally, we study the most common use-case of Batch Normalization, image classification, which is fundamental to most visual problems in machine learning. In total, these four techniques can have a surprisingly large effect, improving accuracy by over 6% on one of our benchmark datasets while only changing the usage of Batch Normalization layers.

We have released code at `https://github.com/ceciliaresearch/four_things_batch_norm`.

2 Related Work/Background on Normalization Methods

Most normalization approaches in neural networks, including Batch Normalization, have the general form of normalizing their inputs x_i to have a learnable mean and standard deviation:

$$\hat{x}_i = \gamma \frac{x_i - \mu_i}{\sqrt{\sigma_i^2 + \epsilon}} + \beta \tag{1}$$

where γ and β are the learnable parameters, typically initialized to 1 and 0, respectively. Where approaches typically differ is in how the mean μ_i and variance σ_i^2 are calculated.

Batch Normalization (Ioffe & Szegedy, 2015), the pioneering work in normalization layers, defined μ_i and σ_i^2 to be calculated for each channel or feature map separately across a minibatch of data. For example, in a convolutional layer, the mean and variance are computed across all spatial locations and training examples in a minibatch. During inference, these statistics are replaced with an exponential moving average of the mean and variance, making inference behavior independent of inference batch statistics. The effectiveness of Batch Normalization is undeniable, playing a key role in nearly all state-of-the-art convolutional neural networks since its discovery (Szegedy et al., 2016; 2017; He et al., 2016a;b; Zoph & Le, 2017; Zoph et al., 2018; Hu et al., 2018; Howard et al., 2017; Sandler et al., 2018). Despite this, there is still a fairly limited understanding of Batch Normalization's efficacy — while Batch Normalization's original motivation was to reduce internal covariate shift during training (Ioffe & Szegedy, 2015), recent work has instead proposed that its true effectiveness stems from making the optimization landscape smoother (Santurkar et al., 2018).

One weakness of Batch Normalization is its critical dependence on having a reasonably large batch size, due to the inherent approximation of estimating the mean and variance with a single batch of data. Several works propose methods without this limitation: Layer Normalization (Ba et al., 2016), which has found use in many natural language processing tasks (Vaswani et al., 2017), tackles this by calculating μ_i and σ_i^2 over all channels, rather than normalizing each channel independently, but does not calculate statistics across examples in each batch. Instance Normalization (Ulyanov et al., 2016), in contrast, only calculates μ_i and σ_i^2 using the information present in each channel, relying on the content of each channel at different spatial locations to provide effective normalization statistics. Group Normalization (Wu & He, 2018) generalizes Layer and Instance Normalization, calculating statistics in "groups" of channels, allowing for stronger normalization power than Instance Normalization, but still allowing for each channel to contribute significantly to the statistics used for its own normalization. The number of normalization groups per normalization layer is typically set to a global constant in group normalization, though alternatives such as specifying the number of channels per group have also been tried (Wu & He, 2018).

Besides these most common approaches, many other forms of normalization also exist: Weight Normalization (Salimans & Kingma, 2016) normalizes the weights of each layer instead of the inputs,

parameterizing them in terms of a vector giving the direction of the weights and an explicit scale, which must be initialized very carefully. Decorrelated Batch Normalization (Huang et al., 2018) performs ZCA whitening in its normalization layer, and Iterative Normalization (Huang et al., 2019) makes it more efficient via a Newton iteration approach. Cho & Lee (2017) analyze the weights in Batch Normalization from the perspective of a Riemannian manifold, yielding new optimization and regularization methods that utilize the manifold's geometry.

Targeting the small batch problem, Batch Renormalization (Ioffe, 2017) uses the moving average of batch statistics to normalize during training, parameterized in such a way that gradients still propagate through the minibatch mean and standard deviation, but introduces two new hyperparameters and still suffers somewhat diminished performance in the small-batch setting. Guo et al. (2018) tackle the small batch setting by aggregating normalization statistics over multiple forward passes.

Recently, Switchable Normalization (Luo et al., 2019) aims to learn a more effective normalizer by calculating μ_i and σ_i^2 as learned weighted combinations of the statistics computed from other normalization methods. While flexible, care must be taken for two reasons: First, as the parameters are learned differentiably, they are fundamentally aimed at minimizing the training loss, rather than improved generalization, which typical hyperparameters are optimized for on validation sets. Second, the choice of which normalizers to include in the weighted combination remains important, manifesting in Switchable Normalization's somewhat worse performance than Group Normalization for small batch sizes. Differentiable Dynamic Normalization (Luo et al., 2019) fixes the latter point, learning an even more flexible normalization layer. Beyond these, there are many approaches we omit for lack of space (Littwin & Wolf, 2018; Deecke et al., 2019; Hoffer et al., 2018; Klambauer et al., 2017; Xiao et al., 2018; Zhang et al., 2019).

3 Improving Normalization: What Everyone Should Know

In this section we detail four methods for improving Batch Normalization. We also refer readers to the Appendix for a discussion of methods which do *not* improve normalization layers (sometimes surprisingly so). For clarity, we choose to interleave descriptions of the methods with experimental results, which aids in understanding each of the approaches as they are presented. We experiment with four standard image-centric datasets in this section: CIFAR-100, SVHN, Caltech-256, and ImageNet, and report results on validation datasets in order to fully describe each approach without contaminating test-set results. We give results on test sets, and experimental details in Sec. 4.

3.1 Inference Example Weighing

Batch Normalization has a disparity in function between training and inference: As previously noted, Batch Normalization calculates its normalization statistics over each minibatch of data separately while training, but during inference a moving average of training statistics is used, simulating the expected value of the normalization statistics. Resolving this disparity is a common theme among methods that have sought to replace Batch Normalization (Ba et al., 2016; Ulyanov et al., 2016; Salimans & Kingma, 2016; Wu & He, 2018; Ioffe, 2017). Here we identify a key component of this training versus inference disparity which can be fixed within the context of Batch Normalization itself, improving it in the general case: when using a moving average during inference, each example does not contribute to its own normalization statistics.

To give an example of the effect this has, we consider the output range of Batch Normalization. During training, due to the inclusion of each example in its own normalization statistics, it can be shown[1] that the minimum possible output of a Batch Normalization layer is:

$$\min_{x_0,\ldots,x_{B-1}} \gamma \frac{x_0 - \mu_i}{\sqrt{\sigma_i^2 + \epsilon}} + \beta = -\gamma\sqrt{B-1} + \beta \tag{2}$$

with a corresponding maximum value of $\gamma\sqrt{B-1}+\beta$, where B is the batch size, and we assume for simplicity that Batch Norm is being applied non-convolutionally. In contrast, during inference the output range of Batch Normalization is *unbounded*, creating a discrepancy. Morever, this actually happens for real networks: the output range of a network with Batch Normalization is wider during inference than during training (see Sec. B in Appendix).

[1] Proof in Appendix Sec. A

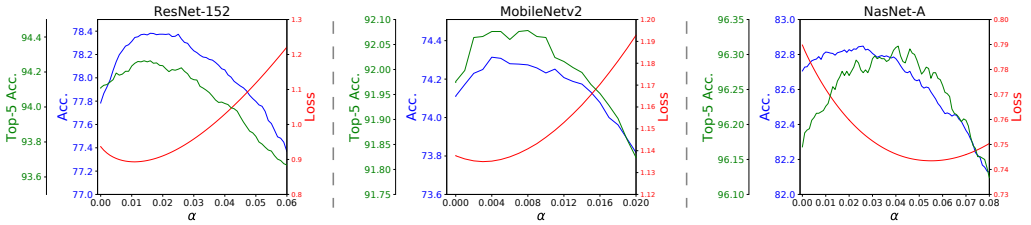


Figure 1: Effect of the example-weighing hyperparameter α on ImageNet for ResNet-152, MobileNetV2, and NASNet-A, measuring top-1 and top-5 accuracies and the cross-entropy loss.

Fortunately, once this problem has been realized, it is possible to fix — we need only figure out how to incorporate example statistics during inference. Denoting m_x as the moving average over x and m_{x^2} the corresponding moving average over x^2, we apply the following normalization:

$$\begin{aligned} \mu_i &= \alpha E[x_i] + (1-\alpha) m_x \\ \sigma_i^2 &= (\alpha E[x_i^2] + (1-\alpha) m_{x^2}) - \mu_i^2 \\ \hat{x}_i &= \gamma \frac{x_i - \mu_i}{\sqrt{\sigma_i^2 + \epsilon}} + \beta \end{aligned} \tag{3}$$

where α is the contribution of x_i to the normalization statistics, and we have reparameterized the variance as $\sigma_i^2 = E[x_i^2] - E[x_i]^2$.

Given this formulation, a natural question is the choice of the parameter α, where $\alpha = 0$ corresponds to the classical inference setting of Batch Normalization and $\alpha = 1$ replicates the setting of techniques which do not use cross-image information in calculating normalization statistics. Intuitively, it would make sense for the optimal value to be $\alpha = \frac{1}{B}$. However, this turns out to not be the case — instead, α is a hyperparameter best optimized on a validation set, whose optimal value may depend the model, dataset, and metric being optimized. While counterintuitive, this can be explained by the remaining set of differences between training and inference: for a basic yet fundamental example, the fact that the model has been fit on the training set (also typically with data augmentation) may produce systematically different normalization statistics between training and inference.

An advantage of this technique is that we can apply it retroactively to any model trained with Batch Normalization, allowing us to verify its efficacy on a wide variety of models. In Fig. 1 we show the effect of α on the ImageNet ILSVRC 2012 validation set (Russakovsky et al., 2015) for three diverse models: ResNet-152 (He et al., 2016b), MobileNetV2 (Sandler et al., 2018), and NASNet-A Large (Zoph et al., 2018)[2]. On ResNet-152, for example, proper setting of α can increase accuracy by up to 0.6%, top-5 accuracy by 0.16%, and loss by a relative 4.7%, which are all quite significant given the simplicity of the approach, the competitiveness of ImageNet as a benchmark, and the fact that the improvement is essentially "free" — it involves only modifying the inference behavior of Batch Normalization layers, and does not require any re-training. Across models, the optimal value for α was largest for NASNet-A, the most memory-intensive (and therefore smallest batch size) model of the three. We refer the reader to the Appendix for additional plots with larger ranges of α.

Surprisingly, it turns out that this approach can have positive effects on models trained without any cross-image normalization at all, such as models trained with Group Normalization (Wu & He, 2018). We demonstrate this in Fig. 2, where we find that adding a tiny amount of information from the moving average statistics can actually result in small improvements, with relatively larger improvements in accuracy on Caltech-256 and cross entropy loss on CIFAR-100 and SVHN. This finding is extremely surprising, since adding in any information from the moving averages at all represents a clear difference from the training setting of Group Normalization. Similar to the unintuitive optimal value for α, we hypothesize that this effect is due to other differences in the settings of training and inference: for example, models are generally trained on images with the application of data augmentation, such as random cropping. During inference, though, images appear unperturbed, and it might be the case that incorporating information from the moving averages is a way of influencing the model's intermediate activations to be more similar to those of data augmented

[2]Models obtained from (Silberman & Guadarrama).

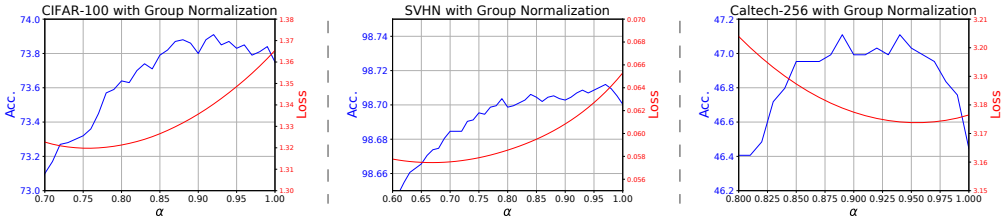


Figure 2: Effect of the example-weighing hyperparameter α for models trained with Group Normalization on CIFAR-100, SVHN, and Caltech-256.

images, which it has been trained on. This mysterious behavior may also point to more general approaches for resolving training-inference discrepancies, and is worthy of further study.

Last, we also note very recent work (Singh & Shrivastava, 2019) which examines a similar approach for incorporating the statistics of an example during inference time, using per-layer weights and optimizing with a more involved procedure that encourages similar outputs to the training distribution.

Summary: Inference example weighing resolves one disparity between training and inference for Batch Normalization, is uniformly beneficial across all models and very easy to tune to metrics of interest, and can be used with any model trained with Batch Normalization, even retroactively.

3.2 Ghost Batch Normalization for Medium Batch Sizes

Ghost Batch Normalization, a technique originally developed for training with very large batch sizes across many accelerators (Hoffer et al., 2017), consists of calculating normalization statistics on disjoint subsets of each training batch. Concretely, with an overall batch size of B and a "ghost" batch size of B' such that B' evenly divides B, the normalization statistics for example i are calculated as

$$\mu_i = \frac{1}{B'} \sum_{j=1}^{B} x_j [\left\lfloor \frac{jB'}{B} \right\rfloor = \left\lfloor \frac{iB'}{B} \right\rfloor]$$
$$\sigma_i^2 = \frac{1}{B'} \sum_{j=1}^{B} x_j^2 [\left\lfloor \frac{jB'}{B} \right\rfloor = \left\lfloor \frac{iB'}{B} \right\rfloor] - \mu_i^2 \tag{4}$$

where $[\cdot]$ is the Iverson bracket, with value 1 if its argument is true and 0 otherwise. Ghost Batch Normalization was previously found to be an important factor in reducing the generalization gap between large-batch and small-batch models (Hoffer et al., 2017), and has since been used by subsequent research rigorously studying the large-batch regime (Shallue et al., 2018). Here, we show that it can also be useful in the medium-batch setting[3].

Why might Ghost Batch Normalization be useful? One reason is its power as a regularizer: due to the stochasticity in normalization statistics caused by the random selection of minibatches during training, Batch Normalization causes the representation of a training example to randomly change every time it appears in a different batch of data. Ghost Batch Normalization, by decreasing the number of examples that the normalization statistics are calculated over, increases the strength of this stochasticity, thereby increasing the amount of regularization. Based on this hypothesis, we would expect to see a unimodal effect of the Ghost Batch Normalization size B' on model performance — a large value of B' would offer somewhat diminished performance as a weaker regularizer, a very low value of B' would have excess regularization and lead to poor performance, and an intermediate value would offer the best tradeoff of regularization strength.

We confirm this intuition in Fig. 3. Surprisingly, just using this one simple technique was capable of improving performance by 5.8% on Caltech-256 and 0.84% on CIFAR-100, which is remarkable given it has no additional cost during training. On SVHN, though, where baseline performance is already a very high 98.79% and models do not overfit much, usage of Ghost Batch Normalization

[3]We experiment with batch sizes up to 128 in this work.

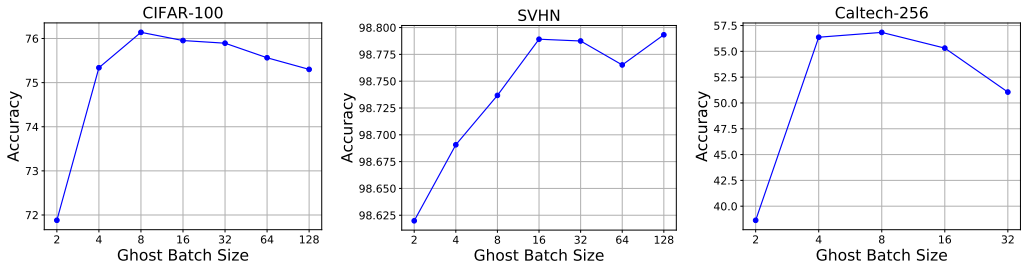


Figure 3: Accuracy vs. Ghost Batch Normalization size for CIFAR-100, SVHN, and Caltech-256.

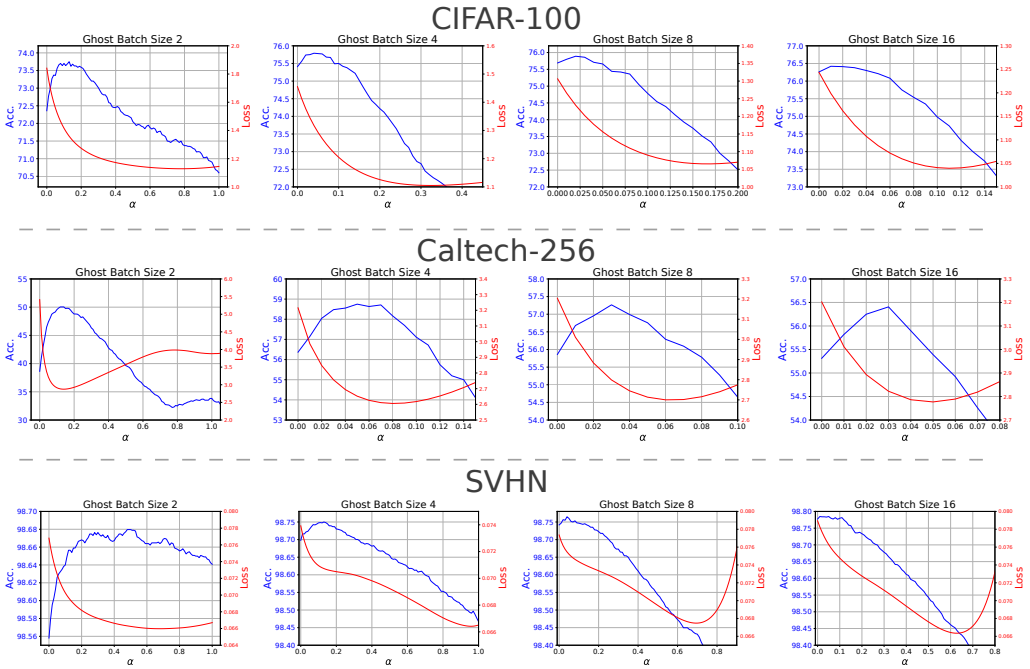


Figure 4: The complementary effects of Inference Example Weighing (Sec. 3.1) and Ghost Batch Normalization (Sec. 3.2) on CIFAR-100, SVHN, and Caltech-256.

did not result in an improvement, giving evidence that at least part of its effect is regularization in nature. In practice, B' may be treated as an additional hyperparameter to optimize.

As a bonus, Ghost Batch Normalization has a synergistic effect with inference example weighing — it has the effect of making each example more important in calculating its own normalization statistics μ_i and σ_i^2, with greater effect the smaller B' is, precisely the setting that inference example weighing corrects for. We show these results in Fig. 4, where we find increasing gain from inference example weighing as B' is made smaller, a gain that compounds from the benefits of Ghost Batch Normalization itself. Interestingly, these examples also demonstrate that accuracy and cross-entropy, the most commonly-used classification loss, are only partially correlated, with the optimal values for the inference example weight α sometimes differing wildly between the two (*e.g.* for SVHN).

Summary: Ghost Batch Normalization is beneficial for all but the smallest of batch sizes, has no computational overhead, is straightforward to tune, and can be used in combination with inference example weighing to great effect.

3.3 BATCH NORMALIZATION AND WEIGHT DECAY

Weight decay (Krogh & Hertz, 1992) is a regularization technique that scales the weight of a neural network after each update step by a factor of $1 - \delta$, and has a complex interaction with Batch Normalization. At first, it may even seem paradoxical that weight decay has any effect in a network trained with Batch Normalization, as scaling the weights immediately before a normalization layer by any non-zero constant has mathematically almost no effect on the output of the normalization layer (and no effect at all when $\epsilon = 0$). However, weight decay actually has a subtle effect on the effective learning rate of networks trained with Batch Normalization — without weight decay, the weights in a batch-normalized network grow to have large magnitudes, which has an inverse effect on the effective learning rate, hampering training (Hoffer et al., 2018; van Laarhoven, 2017).

Here we turn our attention to the less studied scale and bias parameters common in most normalization methods, γ and β. As far as we are aware, the effect of regularization on γ and β has not been studied to any great extent — Wu & He (2018) briefly mention weight decay with these parameters, where weight decay was used when training from scratch, but not fine-tuning, two other papers (Goyal et al., 2017; He et al., 2016a) have this form of weight decay explicitly turned off, and He et al. (2019) encourage disabling weight decay on γ and β, but ultimately find diminished performance by doing so.

Unlike weight decay on weights in *e.g.* convolutional layers, which typically directly precede normalization layers, weight decay on γ and β can have a regularization effect so long as there is a path in the network between the layer in question and the ultimate output of the network, as if such paths do not pass through another normalization layer, then the weight decay is never "undone" by normalization. This structure is only common in certain types of architectures; for example, Residual Networks (He et al., 2016a;b) have such paths for many of their normalization layers due to the chaining of skip-connections. However, Inception-style networks (Szegedy et al., 2016; 2017) have no residual connections, and despite the fact that each "Inception block" branches into multiple paths, every Batch Normalization layer other than those in the very last block do not have a direct path to the network's output.

We evaluated the effects of weight decay on γ and β on CIFAR-100 across 10 runs, where we found that incorporating it improved accuracy by a small but significant 0.3% ($P = 0.002$). Interestingly, even though γ has a multiplicative effect, we did not find it mattered whether γ was regularized to 0 or 1 ($P = 0.46$) — what was important was whether it had weight decay applied at all.

We did the same comparison on Caltech-256 with Inception-v3 and ResNet-50 networks, where we found evidence that the network architecture plays a crucial effect: for Inception-v3, incorporating weight decay on γ and β actually *hurt* performance by 0.13% (mean across 3 trials), while it improved performance for the ResNet-50 network by 0.91%, supporting the hypothesis that the structure of paths between layers and the network's output are what matter in determining its utility.

On SVHN, where the baseline ResNet-18 already had a performance of 98.79%, we found a similar pattern as with Ghost Batch Normalization — introducing this regularization produced no change.

Summary: Regularization in the form of weight decay on the normalization parameters γ and β can be applied to any normalization layer, but is only effective in architectures with particular connectivity properties like ResNets and in tasks for which models are already overfitting.

3.4 GENERALIZING BATCH AND GROUP NORMALIZATION FOR SMALL BATCHES

While Batch Normalization is very effective in the medium to large-batch setting, it still suffers when not enough examples are available to calculate reliable normalization statistics. Although we have shown that techniques such as Inference Example Weighing (Sec. 3.1) can help significantly with this, it is still only a partial solution. At the same time, Group Normalization (Wu & He, 2018) was designed for a batch size of $B = 1$ or greater, but ignores all cross-image information.

In order to generalize Batch and Group Normalization in the batch size $B > 1$ case, we propose to expand the grouping mechanism of Group Normalization from being over only channels to being over both channels and examples — that is, normalization statistics are calculated both *within* groups of channels of each example and *across* examples in groups within each batch[4].

[4]See submitted code for specific implementation details.

In principle, this would appear to introduce an additional hyperparameter on top of the number of channel groups used by Group Normalization, both of which would need to be optimized by expensive end-to-end runs of model training. However, in this case we can actually take advantage of the fact that the target batch size is small: if the batch size B is ever large enough that having multiple groups in the example dimension is useful, then it is *also* large enough to eschew usage of the channel groups from Group Normalization, in a regime where either vanilla Batch Normalization or Ghost Batch Normalization is more effective. Thus, when dealing with a small batch size, in practice we only need to optimize over the same set of hyperparameters as Group Normalization.

To demonstrate, we target the extreme setting of $B = 2$, and incorporate Inference Example Weighing to all approaches. For CIFAR-100, this approach improves validation set performance over a tuned Group Normalization by 0.69% in top-1 accuracy (from 73.91% to 74.60%, average over three runs), and on Caltech-256, performance dramatically improved by 5.0% (from 48.2% to 53.2%, average over two runs). However, this approach has one downside: due to differences in feature statistics across examples, when using only two examples the variability in the normalization statistics can still be quite high, even when using multiple channels within each normalization group. As a result, a regularization effect can occur, which may be undesirable for tasks which models are not overfitting much. As in Sec. 3.2 and Sec. 3.3, we see this effect in SVHN, where this approach is actually ever so slightly worse than Group Normalization on the validation set (from 98.75% to 98.73%). On such datasets and tasks, it may be more fruitful to invest in higher-capacity models.

Summary: Combining Group and Batch Normalization leads to more accurate models in the setting of batch sizes $B > 1$, and can have a regularization effect due to Batch Normalization's variability in statistics when calculated over small batch sizes.

4 Additional Experiments

4.1 Experimental Details

All results in Sec. 3 were performed on the validation datasets of each respective dataset (this section examines test set performance after hyperparameters have been optimized). Of the six datasets we experiment with, only ImageNet (Russakovsky et al., 2015) and Flowers-102 (Nilsback & Zisserman, 2008) have their own pre-defined validation split, so we constructed validation splits for the other datasets as follows: for CIFAR-100 (Krizhevsky & Hinton, 2009), we randomly took 40,000 of the 50,000 training images for the training split, and the remaining 10,000 as a validation split. For SVHN (Netzer et al., 2011), we similarly split the 604,388 non-test images in a 80-20% split for training and validation. For Caltech-256, no canonical splits of any form are defined, so we used 40 images of each of the 256 categories for training, 10 images for validation, and 30 for testing. For CUB-2011, we used 25% of the given training data as a validation set.

The model used for CIFAR-100 and SVHN was ResNet-18 (He et al., 2016b;a) with 64, 128, 256, and 512 filters across blocks. For Caltech-256, a much larger Inception-v3 (Szegedy et al., 2016) model was used, and we additionally experiment with ResNet-152 (He et al., 2016b) on Flowers-102 and CUB-2011 in Sec. 4.3. All experiments were done on two Nvidia Geforce GTX 1080 Ti GPUs.

4.2 Combining All Four: Improvements Across Batch Sizes

Here we show the end-to-end effect of these four improvements on the test sets of each dataset, comparing against both Batch and Group Normalization, with a batch size $B = 128$. We plot results for CIFAR-100 and Caltech-256 in Fig. 5 (a), comparing against Group Normalization and an idealized Batch Normalization with constant performance across batch sizes (simulating if the problematic dependence of Batch Norm on the batch size were completely solved). On CIFAR-100, we see improvements against the best available baseline across all batch sizes.

For medium to large batch sizes ($B \geq 4$), improvements are driven by the combination of Ghost Batch Normalization (Sec. 3.2), Inference Example Weighing (Sec. 3.1), and weight decay introduced on γ and β (Sec. 3.3). To aid in distinguishing between these effects, we also plot the impact of Ghost Batch Normalization alone, which we find particularly impactful as long as long as the batch size isn't too small ($B > 2$). Turning to very small batch sizes, for $B = 1$ improvements

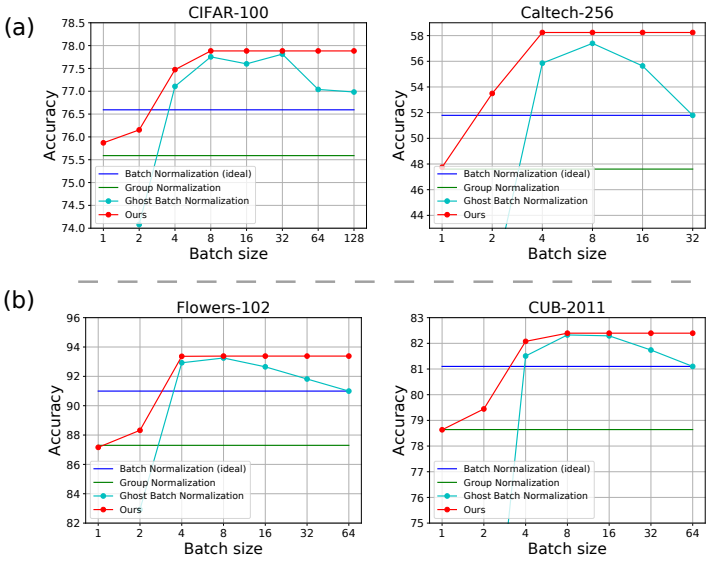


Figure 5: Total performance changes across batch sizes for CIFAR-100 and Caltech-256 (a) when training from scratch, incorporating all proposed improvements to Batch Normalization. On the bottom (b) is the same on Flowers-102 and CUB-2011, which employs transfer learning via fine-tuning from ImageNet. Also shown within each plot is the performance of Group Normalization, an idealized Batch Normalization that scales perfectly across batch sizes, and Ghost Batch Normalization (Sec. 3.2) by itself, for which the x-axis represents the Ghost Batch Size B'.

are due to the introduced weight decay, and for $B = 2$ the generalization of Batch and Group Normalization leads to the improvement (Sec. 3.4), with some additional effect from weight decay.

Improvements on Caltech-256 follow the same trends, but to greater magnitude, with a total increase in performance of 6.5% over Batch Normalization and an increase of 5.9% over Group Normalization for $B = 2$.

4.3 Transfer Learning

We also show the applicability of these approaches in the context of transfer learning, which we demonstrate on the Flowers-102 (Nilsback & Zisserman, 2008) and CUB-2011 (Wah et al., 2011) datasets via fine-tuning a ResNet-152 model from ImageNet. These tasks presents several challenges: 1) the Flowers-102 data only contains 10 images per category in the training set (and CUB-2011 only 30 examples per class), 2) pre-training models on ImageNet is a very strong form of prior knowledge, and despite the small dataset size may heavily reduce the regularization effects of some of the techniques, and 3) we examine the setting of pre-training with generic ImageNet models trained *without* any of these modifications, which gives an advantage to both the generic Batch Normalization and Group Normalization, for which pre-trained models exist.

We plot results in Fig. 5 (b), where we find remarkable qualitative agreement of our non-transfer learning results to this setting, despite the challenges. In total, on Flowers-102 our techniques were able to improve upon Batch Normalization by 2.4% (from 91.0% to 93.4% top-1 accuracy, a 27% relative reduction in error), and upon Group Normalization by 6.1% (from 87.3%, a 48% relative reduction in error). On CUB-2011, which has more training data, we improved upon Batch Normalization by 1.4% (from 81.1% to 82.4%) and Group Normalization by 3.8% (from 78.6%).

We anticipate that even further improvements might arise by additionally pre-training models with some of these techniques (particularly Ghost Batch Normalization), as we were able to see a large impact (roughly 5%) on Group Normalization by pre-training with a Group Normalization-based model instead of Batch Normalization.

Table 1: Accuracy on CIFAR-100 with non-i.i.d. minibatches. B' refers to the Ghost Batch Normalization size (equivalent to the batch size for Batch Normalization and Batch Renormalization), and "Batch Group Norm." refers to our approach in Sec. 3.4. "Inf. Ex. Weight: Off" refers to using only the moving averages for normalization statistics (*i.e.* $\alpha = 0$), while "On" refers to tuning α based on the validation set.

Method	B'	Inf. Ex. Weight: Off	Inf. Ex. Weight: On
Batch Norm	128	40.1	62.2
Batch ReNorm	128	69.0	69.0
Ghost Batch Norm	64	42.3	50.8
	32	57.8	70.9
	16	64.3	72.2
	8	68.7	72.0
	4	70.4	71.5
	2	68.4	71.4
Batch Group Norm.	2	75.2	76.1

4.4 Non-i.i.d. minibatches

An implicit assumption in Batch Normalization is that training examples are sampled independently, so that minibatch normalization statistics all follow roughly the same distribution and training statistics are faithfully represented in the moving averages. However, in applications where training batches are *not* sampled i.i.d., such as metric learning (Oh Song et al., 2016; Movshovitz-Attias et al., 2017) or hard negative mining (Shrivastava et al., 2016), violating this assumption may lead to undesired consequences in the model. Here, we test our approaches in this challenging setting.

Following Batch Renormalization (Ioffe, 2017), we study the case where examples in a minibatch are sampled from a small number of classes — specifically, we consider CIFAR-100, and study the extreme case where each minibatch ($B = 128$) is comprised of examples from only four random categories (sampled with replacement), each of which is represented with 32 examples in the minibatch. We present results for Batch Normalization, Batch Renormalization, our generalization of Batch and Group Normalization from Sec. 3.4 ("Batch Group Norm."), and the full interaction of Ghost Batch Normalization and Inference Example Weighing in Table 1. In this challenging setting, Inference Example Weighing, Ghost Batch Normalization, and Batch Group Norm all have large effect, in many cases halving the error rate of Batch Normalization. For example, Inference Example Weighing was able to reduce the error rate by 20% without any retraining, and tuning Ghost Batch Normalization, even without any inference modifications, was just as effective as Batch Renormalization, a technique partially designed for the non-i.i.d. case. Even further, Batch Group Normalization was hardly affected at all by the non-i.i.d. training distribution (76.1 vs 76.2 for i.i.d.). Last, it is interesting to note that Inference Example Weighing had practically no effect on Batch Renormalization (improvement $\leq 0.1\%$), confirming Batch Renormalization's effect in making models more robust to the use of training vs moving average normalization statistics.

5 Conclusion

In this work, we have demonstrated four improvements to Batch Normalization that should be known by all who use it. These include: a method for leveraging the statistics of inference examples more effectively in normalization statistics, fixing a discrepancy between training and inference with Batch Normalization; demonstrating the surprisingly powerful effect of Ghost Batch Normalization for improving generalization of models without requiring very large batch sizes; investigating the previously unstudied effect of weight decay on the scaling and shifting parameters γ and β; and introducing a new approach for normalization in the small batch setting, generalizing and leveraging the strengths of both Batch and Group Normalization. In each case, we have done our best to not only demonstrate the effect of the method, but also provide guidance and evidence for precisely which cases in which it may be effective, which we hope will aid in their applicability.

A PROOF OF BATCH NORMALIZATION OUTPUT BOUNDS

Here we present a proof of Eq. 2. We first prove the bound as an inequality and then show that it is tight. Without loss of generality, we assume that x_0 is the minimum of $\{x_i\}_{i=0}^{B-1}$ and that $\gamma \geq 0$. Then we want to show that

$$\min_{x_0,\ldots,x_{B-1}} \gamma \frac{x_0 - \mu_i}{\sqrt{\sigma_i^2 + \epsilon}} + \beta = -\gamma\sqrt{B-1} + \beta \tag{5}$$

Expanding μ_i and σ_i^2 (using the maximum likelihood estimator for σ_i^2), and canceling the scaling and offset terms γ and β, we want to show

$$\min_{x_0,\ldots,x_{B-1}} \frac{x_0 - \frac{1}{B}\sum_{i=0}^{B-1} x_i}{\sqrt{\frac{1}{B}\sum_{i=0}^{B-1}(x_i - \frac{1}{B}\sum_{j=0}^{B-1} x_j)^2 + \epsilon}} = -\sqrt{B-1} \tag{6}$$

From here we assume without loss of generality that $x_0 = 0$ – since the output of Batch Normalization is invariant to an additive constant on all x_i, we can subtract x_0 from all x_i and maintain the same value. We also assume that all $x_i \geq 0$, then frame the minimum as a bound

$$\frac{-\frac{1}{B}\sum_{i=0}^{B-1} x_i}{\sqrt{\frac{1}{B}\sum_{i=0}^{B-1}(x_i - \frac{1}{B}\sum_{j=0}^{B-1} x_j)^2 + \epsilon}} \geq -\sqrt{B-1} \tag{7}$$

$$-\frac{1}{B}\sum_{i=0}^{B-1} x_i \geq -\sqrt{\frac{B-1}{B}\sum_{i=0}^{B-1}\left(x_i - \frac{1}{B}\sum_{j=0}^{B-1} x_j\right)^2 + \epsilon} \tag{8}$$

$$-\frac{1}{B}\sum_{i=0}^{B-1} x_i \geq -\sqrt{\frac{B-1}{B}\sum_{i=0}^{B-1}\left(x_i^2 - \frac{2x_i}{B}\sum_{j=0}^{B-1} x_j + \frac{1}{B^2}\left(\sum_{j=0}^{B-1} x_j\right)^2\right) + \epsilon} \tag{9}$$

$$\sum_{i=0}^{B-1} x_i \leq \sqrt{\sum_{i=0}^{B-1}\left((B-1)Bx_i^2 - 2(B-1)x_i\sum_{j=0}^{B-1} x_j + \frac{B-1}{B}\left(\sum_{j=0}^{B-1} x_j\right)^2\right) + \epsilon} \tag{10}$$

$$\sum_{i=0}^{B-1} x_i \leq \sqrt{(B-1)B\sum_{i=0}^{B-1} x_i^2 - 2(B-1)\left(\sum_{i=0}^{B-1} x_i\right)^2 + (B-1)\left(\sum_{i=0}^{B-1} x_i\right)^2 + \epsilon} \tag{11}$$

$$\sum_{i=0}^{B-1} x_i \leq \sqrt{(B-1)B\sum_{i=0}^{B-1} x_i^2 - (B-1)\left(\sum_{i=0}^{B-1} x_i\right)^2 + \epsilon} \tag{12}$$

$$\left(\sum_{i=0}^{B-1} x_i\right)^2 \leq (B-1)B\sum_{i=0}^{B-1} x_i^2 - (B-1)\left(\sum_{i=0}^{B-1} x_i\right)^2 + \epsilon \tag{13}$$

$$B\left(\sum_{i=0}^{B-1} x_i\right)^2 \leq (B-1)B\sum_{i=0}^{B-1} x_i^2 + \epsilon \tag{14}$$

$$\left(\sum_{i=0}^{B-1} x_i\right)^2 \leq (B-1)\sum_{i=0}^{B-1} x_i^2 + \epsilon \tag{15}$$

Using the fact that $x_0 = 0$ and $\epsilon > 0$, it suffices to show

$$\left(\sum_{i=1}^{B-1} x_i\right)^2 \leq (B-1)\sum_{i=1}^{B-1} x_i^2 \tag{16}$$

With a change of variables, we have the more general

$$\left(\sum_{i=0}^{N-1} x_i\right)^2 \leq N \sum_{i=0}^{N-1} x_i^2 \tag{17}$$

$$\frac{1}{N^2}\left(\sum_{i=0}^{N-1} x_i\right)^2 \leq \frac{1}{N} \sum_{i=0}^{N-1} x_i^2 \tag{18}$$

$$\mathbb{E}[x]^2 \leq \mathbb{E}[x^2] \tag{19}$$

$$\mathbb{E}[x^2] - \mathbb{E}[x]^2 \geq 0 \tag{20}$$

which is simply an alternate form for the variance of x, which is always non-negative, completing the bound.

To show that the bound is tight, we can set $x_0 = 0$ and $x_i = a$ for all $i > 0$, where a is a non-negative constant:

$$\frac{-\frac{1}{B}\sum_{i=0}^{B-1} x_i}{\sqrt{\frac{1}{B}\sum_{i=0}^{B-1}(x_i - \frac{1}{B}\sum_{j=0}^{B-1} x_j)^2 + \epsilon}} \tag{21}$$

$$\frac{-\frac{1}{B}\sum_{i=1}^{B-1} a}{\sqrt{\frac{1}{B}\left(\frac{(B-1)^2}{B^2}a^2 + \sum_{i=1}^{B-1}(a - \frac{B-1}{B}a)^2\right) + \epsilon}} \tag{22}$$

$$\frac{-\frac{B-1}{B}a}{\sqrt{\frac{1}{B}\left(\frac{(B-1)^2}{B^2}a^2 + (B-1)a^2\left(1 - \frac{2(B-1)}{B} + \frac{(B-1)^2}{B^2}\right)\right) + \epsilon}} \tag{23}$$

$$\frac{-(B-1)a}{B\sqrt{\frac{a^2(B-1)}{B}\left(\frac{B-1}{B^2} + 1 - 2\frac{B-1}{B} + \frac{(B-1)^2}{B^2}\right) + \epsilon}} \tag{24}$$

$$\frac{-(B-1)a}{\sqrt{a^2(B-1)\left(\frac{B-1}{B} + B - 2B + 2 + \frac{(B-1)^2}{B}\right) + \epsilon}} \tag{25}$$

$$\frac{-(B-1)a}{\sqrt{a^2(B-1)\left(\frac{B^2-2B+1+B-1-B^2+2B}{B}\right) + \epsilon}} \tag{26}$$

$$\frac{-(B-1)a}{\sqrt{a^2(B-1) + \epsilon}} \tag{27}$$

As $a \to \infty$ (or if $\epsilon = 0$), then this approaches

$$\frac{-(B-1)a}{a\sqrt{(B-1)}} \tag{28}$$

which is simply

$$-\sqrt{(B-1)} \tag{29}$$

completing the proof.

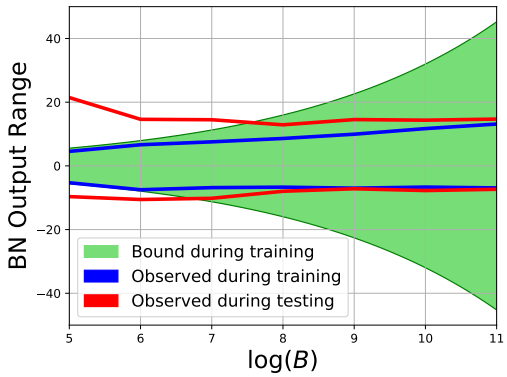


Figure 6: Range of output values obtained during inference on the CIFAR-10 test set, compared with the range observed during training and the bound of Eq. 2. See text for details.

B Empirical Evidence of Batch Normalization Output Bounds

In Fig. 6 we show the observed output ranges for the last Batch Normalization layer in our CIFAR-10 network (spatial resolution: 4×4), plotting both the range during training and at inference time on the CIFAR-10 test set. Different values of B were obtained by using different Ghost Batch Normalization sizes, keeping in mind that B is determined by the product of the batch size and spatial dimensions.

At large values of B, it is unlikely that any network obtains a value even close to the bound of Eq. 2, but as B gets smaller, the output range of the network during training becomes smaller in magnitude, eventually being nearly tight with our bound — for example, for $\log(B) = 5$, the theoretical minimum is -5.57, while the network obtained a minimum of -5.30. However, the maximum and minimum values obtained during inference on the test set show no clear pattern as B changes, and are not subject to the training time bound, which is particularly noticeable for small values of B, where values fall outside the training-time bounds.

C Negative Results: Approaches That Didn't Work.

Here we detail a handful of approaches which seemed intuitively promising but ultimately failed to produce positive results.

Batch Normalization Moving Averages. In an attempt to resolve the other disparities Batch Normalization has between its training and inference behaviors, we experimented with a handful of different approaches for modifying the moving averages used during inference. First, since examples at inference time do not have data augmentation applied to them, we tried computing the moving averages over examples without data augmentation (implemented by training the model for a few extra epochs over non-augmented examples with a learning rate of 0, but while still updating the moving average variables). This decreased accuracy on CIFAR-100 by roughly half a percent, though it did yield mild improvements to the test set cross-entropy loss.

Next, we experimented with calculating the moving averages over the *test set*, not making use of any of the test labels. Perhaps surprisingly, this behaved very similar to when moving averages were calculated over the training examples (within 0.1% in accuracy and within 1% in cross-entropy), with trends holding regardless of whether data augmentation was applied or not.

Adding Batch Normalization-like Stochasticity to Group Normalization. One of the hypotheses for why Group Normalization generally performs slightly worse than Batch Normalization is the regularization effect of Batch Normalization due to random minibatches producing variability in the normalization statistics. Therefore, we tried introducing stochasticity to Group Normalization in a variety of ways, none of which we could get to work well: 1) Adding gaussian

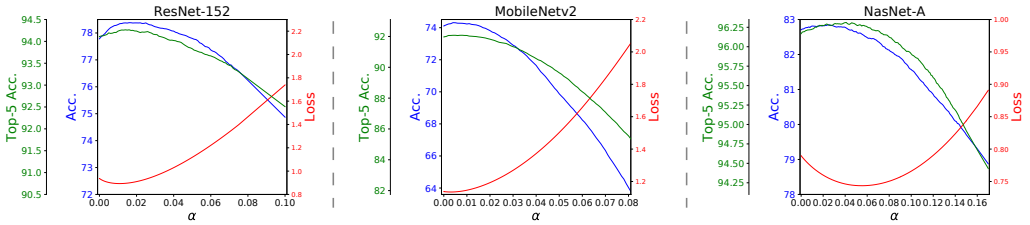


Figure 7: Effect of the example-weighing hyperparameter α on ImageNet; supplemental version of Fig. 1 with a larger range of α.

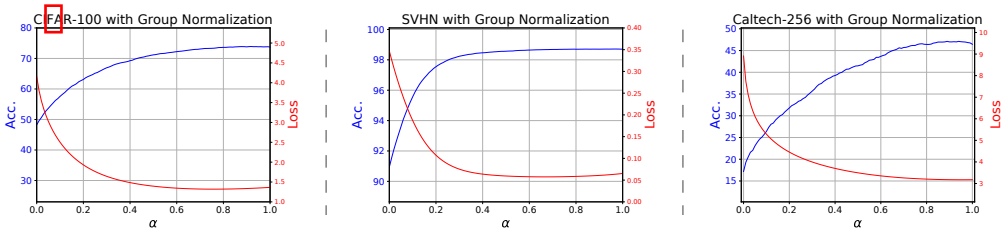


Figure 8: Effect of the example-weighing hyperparameter α for models trained with Group Normalization on CIFAR-100, SVHN, and Caltech-256; supplemental version of Fig. 2 with a larger range of α.

noise to the normalization statistics, where the noise is based on a moving average of the normalization statistics, 2) Using random groupings of channels for calculating normalization statistics (optionally only doing randomization a fraction of the time), and 3) changing the number of groups throughout the training procedure, either as increasing or decreasing functions of training steps.

MORE PRINCIPLED GROUP SIZE COMPUTATION. As part of generalizing Batch and Group Normalization, we examined whether it was possible to determine the number of groups in each normalization layer in a more principled way that simply specifying it as a constant throughout the network. For example, one approach we had mild success with was setting the number of elements per group (height $\times$ width $\times$ group size) to a constant, making the number of elements contributing to the normalization statistics uniform across layers. However, we were unable to get any of these ideas to work in a way that generalized properly across datasets. We also tried learning group sizes in a differentiable way with Switchable Normalization, but found that this made models overfit too much.

D SUPPLEMENTAL INFERENCE EXAMPLE WEIGHING PLOTS

In Figures 7, 8, and 9 we present plots corresponding to Figures 1, 2, and 4 of the main text, with larger ranges of the inference weight α. In the main text, we restricted the range of α to values which showed off the tradeoff of α versus performance at a reasonably local scale, and these figures show a larger scale for completeness in characterizing model behavior. While this behavior can largely be extrapolated from the behavior for a smaller range of α, there are some interesting trends.

On ImageNet 7, we see that only a small amount of inference example weighing is necessary to get most of its benefit, and setting α to larger values corresponds to a regime quite different than in training, smoothly decaying model performance as α becomes less and less appropriate. Similarly, when applying inference example weighing to Group Normalization (Fig. 8, while performance intuitively decays as α moves farther and farther away from 1, a surprisingly large range of values for α result in similar performance to Group Normalization, especially on SVHN. Lastly, when comparing the effect of α on models trained with Ghost Batch Normalization (Fig. 9, we clearly see that the optimal value for α is decreasing with respect to the Ghost Batch Normalization size, with the possible unusual exception of optimizing for loss on SVHN.

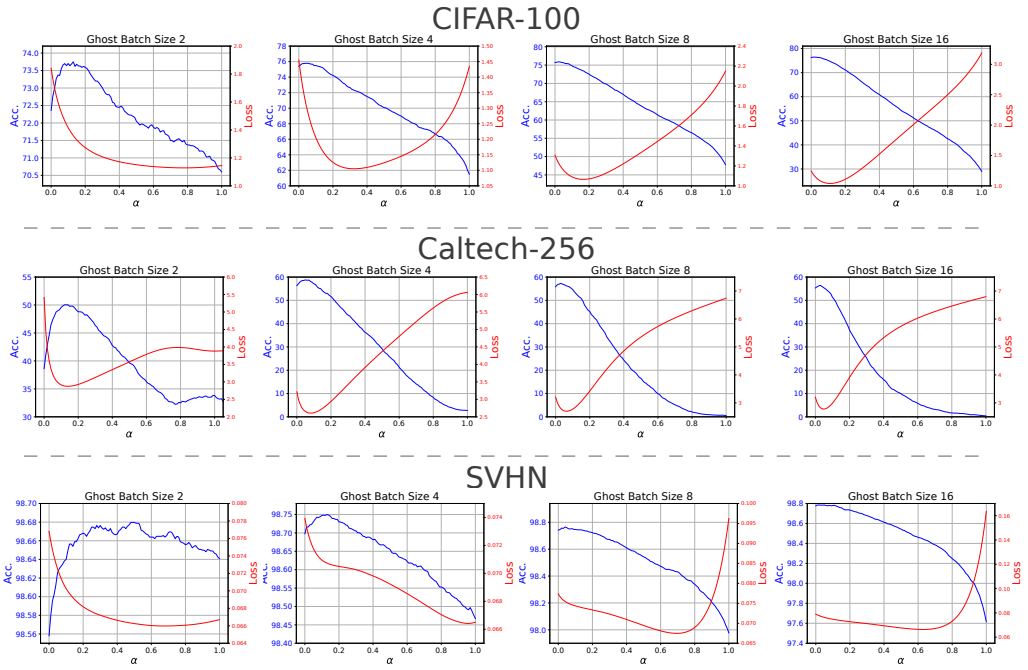


Figure 9: The complementary effects of Inference Example Weighing and Ghost Batch Normalization on CIFAR-100, SVHN, and Caltech-256; supplemental version of Fig. 4 with a larger range of α.

EvalNorm: Estimating Batch Normalization Statistics for Evaluation

Saurabh Singh
Google Research
saurabhsingh@google.com

Abhinav Shrivastava*
University of Maryland, College Park
abhinav@cs.umd.edu

Abstract

Batch normalization (BN) has been very effective for deep learning and is widely used. However, when training with small minibatches, models using BN exhibit a significant degradation in performance. In this paper we study this peculiar behavior of BN to gain a better understanding of the problem, and identify a cause. We propose 'EvalNorm' to address the issue by estimating corrected normalization statistics to use for BN during evaluation. EvalNorm supports online estimation of the corrected statistics while the model is being trained, and does not affect the training scheme of the model. As a result, EvalNorm can also be used with existing pre-trained models allowing them to benefit from our method. EvalNorm yields large gains for models trained with smaller batches. Our experiments show that EvalNorm performs 6.18% (absolute) better than vanilla BN for a batchsize of 2 on ImageNet validation set and from 1.5 to 7.0 points (absolute) gain on the COCO object detection benchmark across a variety of setups.

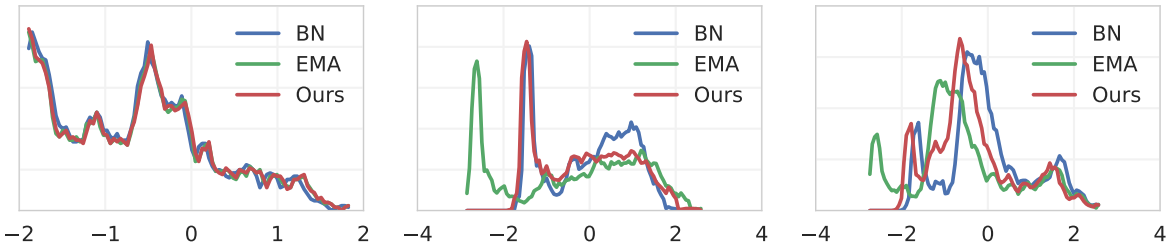
1. Introduction

Batch Normalization (BN) [11] has significantly contributed to the wide application and success of deep learning. It has enabled training of larger and deeper models [7, 8, 22, 23] leading to significant advances in computer vision. However, a well-acknowledged drawback of BN is that it requires sufficiently large minibatches [7, 10, 23], and results in drastically reduced model performance for smaller mini-batches. In this paper we study this peculiar behavior of BN to gain a better understanding of the source of the problem. We identify a cause and propose *EvalNorm* to address the issue. EvalNorm estimates corrected normalization statistics to use for BN during evaluation only. EvalNorm supports online estimation of the corrected statistics while the model is being trained and does not affect the training scheme of the model. As a result, EvalNorm can also be used with existing pre-trained models allowing them to benefit from our method without retraining. For pre-trained models EvalNorm suggests: 1) a rule of thumb that is trivial to implement and, 2) an offline estimation method that requires a pass through data (but doesn't update the model). The ability to estimate corrected statistics online for new models helps avoid an otherwise two step process. The model is still trained as it would be without our method. Note that proposing a new normalization technique is not a goal of this paper. Instead, the goal is to gain a better understanding of the behavior of BN for small minibatches. We limit ourselves to explorations that only affect the evaluation of a model trained with BN and refer to our method as 'Eval Normalization' (EvalNorm or EN).

Reliance of BN on large minibatches is prohibitive in several settings due to the hardware memory limitations. For example, applications requiring high-resolution inputs (object detection, segmentation, medical image analysis, etc.) or high-capacity (deeper and wider) networks are constrained to using smaller minibatches and thus take a performance hit. As a result, several normalization techniques, such as Group Normalization [25] and Batch Re-normalization [10], have been proposed to address the problem due to smaller minibatches. However, these approaches don't shed any light on the source of the problem with BN. Instead, they propose alternatives that make changes to the model and require retraining. As a result, many already trained and deployed models that use BN, where retraining is not possible, can not benefit from these alternatives. Our experimental evaluation of EN shows that it addresses the shortcomings of BN for small batches and provides a feasible alternative when retraining isn't an option.

Training and evaluation discrepancy in BN: During training, BN normalizes each channel for an example using the mean and variance of that channel aggregated across the full minibatch. This aggregation across the minibatch introduces a dependency on other minibatch samples. However, during evaluation each example is evaluated by itself and thus an approximation of the minibatch statistics is required. Typically, an exponential moving average (EMA) of minibatch moment statistics is maintained during training and used as a substitute during evaluation. Normalization using EMA

*Work done while at Google.



(a) 128 samples/minibatch (b) 2 samples/minibatch (c) 2 samples/minibatch

Figure 1: **Discrepancy in train vs. test distributions:** In each plot we show normalized activation distributions using different normalization operations and different normalization minibatch sizes for an arbitrary channel of a ResNet-20 trained on CIFAR-100. 'BN' denotes result of batch normalization during *training*, 'EMA' denotes normalization used during *evaluation* using EMA, and 'Ours' is using our EvalNorm adjustment during *evaluation*. Notice that for larger minibatches (128 samples) in (a), all three normalization operation result in similar distributions. However, for small minibatches (2 samples) in (b) and (c), 'Ours' using test time EN is much closer to the train time 'BN' than the standard 'EMA' statistics.

statistics is assumed to provide an accurate approximation to normalization observed during training. While reasonable for larger minibatches, we demonstrate that this assumption is erroneous for small minibatches (Section 3.2). This is because the mean and variance used to normalize a sample in a minibatch during training depend on that sample itself. For small minibatches this dependency is significant but ignored by the default method of using EMA. We qualitatively illustrate the discrepancy in train and test normalization in Figure 1, where we plot the distribution of normalized features at train ('BN') and test ('EMA') time. Each plot shows smoothed histograms of activations for an arbitrary channel of an arbitrary layer (in a ResNet-20 model) using different methods. For larger minibatches (Figure 1a), we observe that the distribution of normalized activations during training and testing match well. However, for small minibatches (Figures 1b and 1c), normalized feature distributions are quite different. Lastly, note that our method ('Ours') reduces this discrepancy and brings the distribution of normalized activations during testing closer to that during training ('BN').

To summarize our contributions, we: 1) quantify the dependence of the normalization statistics used by BN on a particular minibatch sample leading to an insight into the source of poor small minibatch performance, 2) propose two methods for estimating a correction without retraining existing models that use BN, and 3) propose a method to estimate corrections online during training of new models without affecting the training. We have included an extensive experimental section that analyzes and validates our insight and demonstrates that our insight leads to an improved evaluation performance of BN on a variety of common benchmarks.

2. Related work

Normalization of data for training is regularly used in machine learning. For example, it's a common practice to normalize features (scale or whiten) before learning classifiers like SVM. Similarly, for deep networks, many techniques have been proposed to normalize both inputs and intermediate representations, which make the training better and faster [6, 13]. Batch Normalization or BatchNorm (BN) is one such technique which aims to stabilize latent feature distributions in a deep network. BN normalizes features using the statistics computed from a minibatch; and has been shown to ease the learning problem and enable fast convergence of very deep network architectures. However, with BN's widespread adoption, it has been observed that models using BN exhibit severe degradation in performance when trained with smaller minibatches (as discussed in Section 1).

To address the stochasticity due to small minibatches and bias due to non-iid samples, Ioffe [10] introduced batch renormalization (Renorm) which constrains the minibatch moments to a specific range. This limits the variation in minibatch statistics during training. Ioffe [10] also introduced hyperparameters to prevent drift in the EMA statistics due to the variability of small minibatches. However, Renorm is still dependent on minibatch statistics with small minibatches, leading to worse performance. For tasks where small minibatches are standard (e.g., object detection), another approach is to engineer systems that can circumvent the issue. Peng et al. [17] proposed to perform synchronized computation of BN statistics across GPUs (Cross-GPU BN) to obtain better statistics. However, since images/GPU for object detection is small (1-2 images), this approach requires $\sim$128 GPUs to compute reasonable BN estimates. Further, this does not address the original problem of BN with small minibatches. Moreover, the need for synchronized computation prohibits the use of asynchronous training, which is a standard and practical tool for large-scale problems.

Instead of dealing with the small minibatch problem, several normalization techniques have been proposed that do not utilize the construct of a 'minibatch.' Instance Normalization [24] performs normalization similar to BN but only for a single sample and was shown to be effective on image style

transfer applications. Similarly, Layer Normalization [2] utilizes the entire layer (all channels) to estimate the normalization statistics. These approaches [2, 24] have not shown benefits on image recognition tasks, which is the application we focus on. Instead of normalizing the activations, Weight Normalization [20] reparameterizes the weights in the neural network to accelerate convergence. Normalization Propagation [1] uses data independent moment estimates in every layer, instead of computing them from minibatches during training. Group Normalization (GN) [25] divides the channels into groups and, within each group, computes the moments for normalization. GN alleviates the small minibatch problem to some extent, but it performs worse than BN for larger minibatches. Ren et al. [18] provide a unifying view of the different normalization approaches by characterizing them as the same transformation but along different dimensions (layers, samples, filters, etc.).

Unlike many approaches discussed above, the proposed EN does not modify the training scheme of batch normalization. It only estimates a different set of evaluation time statistics. The parameters used in EN are independent of the deep network, and training the parameters does not impact the network's training in any way. We conjecture that EN may be complementary to some of the above-mentioned approaches and may be used in conjunction with them to normalize across batch dimension. However, we consider this beyond the focus of this study.

3. Eval Normalization (EN)

We first briefly describe the relevant aspects of batch normalization and then present our method.

3.1. Batch Normalization (BN)

BN normalizes a particular channel of a sample in a minibatch using the mean and variance of that channel computed across the whole minibatch. Since the normalization of the channels is decoupled, we analyze the normalization of a single (but arbitrary) channel. Consider the set of activations $\mathcal{B} = \{\boldsymbol{x_1}, \ldots, \boldsymbol{x_B}\}$ of a particular channel in an arbitrary layer for a minibatch of size B. Typically, $\boldsymbol{x_i}$ is a two dimensional field of scalars. During training, BN uses the minibatch mean $\mu_\mathcal{B}$ and variance $\sigma^2_\mathcal{B}$ to compute the normalized activations $\hat{\boldsymbol{x}}_\boldsymbol{i}$ as

$$\hat{\boldsymbol{x}}_\boldsymbol{i} = \frac{(\boldsymbol{x_i} - \mu_\mathcal{B})}{\sigma_\mathcal{B}}. \tag{1}$$

This has two notable ramifications during training: 1) For activations $\boldsymbol{x_i}$ the normalization statistics $\mu_\mathcal{B}$ and $\sigma^2_\mathcal{B}$ are a function of the statistics of activations $\boldsymbol{x_i}$ themselves, 2) the normalized activations $\hat{\boldsymbol{x}}_\boldsymbol{i}$ for a particular sample are stochastic as they depend on the statistics of the other samples in a stochastic minibatch.

During evaluation randomness of $\hat{\boldsymbol{x}}_\boldsymbol{i}$ due to stochastic dependency on other minibatch elements poses a difficulty for BN. To keep the evaluation deterministic and remove the dependency on other test samples BN substitutes $\hat{\boldsymbol{x}}_\boldsymbol{i}$ by an estimate of its expected value $\mathbb{E}[\hat{\boldsymbol{x}}_\boldsymbol{i}]$. This is done by treating $\mu_\mathcal{B}$ and $\sigma^2_\mathcal{B}$ encountered during training as random variables and substituting them by an estimate of their expected values as $\mu_\mathcal{E} \approx \mathbb{E}[\mu_\mathcal{B}]$ and $\sigma^2_\mathcal{E} \approx \mathbb{E}[\sigma^2_\mathcal{B}]$ to construct a first order approximation of $\mathbb{E}[\hat{\boldsymbol{x}}_\boldsymbol{i}]$ as

$$\mathbb{E}[\hat{\boldsymbol{x}}_\boldsymbol{i}] \approx \frac{(\boldsymbol{x_i} - \mathbb{E}[\mu_\mathcal{B}])}{\sqrt{\mathbb{E}[\sigma^2_\mathcal{B}]}} \approx \frac{(\boldsymbol{x_i} - \mu_\mathcal{E})}{\sqrt{\sigma^2_\mathcal{E}}}. \tag{2}$$

The estimates $\mu_\mathcal{E}$ and $\sigma^2_\mathcal{E}$ are typically maintained as exponential moving averages (EMA) during training.

Note that BN also employs a learned affine transform after normalization. We omit this affine transform in the text as it is not relevant to the discussion of normalization.

3.2. Source of stochasticity in $\hat{\boldsymbol{x}}_\boldsymbol{i}$

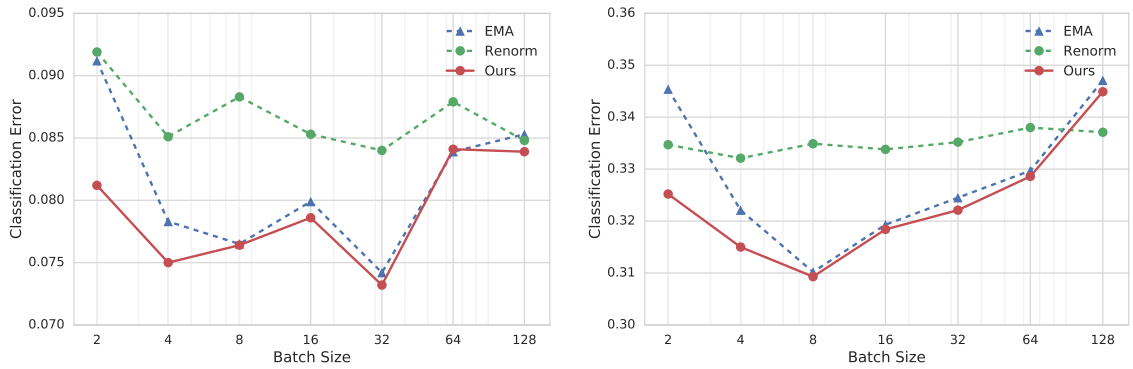
As noted earlier, $\mu_\mathcal{B}$ and $\sigma^2_\mathcal{B}$ are a function of $\boldsymbol{x_i}$. However, eq. (2) ignores this dependency entirely. Let us first make this dependency explicit. Let μ_i and σ^2_i be the mean and variance respectively of $\boldsymbol{x_i}$. Denote $\mathcal{C} = \mathcal{B} - \boldsymbol{x_i}$ as the set of $B-1$ mini-batch elements excluding $\boldsymbol{x_i}$ with combined mean $\mu_\mathcal{C}$ and variance $\sigma^2_\mathcal{C}$. The full minibatch mean $\mu_\mathcal{B}$ and variance $\sigma^2_\mathcal{B}$ can be expressed in terms of the above with the combined population mean and variance formulae. Let $\alpha = 1/B$, then

$$\mu_\mathcal{B} = \alpha\mu_i + (1-\alpha)\mu_\mathcal{C} \tag{3}$$

$$\sigma^2_\mathcal{B} = \alpha\sigma^2_i + (1-\alpha)\sigma^2_\mathcal{C} + \alpha(1-\alpha)(\mu_i - \mu_\mathcal{C})^2. \tag{4}$$

First note that $0 < \alpha \leqslant 1$, since $B \geqslant 1$. Therefore, α can be viewed as interpolating between the contributions from the sample being normalized and the other minibatch samples. For normalized activations $\hat{\boldsymbol{x}}_\boldsymbol{i}$ it is evident that the true source of stochasticity are $\mu_\mathcal{C}$ and $\sigma^2_\mathcal{C}$ due to randomized minibatch. Therefore, we assert that $\mathbb{E}[\hat{\boldsymbol{x}}_\boldsymbol{i}]$ in eq. (2) for the minibatch sample $\boldsymbol{x_i}$ should be approximated using eqs. (3) and (4) as opposed to approximating $\mu_\mathcal{B}$ and $\sigma^2_\mathcal{B}$ directly.

Note that for large minibatches $\alpha \approx 0$, resulting in a nominal contribution of μ_i and σ^2_i to the normalizing statistics (i.e., $\mathbb{E}[\mu_\mathcal{B}] \approx \mathbb{E}[\mu_\mathcal{C}]$ and $\mathbb{E}[\sigma^2_\mathcal{B}] \approx \mathbb{E}[\sigma^2_\mathcal{C}]$). Therefore, $\mathbb{E}[\mu_\mathcal{B}] \approx \mathbb{E}[\mu_\mathcal{C}] \approx \mu_\mathcal{E}$ and $\mathbb{E}[\sigma^2_\mathcal{B}] \approx \mathbb{E}[\sigma^2_\mathcal{C}] \approx \sigma^2_\mathcal{E}$ (that is, using the EMA statistics for normalization) are reasonable approximations. However, for small minibatches the contributions of μ_i and σ^2_i in eqs. (3) and (4) can not be ignored and these approximations are not accurate. We argue that this inaccuracy is the primary reason for observed distribution mis-matches between BN and EMA in Figure 1, and poor performance of BN for small minibatches.



(a) CIFAR-10 (b) CIFAR-100

Figure 2: **Classification error on CIFAR-10 and CIFAR-100**. EN consistently outperforms both EMA and Renorm for all minibatch sizes. Notice that the error does not decrease monotonically with increasing minibatch sizes. One intuition behind this is that at the inflection point, the stochasticity of the estimates act as a good regularizer (similar trends were observed by Masters and Luschi [16]).

3.3. Approximating $\mathbb{E}[\hat{x}_i]$ for small minibatches

Simple approximation: At first glance a simple solution would be to directly use $\alpha = 1/B$ and approximate $\mathbb{E}[\mu_\mathcal{B}]$ and $\mathbb{E}[\sigma^2_\mathcal{B}]$ using eqs. (3) and (4). More specifically, $\mathbb{E}[\mu_\mathcal{B}] \approx \alpha\mu_i + (1-\alpha)\mu_\mathcal{E}$ and $\mathbb{E}[\sigma^2_\mathcal{B}] \approx \alpha\sigma^2_i + (1-\alpha)\sigma^2_\mathcal{E} + \alpha(1-\alpha)(\mu_i - \mu_\mathcal{E})^2$. These can then be substituted in eq. (2) for normalization. Note that we have made the additional approximations of $\mathbb{E}[\mu_\mathcal{C}] \approx \mu_\mathcal{E}, \mathbb{E}[\sigma^2_\mathcal{C}] \approx \sigma^2_\mathcal{E}$. However, for small minibatches $\mu_\mathcal{C}$ and $\sigma^2_\mathcal{C}$ exhibit high variance and these approximations may be inaccurate. Unfortunately, higher order approximations to account for the variance either require tracking higher order moments, whose estimates would themselves be unreliable, or making strong assumptions about the distributions of $\boldsymbol{x_i}$. Nevertheless, we explore this alternative in experiments where it turns out that $\alpha = 1/B$ is less than ideal. Experimenting with a few heuristic choices for the value of α leads to a simple *rule-of-thumb* of $\alpha = 1/B^2$ (Figure 4 and Table 1) that is found to work well empirically.

Estimated approximation: Instead of using a fixed α, we propose to turn α into an estimated variable. We instantiate two separate copies of α, namely $\hat{\alpha}$ and $\hat{\beta}$, and construct the following approximations to eqs. (3) and (4)

$$\mathbb{E}[\mu_\mathcal{B}] \approx \hat{\alpha}\mu_i + (1-\hat{\alpha})\mu_\mathcal{E} \tag{5}$$

$$\mathbb{E}[\sigma^2_\mathcal{B}] \approx \hat{\beta}\sigma^2_i + (1-\hat{\beta})\sigma^2_\mathcal{E} + \hat{\beta}(1-\hat{\beta})(\mu_i - \mu_\mathcal{E})^2. \tag{6}$$

Our method estimates $\hat{\alpha}$ and $\hat{\beta}$ (Section 3.4) such that normalized activations using above approximations match the normalized activations using minibatch statistics. Note that, decoupled $\hat{\alpha}$ and $\hat{\beta}$ lead to slightly better empirical results than a single value.

3.4. Estimating $\hat{\alpha}$ and $\hat{\beta}$

Offline estimation: Given a trained model, we propose to estimate $\hat{\alpha}$ and $\hat{\beta}$ for a particular layer by minimizing an auxiliary loss $\mathcal{L}_\text{aux}$ as below. Let $\langle\cdot\rangle$ represent a `stop_gradient` operation that does not allow flow of gradients to its argument in automatic differentiation frameworks. Then $\mathcal{L}_\text{aux}$ is computed as

$$\hat{\mu} = \hat{\alpha}\langle\mu_i\rangle + (1-\hat{\alpha})\mu_\mathcal{E} \tag{7}$$

$$\hat{\sigma}^2 = \hat{\beta}\langle\sigma_i\rangle^2 + (1-\hat{\beta})\sigma^2_\mathcal{E} + \hat{\beta}(1-\hat{\beta})(\langle\mu_i\rangle - \mu_\mathcal{E})^2 \tag{8}$$

$$\mathcal{L}_\text{aux} = \left\| \left\langle \frac{\boldsymbol{x_i} - \mu_\mathcal{B}}{\sigma_\mathcal{B}} \right\rangle - \frac{\langle \boldsymbol{x_i} \rangle - \hat{\mu}}{\hat{\sigma}} \right\|_1 \tag{9}$$

Minimizing this objective allows us to estimate $\hat{\alpha}, \hat{\beta}$ such that evaluation time normalization produces activations that are similar to those observed using minibatch statistics for normalization. Further, the `stop_gradient` operation prevents the estimation from affecting the model parameters.

Online estimation: For training a new model a naïve approach would be to first train the model as usual with BN and then estimate $\hat{\alpha}$ and $\hat{\beta}$ in a second pass while freezing rest of the parameters using $\mathcal{L}_\text{aux}$ above. However, use of `stop_gradient` as above allows us to collapse the two steps into a single one without affecting the training of the model parameters.

4. Experiments

We evaluate our approach on two tasks and four datasets: image classification on CIFAR-10, CIFAR-100 [12], and ImageNet [3], and object detection on COCO [14]. We use CIFAR-100 as a test-bed to perform ablation experiments and study various aspects of our method. The results on ImageNet and COCO demonstrate the utility of using EvalNorm

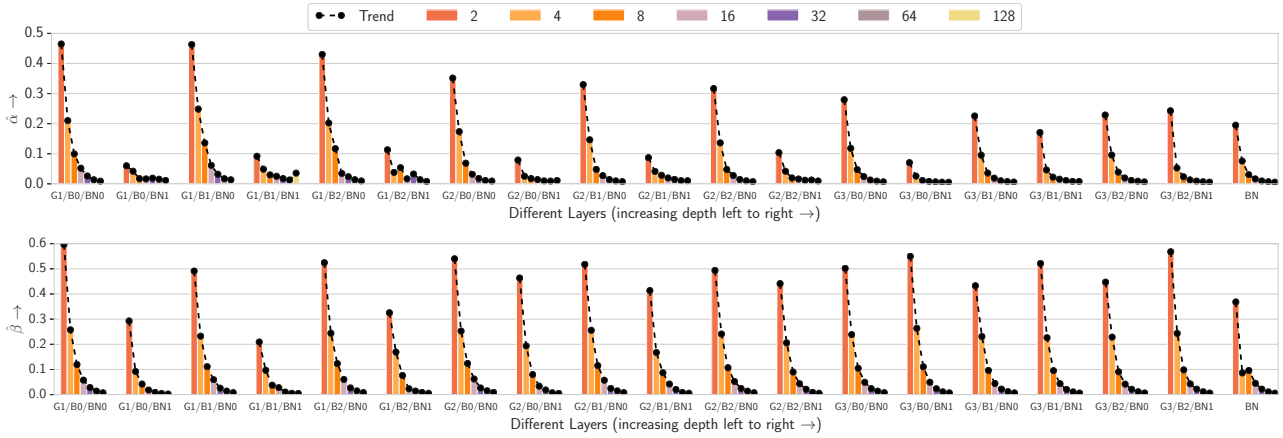


Figure 3: Estimated $\hat{\alpha}$ (top) and $\hat{\beta}$ (bottom) values for different batchsizes on CIFAR-100. Each set of bars corresponds to a BN operation (e.g. G1/B0 represents group 1, block 0; BN0 and BN1 represent first and second BN operation) and each color corresponds to a batch size (increasing left to right in a set). Both $\hat{\alpha}$ and $\hat{\beta}$ take larger values for smaller batchsizes indicating that individual statistics need to be included in the normalization statistics and weighted in accordance with the batchsize.

(EN), especially for models trained with small mini-batches. Note that, unless explicitly stated, $\hat{\alpha}$ and $\hat{\beta}$ in all the experiments are estimated online for ease of experimentation.

4.1. Image Classification on CIFAR-10/100

Experimental setup. CIFAR-10 and CIFAR-100 contain images from 10 and 100 classes respectively. For both CIFAR-10 and CIFAR-100, we train on the 50000 training images and evaluate on the 10000 test images. Unless specified otherwise, we use a 20 layer ResNetv2 CIFAR variant [8] for both datasets. The base CIFAR variant contains three groups of residual blocks with widths $\{16, 32, 64\}$ respectively. Wider variants use multiples of these sizes. All networks are trained using SGD with momentum for 128k updates, with an initial learning rate of 0.1 and a cosine decay schedule [15] unless otherwise specified.

Small minibatches. In this section we study the effect of minibatch size used to compute the normalization statistics. For fair comparison all models need to be trained for the same number of epochs. However, this leads to smaller minibatch models getting more gradient updates. Moreover, gradients from small minibatches have higher variance in comparison to larger minibatches requiring an adjustment in learning rate. To isolate the impact of small minibatches on normalization from these confounding factors, we use the same minibatch size (128 samples) for gradient updates and vary the number of samples used for normalization (from 2 to 128) (refer to the "microbatch" setup in [10]).

Figures 2a and 2b compare our method (EN) with the default EMA statistics used by standard BN [11] and batch re-normalization (Renorm) [10]. For Renorm, we set $r_{\max} = 2.0$ and $d_{\max} = 1.0$. For larger minibatch sizes (64 and 128),

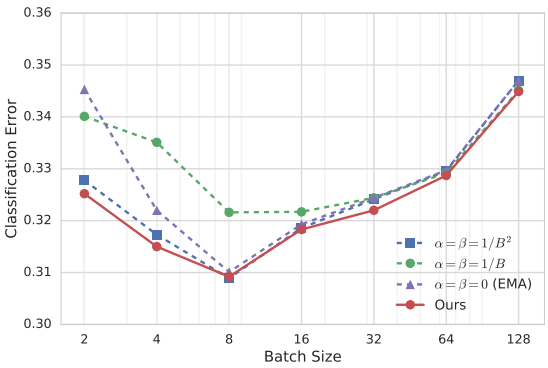
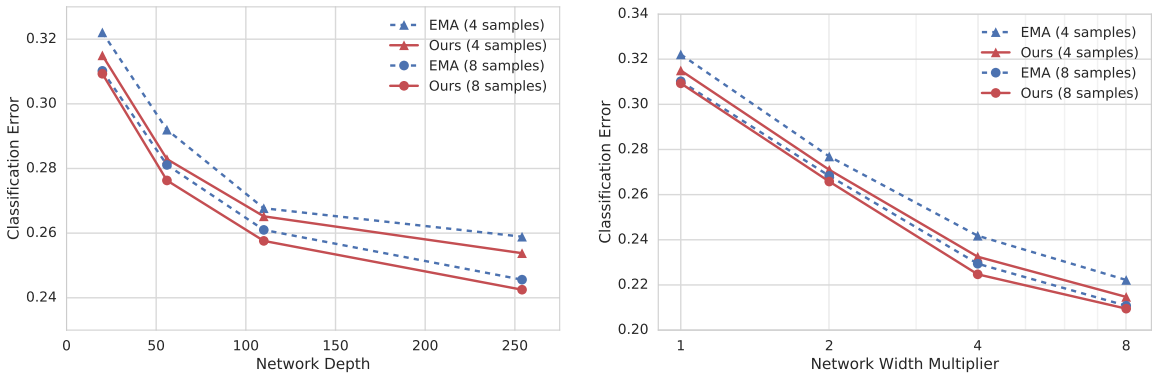


Figure 4: Comparison of a set of hand picked schedules for $\hat{\alpha}, \hat{\beta}$ as a function of batch size B. Although, the estimated values using our method perform the best, $\hat{\alpha} = \hat{\beta} = 1/B^2$ appears to be a good rule of thumb.

we notice that all methods are comparable, but for smaller minibatch sizes (2 and 4) using EN statistics leads to a lower classification error compared to both BN and Renorm. Even though Renorm is less sensitive to varying minibatch sizes, it also lead to worse performance.

Interestingly, the performance of BN and EN does not decrease monotonically with the decreasing minibatch size used for normalization. Instead, it seems to improve before becoming worse. Similar trends were reported by [16]. In depth study of this is beyond the scope of this paper (refer to [16] for an empirical study). One intuition is that smaller minibatches introduce stochasticity that acts as a regularizer before it starts to hurt performance.

Visualization of estimated $\hat{\alpha}, \hat{\beta}$. Figure 3 visualizes the estimated $\hat{\alpha}, \hat{\beta}$ across different layers for various batch sizes.



(a) Depth (b) Width

Figure 5: Performance of EN for deeper and wider network variants on CIFAR-100. EN consistently leads to lower classification error compared to EMA statistics.

Larger value of $\hat{\alpha}, \hat{\beta}$ for smaller batches confirms our insight that statistics of the sample being normalized need to be included and weighted in accordance with batchsize. Both $\hat{\alpha}, \hat{\beta}$ follow the trend of taking on smaller values for larger batches indicating a decreasing influence of individual statistics for larger batches in agreement with eqs. (3) and (4).

Simple approximation for $\hat{\alpha}, \hat{\beta}$. Figure 4 compares a set of hand picked schedules for $\hat{\alpha}, \hat{\beta}$ as a function of batch size B. Although, the estimated values using our method perform the best, $\hat{\alpha} = \hat{\beta} = 1/B^2$ appears to be a good rule of thumb. Note that this is counter intuitive to the more natural seeming $\hat{\alpha} = \hat{\beta} = 1/B$. We also validate this on ImageNet in Table 1.

Deeper and wider networks. Figure 5 compares EN and EMA in deeper (20, 56, 110, and 254 layers) and wider (1, 2, 4, and 8 $\times$ the original width) ResNetv2 models, with normalization minibatch size of 4 and 8 on CIFAR-100. EN consistently leads to lower classification error compared to using EMA statistics indicating its general applicability.

4.2. Image Classification on ImageNet

We report results on the ImageNet classification dataset [3] with 1000 classes. We train on $\sim$1.28M training images and evaluate on 50k validation images. All methods in this section use a 50 layer ResNetv2 [8] network architecture. We resize all images to 229$\times$229 and use training time data augmentation from [23].

As is standard practice [7, 8], we use synchronous SGD across 8 GPUs to train all models in this section. We compute the BN and EN statistics per GPU, whereas gradients are averaged across all GPUs. Therefore, the effective minibatch size for gradient computation (SGD 'batchsize') is 8$\times$ the per GPU normalization minibatch size ('samples/GPU'). This is the standard synchronized multi-GPU training setup in popular libraries, such as PyTorch and Tensorflow.

We study normalization minibatch sizes (samples/GPU) of 2, 4, 8, 16, and 32, leading to an effective SGD batchsize (N) of 16, 32, 64, 128, and 256 respectively. All models are trained for 60 epochs with an initial learning rate of $0.1 \times N/256$ and a cosine decay schedule. Other implementation details (such as initialization) follows [8]. We report results on two image classification metrics: 'Prec@1' and 'Recall@5'. Prec@1 measures the classification accuracy of the top-1 prediction, and Recall@5 measures the recall accuracy for top-5 predictions.

We evaluate using EN and EMA statistics for inference time normalization and report the results in Table 1. We observe that using EN statistics *consistently* perform better than EMA statistics across all minibatch sizes. For larger minibatches, where EMA statistics provide an accurate approximation, the difference in performance is marginal. However, for small minibatches EN statistics have a significant impact. For example, for 2 samples/GPU, Prec@1 metric improves by more than 8 points and Recall@5 metric improves by 6.27 points. Also note that the performance gap between small and large minibatch size is much lower with EN. For example, reducing samples/GPU from 32 to 4, Prec@1 for EMA drops from 75.98 to 71.40 (-4.58 points), whereas for EN, it only drops by 2.7 points.

Next, in Table 1 we also compare our method with two recent normalization methods namely Group normalization [25] and Batch renormalization [10] that circumvent minibatch dependence. Note that these methods change the model or the training method itself. EN performs similar to or better than alternatives that normalize across batch (EMA and Renorm), indicating that it properly accounts for individual sample contributions. For large batch sizes EN outperforms other methods while for small batch sizes Group normalization tends to perform better. However, the gain from retraining using GroupNorm is reduced from 10% to 3.75% (for batchsize 2), and EN does not suffer the side

Table 1: **ImageNet classification results** for ResNet-50 [8] using EMA and EN statistics for different normalization minibatch sizes (samples/GPU). We show the classification accuracy for top-1 prediction (Prec@1) and the recall for top-5 prediction (Recall@5). Δ represents improvement due to EN over EMA. EN consistently performs better than EMA across all minibatch sizes, and provides significant gains for small minibatches. See section 4.2 for details.

	samples/GPU →	32	16	8	4	2
Prec@1	Renorm	76.04	75.78	74.21	73.33	70.87
	Groupnorm	75.87	75.73	75.75	75.61	74.78
	EMA	75.98	75.26	73.68	71.40	64.80
	EN [**ours**]	76.20	75.89	74.87	73.49	70.98
	Δ	+0.22	+0.63	+1.19	+2.09	**+6.18**
	$\alpha = \beta = 1/B^2$ [**ours**]	76.28	75.77	74.52	73.17	70.12
	EN-Offline [**ours**]	76.18	75.85	74.92	73.48	71.03
Recall@5	Renorm	92.81	92.65	92.18	91.68	90.03
	Groupnorm	92.59	92.31	92.28	92.10	91.81
	EMA	92.88	92.61	92.17	90.70	86.13
	EN [**ours**]	92.97	92.78	92.38	91.75	90.18
	Δ	+0.09	+0.17	+0.21	+1.05	**+4.05**
	$\alpha = \beta = 1/B^2$ [**ours**]	92.96	92.81	92.17	91.46	89.51
	EN-Offline [**ours**]	92.93	92.79	92.30	91.81	90.25

effect of reduced performance for large batch sizes.

Table 1 also reports the performance using the suggested rule-of-thumb ($\alpha = \beta = 1/B^2$) as well as offline estimation. Rule-of-thumb significantly improves over EMA but underperforms EN. Nevertheless, it is an attractive and easier alternative to estimation at the cost of performance. Offline estimation performs similar to online estimation as expected.

4.3. Object Detection on COCO

Finally, we evaluate our method on the task of object detection, and demonstrate consistent improvements when using EN statistics as opposed to EMA. Object detection frameworks are typically trained with high resolution inputs, and hence use small SGD minibatch sizes. As a result object detection is a good benchmark to evaluate the differences between EN and EMA.

Experimental setup. All experiments in this section use the COCO dataset [14] with 80 object classes. We train using the trainval35k set [4] with ∼75k images and evaluate on a held out minival set [4] with 5k images. We report the standard COCO evaluation metrics for mean average prevision with varying IoU thresholds (AP, AP^{50}, AP^{75}) (see [14] for details).

We use the Faster R-CNN (FRCN) [9, 19] object detection framework, built with a 50 layer ResNetv1 [7] (ResNet-50). The FRCN system has three components: 1) backbone feature extractor (base network), which operates on high resolution images (we resize images to 600×600 for all experiments), 2) region proposal network (RPN), which proposes regions of interest (ROIs), and 3) region classification network (RCN), which classifies regions proposed by RPN using cropped features from the base network. All but the last residual blocks from ResNet-50 are used as the base network and the last residual block is used in the RCN network. Refer to [9, 19, 21] for architecture details.

We study the SGD minibatch sizes (images/GPU or N) of 2, 4, and 8. As is standard practice [9], we use minibatch size of 300 and $64N$ ROIs for the RPN and RCN respectively. Note that this results in different normalization minibatches for different BN layers in the network (N for base network and $64N$ for RCN). All models are trained for 64 epochs (in terms of images and not ROIs) with an initial learning rate of $0.015 \times N/8$ and a cosine decay schedule. All methods use asynchronous SGD with 11 parallel GPU workers. We use the publicly released code from [9].

We investigate two different training paradigms: training from a random initialization and transfer learning (or fine-tuning). The random initialization setup is similar to the experiments reported on image classification in Sections 4.1 and 4.2. The transfer learning setup is more common in practice because it generally leads to higher performance. Next, we discuss the trade-offs and training flexibility in each paradigm and report results in Table 2 (blocks (a)-(d)).

Random initialization. We study the impact of EN by training a ResNet-50 Faster R-CNN model with all parameters initialized randomly. The results for using both EMA and EN statistics are reported Table 2(a). We observe that when using 4 and 8 images/minibatch, both methods are on-par; but when training with 2 images/minibatch, we see consistent and significant gains of more than 1.5 points on all AP metrics. Note that in this setup, the SGD and normalization minibatch sizes for the base network are same; unlike the ImageNet setup, where we average gradients across 8 GPUs resulting in SGD minibatch size being 8× the normalization minibatch size.

Transfer learning. The standard paradigm of training object detection models (like Faster R-CNN) is to use transfer learning or fine-tuning [4, 5, 9, 19]. In this setup, parameters from a model trained on ImageNet classification are transferred to the detection model, which is then trained on object detection. We use the model checkpoint from [7] as is standard practice. The transfer learning paradigm allows us to study the impact of EN statistics compared to EMA in a variety of settings.

First, we investigate which method is able to better estimate BN statistics in the absence of a prior. For this, we use the ImageNet trained model to initialize the network parameters (except BN parameters), but we use initial values of BN parameters from [11]; and both sets of parameters

Table 2: **Object detection results on COCO** using Faster R-CNN [19] with ResNet50 [7]. We report EMA and EN performance for different minibatch sizes (images/batch). EN performs better across all AP metrics and all minibatch sizes, specially for 2 images/minibatch, where we see significant gains. Different blocks (a)-(d) are described in Section 4.3. ($\Delta > 1$ point are shown in bold)

	Network Weights		BN EMA Stats			AP			AP^{50}			AP^{75}		
	Init	Train?	Init	Train?	images/batch →	8	4	2	8	4	2	8	4	2
(a)	Random	✓	Random	✓	EMA	25.2	25.6	22.0	40.9	41.2	36.1	26.7	27.5	22.4
					EN [**ours**]	25.2	25.8	23.5	41.0	41.3	38.1	26.8	27.6	25.0
					Δ	0.0	+0.1	**+1.5**	+0.1	+0.1	**+2.0**	+0.1	+0.1	**+2.6**
(b)	ImageNet	✓	Random	✓	EMA	30.4	29.9	27.6	48.4	47.2	43.5	32.8	31.9	29.6
					EN [**ours**]	30.7	30.5	29.6	48.5	47.9	46.5	33.0	32.8	31.3
					Δ	+0.2	+0.6	**+2.0**	+0.1	+0.7	**+3.1**	+0.2	+0.9	**+1.7**
(c)	ImageNet	✓	ImageNet	×	EMA	27.5	28.8	28.2	45.1	47.3	46.6	28.9	30.5	30.5
					EN [**ours**]	30.2	30.3	30.5	48.2	48.6	48.7	32.2	32.1	32.7
					Δ	**+2.7**	**+1.5**	**+2.3**	**+3.1**	**+1.3**	**+2.1**	**+3.3**	**+1.6**	**+2.2**
(d)	ImageNet	✓	ImageNet	✓	EMA	31.7	30.1	24.8	50.6	47.6	40.0	33.9	32.2	26.1
					EN [**ours**]	31.9	31.6	30.2	50.7	49.6	46.9	34.1	34.0	31.0
					Δ	+0.2	**+1.5**	**+5.4**	+0.2	**+2.1**	**+7.0**	+0.2	**+1.8**	**+4.9**

are trained simultaneously. We report the results for EN and EMA in Table 2(b). We notice that EN is consistently better than EMA across all minibatch sizes and AP metrics. In fact, as opposed to random initialization, we see gains for both 2 and 4 images/minibatch, with the improvements for the smaller minibatch being much higher.

Next, we study the standard setup [9, 19] of initializing both, the network and the BN EMA parameters, using the ImageNet model. Since standard BN performs poorly with smaller minibatches [7, 10, 25], the BN parameters are not updated during training in this setup. The results for this setup are reported in Table 2(c). Notice that EN performs significantly better than EMA across all minibatch sizes and AP metrics. EN allows adjustment of statistics by using the learned features (as opposed to 'stale' features from initialization). Since this is the standard training paradigm used by almost all detection systems, these consistent and significant improvements are doubly important.

Finally, to demonstrate the effectiveness of EN in adjusting statistics, we initialize both network and BN parameters from ImageNet, and fine-tune **both** for object detection. In practice, this setup performs poorly because BN is unable to train for small minibatches, and is generally not used (for example, [25] ignore this variant because of significant drop in performance). We also notice this in Table 2(c) and (d), where the EMA performance for 2 images/minibatch drops by 3.4 AP, 6.6 AP^{50}, and 4.4 AP^{75}. Because of these drastic drops in performance, methods do not fine-tune BN parameters. Compare this to using the proposed EN statistics, which improves over EMA by **5.4** AP, **7.0** AP^{50}, and **4.9** AP^{75}. Therefore, using EN allows us to train BN parameters for smaller minibatches yielding significant improvements.

In this section, we showed that for the object detection task, which is generally trained with small minibatches, using EN statistics performs markedly better across all training setups. In Table 2, notice that we get a healthy performance improvement for 2 images/minibatch across training setups (ranging from **+1.5** points to **+7** points). In the standard paradigm (Table 2(c)), we improve for all minibatch sizes. Moreover, when training BN parameters for small minibatches, (Table 2(c) and (d)), we improve both 2 and 4 images/minibatch.

5. Conclusion

Our goal in this work was to gain a better understanding of the problem with BN for small minibatches. We found that for models trained with small minibatches, normalization using EMA statistics during evaluation provides inaccurate approximation for normalization using minibatch statistics during training. This leads to a discrepancy between training and evaluation and is the main reason of performance degradation of BN for small batch sizes. We proposed EvalNorm, which provides a corrected normalization term for use at evaluation. EN is fully compatible with the existing pre-trained models using BN and yields large gains for models trained with smaller batches.

Acknowledgement. We would like to thank Chen Sun, Sergey Ioffe, and Rahul Sukthankar for helpful discussions and comments; and Ishan Misra and Larry Davis for feedback on the draft.

TTN: A DOMAIN-SHIFT AWARE BATCH NORMALIZATION IN TEST-TIME ADAPTATION

Hyesu Lim[1,2*], **Byeonggeun Kim**[*], **Jaegul Choo**[2], **Sungha Choi**[1‡]
[1]Qualcomm AI Research[†], [2]KAIST

ABSTRACT

This paper proposes a novel batch normalization strategy for test-time adaptation. Recent test-time adaptation methods heavily rely on the modified batch normalization, *i.e.,* transductive batch normalization (TBN), which calculates the mean and the variance from the current test batch rather than using the running mean and variance obtained from source data, *i.e.,* conventional batch normalization (CBN). Adopting TBN that employs test batch statistics mitigates the performance degradation caused by the domain shift. However, re-estimating normalization statistics using test data depends on impractical assumptions that a test batch should be large enough and be drawn from i.i.d. stream, and we observed that the previous methods with TBN show critical performance drop without the assumptions. In this paper, we identify that CBN and TBN are in a trade-off relationship and present a new *test-time normalization* (TTN) method that interpolates the standardization statistics by adjusting the importance between CBN and TBN according to the *domain-shift sensitivity* of each BN layer. Our proposed TTN improves model robustness to shifted domains across a wide range of batch sizes and in various realistic evaluation scenarios. TTN is widely applicable to other test-time adaptation methods that rely on updating model parameters via backpropagation. We demonstrate that adopting TTN further improves their performance and achieves state-of-the-art performance in various standard benchmarks.

1 INTRODUCTION

When we deploy deep neural networks (DNNs) trained on the source domain into test environments (*i.e.,* target domains), the model performance on the target domain deteriorates due to the domain shift from the source domain. For instance, in autonomous driving, a well-trained DNNs model may exhibit significant performance degradation at test time due to environmental changes, such as camera sensors, weather, and region (Choi et al., 2021; Lee et al., 2022; Kim et al., 2022b).

Test-time adaptation (TTA) has emerged to tackle the distribution shift between source and target domains during test time (Sun et al., 2020; Wang et al., 2020). Recent TTA approaches (Wang et al., 2020; Choi et al., 2022; Liu et al., 2021) address this issue by 1) (re-)estimating normalization statistics from current test input and 2) optimizing model parameters in unsupervised manner, such as entropy minimization (Grandvalet & Bengio, 2004; Long et al., 2016; Vu et al., 2019) and self-supervised losses (Sun et al., 2020; Liu et al., 2021). In particular, the former focused on the weakness of conventional batch normalization (CBN) (Ioffe & Szegedy, 2015) for domain shift in a test time. As described in Fig. 1(b), when standardizing target feature activations using source statistics, which are collected from the training data, the activations can be transformed into an unintended feature space, resulting in misclassification. To this end, the TTA approaches (Wang et al., 2020; Choi et al., 2022; Wang et al., 2022) have heavily depended on the direct use of test batch statistics to fix such an invalid transformation in BN layers, called transductive BN (TBN) (Nado et al., 2020; Schneider et al., 2020; Bronskill et al., 2020) (see Fig. 1(c)).

The approaches utilizing TBN showed promising results but have mainly been assessed in limited evaluation settings (Wang et al., 2020; Choi et al., 2022; Liu et al., 2021). For instance, such evaluation settings assume large test batch sizes (*e.g.,* 200 or more) and a single stationary distribution shift

[*]Work completed while at Qualcomm Technologies, Inc. [‡]Corresponding author.
[†]Qualcomm AI Research is an initiative of Qualcomm Technologies, Inc.

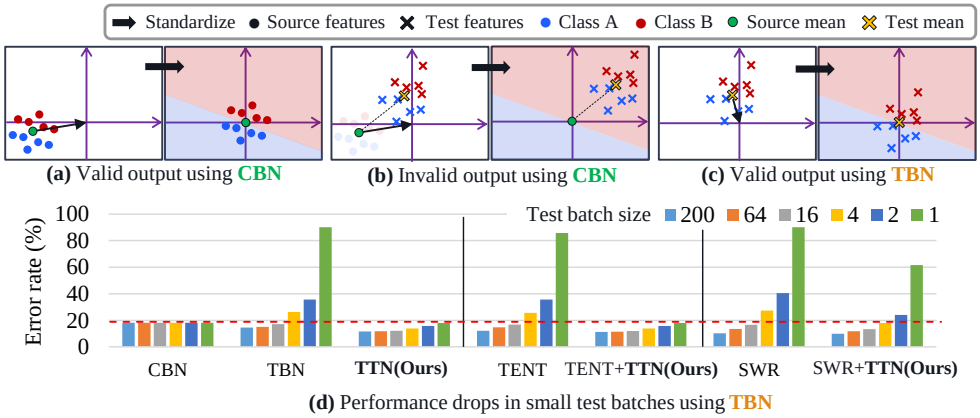


Figure 1: **Trade-off between CBN & TBN.** In conceptual illustrations (a), (b), and (c), the depicted standardization only considers making the feature distribution have a zero mean, disregarding making it have unit variance. When the source and test distributions are different, and the test batch size is large, **(b)** test features can be wrongly standardized when using CBN (Ioffe & Szegedy, 2015), but **(c)** TBN (Nado et al., 2020) can provide a valid output. **(d)** Error rates (↓) on shifted domains (CIFAR-10-C). TBN and TBN applied (TENT (Wang et al., 2020), SWR (Choi et al., 2022)) methods suffer from severe performance drop when the batch size becomes small, while TTN (Ours) improves overall performance.

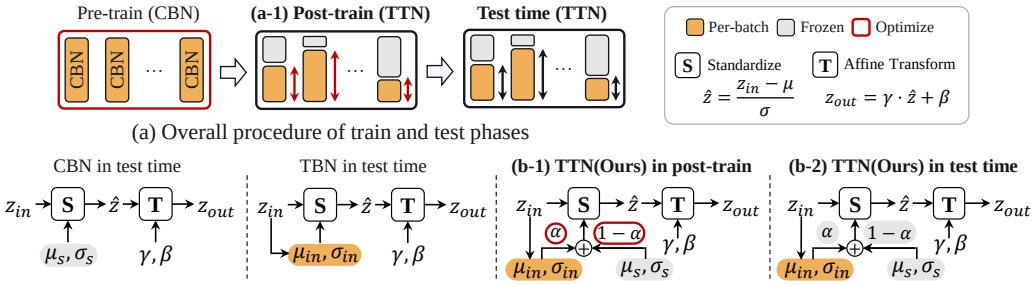
(*i.e.,* single corruption). Recent studies suggest more practical evaluation scenarios based on small batch sizes (Mirza et al., 2022; Hu et al., 2021; Khurana et al., 2021) or continuously changing data distribution during test time (Wang et al., 2022). We show that the performance of existing methods significantly drops once their impractical assumptions of the evaluation settings are violated. For example, as shown in Fig. 1(d), TBN (Nado et al., 2020) and TBN applied methods suffer from severe performance drop when the test batch size becomes small, while CBN is irrelevant to the test batch sizes. We identify that CBN and TBN are in a trade-off relationship (Fig. 1), in the sense that one of each shows its strength when the other falls apart.

To tackle this problem, we present a novel *test-time normalization (TTN)* strategy that controls the trade-off between CBN and TBN by adjusting the importance of source and test batch statistics according to the *domain-shift sensitivity* of each BN layer. Intuitively, we linearly interpolate between CBN and TBN so that TBN has a larger weight than CBN if the standardization needs to be adapted toward the test data. We optimize the interpolating weight after the pre-training but before the test time, which we refer to as the post-training phase. Specifically, given a pre-trained model, we first estimate channel-wise sensitivity of the affine parameters in BN layers to domain shift by analyzing the gradients from the back-propagation of two input images, clean input and its augmented one (simulating unseen distribution). Afterward, we optimize the interpolating weight using the channel-wise sensitivity replacing BN with the TTN layers. It is noteworthy that none of the pre-trained model weights are modified, but we only train newly added interpolating weight.

We empirically show that TTN outperforms existing TTA methods in realistic evaluation settings, *i.e.,* with a wide range of test batch sizes for single, mixed, and continuously changing domain adaptation through extensive experiments on image classification and semantic segmentation tasks. TTN as a stand-alone method shows compatible results with the state-of-the-art methods and combining our TTN with the baselines even boosts their performance in overall scenarios. Moreover, TTN applied methods flexibly adapt to new target domains while sufficiently preserving the source knowledge. No action other than computing per batch statistics (which can be done simultaneously to the inference) is needed in test-time; TTN is compatible with other TTA methods without requiring additional computation cost.

Our contributions are summarized as follows:

- We propose a novel domain-shift aware test-time normalization (TTN) layer that combines source and test batch statistics using channel-wise interpolating weights considering the sensitivity to domain shift in order to flexibly adapt to new target domains while preserving the well-trained source knowledge.



(a) Overall procedure of train and test phases

(b) Comparison of BN layers

Figure 2: **Method overview.** **(a)** We introduce an additional training phase between pre-train and test time called **(a-1)** post-training phase. **(b)** Our proposed TTN layer combines per-batch statistics and frozen source statistics with interpolating weight α, which is **(b-1)** optimized in post-training phase and **(b-2)** fixed in test time.

- To show the broad applicability of our proposed TTN, which does not alter training or test-time schemes, we show that adding TTN to existing TTA methods significantly improves the performance across a wide range of test batch sizes (from 200 to 1) and in three realistic evaluation scenarios; stationary, continuously changing, and mixed domain adaptation.
- We evaluate our method through extensive experiments on image classification using CIFAR-10/100-C, and ImageNet-C (Hendrycks & Dietterich, 2018) and semantic segmentation task using CityScapes (Cordts et al., 2016), BDD-100K (Yu et al., 2020), Mapillary (Neuhold et al., 2017), GTAV (Richter et al., 2016), and SYNTHIA (Ros et al., 2016).

2 Methodology

In this section, we describe our method, the *Test-Time Normalization* (TTN) layer, whose design is suitable for test-time adaptation (TTA) in practical usages out of the large batch size and i.i.d assumptions during a test time. We first define the problem setup in Section 2.1 and present our proposed TTN layers in Section 2.2. Finally, we discuss how we optimize TTN layers in Sections 2.3.

2.1 Problem Setup

Let the train and test data be $\mathcal{D}_S$ and $\mathcal{D}_T$ and the corresponding probability distributions be P_S and P_T, respectively, where $\mathcal{D}_S$ and $\mathcal{D}_T$ share the output space, *i.e.*, $\{y_i\} \sim \mathcal{D}_S = \{y_i\} \sim \mathcal{D}_T$. The covariate shift in TTA is defined as $P_S(x) \neq P_T(x)$ where $P_S(y|x) = P_T(y|x)$ (Quinonero-Candela et al., 2008). A model, f_θ, with parameters θ, is trained with a mini-batch, $\mathcal{B}^S = \{(x_i, y_i)\}_{i=1}^{|\mathcal{B}^S|}$, from source data $\mathcal{D}_S$, where x_i is an example and y_i is the corresponding label. During the test, f_θ encounters a test batch $\mathcal{B}^T \sim \mathcal{D}_T$, and the objective of TTA is correctly managing the test batch from the different distribution.

To simulate more practical TTA, we mainly consider two modifications: (1) various test batch sizes, $|\mathcal{B}^T|$, where small batch size indicates small latency while handling the test data online, and (2) multi, N-target domains, $\mathcal{D}_T = \{\mathcal{D}_{T,i}\}_{i=1}^N$. Under this setting, each test batch $\mathcal{B}^T$ is drawn by one of the test domains in $\mathcal{D}_T$, where $\mathcal{D}_T$ may consist of a single target domain, multiple target domains, or mixture of target domains.

2.2 Test-Time Normalization Layer

We denote an input of a BN layer as $\mathbf{z} \in \mathbb{R}^{BCHW}$, forming a mini-batch size of B. The mean and variance of $\mathbf{z}$ are μ and σ^2, respectively, which are computed as follows:

$$\mu_c = \frac{1}{BHW} \sum_b^B \sum_h^H \sum_w^W \mathbf{z}_{bchw}, \quad \sigma_c^2 = \frac{1}{BHW} \sum_b^B \sum_h^H \sum_w^W (\mathbf{z}_{bchw} - \mu_c)^2, \tag{1}$$

where μ and σ^2 are in $\mathbb{R}^C$, and C, H, and W stand for the number of channels, dimension of height, and that of width, respectively. Based on μ and σ^2, the source statistics $\mu_s, \sigma_s^2 \in \mathbb{R}^C$ are usually estimated with exponential moving average over the training data.

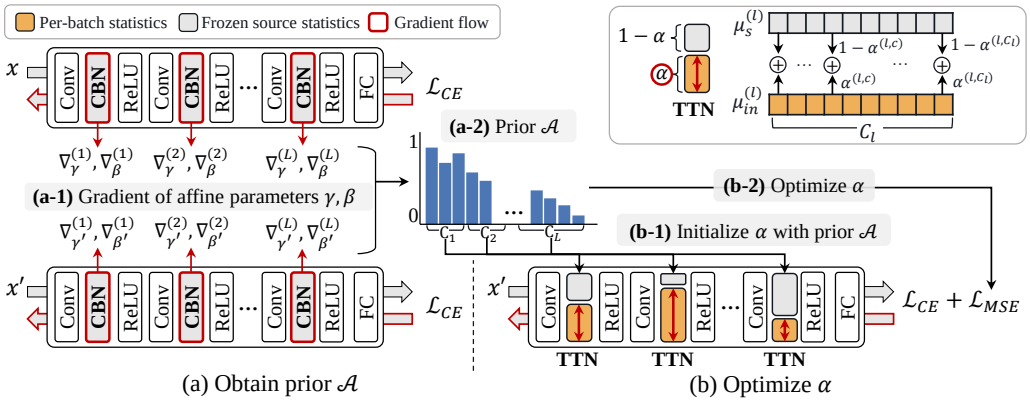


Figure 3: **Two stages in post-training phase. (a)** Given a pre-trained model, which uses CBN, and its training data, we obtain prior knowledge of each BN layer. **(a-1)** We first compute gradients of affine parameters in each BN layer from clean x and augmented input x' and obtain the gradient distance score (Eq. 4). **(a-2)** For BN layers with larger distance score, we put more importance on current batch statistics than source statistics (*i.e.,* large α), and we define prior $\mathcal{A}$ accordingly (Eq. 5). **(b)** After obtaining prior $\mathcal{A}$, we substitute BN layers from CBN to TTN. **(b-1)** Initializing α with prior $\mathcal{A}$, **(b-2)** we optimize α using CE and MSE loss (Eq. 6) with augmented training data x'.

In BN layers, input $\mathbf{z}$ is first standardized with statistics μ and σ^2 and then is scaled and shifted with learnable parameters γ and β in $\mathbb{R}^C$. The standardization uses current input batch statistics during training and uses estimated source statistics μ_s and σ_s^2 at test time (Fig. 2(b)). To address domain shifts in test time, we adjust the source statistics by combining the source and the test mini-batch statistics (Singh & Shrivastava, 2019; Summers & Dinneen, 2019) with a learnable interpolating weight $\alpha \in \mathbb{R}^C$ ranges $[0, 1]$. Precisely, TTN standardizes a feature with

$$\tilde{\mu} = \alpha\mu + (1-\alpha)\mu_s, \quad \tilde{\sigma}^2 = \alpha\sigma^2 + (1-\alpha)\sigma_s^2 + \alpha(1-\alpha)(\mu - \mu_s)^2, \tag{2}$$

while using the same affine parameters, γ and β. Note that we have different mixing ratios α_c for every layer and channel.

2.3 Post Training

Like Choi et al. (2022), we introduce an additional training phase, the post-training (after pre-training but before testing), to optimize the mixing parameters α in Eq. 2 (Fig. 2(a)). Note that all parameters except α are frozen and we have access to the labeled source data during the post-training. We first obtain prior knowledge $\mathcal{A}$ of α by identifying which layers and their channels are sensitive to domain shifts. Then, we optimize α with the prior knowledge and an additional objective term. The overall procedure is depicted in Fig. 3 and the pseudocode is provided in appendix A.3.

Obtain Prior $\mathcal{A}$. To identify which BN layers and corresponding channels are sensitive to domain shifts, we simulate the domain shifts by augmenting[1] the clean image, *i.e.,* original training data, and make a pair of (clean x, domain-shifted x') images, where the semantic information is shared. To analyze in which layer and channel the standardization statistics should be corrected, we consider the standardized features $\hat{z}^{(l,c)}$ of $z^{(l,c)}$, for a channel index c at a layer l, whose input is clean x. We compare $\hat{z}^{(l,c)}$ to that of domain-shifted one, $\hat{z}'^{(l,c)}$ from x'. Since the pre-trained CBN uses the same $\mu_s^{(l,c)}$ and $\sigma_s^{(l,c)}$ for both inputs, the difference between $\hat{z}^{(l,c)}$ and $\hat{z}'^{(l,c)}$ is caused by the domain discrepancy between x and x'. We argue that if the difference is significant, the parameter at (l, c) is sensitive to the domain shift, *i.e.,* intensely affected by the domain shift, and hence the standardization statistics at (l, c) should be adapted towards the shifted input.

Drawing inspiration from Choi et al. (2022), we measure the domain-shift sensitivity by comparing gradients. Since the standardized feature $\hat{z}$ is scaled and shifted by γ and β in each BN layer, we compare the gradients of affine parameters γ and β, ∇_γ and ∇_β, respectively, to measure the dissimilarity of $\hat{z}$ and $\hat{z}'$. As described in Fig. 3(a-1), we collect the ∇_γ and ∇_β using cross-entropy

[1] It is noteworthy that the post-training phase is robust to the choice of data augmentation types. Ablation study results and discussions are provided in the appendix B.4.

loss, $\mathcal{L}_{\text{CE}}$. To this end, we introduce a *gradient distance score*, $d^{(l,c)} \in \mathbb{R}$ for each channel c at layer l as follows:

$$s = \frac{1}{N}\sum_{i=1}^{N}\frac{g_i \cdot g_i'}{\|g_i\|\|g_i'\|}, \tag{3}$$

$$d^{(l,c)} = 1 - \frac{1}{2}\left(s_{\gamma}^{(l,c)} + s_{\beta}^{(l,c)}\right), \tag{4}$$

where (g, g') is $(\nabla_{\gamma}^{(l,c)}, \nabla_{\gamma'}^{(l,c)})$ and $(\nabla_{\beta}^{(l,c)}, \nabla_{\beta'}^{(l,c)})$ for $s_{\gamma}^{(l,c)}$ and $s_{\beta}^{(l,c)}$, respectively, N is the number of training data, and the resulting $d^{(l,c)} \in [0, 1]$. Once we obtain s_{γ} and s_{β} from Eq. 3, we conduct min-max normalization over all $s_{\gamma}^{(l,c)}$ and $s_{\beta}^{(l,c)}$, before computing Eq. 4.

To magnify the relative difference, we take the square as a final step and denote the result as a prior $\mathcal{A}$ (Fig. 3(a-2)):

$$\mathcal{A} = [d^{(1,\cdot)}, d^{(2,\cdot)}, \ldots, d^{(L,\cdot)}]^2, \tag{5}$$

where $d^{(l,\cdot)} = [d^{(l,c)}]_{c=1}^{C_l}$.

Optimize α. The goal of optimizing α is to make the combined statistics correctly standardize the features when the input is sampled from an arbitrary target domain. After obtaining the prior $\mathcal{A}$, we replace CBN with TTN layers while keeping the affine parameters. Then, we initialize the interpolating weights α with $\mathcal{A}$, which represents in which layer and channel the standardization statistics need to be adapted using test batch statistics (see Fig. 3(b-1)). To simulate distribution shifts, we use augmented training data. Expecting the model to make consistent predictions either given clean or augmented inputs, we use cross-entropy loss $\mathcal{L}_{\text{CE}}$. Furthermore, to prevent α from moving too far from the initial value $\mathcal{A}$, we use mean-squared error loss $\mathcal{L}_{\text{MSE}}$ between α and the prior $\mathcal{A}$, *i.e.,* $\mathcal{L}_{\text{MSE}} = \|\alpha - \mathcal{A}\|^2$ as a constraint. Total loss $\mathcal{L}$ can be written as $\mathcal{L} = \mathcal{L}_{\text{CE}} + \lambda\mathcal{L}_{\text{MSE}}$ (6), where λ is a weighting hyperparameter (Details are provided in the appendix A.1 & B.1).

3 Experiments

In image classification, we evaluate TTN for corruption robustness in realistic evaluation settings, *i.e.,* where the test batch size can be variant and where the target domain can be either stationary, continuously changing, or mixed with multiple domains. Additionally, we further validate TTN on domain generalization benchmarks incorporating natural domain shifts (*e.g.,* changes in camera sensors, weather, time, and region) in semantic segmentation.

3.1 Experimental Setup

Given models pre-trained on clean source data, we optimize TTN parameter α with the augmented source data in the post-training phase. Afterward, we evaluate our post-trained model on the corrupted target data. **Implementation details are provided in the appendix A.1.**

Datasets and models. We use corruption benchmark datasets CIFAR-10/100-C and ImageNet-C, which consist of 15 types of common corruptions at five severity levels (Hendrycks & Dietterich, 2018). Each corruption is applied to test images of the clean datasets (CIFAR-10/100 and ImageNet). We use a training set of the clean dataset for post-training and the corrupted dataset for evaluation. As backbone models, we used WideResNet-40-2 (Hendrycks et al., 2019) trained on CIFAR-10/100, and ResNet-50 (He et al., 2016) trained on ImageNet. To validate our method in semantic segmentation, we conduct experiments on Cityscapes (Cordts et al., 2016), BDD-100K (Yu et al., 2020), Mapillary (Neuhold et al., 2017), GTAV (Richter et al., 2016), and SYNTHIA (Ros et al., 2016) datasets, in accordance with the experimental setup for domain generalization proposed in RobustNet (Choi et al., 2021).

Baselines. To demonstrate that TTN successfully controls the trade-off between CBN and TBN, we compare TTN with (1) AdaptiveBN (Schneider et al., 2020), (2) α-BN (You et al., 2021) and (3) MixNorm (Hu et al., 2021), which combines or takes the moving average of the source and the test batch statistics with a pre-defined hyperparameter (*i.e.,* a constant α). The following baselines are suggested on top of TBN (Nado et al., 2020); (4) TENT (Wang et al., 2020) optimizes BN affine parameters via entropy minimization. (5) SWR (Choi et al., 2022) updates the entire model parameters considering the domain-shift sensitivity. (6) CoTTA (Wang et al., 2022) ensembles the output of

Table 1: **Single domain adaptation on corruption benchmark.** Error rate (↓) averaged over 15 corruptions with severity level 5 using WideResNet-40-2 as a backbone for each test batch size. We used reported results of MixNorm with fixed parameters from the original paper and denoted as $*$. In appendix B.3, we provide variants of TTN, which show stronger performance for small test batch.

	Method	CIFAR-10-C 200	64	16	4	2	1	Avg.	CIFAR-100-C 200	64	16	4	2	1	Avg.
	Source (CBN)	18.27	18.27	18.27	18.27	18.27	18.27	18.27	46.75	46.75	46.75	46.75	46.75	46.75	46.75
Norm	TBN	14.49	15.02	17.10	26.28	35.65	90.00	33.09	39.25	40.21	44.03	59.10	80.65	99.04	60.38
Norm	AdaptiveBN	12.21	12.31	12.89	14.51	15.79	16.14	13.98	36.56	36.85	38.19	41.18	43.26	**44.01**	**40.01**
Norm	α-BN	13.78	13.77	13.89	14.54	**15.16**	15.47	14.44	39.72	39.85	39.99	41.34	**42.66**	45.64	41.53
Norm	MixNorm*	13.85	14.41	14.23	14.60 (B=5)	-	**15.09**	14.44	-	-	-	-	-	-	-
Norm	**Ours (TTN)**	**11.67**	**11.80**	**12.13**	**13.93**	15.83	17.99	**13.89**	**35.58**	**35.84**	**36.73**	**41.08**	46.67	57.71	42.27
Optim.	TENT	12.08	14.78	16.90	25.61	35.69	90.00	32.51	35.52	39.90	43.78	59.02	80.68	99.02	59.65
Optim.	**+Ours (TTN)**	**11.28**	**11.52**	**12.04**	**13.95**	**15.84**	**17.94**	**13.77**	**35.16**	**35.57**	**36.55**	**41.18**	**46.63**	**58.33**	**42.24**
Optim.	SWR	10.26	13.51	16.61	27.33	40.48	90.04	33.04	**32.68**	37.41	43.15	59.90	87.07	99.05	59.88
Optim.	**+Ours (TTN)**	**9.92**	**11.77**	**13.41**	**18.02**	**24.09**	**61.56**	**23.13**	32.86	**35.13**	**38.66**	**49.80**	**60.72**	**80.90**	**49.68**

Table 2: **Continuously changing domain adaptation on corruption benchmark.** Error rate (↓) averaged over 15 corruptions with severity level 5 using WideResNet-40-2 as backbone for each test batch size. We omitted 'Norm' methods results in this table since they are eqaul to that of Table 1.

	Method	CIFAR-10-C 200	64	16	4	2	1	Avg.	CIFAR-100-C 200	64	16	4	2	1	Avg.
	Source (CBN)	18.27	18.27	18.27	18.27	18.27	18.27	18.27	46.75	46.75	46.75	46.75	46.75	46.75	46.75
	Ours (TTN)	**11.67**	**11.80**	**12.13**	**13.93**	**15.83**	**17.99**	**13.89**	**35.58**	**35.84**	**36.73**	**41.08**	**46.67**	**57.71**	**42.27**
Optim.	CoTTA	12.46	14.60	21.26	45.69	58.87	90.00	40.48	39.75	42.20	52.94	73.69	87.66	98.99	65.87
Optim.	TENT	12.54	13.52	15.69	26.23	35.77	90.00	32.29	**36.11**	37.90	43.78	58.71	81.76	99.04	59.55
Optim.	**+Ours (TTN)**	**11.44**	**11.60**	**12.08**	**16.14**	**18.36**	**22.40**	**15.33**	43.50	**37.60**	**38.28**	**44.60**	**54.29**	**80.63**	**49.82**
Optim.	SWR	11.04	11.53	13.90	23.99	34.02	90.00	30.75	34.16	35.79	40.71	58.15	80.55	99.03	62.56
Optim.	**+Ours (TTN)**	**10.09**	**10.51**	**11.28**	**14.29**	**16.67**	**84.12**	**24.49**	**33.09**	**34.07**	**36.15**	**42.41**	**53.63**	**93.08**	**48.74**

augmented test inputs, updates the entire model parameters using a consistency loss between student and teacher models, and stochastically restores the pre-trained model. We refer to TBN, (1), (2), and (3) as *normalization*-based methods (Norm), the other as *optimization*-based methods (Optim.), and denote the pre-trained model using CBN as 'source'.

Evaluation scenarios. To show that TTN performs robust on various test batch sizes, we conduct experiments with test batch sizes of 200, 64, 16, 4, 2, and 1. We evaluate our method in three evaluation scenarios; *single*, *continuously changing*, and *mixed* domain adaptation. In the single domain adaptation, the model is optimized for one corruption type and then reset before adapting to the subsequent corruption, following the evaluation setting from TENT and SWR. In the continuously changing adaptation (Wang et al., 2022), the model is continuously adapted to 15 corruption types (w/o the reset), which is more realistic because it is impractical to precisely indicate when the data distribution has shifted in the real world. Finally, to simulate the non-stationary target domain where various domains coexist, we evaluate methods in the mixed domain adaptation setting, where a single batch contains multiple domains. We use a severity level of 5 (Hendrycks & Dietterich, 2018) for all experiments. It is noteworthy that we use a single checkpoint of TTN parameter α for each dataset across all experimental settings.

3.2 Experiments on Image Classification

Tables 1, 2, and 3 show error rates on corruption benchmark datasets in three different evaluation scenarios; single domain, continuously changing, and mixed domain adaptation, respectively. Note that the performance of normalization-based methods in the single (Table 1) and in the continuously changing (Table 2) settings are identical. Tables 4 and 5 show the adaptation performance on the source and class imbalanced target domains, respectively. **More results and discussions are provided in the appendix B, importantly, including results on ImageNet-C (B.5).**

Robustness to practical settings. In Table 1, 2, and 3, TTN and TTN applied methods show robust performance over the test batch size ranges from 200 to 1. Comparing with normalization-based baselines, we demonstrate that TTN, which uses channel-wisely optimized combining rate α, shows better results than defining α as a constant hyper-parameter, which can be considered as a special

Table 3: **Mixed domain adaptation on corruption benchmark.** Error rate (↓) of mixed domain with severity level 5 using WideResNet-40-2 as backbone for each test batch size. We used the reported results of MixNorm with fixed parameters from the original paper and denoted them as $*$.

	Method	CIFAR-10-C 200	64	16	4	2	1	Avg.	CIFAR-100-C 200	64	16	4	2	1	Avg.
	Source (CBN)	18.27	18.27	18.27	18.27	18.27	18.27	18.27	46.75	46.75	46.75	46.75	46.75	46.75	46.75
Norm	TBN	14.99	15.29	17.38	26.65	35.59	90.00	33.31	39.88	40.48	43.73	59.11	80.30	98.91	60.40
Norm	AdaptiveBN	12.62	12.48	12.97	14.59	15.74	16.02	14.07	36.88	36.86	38.49	41.43	43.38	44.31	40.23
Norm	α-BN	13.78	13.78	13.99	14.61	**15.07**	**15.20**	14.41	40.25	40.11	40.47	41.64	**42.39**	**43.81**	**41.45**
Norm	MixNorm*	18.80	18.80	18.80	18.80	18.80	18.80	18.80	-	-	-	-	-	-	-
Norm	**Ours (TTN)**	**12.16**	**12.19**	**12.34**	**13.96**	15.55	17.83	**14.00**	**36.24**	**36.23**	**36.85**	**41.01**	45.85	55.52	41.95
Optim.	TENT	14.33	14.97	17.30	26.07	35.37	90.00	33.01	39.36	40.01	43.33	58.98	80.55	98.92	60.19
Optim.	**+Ours (TTN)**	**12.02**	**12.04**	**12.20**	**13.77**	**15.42**	**16.40**	**13.64**	**36.29**	**36.23**	**36.89**	**41.38**	**46.65**	**57.95**	**42.57**
Optim.	SWR	13.24	13.06	16.57	26.08	38.65	91.03	59.54	37.84	37.93	44.37	59.50	78.66	98.95	33.10
Optim.	**+Ours (TTN)**	**11.89**	**11.65**	**13.37**	**17.05**	**23.50**	**64.10**	**50.29**	**36.49**	**36.51**	**39.60**	**46.20**	**58.20**	**84.76**	**23.59**

case of TTN; TBN and α-BN corresponds to $\alpha = 1$ and 0.1, respectively. More comparisons with different constant α are provided in the appendix B.2. It is noteworthy that TTN as a stand-alone method favorably compares with optimization-based baselines in all three scenarios.

Table 4: **Source domain adaptation.** Error rate (↓) on CIFAR-10 using WideResNet-40-2.

	Method	Test batch size 200	64	16	4	2	1	Avg.
	Source (CBN)	4.92	4.92	4.92	4.92	4.92	4.92	4.92
Norm	TBN	6.41	6.60	8.64	17.65	26.08	90.00	25.90
Norm	**Ours (TTN)**	**4.88**	**5.11**	**5.35**	**7.27**	**9.45**	**9.96**	**7.00**
Optim.	TENT	6.15	6.45	8.61	17.61	26.20	90.00	32.2
Optim.	**+Ours (TTN)**	**4.93**	**5.11**	**5.32**	**7.22**	**9.38**	**10.21**	**7.02**
Optim.	SWR	5.63	6.01	8.25	17.49	26.32	90.00	25.62
Optim.	**+Ours (TTN)**	**4.79**	**5.02**	**5.51**	**6.68**	**7.91**	**9.34**	**6.54**

Table 5: **Class imbalanced target domain.** Error rate (↓) averaged over 15 corruptions of CIFAR-10-C with severity level 5 using WideResNet-40-2. Details are provided in the appendix A.2.

Method	Test batch size 200	64	16	4	2	1	Avg.
Source (CBN)	18.27	18.27	18.27	18.27	18.27	18.27	18.27
TBN	77.60	76.66	77.72	78.59	77.84	90.00	79.74
Ours (TTN)	**35.75**	**35.13**	**34.92**	**32.51**	**28.60**	**17.99**	**30.82**

Adopting TTN improves other TTA methods. We compare optimization-based methods with and without TTN layers. Since TENT, SWR, and CoTTA optimize model parameters on top of using TBN layers, they also suffer from performance drops when the test batch size becomes small. Adopting TTN reduces the dependency on large test batch size, *i.e.,* makes robust to small batch size, and even improves their performance when using large test batch. Furthermore, in continual (Table 2) and mixed domain (Table 3) adaptation scenario, TENT and SWR shows higher error rate than in single domain (Table 1) adaptation. We interpret that because they update the model parameters based on the current output and predict the next input batch using the updated model, the model will not perform well if the consecutive batches have different corruption types (*i.e.,* mixed domain adaptation). Moreover, the error from the previous input batch propagates to the future input stream, and thus they may fall apart rapidly once they have a strongly wrong signal, which can happen in continual adaptation (*i.e.,* long-term adaptation without resetting). Applying TTN significantly accelerates their model performance regardless of the evaluation scenarios.

TTN preserves knowledge on source domain. In practice, data driven from the source domain (or a merely different domain) can be re-encountered in test time. We used clean domain test data in the single domain adaptation scenario to show how TTN and other TTA methods adapt to the seen source domain data (but unseen instance). As shown in Table 4, all baseline methods using TBN layers, show performance drops even with large batch sizes. We can conclude that it is still better to rely on source statistics collected from the large training data than using only current input statistics, even if its batch size is large enough to obtain reliable statistics (*i.e.,* 200). However, since TTN utilizes source statistics while leveraging the current input, TTN itself and TTN adopted methods well preserve the source knowledge compared to the TBN-based methods. With a batch size of 200, we observe that combining the source and a current test batch statistics outperforms the source model (see 3rd row of Table 4).

TTN is robust to class imbalanced scenario. Heavily depending on current test batch statistics are especially vulnerable when the class labels are imbalanced (Boudiaf et al., 2022; Gong et al., 2022). To simulate this situation, we sorted test images in class label order and then sampled test batches following the sorted data order. In Table 5, we observe that TTN is more robust to the class imbalanced scenario than utilizing only test batch statistics (*i.e.,* TBN). As explained in Section 3.5,

Table 6: **Adaptation on DG benchmarks in semantic segmentation.** mIoU(↑) on four unseen domains with test batch size of 2 using ResNet-50 based DeepLabV3+ as a backbone.

	Method (Cityscapes→)	BDD-100K	Mapillary	GTAV	SYNTHIA	Cityscapes
	Source (Chen et al., 2018)	43.50	54.37	43.71	22.78	76.15
Norm	TBN	43.12	47.61	42.51	25.71	72.94
Norm	**Ours (TTN)**	**47.40**	**56.92**	**44.71**	**26.68**	**75.09**
Optim.	TENT	43.30	47.80	43.57	25.92	72.93
Optim.	**+ Ours (TTN)**	**47.89**	**57.84**	**46.18**	**27.29**	**75.04**
Optim.	SWR	43.40	47.95	42.88	25.97	72.93
Optim.	**+ Ours (TTN)**	**48.85**	**59.09**	**46.71**	**29.16**	**74.89**

we are putting more importance on CBN than TBN, where semantic information is mainly handled, *i.e.,* in deeper layers, so we can understand that TTN is less impacted by skewed label distribution.

3.3 Experiments on Semantic Segmentation

We additionally conduct experiments on domain generalization (DG) benchmarks (Choi et al., 2021; Pan et al., 2018) for semantic segmentation, including natural domain shifts (*e.g.,* Cityscapes→BDD-100K), to demonstrate the broad applicability of TTN. Table 6 shows the results of evaluating the ResNet-50-based DeepLabV3+ (Chen et al., 2018) model trained on the Cityscapes training set using the validation set of real-world datasets such as Cityscapes, BDD-100K, and Mapillary, and synthetic datasets including GTAV and SYNTHIA. We employ a test batch size of 2 for test-time adaptation in semantic segmentation. We observe that even when exploiting test batch statistics for standardization in BN layers (TBN) or updating the model parameters on top of TBN (TENT, SWR) does not improve the model performance (*i.e.*, perform worse than the source model), adopting TTN helps the model make good use of the strength of the test batch statistics. Implementation details and additional results are provided in the appendix A.1 and B.7, respectively.

3.4 Ablation Studies

Prior $\mathcal{A}$ regularizes α to be robust to overall test batch sizes. We conduct an ablation study on the importance of each proposed component, *i.e.,* initializing α with prior $\mathcal{A}$, optimizing α using CE and MSE losses, and the results are shown in Table 7. Using $\mathcal{A}$ for initialization and MSE loss aims to optimize α following our intuition that we discussed in Section 2.3. Optimizing α using CE loss improves the overall performance, but without regularizing with MSE loss, α may overfit to large batch size (rows 2 & 3). Initialization with $\mathcal{A}$ or not does not show a significant difference, but $\mathcal{A}$ provides a better starting point than random initialization when comparing the left and right of the 2nd row. We observe that when using MSE loss, regardless of initialization using $\mathcal{A}$, the optimized α sufficiently reflects our intuition resulting in a low error rate to overall batch sizes (row 3).

Table 7: **Ablation study on importance of each component**

Method			**Test batch size**						**Avg.**	**Method**			**Test batch size**						**Avg.**
Init.	CE	MSE	200	64	16	4	2	1		Init.	CE	MSE	200	64	16	4	2	1	
-	-	✓	13.36	13.43	13.85	15.50	17.43	20.07	15.61	✓	-	-	13.37	13.43	13.85	15.50	17.44	20.07	15.61
-	✓	-	**11.64**	**11.73**	12.26	14.46	16.94	19.88	14.49	✓	✓	-	11.73	11.82	12.23	14.18	16.41	19.27	14.27
-	✓	✓	**11.64**	11.78	**12.21**	**13.97**	**15.86**	**18.00**	**13.91**	✓	✓	✓	**11.67**	**11.80**	**12.13**	**13.93**	**15.83**	**17.99**	**13.89**

3.5 Visualization of α

Fig. 4 shows the visualization of optimized α for CIFAR-10 using WideResNet-40-2. We observe that α decreases from shallow to deep layers (left to right), which means CBN is more active in deeper layers, and TBN is vice versa. As shown in Table 4 and 6, CBN employing source statistics is superior to TBN when the distribution shift between source and target domains is small. Assuming that the α we obtained is optimal, we can conjecture that CBN is more active (*i.e.,* α closer to 0) in deeper layers because domain information causing the distribution shift has been diminished. In contrast, TBN has a larger weight (*i.e.,* α closer to 1) in shallower layers since the domain information induces a large distribution shift. This interpretation is consistent with the observations of previous studies (Pan et al., 2018; Wang et al., 2021; Kim et al., 2022a) that style information mainly exists in shallower layers, whereas only content information remains in deeper layers.

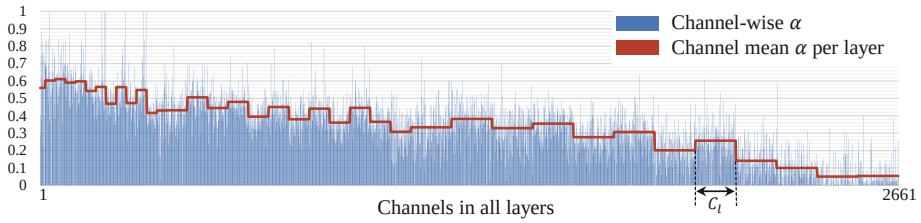


Figure 4: **Optimized α.** x- and y-axes indicate all channels in order from shallow to deep layers and the interpolating weight α, respectively. C_l denotes the channel size of layer l.

4 Related Work

Test-time adaptation/training (TTA) aims to adapt models towards test data to overcome the performance degradation caused by distribution shifts (Sun et al., 2020; Wang et al., 2020). There are other related problems, unsupervised domain adaptation (UDA) (Sun & Saenko, 2016; Ganin et al., 2016) and source-free domain adaptation (SFDA) (Liang et al., 2020; Huang et al., 2021; Liu et al., 2021). Both UDA and SFDA have access to sufficiently large enough unlabeled target datasets, and their objective is to achieve high performance on that particular target domain. Unlike UDA and SFDA, TTA utilizes test data in an online manner. There are two key factors of recent approaches: adapting standardization statistics in normalization layers and adapting model parameters.

Normalization in Test Time. Nado et al. (2020) suggested prediction-time BN, which uses test batch statistics for standardization and Schneider et al. (2020) introduced to adapt BN statistics by combining source and test batch statistics considering the the test batch size to mitigate the intermediate covariate shift. In this paper, we refer to the former method as TBN. Similarly, You et al. (2021) and Khurana et al. (2021) mixed the statistics using predefined hyperparameter. Also, Mirza et al. (2022) and Hu et al. (2021) adapted the statistics using moving average while augmenting the input to form a pseudo test batch when only a single instance is given. The primary difference with the existing approaches is that we consider the channel-wise domain-shift sensitivity of BN layers to optimize the interpolating weights between CBN and TBN. Concurrently, Zou et al. (2022) proposed to adjust the standardization statistics using a learnable calibration strength and showed its effectiveness focusing on the semantic segmentation task.

Optimization in Test Time. TENT (Wang et al., 2020), SWR (Choi et al., 2022), and CoTTA (Wang et al., 2022) updated model parameters while using TBN. TENT optimized affine parameters in BN layers using entropy minimization while freezing the others. To maximize the adaptability, SWR updated the entire model parameters minimizing the entropy loss based on the domain-shift sensitivity. To stabilize the adaptation in continuously changing domains, CoTTA used consistency loss between student and teacher models and stochastically restored random parts of the pre-trained model. Liu et al. (2021) and Chen et al. (2022) suggested to update the model through contrastive learning.

We focus on correcting the standardization statistics using domain-shift aware interpolating weight α. Similar to Choi et al. (2022), we measure the domain-shift sensitivity by comparing gradients. The principle difference is that we use channel-wise sensitivity when optimizing α in post-training, while SWR used layer-wise sensitivity regularizing the entire model update in test time.

5 Conclusions

This paper proposes TTN, a novel domain-shift aware batch normalization layer, which combines the benefits of CBN and TBN that have a trade-off relationship. We present a strategy for mixing CBN and TBN based on the interpolating weight derived from the optimization procedure utilizing the sensitivity to domain shift and show that our method significantly outperforms other normalization techniques in various realistic evaluation settings. Additionally, our method is highly practical because it can complement other optimization-based TTA methods. The oracle mixing ratio between CBN and TBN can vary depending on the domain gap difference. However, our proposed method employs a fixed mixing ratio during test time, where the mixing ratio is optimized before model deployment. If we could find the optimal mixing ratio according to the distribution shift during test time, we can expect further performance improvement. We consider it as future work. In this regard, our efforts encourage this field to become more practical and inspire new lines of research.

A APPENDIX: FOR REPRODUCIBILITY

This section provides supplemental material for Section 2 and 3.1.

A.1 IMPLEMENTATION DETAILS

Datasets and Models. For CIFAR-10/100-C, we optimized α using augmented CIFAR-10/100 training set on the pre-trained WideResNet-40-2 (WRN-40) (Hendrycks et al., 2019). For ImageNet-C, we used augmented ImageNet training set (randomly sampled 64000 instances per epoch) on the pre-trained ResNet-50.

Augmentation. Following the setting of SWR (Choi et al., 2022), we used color jittering, random invert and random grayscale when obtaining the prior $\mathcal{A}$. When optimizing α, we followed the augmentation choice of CoTTA (Wang et al., 2022), which are color jittering, padding, random affine, gaussian blur, center crop and random horizontal flip. We excluded for gaussian noise to avoid any overlap with corruption type of common corruptions (Hendrycks & Dietterich, 2018). For ImageNet, we only used the same augmentation both for obtaining $\mathcal{A}$ and optimizing α following the SWR augmentation choices.

Post-training. When obtaining prior, we used randomly selected 1024 samples from the training set, following the setting of SWR. We used Adam (Kingma & Ba, 2015) optimizer using a learning rate (LR) of 1e-3, which is decayed with cosine schedule (Loshchilov & Hutter, 2017) for 30 epochs and used 200 training batch for CIFAR-10/100. For ImageNet, we lowered LR to 2.5e-4 and used 64 batch size. For semantic segmentation task, we trained TTN using Cityscapes training set, and training batch size of 2. We resized the image height to 800 while preserving the original aspect ratio Cordts et al. (2016). We can terminate the training when MSE loss saturates (We observed that α does not show significant difference after the MSE loss is saturated). We used the weighting hyperparmeter to MSE loss λ as 1.

Test time. For AdaptiveBN, which adjusts the interpolating weight using two factors: hyperparameter N and test batch size n, we followed the suggested N, which is empirically obtained best hyperparameter from the original paper, for each n (Figure 11, ResNet architecture from Schneider et al. (2020)). In detail, we set N as 256, 128, 64, 32, and 16 for test batch size 200, 64, 16, 4, 2, and 1, which yields α as 0.44, 0.33, 0.2, 0.11, 0.06, and 0.06, respectively.

For optimization-based TTA methods, we followed default setting in TENT, SWR, and CoTTA for test-time adaptation. We used LR of 1e-3 to test batch size of 200 for CIFAR-10/100-C in single domain (TENT, SWR) and continuously changing domain (CoTTA) scenarios. To avoid rapid error accumulation, we lowered LR to 1e-4 for TENT and SWR in continual and mixed domain scenarios. Moreover, we updated model parameters after accumulating the gradients for 200 samples for CIFAR-10/100-C. In other words, we compute gradients per batch, but update, *i.e.,* `optimizer.step()`, after seeing 200 data samples. Exceptionally, we used LR of 5e-5 for SWR and SWR+TTN in mixed domain setting. Additionally, in continuously changing and mixed domain scenarios, we used the stable version of SWR, which updates the model parameter based on the frozen source parameters instead of previously updated parameters (original SWR).

For semantic segmentation, we set the test batch size as 2 and learning rate for optimization-based methods as 1e-6 for all datasets. For SWR, we set the importance of the regularization term λ_r as 500. The other hyperparameters are kept the same as Choi et al. (2022) choices. For all test-time adaptation, we used constant learning rate without scheduling.

A.2 EVALUATION SCENARIO DETAILS

Class imbalanced setting. In the main paper Table 5, we show the results under class imbalanced settings. In the setting, we sorted the test dataset of each corruption in the order of class labels, i.e., from class 0 to 9 for CIFAR-10-C. Then, we comprised test batches following the sorted order. Therefore, most batches consist of single class input data, which leads to biased test batch statistics. For larger batch size, the statistics are more likely to be extremely skewed, and that explains why error rates are higher with larger batch sizes than with the small ones.

A.3 PSEUDOCODE

Pseudocode for post-training, *i.e.,* obtaining $\mathcal{A}$ and optimizing α, is provided in Algorithms 1 and 2, respectively, and that for test time is in Algorithm 3. Moreover, we provide PyTorch-friendly pseudocode for obtaining $\mathcal{A}$ in Listing 1. Please see Section 2 for equations and terminologies used in the algorithms.

Algorithm 1 Obtain prior $\mathcal{A}$

Require: Pre-trained model f_θ; source training data $\mathcal{D}_S = (X, Y)$
Output: Prior $\mathcal{A}$
for all (x, y) in $\mathcal{D}_S$ **do**
    Augment x: x'
    Collect gradients $(\nabla_\gamma, \nabla_{\gamma'})$ and $(\nabla_\beta, \nabla_{\beta'})$ from f_θ using clean x and augmented x'
end for
Compute gradient distance score d using Eq. 4
Define prior $\mathcal{A}$ using Eq. 5

Algorithm 2 Post-train

Require: Pre-trained model f_θ; source training data $\mathcal{D}_S = (X, Y)$;
step size hyperparameter η; regularization weight hyperparameter λ
Output: Optimized interpolating weight α
Obtain prior $\mathcal{A}$ using Algorithm 1
Initialize α with prior $\mathcal{A}$
Replace all BN layers of f_θ to TTN layers using α and Eq. 2
while not done **do**
    Sample minibatches $\mathcal{B}^S$ from $\mathcal{D}_S$
    for all minibatches **do**
        Augment all x in $\mathcal{B}^S$: x'
        Evaluate $\nabla_\alpha \mathcal{L}$ using f_θ given $\{(x'_i, y_i)\}_{i=1}^{|\mathcal{B}^S|}$ using Eq. 6 while adapting standardization statistics using Eq. 2
        Update $\alpha \leftarrow \alpha - \eta \nabla_\alpha \mathcal{L}$
    end for
end while

Algorithm 3 Inference (Test time)

Require: Pre-trained model f_θ; optimized α, target test data $\mathcal{D}_T = (X)$;
Replace all BN layers of f_θ to TTN layers using α and Eq. 2
Sample minibatches $\mathcal{B}^T$ from $\mathcal{D}_T$
for all minibatches **do**
    Make prediction using f_θ given $\mathcal{B}^T$ while adapting standardization statistics using Eq. 2
end for

```
def obtain_prior(model, train_data):
  # make weight and bias of BN layers requires_grad=True, otherwise False
  params = {n: p for n, p in model.named_parameters() if p.requires_grad}

  grad_sim = {}
  for x, y in train_data:
    # collect gradients for clean and augmented input
    grad_org = collect_grad(model, x, y, params)
    grad_aug = collect_grad(model, augment(x), y, params)

    # compute grad similarity
    for n, p in params.items():
      grad_sim[n].data = cosine_sim(grad_org, grad_aug)

```

```
  # average over data samples
  for n, p in params.items():
    grad_sim[n].data /= len(train_data)

  # min max normalization
  max_grad = get_max_value(grad_sim) # scalar
  min_grad = get_min_value(grad_sim) # scalar
  grad_dist = {}
  for n, p in grad_sim.items():
    grad_dist[n] = (p - min_grad) / (max_grad - min_grad)

  prior = []
  j = 0
  # integrate gradients of weight(gamma) and bias(beta) for each BN layer
  for n, p in grad_dist.items():
      if "weight" in n:
          prior.append(p)
      elif "bias" in n:
          prior[j] += p
          prior[j] /= 2
          prior[j] = (1-prior[j])**2
          j += 1

  return prior

def collect_grad(model, x, y, params):
  model.zero_grad()
  out = model(x)
  loss = ce_loss(out, y)
  loss.backward()

  grad = {}
  for n, p in params.items():
    grad[n].data = p.data

  return grad
```

Listing 1: PyTorch-friendly pseudo code for obtaining prior

B Appendix: Experimental Results

B.1 Ablation Studies

Channel-wise α. In Table 8, we compared different granularity levels of interpolating weight α, *i.e.,* channel-wise, layer-wise, and a constant value with CIFAR-10-C and backbone WRN-40. We observed that channel-wise optimized α shows the best performance. We take average of the channel-wisely optimized α (the 1st row) over channels to make a layer-wise α (the 2nd row), and take average over all channels and layers to make a constant value (the 3rd row). α of the 1st and 2nd rows are visualized in the main paper Figure 4 colored in blue and red, respectively. The constant value α (the 3rd row) is 0.3988. We also optimized α layer-wisely (the 4th row).

Table 8: **Ablation study on granularity level of α**

#	Method	Test batch size						Avg.
		200	64	16	4	2	1	
1	**Channel-wise (Optimized)**	**11.67**	**11.80**	**12.13**	**13.93**	**15.83**	**17.99**	**13.89**
2	Layer-wise (Channel mean)	12.75	12.84	13.16	14.66	16.40	18.82	14.77
3	Constant (Mean)	12.07	12.21	13.05	16.72	20.04	21.26	15.89
4	Layer-wise (Optimized)	13.11	13.21	13.51	14.84	16.46	18.62	14.96

MSE loss strength λ. We empirically set the MSE loss strength λ in Eq. 6 as 1 through the ablation study using CIFAR-10-C and WRN-40 (Table 9). Using the MSE regularizer prevents the learnable α from moving too far from the prior, thus letting α follow our intuition, i.e., putting smaller importance on the test batch statistics if the layer or channel is invariant to domain shifts. However, with too strong regularization (λ=10.0), the overall error rates are high, which means the α needs to be sufficiently optimized. On the other hand, with too small regularization, the α may overfit to the training batch size (B=200) and lose the generalizability to the smaller batch size.

Table 9: **MSE loss strength λ**

λ	Test batch size						Avg.
	200	64	16	4	2	1	
0.0	11.73	11.82	12.23	14.18	16.41	19.27	14.27
0.1	11.65	11.91	12.09	13.84	15.71	18.24	13.91
1.0	11.67	11.80	12.13	13.93	15.83	17.99	13.89
10.0	12.45	12.63	12.82	14.55	16.32	18.56	14.56

B.2 More comparisons on CIFAR10-C

Constant α. Table 10 shows results of simple baseline for normalization-based methods where the α is a constant value ranging from 0 to 1. α = 0 is identical to CBN (Ioffe & Szegedy, 2015), and α = 1 is identical to TBN (Nado et al., 2020). We observe that the lower α, i.e., using less test batch statistics, shows better performance for small test batch sizes. This observation is analogous to the finding of the previous work (Schneider et al., 2020).

Table 10: **Constant α.** Error rate (↓) averaged over 15 corruptions of CIFAR-10-C (WRN-40).

α	Test batch size						Avg.
	200	64	16	4	2	1	
0	18.26	18.39	18.26	18.26	18.25	18.25	18.28
0.1	13.95	14.1	14.05	14.65	**15.14**	**15.45**	14.56
0.2	12.46	12.66	12.89	**14.30**	15.53	15.64	**13.91**
0.3	**12.05**	**12.29**	**12.72**	15.18	17.35	17.42	14.50
0.4	12.13	12.41	13.12	16.69	19.81	20.51	15.78
0.5	12.42	12.78	13.73	18.32	22.52	24.88	17.44
0.6	12.88	13.32	14.48	20.02	25.17	31.97	19.64
0.7	13.37	13.9	15.23	21.75	27.91	46.65	23.14
0.8	13.82	14.37	15.94	23.44	30.59	77.15	29.22
0.9	14.18	14.8	16.58	24.94	33.12	89.81	32.24
1	14.50	15.15	17.10	26.29	35.67	90.00	33.12

B.3 Variants of TTN for small test batch size

Online TTN. TTN interpolating weight α can also be adapted during test time. Table 11 shows the results of the TTN online version, which further optimizes the post-trained alpha using the entropy minimization and mean-squared error (MSE) loss between the updated alpha and the initial post-trained alpha. We followed entropy minimization loss used in TENT (Wang et al., 2020), and the MSE loss can be written as $\mathcal{L}_{\text{MSE}} = \|\alpha - \alpha_0\|^2$, where α_0 is the post-trained α. We set the learning rate as 1e-2, 2.5e-3, 5e-4, 1e-4, 5e-5, and 2.5e-5 by linearly decreasing according to the test batch size of 200, 64, 16, 4, 2, and 1 (Goyal et al., 2017). The online TTN shows improvements compared to the offline TTN in all three evaluation scenarios (single, continuously changing, and mixed).

Scaled TTN. Following the observation from Table 10, we lowered the interpolating weight by multiplying a constant scale value, ranging $(0, 1)$, to the optimized TTN α. In Table 11, we empirically set the scale value as 0.4.

Dynamic training batch size. We observe that using the dynamic batch size in the post-training stage also improves the performance for small test batch sizes (2 or 1), while slightly deteriorating the performance for large test batch sizes (200 or 64). We randomly sampled training batch size from the range of $[4, 200]$ for each iteration. Other hyperparameters are kept as the same.

Table 11: **TTN variants.** Error rate (↓) averaged over 15 corruptions of CIFAR-10-C (WRN-40).

Method	Eval. setting	Test batch size						Avg.
		200	64	16	4	2	1	
TTN (offline, default)	Single & Cont.	11.67	11.80	12.13	13.93	15.83	17.99	13.89
TTN (offline, default)	Mixed	12.16	12.19	12.34	13.96	15.55	17.83	14.00
TTN (online)	Single	11.39	11.64	11.97	13.70	15.41	17.49	13.60
TTN (online)	Cont.	11.67	11.96	12.15	13.90	15.67	17.72	13.85
TTN (online)	Mixed	12.04	12.04	12.06	13.90	15.46	17.62	13.85
TTN (scaled)	Single & Cont.	13.20	13.38	13.35	13.88	14.54	15.17	13.92
TTN (scaled)	Mixed	13.17	13.05	13.17	13.74	14.36	15.09	13.76
TTN (dynamic train batch size)	Single & Cont.	11.82	12.04	12.17	13.60	15.13	17.22	13.66
TTN (dynamic train batch size)	Mixed	12.12	12.01	11.91	13.43	14.76	17.23	13.58

B.4 Study on augmentation type

TTN is robust to the data augmentation. We used data augmentation in the post-training phase to simulate domain shifts. The rationale for the simulation is to expose the model to different input domains. Especially when obtaining the prior, we compare the outputs from shifted domains with the clean (original) domain in order to analyze which part of the model is affected by the domain discrepancy. Therefore, changing the domain itself is what matters, not *which* domain the input is shifted to. We demonstrated this by conducting ablation studies by varying the augmentation type.

We analyzed various augmentation types when obtaining prior and when optimizing alpha and the results are shown in Figure 5 (a) and (b), respectively. We use a true corruption, *i.e.,* one of 15 corruption types in the corruption benchmark, as an augmentation type in the post-training phase to analyze how TTN works if the augmentation type and test corruption type are misaligned or perfectly aligned. Specifically, we used the true corruption when obtaining the prior while keeping the augmentation choices when optimizing alpha as described in the appendix A.1, and vice versa. Expectedly, the diagonal elements, i.e., where the same corruption type is used both for in post-training and in test time, tend to show the lowest error rate in Figure 5(b). The row-wise standard deviation is lower than 1 in most cases and even lower than 0.5 in Figure 5(a), which means the prior is invariant to the augmentation choice. Moreover, we observe that the average error rate over all elements, 11.74% in ablation for obtaining prior and 11.67% in ablation for optimizing alpha, is almost as same as TTN result 11.67% (See Table 14 and Figure 5). Moreover, we conducted an ablation study on the choice of the augmentation type using CIFAR-10-C and WRN-40 (Table12). We observe that obtaining prior and optimizing alpha steps are robust to the augmentation types.

Table 12: **Ablation study on augmentation choice.** From left to right one augmentation type is added at a time. Default setting, which we used in all experiments, is colored in gray.

Prior augmentation type	color jitter	+ grayscale	**+ invert**	+ gaussian blur	+ horizontal flip
	12.03	11.83	11.67	11.59	11.58
Optimizing α augmentation type	color jitter	+ padding	+ affine	+ gaussian blur	**+ horizontal flip**
	11.78	11.78	11.7	11.70	11.67

B.5 Results on ImageNet-C

Table 13 shows the experimental results using ResNet-50 (He et al., 2016) on ImageNet-C (Hendrycks & Dietterich, 2018) dataset. We emphasized the effectiveness of our proposed method by showing the significant improvement in large-scale dataset experiment. Similar to the results in CIFAR-10/100-C, TTN showed the best performance compared to normalization-based methods (TBN (Nado et al., 2020), AdaptiveBN Schneider et al. (2020), and α-BN (You et al., 2021)) and improved TTA performance when it is applied to optimization-based methods (TENT (Wang et al., 2020) and SWR (Choi et al., 2022)).

During post-training, we used LR of 2.5e-4 and batch size of 64. In test time, we used the learning rate of 2.5e-4 for TENT following the setting from the original paper, and used 5e-6 for TENT+TTN. For SWR and SWR+TTN, we used learning rate of 2.5e-4 for B=64, and 2.5e-6 for B=16, 4, 2, and 1. We had to carefully tune the learning rate especially for SWR, since the method updates the

entire model parameters in an unsupervised manner and hence is very sensitive to the learning rate when the test batch size becomes small. Other hyperparameters and details are kept same (See the appendix A.1 for more details). Moreover, to avoid rapid accumulation, we stored gradients for sufficiently large test samples and then updated the model parameters (for example, we conducted adaptation in every 64 iterations in the case of batch size of 1) in both TENT and SWR.

Table 13: **Single domain adaptation on ImageNet-C (ResNet-50).** Error rate (↓) averaged over 15 corruption types with severity level 5 is reported for each test batch size.

	Method	Test batch size					Avg. Error
		64	16	4	2	1	
	Source (CBN)	93.34	93.34	93.34	93.34	93.34	93.34
Norm	TBN	74.24	76.81	85.74	95.35	99.86	86.40
Norm	AdaptiveBN	77.86	81.47	86.71	90.15	91.11	85.46
Norm	α-BN	86.06	86.32	87.16	88.33	90.45	87.66
Norm	**Ours (TTN)**	**72.21**	**73.18**	**76.98**	**81.52**	**88.49**	**78.48**
Optim.	TENT	**66.56**	72.61	93.37	99.46	99.90	86.41
Optim.	**+Ours (TTN)**	71.42	**72.45**	**76.66**	**81.89**	**91.00**	**78.68**
Optim.	SWR	64.41	74.19	84.30	93.05	99.86	83.16
Optim.	**+Ours (TTN)**	**55.68**	**69.25**	**78.48**	**86.37**	**94.08**	**76.77**

B.6 Error Rates of Each Corruption

In Table 14, we show the error rates of TTN for each corruption of CIFAR-10-C using the WRN-40 backbone. The averaged results over the corruptions are in Table 1.

Table 14: **Results of each corruption (CIFAR-10-C)**

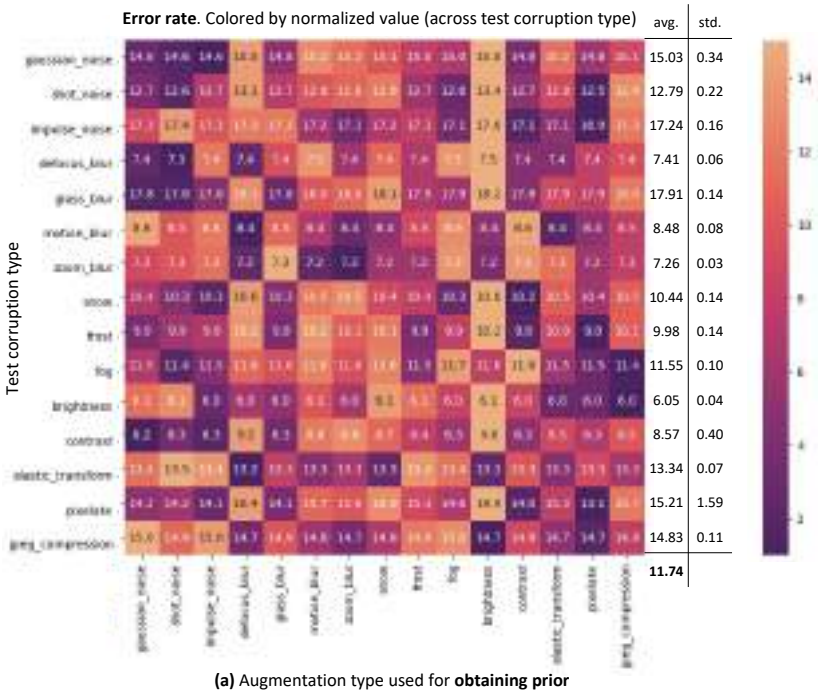
batch size	gauss.	shot	impulse	defocus	glass	motion	zoom	snow	frost	fog	brightness	contrast	elastic	pixel.	jpeg.	Avg.
200	14.81	12.78	17.32	7.37	17.87	8.51	7.23	10.29	9.88	11.29	6.06	8.36	13.42	14.89	14.94	11.67
64	14.81	12.81	17.42	7.41	18.21	8.66	7.41	10.44	9.93	11.63	6.11	8.35	13.59	15.18	15.02	11.80
16	15.23	13.00	17.98	7.71	18.46	8.95	7.68	10.85	10.17	12.21	6.25	8.54	13.95	15.67	15.34	12.13
4	17.03	15.29	19.75	9.26	20.93	10.01	9.62	12.58	11.85	13.65	7.69	9.25	16.21	18.08	17.70	13.93
2	19.21	16.80	21.86	10.70	23.74	11.39	11.92	14.00	13.39	15.38	8.95	10.13	18.92	20.68	20.40	15.83
1	22.03	19.97	25.24	12.41	26.99	12.22	14.39	14.73	14.60	17.27	10.16	9.51	22.10	24.12	24.09	17.99

B.7 Segmentation Results

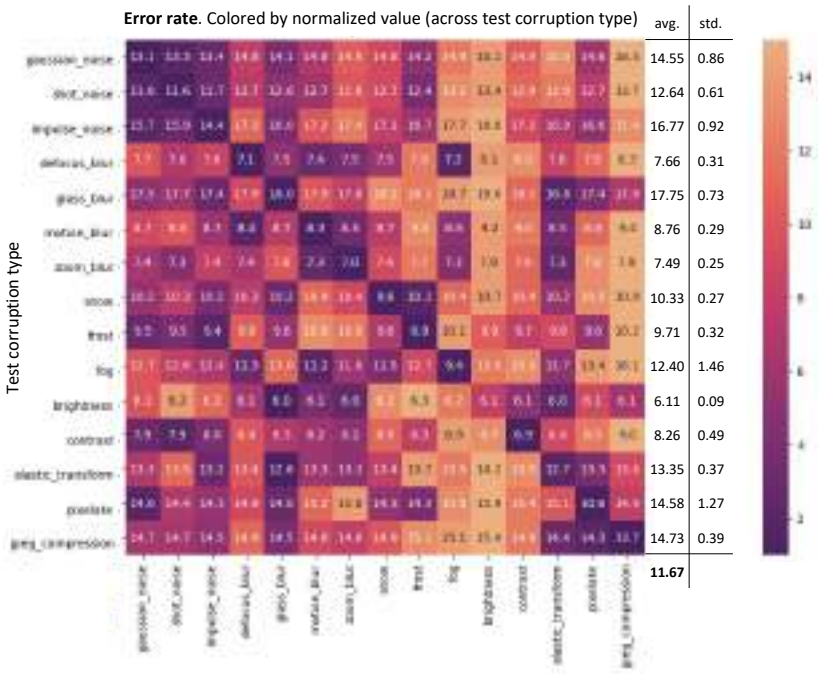
For more comparisons, we add segmentation results for TBN and TTN using test batch size (B) of 4 and 8 (Table 15). The results demonstrate that TTN is robust to the test batch sizes. In other words, the performance difference across the test batch size is small when using TTN (TTN with B=2, 4, and 8). The results are averaged over 3 runs (i.e., using 3 independently optimized α), and the standard deviation is denoted with a $\pm$ sign. We omitted the standard deviation for TBN, which is 0.0 for every result since no optimization is required. Since the backbone network is trained with a train batch size of 8, we assume that test batch statistics estimated from the test batch with B=8 are sufficiently reliable. Accordingly, TBN with B=8 shows compatible results. However, when B becomes small (i.e., in a more practical scenario), problematic test batch statistics are estimated, and thus TBN suffers from the performance drop while TTN keeps showing robust performance. It is worth noting that TTN outperforms TBN by 3.77% in average accuracy when B=2, i.e., in the most practical evaluation setting, and by 2.54% and 1.99% for B=4 and 8, respectively.

Table 15: **Adaptation on DG benchmarks in semantic segmentation.** mIoU(↑) on four unseen domains with test batch size of 2, 4, and 8 using ResNet-50 based DeepLabV3+ as a backbone.

	Method (Cityscapes→)	BDD-100K	Mapillary	GTAV	SYNTHIA	Cityscapes
Norm	TBN (B=2)	43.12	47.61	42.51	25.71	72.94
Norm	**Ours (TTN)** (B=2)	**47.40**($\pm$ 0.02)	**56.88**($\pm$ 0.04)	**44.69**($\pm$ 0.03)	**26.68**($\pm$ 0.01)	**75.09**($\pm$ 0.01)
Norm	TBN (B=4)	45.64	49.17	44.26	25.96	74.29
Norm	**Ours (TTN)** (B=4)	**47.72**($\pm$ 0.01)	**57.11**($\pm$ 0.01)	**45.08**($\pm$ 0.02)	**26.52**($\pm$ 0.01)	**75.56**($\pm$ 0.01)
Norm	TBN (B=8)	46.42	49.38	44.81	25.97	75.42
Norm	**Ours (TTN)** (B=8)	**47.25**($\pm$ 0.02)	**57.28**($\pm$ 0.02)	**45.13**($\pm$ 0.03)	**26.45**($\pm$ 0.01)	**75.82**($\pm$ 0.01)



(a) Augmentation type used for **obtaining prior**



(b) Augmentation type used for **optimizing alpha**

Figure 5: **True test corruption as augmentation.** Each column represents the augmentation type used **(a)** when obtaining prior or **(b)** when optimizing α. Each row represents the test corruption type. The error rate ($\downarrow$) of CIFAR-10-C with severity level 5 and test batch size 200 using WRN-40 is annotated in each element. Element (i, j) represents the error rate when j-th corruption type is used for augmenting the clean train data during post-training and tested on i-th corruption test set. The heatmap is colored by error rate normalized across the test corruption type (row-wise normalization).

Group Normalization

Yuxin Wu Kaiming He

Facebook AI Research (FAIR)

{yuxinwu,kaiminghe}@fb.com

Abstract

Batch Normalization (BN) is a milestone technique in the development of deep learning, enabling various networks to train. However, normalizing along the batch dimension introduces problems — BN's error increases rapidly when the batch size becomes smaller, caused by inaccurate batch statistics estimation. This limits BN's usage for training larger models and transferring features to computer vision tasks including detection, segmentation, and video, which require small batches constrained by memory consumption. In this paper, we present Group Normalization (GN) as a simple alternative to BN. GN divides the channels into groups and computes within each group the mean and variance for normalization. GN's computation is independent of batch sizes, and its accuracy is stable in a wide range of batch sizes. On ResNet-50 trained in ImageNet, GN has 10.6% lower error than its BN counterpart when using a batch size of 2; when using typical batch sizes, GN is comparably good with BN and outperforms other normalization variants. Moreover, GN can be naturally transferred from pre-training to fine-tuning. GN can outperform its BN-based counterparts for object detection and segmentation in COCO,[1] and for video classification in Kinetics, showing that GN can effectively replace the powerful BN in a variety of tasks. GN can be easily implemented by a few lines of code in modern libraries.

1. Introduction

Batch Normalization (Batch Norm or BN) [26] has been established as a very effective component in deep learning, largely helping push the frontier in computer vision [59, 20] and beyond [54]. BN normalizes the features by the mean and variance computed within a (mini-)batch. This has been shown by many practices to ease optimization and enable very deep networks to converge. The stochastic uncertainty of the batch statistics also acts as a regularizer that can benefit generalization. BN has been a foundation of many state-of-the-art computer vision algorithms.

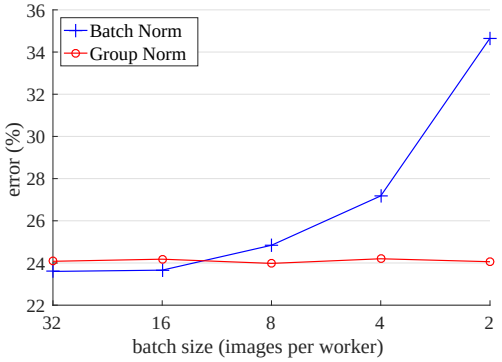


Figure 1. **ImageNet classification error *vs*. batch sizes**. This is a ResNet-50 model trained in the ImageNet training set using 8 workers (GPUs), evaluated in the validation set.

Despite its great success, BN exhibits drawbacks that are also caused by its distinct behavior of normalizing along the batch dimension. In particular, it is required for BN to work with a *sufficiently large batch size* (*e.g.*, 32 per worker[2] [26, 59, 20]). A small batch leads to inaccurate estimation of the batch statistics, and *reducing BN's batch size increases the model error dramatically* (Figure 1). As a result, many recent models [59, 20, 57, 24, 63] are trained with non-trivial batch sizes that are memory-consuming. The heavy reliance on BN's effectiveness to train models in turn prohibits people from exploring higher-capacity models that would be limited by memory.

The restriction on batch sizes is more demanding in computer vision tasks including detection [12, 47, 18], segmentation [38, 18], video recognition [60, 6], and other high-level systems built on them. For example, the Fast/er and Mask R-CNN frameworks [12, 47, 18] use a batch size of 1 or 2 images because of higher resolution, where BN is "frozen" by transforming to a linear layer [20]; in video classification with 3D convolutions [60, 6], the presence of spatial-temporal features introduces a trade-off between the temporal length and batch size. The usage of BN often requires these systems to compromise between the model design and batch sizes.

[1] https://github.com/facebookresearch/Detectron/blob/master/projects/GN.

[2] In the context of this paper, we use "batch size" to refer to the number of samples *per worker* (*e.g.*, GPU). BN's statistics are computed for each worker, but *not* broadcast across workers, as is standard in many libraries.

This paper presents Group Normalization (GN) as a simple alternative to BN. We notice that many classical features like SIFT [39] and HOG [9] are *group-wise* features and involve *group-wise normalization*. For example, a HOG vector is the outcome of several spatial cells where each cell is represented by a normalized orientation histogram. Analogously, we propose GN as a layer that divides channels into groups and normalizes the features within each group (Figure 2). GN does not exploit the batch dimension, and its computation is independent of batch sizes.

GN behaves very stably over a wide range of batch sizes (Figure 1). With a batch size of 2 samples, GN has 10.6% lower error than its BN counterpart for ResNet-50 [20] in ImageNet [50]. With a regular batch size, GN is comparably good as BN (with a gap of ~0.5%) and outperforms other normalization variants [3, 61, 51]. Moreover, although the batch size may change, GN can naturally transfer from pre-training to fine-tuning. GN shows improved results *vs*. its BN counterpart on Mask R-CNN for COCO object detection and segmentation [37], and on 3D convolutional networks for Kinetics video classification [30]. The effectiveness of GN in ImageNet, COCO, and Kinetics demonstrates that GN is a competitive alternative to BN that has been dominant in these tasks.

There have been existing methods, such as Layer Normalization (LN) [3] and Instance Normalization (IN) [61] (Figure 2), that also avoid normalizing along the batch dimension. These methods are effective for training sequential models (RNN/LSTM [49, 22]) or generative models (GANs [15, 27]). But as we will show by experiments, both LN and IN have limited success in visual recognition, for which GN presents better results. Conversely, GN could be used in place of LN and IN and thus is applicable for sequential or generative models. This is beyond the focus of this paper, but it is suggestive for future research.

2. Related Work

Normalization. It is well-known that normalizing the input data makes training faster [33]. To normalize hidden features, initialization methods [33, 14, 19] have been derived based on strong assumptions of feature distributions, which can become invalid when training evolves.

Normalization layers in deep networks had been widely used before the development of BN. Local Response Normalization (LRN) [40, 28, 32] was a component in AlexNet [32] and following models [64, 53, 58]. Unlike recent methods [26, 3, 61], LRN computes the statistics in a small neighborhood for each pixel.

Batch Normalization [26] performs more global normalization along the batch dimension (and as importantly, it suggests to do this for all layers). But the concept of "batch" is not always present, or it may change from time to time. For example, batch-wise normalization is not legitimate at inference time, so the mean and variance are pre-computed from the training set [26], often by running average; consequently, there is no normalization performed when testing. The pre-computed statistics may also change when the target data distribution changes [45]. These issues lead to inconsistency at training, transferring, and testing time. In addition, as aforementioned, reducing the batch size can have dramatic impact on the estimated batch statistics.

Several normalization methods [3, 61, 51, 2, 46] have been proposed to avoid exploiting the batch dimension. Layer Normalization (LN) [3] operates along the channel dimension, and Instance Normalization (IN) [61] performs BN-like computation but only for each sample (Figure 2). Instead of operating on features, Weight Normalization (WN) [51] proposes to normalize the filter weights. These methods do not suffer from the issues caused by the batch dimension, but they have not been able to approach BN's accuracy in many visual recognition tasks. We provide comparisons with these methods in context of the remaining sections.

Addressing small batches. Ioffe [25] proposes Batch Renormalization (BR) that alleviates BN's issue involving small batches. BR introduces two extra parameters that constrain the estimated mean and variance of BN within a certain range, reducing their drift when the batch size is small. BR has better accuracy than BN in the small-batch regime. But BR is also batch-dependent, and when the batch size decreases its accuracy still degrades [25].

There are also attempts to *avoid* using small batches. The object detector in [43] performs synchronized BN whose mean and variance are computed across multiple GPUs. However, this method does not solve the problem of small batches; instead, it migrates the algorithm problem to engineering and hardware demands, using a number of GPUs proportional to BN's requirements. Moreover, the synchronized BN computation prevents using *asynchronous* solvers (ASGD [10]), a practical solution to large-scale training widely used in industry. These issues can limit the scope of using synchronized BN.

Instead of addressing the batch statistics computation (*e.g*., [25, 43]), our normalization method inherently avoids this computation.

Group-wise computation. *Group convolutions* have been presented by AlexNet [32] for distributing a model into two GPUs. The concept of *groups* as a dimension for model design has been more widely studied recently. The work of ResNeXt [63] investigates the trade-off between depth, width, and groups, and it suggests that a larger number of groups can improve accuracy under similar computational cost. MobileNet [23] and Xception [7] exploit *channel-wise* (also called "depth-wise") convolutions, which are group convolutions with a group number equal to the channel

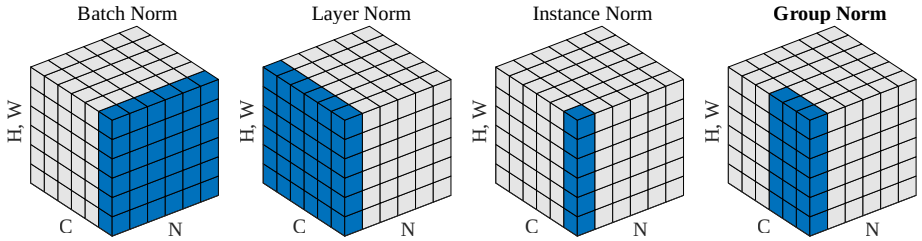


Figure 2. **Normalization methods**. Each subplot shows a feature map tensor, with N as the batch axis, C as the channel axis, and (H, W) as the spatial axes. The pixels in blue are normalized by the same mean and variance, computed by aggregating the values of these pixels.

number. ShuffleNet [65] proposes a channel shuffle operation that permutes the axes of grouped features. These methods all involve dividing the channel dimension into groups. Despite the relation to these methods, GN does *not* require group convolutions. GN is a generic layer, as we evaluate in standard ResNets [20].

3. Group Normalization

The channels of visual representations are not entirely independent. Classical features of SIFT [39], HOG [9], and GIST [41] are *group-wise* representations by design, where each group of channels is constructed by some kind of histogram. These features are often processed by *group-wise normalization* over each histogram or each orientation. Higher-level features such as VLAD [29] and Fisher Vectors (FV) [44] are also group-wise features where a group can be thought of as the sub-vector computed with respect to a cluster.

Analogously, it is not necessary to think of deep neural network features as unstructured vectors. For example, for conv_1 (the first convolutional layer) of a network, it is reasonable to expect a filter and its horizontal flipping to exhibit similar distributions of filter responses on natural images. If conv_1 happens to approximately learn this pair of filters, or if the horizontal flipping (or other transformations) is made into the architectures by design [11, 8], then the corresponding channels of these filters can be normalized together.

The higher-level layers are more abstract and their behaviors are not as intuitive. However, in addition to orientations (SIFT [39], HOG [9], or [11, 8]), there are many factors that could lead to grouping, *e.g.*, frequency, shapes, illumination, textures. Their coefficients can be interdependent. In fact, a well-accepted computational model in neuroscience is to normalize across the cell responses [21, 52, 55, 5], "with various receptive-field centers (covering the visual field) and with various spatiotemporal frequency tunings" (p183, [21]); this can happen not only in the primary visual cortex, but also "throughout the visual system" [5]. Motivated by these works, we propose new generic group-wise normalization for deep neural networks.

3.1. Formulation

We first describe a general formulation of feature normalization, and then present GN in this formulation. A family of feature normalization methods, including BN, LN, IN, and GN, perform the following computation:

$$\hat{x}_i = \frac{1}{\sigma_i}(x_i - \mu_i). \tag{1}$$

Here x is the feature computed by a layer, and i is an index. In the case of 2D images, $i = (i_N, i_C, i_H, i_W)$ is a 4D vector indexing the features in (N, C, H, W) order, where N is the batch axis, C is the channel axis, and H and W are the spatial height and width axes.

μ and σ in (1) are the mean and standard deviation (std) computed by:

$$\mu_i = \frac{1}{m}\sum_{k\in\mathcal{S}_i} x_k, \quad \sigma_i = \sqrt{\frac{1}{m}\sum_{k\in\mathcal{S}_i}(x_k - \mu_i)^2 + \epsilon}, \tag{2}$$

with ϵ as a small constant. $\mathcal{S}_i$ is the set of pixels in which the mean and std are computed, and m is the size of this set. Many types of feature normalization methods mainly differ in how the set $\mathcal{S}_i$ is defined (Figure 2), discussed as follows.

In **Batch Norm** [26], the set $\mathcal{S}_i$ is defined as:

$$\mathcal{S}_i = \{k \mid k_C = i_C\}, \tag{3}$$

where i_C (and k_C) denotes the sub-index of i (and k) along the C axis. This means that the pixels sharing the same channel index are normalized together, *i.e.*, for each channel, BN computes μ and σ along the (N, H, W) axes. In **Layer Norm** [3], the set is:

$$\mathcal{S}_i = \{k \mid k_N = i_N\}, \tag{4}$$

meaning that LN computes μ and σ along the (C, H, W) axes for each sample. In **Instance Norm** [61], the set is:

$$\mathcal{S}_i = \{k \mid k_N = i_N, k_C = i_C\}. \tag{5}$$

meaning that IN computes μ and σ along the (H, W) axes for each sample and each channel. The relations among BN, LN, and IN are in Figure 2.

As in [26], all methods of BN, LN, and IN learn a per-channel linear transform to compensate for the possible lost of representational ability:

$$y_i = \gamma \hat{x}_i + \beta, \tag{6}$$

where γ and β are trainable scale and shift (indexed by i_C in all case, which we omit for simplifying notations).

Group Norm. Formally, a Group Norm layer computes μ and σ in a set $\mathcal{S}_i$ defined as:

$$\mathcal{S}_i = \{k \mid k_N = i_N, \lfloor \frac{k_C}{C/G} \rfloor = \lfloor \frac{i_C}{C/G} \rfloor \}. \tag{7}$$

Here G is the number of groups, which is a pre-defined hyper-parameter ($G = 32$ by default). C/G is the number of channels per group. $\lfloor \cdot \rfloor$ is the floor operation, and "$\lfloor \frac{k_C}{C/G} \rfloor = \lfloor \frac{i_C}{C/G} \rfloor$" means that the indexes i and k are in the same group of channels, assuming each group of channels are stored in a sequential order along the C axis. GN computes μ and σ along the (H, W) axes and along a group of $\frac{C}{G}$ channels. The computation of GN is illustrated in Figure 2 (rightmost), which is a simple case of 2 groups ($G = 2$) each having 3 channels.

Given $\mathcal{S}_i$ in Eqn.(7), a GN layer is defined by Eqn.(1), (2), and (6). Specifically, the pixels in the same group are normalized together by the same μ and σ. GN also learns the per-channel γ and β.

Relation to Prior Work. LN, IN, and GN all perform independent computations along the batch axis. The two extreme cases of GN are equivalent to LN and IN (Figure 2).

Relation to Layer Normalization [3]. GN becomes LN if we set the group number as $G = 1$. LN assumes *all* channels in a layer make "similar contributions" [3]. Unlike the case of fully-connected layers studied in [3], this assumption can be less valid with the presence of convolutions, as discussed in [3]. GN is less restricted than LN, because each group of channels (instead of all of them) are assumed to subject to the shared mean and variance; the model still has flexibility of learning a different distribution for each group. This leads to improved representational power of GN over LN, as shown by the lower training and validation error in experiments (Figure 4).

Relation to Instance Normalization [61]. GN becomes IN if we set the group number as $G = C$ (*i.e.*, one channel per group). But IN can only rely on the spatial dimension for computing the mean and variance and it misses the opportunity of exploiting the channel dependence.

3.2. Implementation

GN can be easily implemented by a few lines of code in PyTorch [42] and TensorFlow [1] where automatic differentiation is supported. Figure 3 shows the code based on TensorFlow. In fact, we only need to specify how the mean and variance ("moments") are computed, along the appropriate axes as defined by the normalization method.

```
def GroupNorm(x, gamma, beta, G, eps=1e−5):
    # x: input features with shape [N,C,H,W]
    # gamma, beta: scale and offset, with shape [1,C,1,1]
    # G: number of groups for GN

    N, C, H, W = x.shape
    x = tf.reshape(x, [N, G, C // G, H, W])

    mean, var = tf.nn.moments(x, [2, 3, 4], keep_dims=True)
    x = (x − mean) / tf.sqrt(var + eps)

    x = tf.reshape(x, [N, C, H, W])

    return x ∗ gamma + beta
```

Figure 3. Python code of Group Norm based on TensorFlow.

4. Experiments

4.1. Image Classification in ImageNet

We experiment in the ImageNet classification dataset [50] with 1000 classes. We train on the ~1.28M training images and evaluate on the 50,000 validation images, using the ResNet models [20].

Implementation details. As standard practice [20, 17], we use 8 GPUs to train all models, and the batch mean and variance of BN are computed *within* each GPU. We use the method of [19] to initialize all convolutions for all models. We use 1 to initialize all γ parameters, except for each residual block's last normalization layer where we initialize γ by 0 following [16] (such that the initial state of a residual block is identity). We use a weight decay of 0.0001 for all weight layers, including γ and β (following [17] but unlike [20, 16]). We train 100 epochs for all models, and decrease the learning rate by $10\times$ at 30, 60, and 90 epochs. During training, we adopt the data augmentation of [58] as implemented by [17]. We evaluate the top-1 classification error on the center crops of 224×224 pixels in the validation set. To reduce random variations, we report the median error rate of the final 5 epochs [16]. Other implementation details follow [17].

Our baseline is the ResNet trained with BN [20]. To compare with LN, IN, and GN, we replace BN with the specific variant. We use the same hyper-parameters for all models. We set $G = 32$ for GN by default.

Comparison of feature normalization methods. We first experiment with a regular batch size of **32 images** (per GPU) [26, 20]. BN works successfully in this regime, so this is a strong baseline to compare with. Figure 4 shows the error curves, and Table 1 shows the final results.

Figure 4 shows that *all* of these normalization methods are able to converge. LN has a small degradation of 1.7%

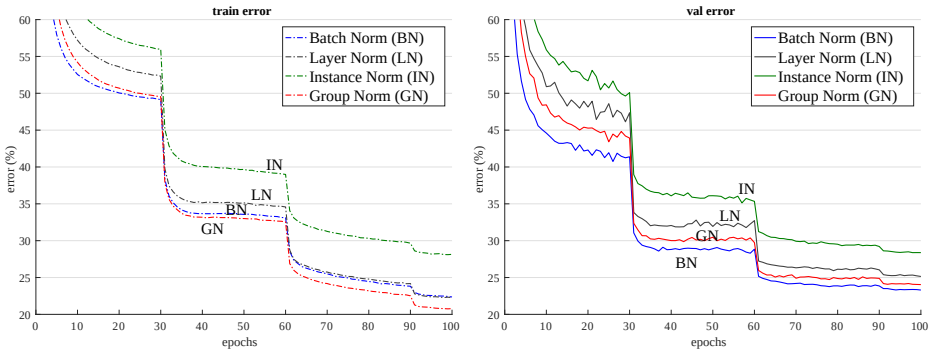


Figure 4. **Comparison of error curves** with a batch size of **32 images**/GPU. We show the ImageNet training error (left) and validation error (right) *vs*. numbers of training epochs. The model is ResNet-50.

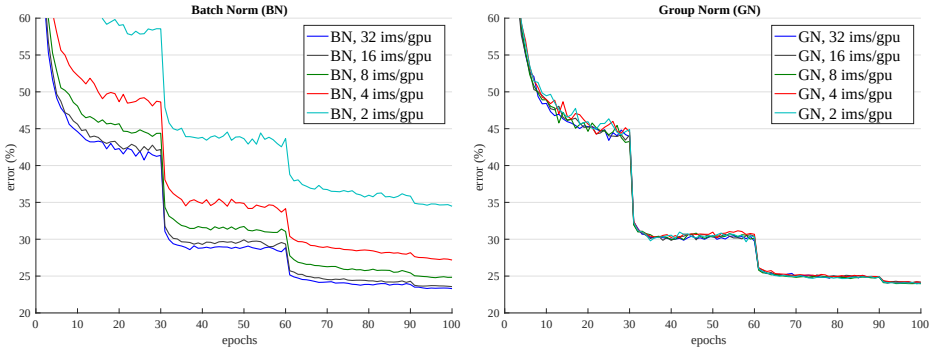


Figure 5. **Sensitivity to batch sizes**: ResNet-50's validation error of BN (left) and GN (right) trained with 32, 16, 8, 4, and 2 images/GPU.

	BN	LN	IN	GN
val error	**23.6**	25.3	28.4	24.1
△ (*vs*. BN)	-	*1.7*	*4.8*	***0.5***

Table 1. **Comparison of error rates** (%) of ResNet-50 in the ImageNet validation set, trained with a batch size of **32 images**/GPU. The error curves are in Figure 4.

comparing with BN. This is an encouraging result, as it suggests that normalizing along *all* channels (as done by LN) of a *convolutional* network is reasonably good. IN also makes the model converge, but is 4.8% worse than BN.[3]

In this regime where BN works well, GN is able to approach BN's accuracy, with a decent degradation of 0.5% in the validation set. Actually, Figure 4 (left) shows that GN has *lower training error* than BN, indicating that GN is effective for easing *optimization*. The slightly higher validation error of GN implies that GN loses some regularization ability of BN. This is understandable, because BN's mean and variance computation introduces uncertainty caused by the stochastic batch sampling, which helps regularization [26]. This uncertainty is missing in GN (and LN/IN). But it is possible that GN combined with a suitable regularizer will improve results. This can be a future research topic.

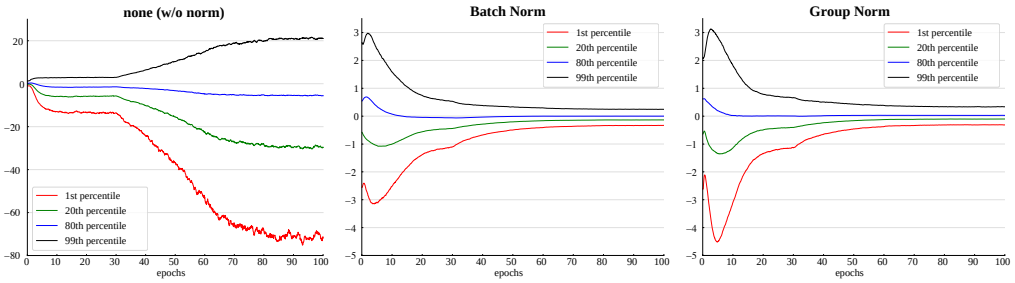
[3]For completeness, we have also trained ResNet-50 with WN [51], which is *filter* (instead of *feature*) normalization. WN's result is 28.2%.

batch size	32	16	8	4	2
BN	**23.6**	**23.7**	24.8	27.3	34.7
GN	24.1	24.2	**24.0**	**24.2**	**24.1**
△	*0.5*	*0.5*	*-0.8*	*-3.1*	*-10.6*

Table 2. **Sensitivity to batch sizes**. We show ResNet-50's validation error (%) in ImageNet. The last row shows the differences between BN and GN. The error curves are in Figure 5. This table is visualized in Figure 1.

Small batch sizes. Although BN benefits from the stochasticity under some situations, its error increases when the batch size becomes smaller and the uncertainty gets bigger. We show this in Figure 1, Figure 5, and Table 2.

We evaluate batch sizes of 32, 16, 8, 4, 2 images per GPU. In all cases, the BN mean and variance are computed within each GPU and not synchronized. All models are trained in 8 GPUs. In this set of experiments, we adopt the linear learning rate scaling rule [31, 4, 16] to adapt to batch size changes — we use a learning rate of 0.1 [20] for the batch size of 32, and $0.1N/32$ for a batch size of N. This linear scaling rule works well for BN if the total batch size changes (by changing the number of GPUs) but the per-GPU batch size does not change [16]. We keep the same number of training epochs for all cases (Figure 5, x-axis). All other hyper-parameters are unchanged.



	error
none	29.2
BN	28.0
GN	**27.6**

Figure 6. **Evolution of feature distributions** of conv_{5_3}'s output (before normalization and ReLU) from VGG-16, shown as the {1, 20, 80, 99} percentile of responses. The table on the right shows the ImageNet validation error (%). Models are trained with 32 images/GPU.

groups (G)

64	32	16	8	4	2	1 (=LN)
24.6	**24.1**	24.6	24.4	24.6	24.7	25.3
0.5	-	*0.5*	*0.3*	*0.5*	*0.6*	*1.2*

channels per group

64	32	16	8	4	2	1 (=IN)
24.4	24.5	**24.2**	24.3	24.8	25.6	28.4
0.2	*0.3*	-	*0.1*	*0.6*	*1.4*	*4.2*

Table 3. **Group division**. We show ResNet-50's validation error (%) in ImageNet, trained with 32 images/GPU. (Top): a given number of groups. (Bottom): a given number of channels per group. The last rows show the differences with the best number.

Figure 5 (left) shows that BN's error becomes considerably higher with small batch sizes. GN's behavior is more stable and insensitive to the batch size. Actually, Figure 5 (right) shows that GN has very similar curves (subject to random variations) across a wide range of batch sizes from 32 to 2. In the case of a batch size of 2, GN has **10.6%** lower error rate than its BN counterpart (24.1% *vs*. 34.7%).

These results indicate that the batch mean and variance estimation can be overly stochastic and inaccurate, especially when they are computed over 4 or 2 images. However, this stochasticity disappears if the statistics are computed from 1 image, in which case BN becomes similar to IN at training time. We see that IN has a better result (28.4%) than BN with a batch size of 2 (34.7%).

The robust results of GN in Table 2 demonstrate GN's strength. It allows to remove the batch size constraint imposed by BN, which can give considerably more memory (*e.g.*, 16× or more). This will make it possible to train higher-capacity models that would be otherwise bottlenecked by memory limitation. We hope this will create new opportunities in architecture design.

Comparison with Batch Renorm (BR). BR [25] introduces two extra parameters (r and d in [25]) that constrain the estimated mean and variance of BN. Their values are controlled by r_{max} and d_{max}. To apply BR to ResNet-50, we have carefully chosen these hyper-parameters, and found that $r_{\text{max}} = 1.5$ and $d_{\text{max}} = 0.5$ work best for ResNet-50. With a batch size of 4, ResNet-50 trained with BR has an error rate of 26.3%. This is better than BN's 27.3%, but still 2.1% higher than GN's 24.2%.

Group division. Thus far all presented GN models are trained with a group number of $G = 32$. Next we evaluate different ways of dividing into groups. With a given fixed group number, GN performs reasonably well for all values of G we studied (Table 3, top panel). In the extreme case of $G = 1$, GN is equivalent to LN, and its error rate is higher than all cases of $G > 1$ studied.

We also evaluate fixing the number of channels per group (Table 3, bottom panel). Note that because the layers can have different channel numbers, the group number G can change across layers in this setting. In the extreme case of 1 channel per group, GN is equivalent to IN. Even if using as few as 2 channels per group, GN has substantially lower error than IN (25.6% *vs*. 28.4%). This result shows the effect of grouping channels when performing normalization.

Deeper models. We have also compared GN with BN on ResNet-101 [20]. With a batch size of 32, our BN baseline of ResNet-101 has 22.0% validation error, and the GN counterpart has 22.4%, slightly worse by 0.4%. With a batch size of 2, GN ResNet-101's error is 23.0%. This is still a decently stable result considering the very small batch size, and it is 8.9% better than the BN counterpart's 31.9%.

Results and analysis of VGG models. To study GN/BN compared to *no normalization*, we consider VGG-16 [56] that can be healthily trained without normalization layers. We apply BN or GN right after each convolutional layer. Figure 6 shows the evolution of the feature distributions of conv_{5_3} (the last convolutional layer). GN and BN behave *qualitatively similar*, while being substantially different with the variant that uses no normalization; this phenomenon is also observed for all other convolutional layers. This comparison suggests that performing normalization is essential for controlling the distribution of features.

For VGG-16, GN is *better* than BN by 0.4% (Figure 6, right). This possibly implies that VGG-16 benefits less from BN's regularization effect, and GN (that leads to lower training error) is superior to BN in this case.

4.2. Object Detection and Segmentation in COCO

Next we evaluate fine-tuning the models for transferring to object detection and segmentation. These computer vision tasks in general benefit from higher-resolution input, so the batch size tends to be small in common practice (1 or 2 images/GPU [12, 47, 18, 36]). As a result, BN is turned into a *linear* layer $y = \frac{\gamma}{\sigma}(x - \mu) + \beta$ where μ and σ are pre-computed from the pre-trained model and frozen [20]. We denote this as BN*, which in fact performs no normalization during fine-tuning. We have also tried a variant that fine-tunes BN (normalization is performed and not frozen) and found it works poorly (reducing ~6 AP with a batch size of 2), so we ignore this variant.

We experiment on the Mask R-CNN baselines [18], implemented in the publicly available codebase of *Detectron* [13]. We use the end-to-end variant with the same hyper-parameters as in [13]. We replace BN* with GN during fine-tuning, using the corresponding models pre-trained from ImageNet.[4] During fine-tuning, we use a weight decay of 0 for the γ and β parameters, which is important for good detection results when γ and β are being tuned. We fine-tune with a batch size of 1 image/GPU and 8 GPUs.

The models are trained in the COCO `train2017` set and evaluated in the COCO `val2017` set (a.k.a `minival`). We report the standard COCO metrics of Average Precision (AP), AP_{50}, and AP_{75}, for bounding box detection (AP^{bbox}) and instance segmentation (AP^{mask}).

Results of C4 backbone. Table 4 shows the comparison of GN *vs*. BN* on Mask R-CNN using a $conv_4$ backbone ("C4" [18]). This C4 variant uses ResNet's layers of up to $conv_4$ to extract feature maps, and ResNet's $conv_5$ layers as the Region-of-Interest (RoI) heads for classification and regression. As they are inherited from the pre-trained model, the backbone and head both involve normalization layers.

On this baseline, GN improves over BN* by 1.1 box AP and 0.8 mask AP. We note that the pre-trained GN model is slightly worse than BN in ImageNet (24.1% *vs*. 23.6%), but GN still outperforms BN* for fine-tuning. BN* creates inconsistency between pre-training and fine-tuning (frozen), which may explain the degradation.

We have also experimented with the LN variant, and found it is 1.9 box AP worse than GN and 0.8 worse than BN*. Although LN is also independent of batch sizes, its representational power is weaker than GN.

Results of FPN backbone. Next we compare GN and BN* on Mask R-CNN using a Feature Pyramid Network (FPN) backbone [35], the currently state-of-the-art framework in COCO. Unlike the C4 variant, FPN exploits all pre-trained layers to construct a pyramid, and appends randomly initialized layers as the head. In [35], the box head consists of two hidden fully-connected layers (2fc). We find that replacing the 2fc box head with 4conv1fc (similar to [48]) can better leverage GN. The resulting comparisons are in Table 5.

As a baseline, BN* has 38.6 box AP using the 4conv1fc head, on par with its 2fc counterpart using the same pre-trained model (38.5 AP). By adding GN to all convolutional layers of the box head (but still using the BN* backbone), we increase the box AP by 0.9 to 39.5 (2nd row, Table 5). This ablation shows that a substantial portion of GN's improvement for detection is from *normalization in the head* (which is also done by the C4 variant). On the contrary, applying BN to the box head (that has 512 RoIs per image) does not provide satisfactory result and is ~9 AP worse — in detection, the batch of RoIs are sampled from the same image and their distribution is not *i.i.d.*, and the *non-i.i.d.* distribution is also an issue that degrades BN's batch statistics estimation [25]. GN does not suffer from this problem.

Next we replace the FPN backbone with the GN-based counterpart, *i.e.*, the GN pre-trained model is used during fine-tuning (3rd row, Table 5). Applying GN to the backbone *alone* contributes a 0.5 AP gain (from 39.5 to 40.0), suggesting that GN helps when transferring features.

backbone	AP^{bbox}	AP^{bbox}_{50}	AP^{bbox}_{75}	AP^{mask}	AP^{mask}_{50}	AP^{mask}_{75}
BN*	37.7	57.9	40.9	32.8	54.3	34.7
GN	**38.8**	**59.2**	**42.2**	**33.6**	**55.9**	**35.4**

Table 4. Detection and segmentation **ablation results in COCO**, using Mask R-CNN with **ResNet-50 C4**. BN* means BN is frozen.

backbone	box head	AP^{bbox}	AP^{bbox}_{50}	AP^{bbox}_{75}	AP^{mask}	AP^{mask}_{50}	AP^{mask}_{75}
BN*	-	38.6	59.5	41.9	34.2	56.2	36.1
BN*	GN	39.5	60.0	43.2	34.4	56.4	**36.3**
GN	GN	**40.0**	**61.0**	**43.3**	**34.8**	**57.3**	**36.3**

Table 5. Detection and segmentation **ablation results in COCO**, using Mask R-CNN with **ResNet-50 FPN** and a 4conv1fc bounding box head. BN* means BN is frozen.

	AP^{bbox}	AP^{bbox}_{50}	AP^{bbox}_{75}	AP^{mask}	AP^{mask}_{50}	AP^{mask}_{75}
R50 BN*	38.6	59.8	42.1	34.5	56.4	36.3
R50 GN	40.3	61.0	44.0	35.7	57.9	37.7
R50 GN, long	**40.8**	**61.6**	**44.4**	**36.1**	**58.5**	**38.2**
R101 BN*	40.9	61.9	44.8	36.4	58.5	38.7
R101 GN	41.8	62.5	45.4	36.8	59.2	39.0
R101 GN, long	**42.3**	**62.8**	**46.2**	**37.2**	**59.7**	**39.5**

Table 6. **Detection and segmentation results in COCO** using Mask R-CNN and FPN. Here BN* is the default Detectron baseline [13], and GN is applied to the backbone, box head, and mask head. "long" means training with more iterations. Code of these results are in https://github.com/facebookresearch/Detectron/blob/master/projects/GN.

[4] Detectron [13] uses pre-trained models provided by the authors of [20]. For fair comparisons, we instead use the models pre-trained in this paper. The object detection and segmentation accuracy is statistically similar between these pre-trained models.

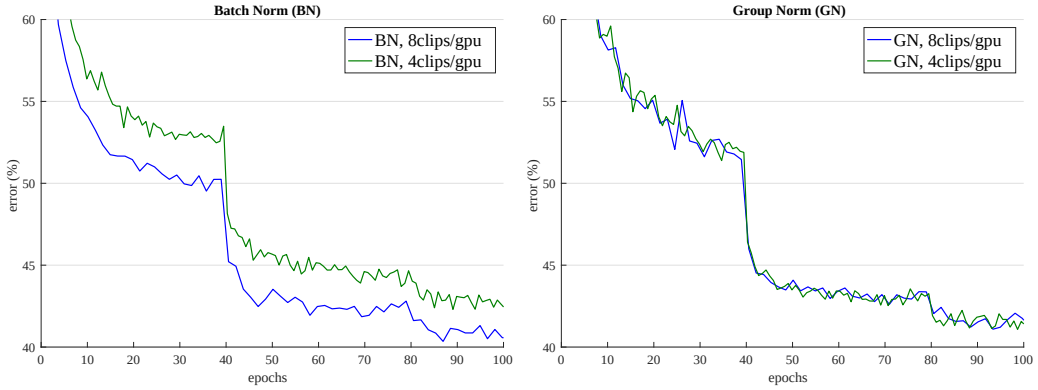


Figure 7. **Error curves in Kinetics with an input length of 32 frames**. We show ResNet-50 I3D's validation error of BN (left) and GN (right) using a batch size of 8 and 4 clips/GPU. The monitored validation error is the 1-clip error under the same data augmentation as the training set, while the final validation accuracy in Table 8 is 10-clip testing without data augmentation.

from scratch	AP^{bbox}	AP^{bbox}_{50}	AP^{bbox}_{75}	AP^{mask}	AP^{mask}_{50}	AP^{mask}_{75}
R50 BN [34]	34.5	55.2	37.7	-	-	-
R50 GN	39.5	59.8	43.6	35.2	56.9	37.6
R101 GN	41.0	61.1	44.9	36.4	58.2	38.7

Table 7. Detection and segmentation results trained **from scratch** in COCO using Mask R-CNN and FPN. Here the BN results are from [34], and BN is synced across GPUs [43] and is *not* frozen. Code of these results are in https://github.com/facebookresearch/Detectron/blob/master/projects/GN.

Table 6 shows the full results of GN (applied to the backbone, box head, and mask head), compared with the standard Detectron baseline [13] based on BN^*. Using the same hyper-parameters as [13], GN increases over BN^* by a healthy margin. Moreover, we found that GN is not fully trained with the default schedule in [13], so we also tried increasing the iterations from 180k to 270k (BN^* does not benefit from longer training). Our final ResNet-50 GN model ("long", Table 6) is **2.2** points box AP and **1.6** points mask AP better than its BN^* variant.

Training Mask R-CNN from scratch. GN allows us to easily investigate training object detectors *from scratch* (without any pre-training). We show the results in Table 7, where the GN models are trained for 270k iterations.[5] To our knowledge, our numbers (**41.0** box AP and **36.4** mask AP) are the best *from-scratch* results in COCO reported to date; they can even compete with the ImageNet-pretrained results in Table 6. As a reference, with synchronous BN [43], a concurrent work [34] achieves a from-scratch result of 34.5 box AP using R50 (Table 7), and 36.3 using a specialized backbone.

[5]For models trained from scratch, we turn off the default StopGrad in Detectron that freezes the first few layers.

clip length	32	32	64
batch size	8	4	4
BN	**73.3 / 90.7**	72.1 / 90.0	73.3 / 90.8
GN	73.0 / 90.6	**72.8 / 90.6**	**74.5 / 91.7**

Table 8. **Video classification results in Kinetics**: ResNet-50 I3D baseline's top-1 / top-5 accuracy (%).

4.3. Video Classification in Kinetics

Lastly we evaluate video classification in the Kinetics dataset [30]. Many video classification models [60, 6] extend the features to 3D spatial-temporal dimensions. This is memory-demanding and imposes constraints on the batch sizes and model designs.

We experiment with Inflated 3D (I3D) convolutional networks [6]. We use the ResNet-50 I3D *baseline* as described in [62]. The models are pre-trained from ImageNet. For both BN and GN, we extend the normalization from over (H, W) to over (T, H, W), where T is the temporal axis. We train in the 400-class Kinetics training set and evaluate in the validation set. We report the top-1 and top-5 classification accuracy, using standard 10-clip testing that averages softmax scores from 10 clips regularly sampled.

We study two different temporal lengths: 32-frame and 64-frame input clips. The 32-frame clip is regularly sampled with a frame interval of 2 from the raw video, and the 64-frame clip is sampled continuously. The model is fully convolutional in spacetime, so the 64-frame variant consumes about $2\times$ more memory. We study a batch size of 8 or 4 clips/GPU for the 32-frame variant, and 4 clips/GPU for the 64-frame variant due to memory limitation.

Results of 32-frame inputs. Table 8 (col. 1, 2) shows the video classification accuracy in Kinetics using 32-frame clips. For the batch size of 8, GN is slightly worse than BN by 0.3% top-1 accuracy and 0.1% top-5. This shows that GN is competitive with BN when BN works well. For

the smaller batch size of 4, GN's accuracy is kept similar (72.8 / 90.6 *vs*. 73.0 / 90.6), but is better than BN's 72.1 / 90.0. BN's accuracy is decreased by 1.2% when the batch size decreases from 8 to 4.

Figure 7 shows the error curves. BN's error curves (left) have a noticeable gap when the batch size decreases from 8 to 4, while GN's error curves (right) are very similar.

Results of 64-frame inputs. Table 8 (col. 3) shows the results of using 64-frame clips. In this case, BN has a result of 73.3 / 90.8. These appear to be acceptable numbers (*vs*. 73.3 / 90.7 of 32-frame, batch size 8), but *the trade-off between the temporal length (64* vs. *32) and batch size (4* vs. *8) could have been overlooked*. Comparing col. 3 and col. 2 in Table 8, we find that the temporal length actually has positive impact (+1.2%), but it is veiled by BN's negative effect of the smaller batch size.

GN does not suffer from this trade-off. The 64-frame variant of GN has 74.5 / 91.7 accuracy, showing healthy gains over its BN counterpart and all BN variants. GN helps the model benefit from temporal length, and the longer clip boosts the top-1 accuracy by 1.7% (top-5 1.1%) with the same batch size.

The improvement of GN on detection, segmentation, and video classification demonstrates that GN is a strong alternative to the powerful and currently dominant BN technique in these tasks.

5. Discussion and Future Work

We have presented GN as an effective normalization layer without exploiting the batch dimension. We have evaluated GN's behaviors in a variety of applications. We note, however, that BN has been so influential that many state-of-the-art systems and their hyper-parameters have been designed for it, which may not be optimal for GN-based models. It is possible that re-designing the systems or searching new hyper-parameters for GN will give better results.

In addition, we have shown that GN is related to LN and IN, two normalization methods that are particularly successful in training recurrent (RNN/LSTM) or generative (GAN) models. This suggests us to study GN in those areas in the future. We will also investigate GN's performance on learning representations for reinforcement learning (RL) tasks, *e.g*., [54], where BN is playing an important role for training very deep models [20].

Acknowledgement. We would like to thank Piotr Dollár and Ross Girshick for helpful discussions.

References

[1] M. Abadi, P. Barham, J. Chen, Z. Chen, A. Davis, J. Dean, M. Devin, S. Ghemawat, G. Irving, M. Isard, et al. Tensorflow: A system for large-scale machine learning. In *Operating Systems Design and Implementation (OSDI)*, 2016.

[2] D. Arpit, Y. Zhou, B. Kota, and V. Govindaraju. Normalization propagation: A parametric technique for removing internal covariate shift in deep networks. In *ICML*, 2016.

[3] J. L. Ba, J. R. Kiros, and G. E. Hinton. Layer normalization. *arXiv:1607.06450*, 2016.

[4] L. Bottou, F. E. Curtis, and J. Nocedal. Optimization methods for large-scale machine learning. *arXiv:1606.04838*, 2016.

[5] M. Carandini and D. J. Heeger. Normalization as a canonical neural computation. *Nature Reviews Neuroscience*, 2012.

[6] J. Carreira and A. Zisserman. Quo vadis, action recognition? a new model and the kinetics dataset. In *CVPR*, 2017.

[7] F. Chollet. Xception: Deep learning with depthwise separable convolutions. In *CVPR*, 2017.

[8] T. Cohen and M. Welling. Group equivariant convolutional networks. In *ICML*, 2016.

[9] N. Dalal and B. Triggs. Histograms of oriented gradients for human detection. In *CVPR*, 2005.

[10] J. Dean, G. Corrado, R. Monga, K. Chen, M. Devin, M. Mao, A. Senior, P. Tucker, K. Yang, Q. V. Le, et al. Large scale distributed deep networks. In *NIPS*, 2012.

[11] S. Dieleman, J. De Fauw, and K. Kavukcuoglu. Exploiting cyclic symmetry in convolutional neural networks. In *ICML*, 2016.

[12] R. Girshick. Fast R-CNN. In *ICCV*, 2015.

[13] R. Girshick, I. Radosavovic, G. Gkioxari, P. Dollár, and K. He. Detectron. https://github.com/facebookresearch/detectron, 2018.

[14] X. Glorot and Y. Bengio. Understanding the difficulty of training deep feedforward neural networks. In *International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2010.

[15] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio. Generative adversarial nets. In *NIPS*, 2014.

[16] P. Goyal, P. Dollár, R. Girshick, P. Noordhuis, L. Wesolowski, A. Kyrola, A. Tulloch, Y. Jia, and K. He. Accurate, large minibatch SGD: Training ImageNet in 1 hour. *arXiv:1706.02677*, 2017.

[17] S. Gross and M. Wilber. Training and investigating Residual Nets. https://github.com/facebook/fb.resnet.torch, 2016.

[18] K. He, G. Gkioxari, P. Dollár, and R. Girshick. Mask R-CNN. In *ICCV*, 2017.

[19] K. He, X. Zhang, S. Ren, and J. Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In *ICCV*, 2015.

[20] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *CVPR*, 2016.

[21] D. J. Heeger. Normalization of cell responses in cat striate cortex. *Visual neuroscience*, 1992.

[22] S. Hochreiter and J. Schmidhuber. Long short-term memory. *Neural computation*, 1997.

[23] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam. MobileNets: Efficient convolutional neural networks for mobile vision applications. *arXiv:1704.04861*, 2017.

FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness

Tri Dao[†], Daniel Y. Fu[†], Stefano Ermon[†], Atri Rudra[‡], and Christopher Ré[†]

[†]Department of Computer Science, Stanford University
[‡]Department of Computer Science and Engineering, University at Buffalo, SUNY

{trid,danfu}@cs.stanford.edu, ermon@stanford.edu, atri@buffalo.edu, chrismre@cs.stanford.edu

June 24, 2022

Abstract

Transformers are slow and memory-hungry on long sequences, since the time and memory complexity of self-attention are quadratic in sequence length. Approximate attention methods have attempted to address this problem by trading off model quality to reduce the compute complexity, but often do not achieve wall-clock speedup. We argue that a missing principle is making attention algorithms *IO-aware*—accounting for reads and writes between levels of GPU memory. We propose FlashAttention, an IO-aware exact attention algorithm that uses tiling to reduce the number of memory reads/writes between GPU high bandwidth memory (HBM) and GPU on-chip SRAM. We analyze the IO complexity of FlashAttention, showing that it requires fewer HBM accesses than standard attention, and is optimal for a range of SRAM sizes. We also extend FlashAttention to block-sparse attention, yielding an approximate attention algorithm that is faster than any existing approximate attention method. FlashAttention trains Transformers faster than existing baselines: 15% end-to-end wall-clock speedup on BERT-large (seq. length 512) compared to the MLPerf 1.1 training speed record, 3× speedup on GPT-2 (seq. length 1K), and 2.4× speedup on long-range arena (seq. length 1K-4K). FlashAttention and block-sparse FlashAttention enable longer context in Transformers, yielding higher quality models (0.7 better perplexity on GPT-2 and 6.4 points of lift on long-document classification) and entirely new capabilities: the first Transformers to achieve better-than-chance performance on the Path-X challenge (seq. length 16K, 61.4% accuracy) and Path-256 (seq. length 64K, 63.1% accuracy).

1 Introduction

Transformer models [82] have emerged as the most widely used architecture in applications such as natural language processing and image classification. Transformers have grown larger [5] and deeper [83], but equipping them with longer context remains difficult [80], since the self-attention module at their heart has time and memory complexity quadratic in sequence length. An important question is whether making attention faster and more memory-efficient can help Transformer models address their runtime and memory challenges for long sequences.

Many approximate attention methods have aimed to reduce the compute and memory requirements of attention. These methods range from sparse-approximation [51, 74] to low-rank approximation [12, 50, 84], and their combinations [3, 9, 92]. Although these methods reduce the compute requirements to linear or near-linear in sequence length, many of them do not display wall-clock speedup against standard attention and have not gained wide adoption. One main reason is that they focus on FLOP reduction (which may not correlate with wall-clock speed) and tend to ignore overheads from memory access (IO).

In this paper, we argue that a missing principle is making attention algorithms *IO-aware* [1]—that is, carefully accounting for reads and writes to different levels of fast and slow memory (e.g., between fast GPU on-chip SRAM and relatively slow GPU high bandwidth memory, or HBM [45], Figure 1 left). On modern

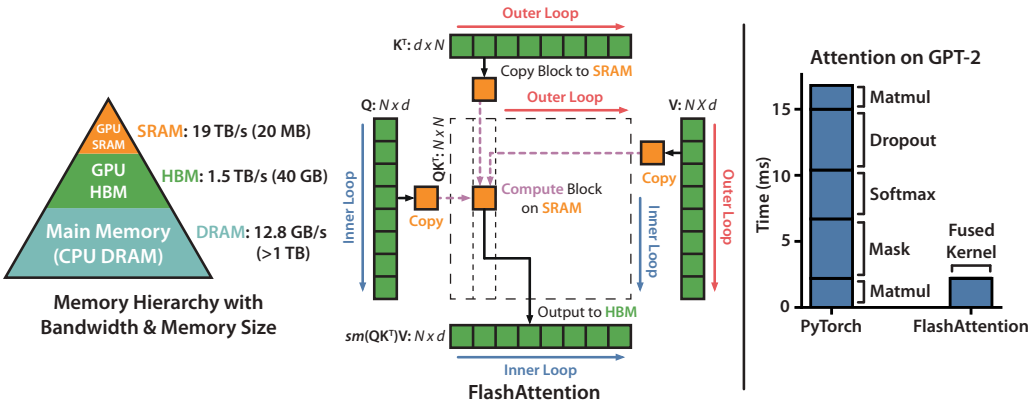


Figure 1: **Left:** FlashAttention uses tiling to prevent materialization of the large $N \times N$ attention matrix (dotted box) on (relatively) slow GPU HBM. In the outer loop (red arrows), FlashAttention loops through blocks of the **K** and **V** matrices and loads them to fast on-chip SRAM. In each block, FlashAttention loops over blocks of **Q** matrix (blue arrows), loading them to SRAM, and writing the output of the attention computation back to HBM. **Right:** Speedup over the PyTorch implementation of attention on GPT-2. FlashAttention does not read and write the large $N \times N$ attention matrix to HBM, resulting in an 7.6× speedup on the attention computation.

GPUs, compute speed has out-paced memory speed [61, 62, 63], and most operations in Transformers are bottlenecked by memory accesses [43]. IO-aware algorithms have been critical for similar memory-bound operations, when reading and writing data can account for a large portion of the runtime—such as database joins [71], image processing [70], numerical linear algebra [4], and more [40, 85]. However, common Python interfaces to deep learning such as PyTorch and Tensorflow do not allow fine-grained control of memory access.

We propose FlashAttention, a new attention algorithm that computes exact attention with far fewer memory accesses. Our main goal is to avoid reading and writing the attention matrix to and from HBM. This requires (i) computing the softmax reduction without access to the whole input (ii) not storing the large intermediate attention matrix for the backward pass. We apply two well-established techniques to address these challenges. (i) We restructure the attention computation to split the input into blocks and make several passes over input blocks, thus incrementally performing the softmax reduction (also known as **tiling**). (ii) We store the softmax normalization factor from the forward pass to quickly **recompute** attention on-chip in the backward pass, which is faster than the standard approach of reading the intermediate attention matrix from HBM. We implement FlashAttention in CUDA to achieve fine-grained control over memory access and fuse all the attention operations into one GPU kernel. Even with the increased FLOPs due to recomputation, our algorithm both **runs faster** (up to 7.6x on GPT-2 [67], Figure 1 right) and **uses less memory**—linear in sequence length—than standard attention, thanks to the massively reduced amount of HBM access.

We analyze the IO complexity [1] of FlashAttention, proving that it requires $O(N^2 d^2 M^{-1})$ HBM accesses where d is the head dimension and M is the size of SRAM, as compared to $\Omega(Nd + N^2)$ of standard attention. For typical values of d and M, FlashAttention requires many times fewer HBM accesses compared to standard attention (up to 9× fewer, as shown in Fig. 2). Moreover, we provide a lower bound, showing that no exact attention algorithm can asymptotically improve on the number of HBM accesses over all SRAM sizes.

We also show that FlashAttention can serve as a useful primitive for realizing the potential of approximate attention algorithms by overcoming their issues with memory access overhead. As a proof of concept, we implement block-sparse FlashAttention, a sparse attention algorithm that is 2-4× faster than even FlashAttention, scaling up to sequence length of 64k. We prove that block-sparse FlashAttention has better IO complexity than FlashAttention by a factor proportional to the sparsity ratio. We discuss further extensions to other operations (attention on multi-GPU, kernel regression, block-sparse matrix

multiply) in Section 5. We open-source FlashAttention to make it easier to build on this primitive.[1]

We empirically validate that FlashAttention speeds up model training and improves model quality by modeling longer context. We also benchmark the runtime and memory footprint of FlashAttention and block-sparse FlashAttention compared to prior attention implementations.

- **Faster Model Training.** FlashAttention trains Transformer models faster in wall-clock time. We train BERT-large (seq. length 512) 15% faster than the training speed record in MLPerf 1.1 [58], GPT2 (seq. length 1K) 3× faster than baseline implementations from HuggingFace [87] and Megatron-LM [77], and long-range arena (seq. length 1K-4K) 2.4× faster than baselines.
- **Higher Quality Models.** FlashAttention scales Transformers to longer sequences, which improves their quality and enables new capabilities. We observe a 0.7 improvement in perplexity on GPT-2 and 6.4 points of lift from modeling longer sequences on long-document classification [13]. FlashAttention enables the first Transformer that can achieve better-than-chance performance on the Path-X [80] challenge, solely from using a longer sequence length (16K). Block-sparse FlashAttention enables a Transformer to scale to even longer sequences (64K), resulting in the first model that can achieve better-than-chance performance on Path-256.
- **Benchmarking Attention.** FlashAttention is up to 3× faster than the standard attention implementation across common sequence lengths from 128 to 2K and scales up to 64K. Up to sequence length of 512, FlashAttention is both faster and more memory-efficient than any existing attention method, whereas for sequence length beyond 1K, some approximate attention methods (e.g., Linformer) start to become faster. On the other hand, block-sparse FlashAttention is faster than all existing approximate attention methods that we know of.

2 Background

We provide some background on the performance characteristics of common deep learning operations on modern hardware (GPUs). We also describe the standard implementation of attention.

2.1 Hardware Performance

We focus here on GPUs. Performance on other hardware accelerators are similar [46, 48].

GPU Memory Hierarchy. The GPU memory hierarchy (Fig. 1 left) comprises multiple forms of memory of different sizes and speeds, with smaller memory being faster. As an example, the A100 GPU has 40-80GB of high bandwidth memory (HBM) with bandwidth 1.5-2.0TB/s and 192KB of on-chip SRAM per each of 108 streaming multiprocessors with bandwidth estimated around 19TB/s [44, 45]. The on-chip SRAM is an order of magnitude faster than HBM but many orders of magnitude smaller in size. As compute has gotten faster relative to memory speed [61, 62, 63], operations are increasingly bottlenecked by memory (HBM) accesses. Thus exploiting fast SRAM becomes more important.

Execution Model. GPUs have a massive number of threads to execute an operation (called a kernel). Each kernel loads inputs from HBM to registers and SRAM, computes, then writes outputs to HBM.

Performance characteristics. Depending on the balance of computation and memory accesses, operations can be classified as either compute-bound or memory-bound. This is commonly measured by the *arithmetic intensity* [85], which is the number of arithmetic operations per byte of memory access.

1. Compute-bound: the time taken by the operation is determined by how many arithmetic operations there are, while time accessing HBM is much smaller. Typical examples are matrix multiply with large inner dimension, and convolution with large number of channels.
2. Memory-bound: the time taken by the operation is determined by the number of memory accesses, while time spent in computation is much smaller. Examples include most other operations: elementwise (e.g., activation, dropout), and reduction (e.g., sum, softmax, batch norm, layer norm).

Kernel fusion. The most common approach to accelerate memory-bound operations is kernel fusion: if there are multiple operations applied to the same input, the input can be loaded once from HBM, instead of multiple times for each operation. Compilers can automatically fuse many elementwise operations [53, 65, 75].

[1] FlashAttention code is available at `https://github.com/HazyResearch/flash-attention`

However, in the context of model training, the intermediate values still need to be written to HBM to save for the backward pass, reducing the effectiveness of naive kernel fusion.

2.2 Standard Attention Implementation

Given input sequences $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ where N is the sequence length and d is the head dimension, we want to compute the attention output $\mathbf{O} \in \mathbb{R}^{N \times d}$:

$$\mathbf{S} = \mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{N \times N}, \quad \mathbf{P} = \mathrm{softmax}(\mathbf{S}) \in \mathbb{R}^{N \times N}, \quad \mathbf{O} = \mathbf{P}\mathbf{V} \in \mathbb{R}^{N \times d},$$

where softmax is applied row-wise.

Standard attention implementations materialize the matrices $\mathbf{S}$ and $\mathbf{P}$ to HBM, which takes $O(N^2)$ memory. Often $N \gg d$ (e.g., for GPT2, $N = 1024$ and $d = 64$). We describe the standard attention implementation in Algorithm 0. As some or most of the operations are memory-bound (e.g., softmax), the large number of memory accesses translates to slow wall-clock time.

This problem is exacerbated by other elementwise operations applied to the attention matrix, such as masking applied to $\mathbf{S}$ or dropout applied to $\mathbf{P}$. As a result, there have been many attempts to fuse several elementwise operations, such as fusing masking with softmax [77].

In Section 3.2, we will show that the standard attention implementation performs HBM accesses quadratic in the sequence length N. We also compare the number of FLOPs and number of HBM accesses of standard attention and of our method (FlashAttention).

Algorithm 0 Standard Attention Implementation

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM.

1: Load $\mathbf{Q}, \mathbf{K}$ by blocks from HBM, compute $\mathbf{S} = \mathbf{Q}\mathbf{K}^\top$, write $\mathbf{S}$ to HBM.
2: Read $\mathbf{S}$ from HBM, compute $\mathbf{P} = \mathrm{softmax}(\mathbf{S})$, write $\mathbf{P}$ to HBM.
3: Load $\mathbf{P}$ and $\mathbf{V}$ by blocks from HBM, compute $\mathbf{O} = \mathbf{P}\mathbf{V}$, write $\mathbf{O}$ to HBM.
4: Return $\mathbf{O}$.

3 FlashAttention: Algorithm, Analysis, and Extensions

We show how to compute exact attention with fewer HBM reads/writes and without storing large intermediate matrices for the backward pass. This yields an attention algorithm that is both memory efficient and faster in wall-clock time. We analyze its IO complexity, showing that our method requires much fewer HBM accesses compared to standard attention. We further show that FlashAttention can serve as a useful primitive by extending it to handle block-sparse attention.

We focus here on the forward pass for ease of exposition; Appendix B contains details for the backward.

3.1 An Efficient Attention Algorithm With Tiling and Recomputation

Given the inputs $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, we aim to compute the attention output $\mathbf{O} \in \mathbb{R}^{N \times d}$ and write it to HBM. Our goal is to reduce the amount of HBM accesses (to sub-quadratic in N).

We apply two established techniques (tiling, recomputation) to overcome the technical challenge of computing exact attention in sub-quadratic HBM accesses. We describe this in Algorithm 1. The main idea is that we split the inputs $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ into blocks, load them from slow HBM to fast SRAM, then compute the attention output with respect to those blocks. By scaling the output of each block by the right normalization factor before adding them up, we get the correct result at the end.

Tiling. We compute attention by blocks. Softmax couples columns of $\mathbf{K}$, so we decompose the large softmax with scaling [51, 60, 66]. For numerical stability, the softmax of vector $x \in \mathbb{R}^B$ is computed as:

$$m(x) := \max_i \; x_i, \quad f(x) := \begin{bmatrix} e^{x_1 - m(x)} & \dots & e^{x_B - m(x)} \end{bmatrix}, \quad \ell(x) := \sum_i f(x)_i, \quad \mathrm{softmax}(x) := \frac{f(x)}{\ell(x)}.$$

For vectors $x^{(1)}, x^{(2)} \in \mathbb{R}^B$, we can decompose the softmax of the concatenated $x = \left[x^{(1)}\ x^{(2)}\right] \in \mathbb{R}^{2B}$ as:

$$m(x) = m(\left[x^{(1)}\ x^{(2)}\right]) = \max(m(x^{(1)}), m(x^{(2)})), \quad f(x) = \left[e^{m(x^{(1)})-m(x)} f(x^{(1)}) \quad e^{m(x^{(2)})-m(x)} f(x^{(2)})\right],$$

$$\ell(x) = \ell(\left[x^{(1)}\ x^{(2)}\right]) = e^{m(x^{(1)})-m(x)}\ell(x^{(1)}) + e^{m(x^{(2)})-m(x)}\ell(x^{(2)}), \quad \text{softmax}(x) = \frac{f(x)}{\ell(x)}.$$

Therefore if we keep track of some extra statistics $(m(x), \ell(x))$, we can compute softmax one block at a time.[2] We thus split the inputs $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ into blocks (Algorithm 1 line 3), compute the softmax values along with extra statistics (Algorithm 1 line 10), and combine the results (Algorithm 1 line 12).

Recomputation. One of our goals is to not store $O(N^2)$ intermediate values for the backward pass. The backward pass typically requires the matrices $\mathbf{S}, \mathbf{P} \in \mathbb{R}^{N \times N}$ to compute the gradients with respect to $\mathbf{Q}, \mathbf{K}, \mathbf{V}$. However, by storing the output $\mathbf{O}$ and the softmax normalization statistics (m, ℓ), we can recompute the attention matrix $\mathbf{S}$ and $\mathbf{P}$ easily in the backward pass from blocks of $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ in SRAM. This can be seen as a form of selective gradient checkpointing [10, 34]. While gradient checkpointing has been suggested to reduce the maximum amount of memory required [66], all implementations (that we know off) have to trade speed for memory. In contrast, even with more FLOPs, our recomputation speeds up the backward pass due to reduced HBM accesses (Fig. 2). The full backward pass description is in Appendix B.

Implementation details: Kernel fusion. Tiling enables us to implement our algorithm in one CUDA kernel, loading input from HBM, performing all the computation steps (matrix multiply, softmax, optionally masking and dropout, matrix multiply), then write the result back to HBM (masking and dropout in Appendix B). This avoids repeatedly reading and writing of inputs and outputs from and to HBM.

Algorithm 1 FlashAttention

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, on-chip SRAM of size M.

1: Set block sizes $B_c = \lceil \frac{M}{4d} \rceil$, $B_r = \min\left(\lceil \frac{M}{4d} \rceil, d\right)$.
2: Initialize $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}$, $\ell = (0)_N \in \mathbb{R}^N$, $m = (-\infty)_N \in \mathbb{R}^N$ in HBM.
3: Divide $\mathbf{Q}$ into $T_r = \left\lceil \frac{N}{B_r} \right\rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ in to $T_c = \left\lceil \frac{N}{B_c} \right\rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
4: Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_i, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_i, \dots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \dots, m_{T_r}$ of size B_r each.
5: **for** $1 \le j \le T_c$ **do**
6: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
7: **for** $1 \le i \le T_r$ **do**
8: Load $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
9: On chip, compute $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
10: On chip, compute $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.
11: On chip, compute $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}$, $\ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$.
12: Write $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1}(\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij} \mathbf{V}_j)$ to HBM.
13: Write $\ell_i \leftarrow \ell_i^{\text{new}}$, $m_i \leftarrow m_i^{\text{new}}$ to HBM.
14: **end for**
15: **end for**
16: Return $\mathbf{O}$.

We show FlashAttention's correctness, runtime, and memory requirement (proof in Appendix C).

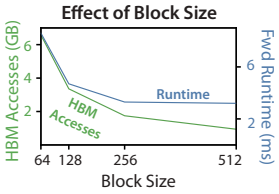
Theorem 1. *Algorithm 1 returns* $\mathbf{O} = \text{softmax}(\mathbf{Q}\mathbf{K}^\top)\mathbf{V}$ *with* $O(N^2 d)$ *FLOPs and requires* $O(N)$ *additional memory beyond inputs and output.*

3.2 Analysis: IO Complexity of FlashAttention

We analyze the IO complexity of FlashAttention, showing significant reduction in HBM accesses compared to standard attention. We also provide a lower bound, proving that no exact attention algorithm can

[2] This style of aggregation is called *algebraic aggregation* [33].

Attention	Standard	FlashAttention
GFLOPs	66.6	75.2
HBM R/W (GB)	40.3	4.4
Runtime (ms)	41.7	7.3



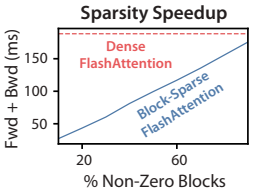


Figure 2: **Left**: Forward + backward runtime of standard attention and FlashAttention for GPT-2 medium (seq. length 1024, head dim. 64, 16 heads, batch size 64) on A100 GPU. HBM access is the primary factor affecting runtime. **Middle**: Forward runtime of FlashAttention (seq. length 1024, head dim. 64, 16 heads, batch size 64) on A100 GPU. Fewer HBM accesses result in faster runtime, up to a point. **Right**: The runtime (for seq. length 4K) of block-sparse FlashAttention is faster than FlashAttention by a factor proportional to the sparsity.

asymptotically improve on HBM accesses over all SRAM sizes. Proofs are in Appendix C.

Theorem 2. *Let N be the sequence length, d be the head dimension, and M be size of SRAM with $d \le M \le Nd$. Standard attention (Algorithm 0) requires $\Theta(Nd + N^2)$ HBM accesses, while* FlashAttention *(Algorithm 1) requires $\Theta(N^2 d^2 M^{-1})$ HBM accesses.*

For typical values of d (64-128) and M (around 100KB), d^2 is many times smaller than M, and thus FlashAttention requires many times fewer HBM accesses than standard implementation. This leads to both faster execution and lower memory footprint, which we validate in Section 4.3.

The main idea of the proof is that given the SRAM size of M, we can load blocks of $\mathbf{K}, \mathbf{V}$ of size $\Theta(M)$ each (Algorithm 1 line 6). For each block of $\mathbf{K}$ and $\mathbf{V}$, we iterate over all blocks of $\mathbf{Q}$ (Algorithm 1 line 8) to compute the intermediate values, resulting in $\Theta(NdM^{-1})$ passes over $\mathbf{Q}$. Each pass loads $\Theta(Nd)$ elements, which amounts to $\Theta(N^2 d^2 M^{-1})$ HBM accesses. We similarly prove that the backward pass of standard attention requires $\Theta(Nd + N^2)$ HBM accesses while the backward pass of FlashAttention requires $\Theta(N^2 d^2 M^{-1})$ HBM accesses (Appendix B).

We prove a lower-bound: one cannot asymptotically improve on the number of HBM accesses for all values of M (the SRAM size) when computing exact attention.

Proposition 3. *Let N be the sequence length, d be the head dimension, and M be size of SRAM with $d \le M \le Nd$. There does not exist an algorithm to compute exact attention with $o(N^2 d^2 M^{-1})$ HBM accesses for all M in the range $[d, Nd]$.*

The proof relies on the fact that for $M = \Theta(Nd)$ any algorithm must perform $\Omega(N^2 d^2 M^{-1}) = \Omega(Nd)$ HBM accesses. This type of lower bound over a subrange of M is common in the streaming algorithms literature [88]. We leave proving parameterized complexity [27] lower bounds in terms of M as exciting future work.

We validate that the number of HBM accesses is the main determining factor of attention run-time. In Fig. 2 (left), we see that even though FlashAttention has higher FLOP count compared to standard attention (due to recomputation in the backward pass), it has much fewer HBM accesses, resulting in much faster runtime. In Fig. 2 (middle), we vary the block size B_c of FlashAttention, which results in different amounts of HBM accesses, and measure the runtime of the forward pass. As block size increases, the number of HBM accesses decreases (as we make fewer passes over the input), and runtime decreases. For large enough block size (beyond 256), the runtime is then bottlenecked by other factors (e.g., arithmetic operations). Moreover, larger block size will not fit into the small SRAM size.

3.3 Extension: Block-Sparse FlashAttention

We extend FlashAttention to approximate attention: we propose block-sparse FlashAttention, whose IO complexity is smaller than FlashAttention by a factor proportional to the sparsity.

Given inputs $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ and a mask matrix $\tilde{\mathbf{M}} \in \{0,1\}^{N \times N}$, we want to compute:

$$\mathbf{S} = \mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{N \times N}, \quad \mathbf{P} = \mathrm{softmax}(\mathbf{S} \odot \mathbb{1}_{\tilde{\mathbf{M}}}) \in \mathbb{R}^{N \times N}, \quad \mathbf{O} = \mathbf{P}\mathbf{V} \in \mathbb{R}^{N \times d},$$

where $(\mathbf{S} \odot \mathbb{1}_{\tilde{\mathbf{M}}})_{kl} = \mathbf{S}_{kl}$ if $\tilde{\mathbf{M}}_{kl} = 1$ and $-\infty$ if $\mathbf{M}_{kl} = 0$. We require $\tilde{\mathbf{M}}$ to have block form: for some block sizes B_r, B_c, for all k, l, $\tilde{\mathbf{M}}_{k,l} = \mathbf{M}_{ij}$ with $i = \lfloor k/B_r \rfloor, j = \lfloor l/B_c \rfloor$ for some $\mathbf{M} \in \{0,1\}^{N/B_r \times N/B_c}$.

Given a predefined block sparsity mask $\mathbf{M} \in \{0,1\}^{N/B_r \times N/B_c}$ we can easily adapt Algorithm 1 to only compute the nonzero blocks of the attention matrix. The algorithm is identical to Algorithm 1, except we skip zero blocks. We reproduce the algorithm description in Algorithm 5 in Appendix B.

We also analyze the IO complexity of block-sparse FlashAttention.

Proposition 4. *Let N be the sequence length, d be the head dimension, and M be size of SRAM with $d \le M \le Nd$. Block-sparse* FlashAttention *(Algorithm 5) requires $\Theta(Nd + N^2 d^2 M^{-1} s)$ HBM accesses where s is the fraction of nonzero blocks in the block-sparsity mask.*

We see that applying block-sparsity yields a direct improvement by the sparsity to the larger term in the IO complexity. For large sequence lengths N, s is often set to $N^{-1/2}$ [11] or $N^{-1} \log N$ [3, 17, 92], resulting in $\Theta(N\sqrt{N})$ or $\Theta(N \log N)$ IO complexity. For downstream experiments, we use the fixed butterfly sparsity pattern [17], which has been shown to be able to approximate arbitrary sparsity [16].

In Fig. 2 (right), we validate that as the sparsity increases, the runtime of block-sparse FlashAttention improves proportionally. On the LRA benchmark, block-sparse FlashAttention achieves 2.8× speedup, while performing on par with standard attention (Section 4).

4 Experiments

We evaluate the impact of using FlashAttention to train Transformer models. We validate two claims about training time and model accuracy, and report attention runtime and memory benchmarks.

- **Training Speed.** FlashAttention outperforms the MLPerf 1.1 [58] speed record for BERT by 15%, and speeds up GPT-2 up to 3× over HuggingFace [87] and 1.8× over Megatron [77] over standard Transformers. FlashAttention speeds up the long-range arena (LRA) benchmark 2.4×.
- **Quality.** FlashAttention scales Transformers to longer sequences, yielding higher quality. FlashAttention trains GPT-2 with context length 4K faster than Megatron trains GPT-2 with context length 1K, while achieving 0.7 better perplexity. Modeling longer sequences yields 6.4 points of lift on two long-document classification tasks. Finally, FlashAttention yields the **first Transformer** that can achieve better-than-random performance on the challenging Path-X task (sequence length 16K), and block-sparse FlashAttention yields the **first sequence model** that we know of that can achieve better-than-random performance on Path-256 (sequence length 64K).
- **Benchmarking Attention.** We measure the runtime and memory performance of FlashAttention and block-sparse FlashAttention based on sequence length. We confirm that the memory footprint of FlashAttention scales linearly with seq. length and is up to 3× faster than standard attention for common seq. lengths (up to 2K). We confirm that runtime of block-sparse FlashAttention scales linearly in seq. length and is faster than all existing approximate attention baselines.

Additional experiment details are in Appendix E.

4.1 Faster Models with FlashAttention

BERT. FlashAttention yields the fastest single-node BERT training speed that we know of. We train a BERT-large [22] model with FlashAttention on Wikipedia. Table 1 compares our training time to the implementation from Nvidia that set the training speed record for MLPerf 1.1 [58]. Our implementation is 15% faster.

Table 1: Training time of BERT-large, starting from the same initialization provided by the MLPerf benchmark, to reach the target accuracy of 72.0% on masked language modeling. Averaged over 10 runs on 8×A100 GPUs.

BERT Implementation	Training time (minutes)
Nvidia MLPerf 1.1 [58]	20.0 ± 1.5
FlashAttention (ours)	**17.4** ± 1.4

GPT-2. FlashAttention yields faster training times for GPT-2 [67] on the large OpenWebtext dataset [32] than the widely used HuggingFace [87] and Megatron-LM [77] implementations. Table 2 shows up to 3× end-to-end speedup compared to Huggingface and 1.7× speedup compared to Megatron-LM. FlashAttention

achieves the same perplexity as the other two implementations, as we do not change the model definition. Appendix E includes plots of the validation perplexity throughout training, confirming that FlashAttention is as numerically stable as the baselines and produces the same training / validation curves.

Table 2: GPT-2 small and medium using FlashAttention achieve up to 3× speed up compared to Huggingface implementation and up to 1.7× compared to Megatron-LM. Training time reported on 8×A100s GPUs.

Model implementations	OpenWebText (ppl)	Training time (speedup)
GPT-2 small - Huggingface [87]	18.2	9.5 days (1.0×)
GPT-2 small - Megatron-LM [77]	18.2	4.7 days (2.0×)
GPT-2 small - FlashAttention	18.2	**2.7 days (3.5×)**
GPT-2 medium - Huggingface [87]	14.2	21.0 days (1.0×)
GPT-2 medium - Megatron-LM [77]	14.3	11.5 days (1.8×)
GPT-2 medium - FlashAttention	14.3	**6.9 days (3.0×)**

Long-range Arena. We compare vanilla Transformer (with either standard implementation or FlashAttention) on the long-range arena (LRA [80]) benchmark. We measure accuracy, throughput, and training time of all models. Each task has a different sequence length varying between 1024 and 4096. We follow the implementation and experimental setting in Tay et al. [80] and Xiong et al. [90].[3] Table 3 shows that FlashAttention achieves up 2.4× speed-up compared to standard attention. Block-sparse FlashAttention is faster than all of the approximate attention methods that we have tested.

Table 3: The performance of standard attention, FlashAttention, block-sparse FlashAttention, and approximate attention baselines on the Long-Range-Arena benchmarks.

Models	ListOps	Text	Retrieval	Image	Pathfinder	Avg	Speedup
Transformer	36.0	63.6	81.6	42.3	72.7	59.3	-
FlashAttention	37.6	63.9	81.4	43.5	72.7	59.8	2.4×
Block-sparse FlashAttention	37.0	63.0	81.3	43.6	73.3	59.6	**2.8×**
Linformer [84]	35.6	55.9	77.7	37.8	67.6	54.9	2.5×
Linear Attention [50]	38.8	63.2	80.7	42.6	72.5	59.6	2.3×
Performer [12]	36.8	63.6	82.2	42.1	69.9	58.9	1.8×
Local Attention [80]	36.1	60.2	76.7	40.6	66.6	56.0	1.7×
Reformer [51]	36.5	63.8	78.5	39.6	69.4	57.6	1.3×
Smyrf [19]	36.1	64.1	79.0	39.6	70.5	57.9	1.7×

4.2 Better Models with Longer Sequences

Language Modeling with Long Context. The runtime and memory-efficiency of FlashAttention allow us to increase the context length of GPT-2 by 4× while still running faster than the optimized implementation from Megatron-LM. Table 4 shows that that GPT-2 with FlashAttention and context length 4K is still 30% faster than GPT-2 from Megatron with context length 1K, while achieving 0.7 better perplexity.

Table 4: GPT-2 small with FlashAttention, with 4× larger context length compared to Megatron-LM, is still 30% faster while achieving 0.7 better perplexity. Training time on 8×A100 GPUs is reported.

Model implementations	Context length	OpenWebText (ppl)	Training time (speedup)
GPT-2 small - Megatron-LM	1k	18.2	4.7 days (1.0×)
GPT-2 small - FlashAttention	1k	18.2	**2.7 days (1.7×)**
GPT-2 small - FlashAttention	2k	17.6	3.0 days (1.6×)
GPT-2 small - FlashAttention	4k	**17.5**	3.6 days (1.3×)

Long Document Classification. Training Transformers with longer sequences with FlashAttention improves performance on the MIMIC-III [47] and ECtHR [6, 7] datasets. MIMIC-III contains intensive care unit patient discharge summaries, each annotated with multiple labels. ECtHR contains legal cases from the

[3] LRA accuracy results are known to be highly dependent on the tuning procedure [90]. Our reproduced baselines perform better than as reported in the original comparison [80].

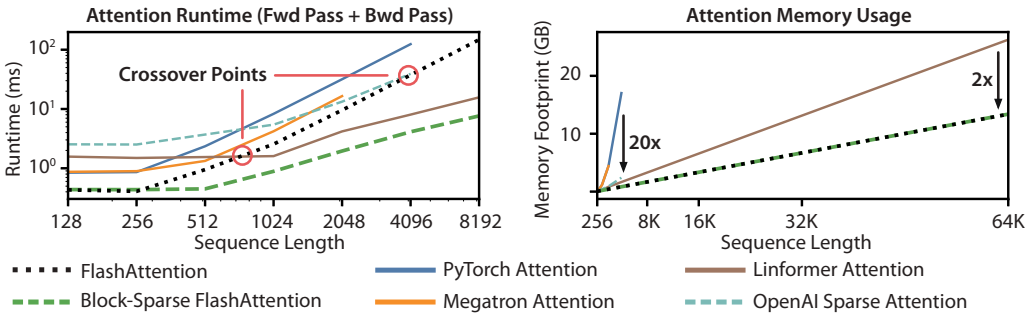


Figure 3: **Left:** runtime of forward pass + backward pass. **Right:** attention memory usage.

European Court of Human Rights, each of which is mapped to articles of the Convention of Human Rights that were allegedly violaged. Both of these datasets contain very long text documents; the average number of tokens in MIMIC is 2,395 tokens, and the longest document contains 14,562 tokens, while the average and longest numbers in ECtHR are 2,197 and 49,392, respectively. We evaluate lift from increasing the sequence length of a pretrained RoBERTa model [56] (we repeat the positional embeddings, as in Beltagy et al. [3]).

Table 5 shows that sequence length 16K outperforms length 512 by 4.3 points on MIMIC, and that length 8K outperforms length 512 by 8.5 points on ECtHR. The discrepancies may be due to subtle distribution shifts: MIMIC-III contains specialized medical text and thus may be more susceptible to a distribution shift in the document length, whereas ECtHR contains general language.

Table 5: Long Document performance (micro F_1) at different sequence lengths using FlashAttention.

	512	1024	2048	4096	8192	16384
MIMIC-III [47]	52.8	50.7	51.7	54.6	56.4	**57.1**
ECtHR [6]	72.2	74.3	77.1	78.6	**80.7**	79.2

Table 6: We report the first Transformer model that can achieve non-random performance on Path-X and Path-256.

Model	Path-X	Path-256
Transformer	✗	✗
Linformer [84]	✗	✗
Linear Attention [50]	✗	✗
Performer [12]	✗	✗
Local Attention [80]	✗	✗
Reformer [51]	✗	✗
SMYRF [19]	✗	✗
FlashAttention	**61.4**	✗
Block-sparse FlashAttention	56.0	**63.1**

Path-X and Path-256. The Path-X and Path-256 benchmarks are challenging tasks from the long-range arena benchmark designed to test long context. The task is to classify whether two points in a black and white 128×128 (or 256×256) image have a path connecting them, and the images are fed to the transformer one pixel at a time. In prior work, all transformer models have either run out of memory, or only achieved random performance [80]. There has been a search for alternative architectures that can model such long context [37]. We present here the first result of Transformer models being able to solve Path-X and Path-256 (Table 6). We pretrain a transformer on Path-64, and then transfer to Path-X by spatially interpolating the positional embeddings. FlashAttention achieves 61.4 accuracy on Path-X. Additionally, block-sparse FlashAttention enables the Transformers to scale to sequence length 64K, achieving 63.1 accuracy[4] on Path-256.

4.3 Benchmarking Attention

We vary sequence length and measure runtime and memory usage of FlashAttention and block-sparse FlashAttention against various attention baselines on one A100 GPU with 40 GB HBM, with dropout and a padding mask. We compare against reference implementations for exact attention, approximate attention, and sparse attention. We report a subset of baselines in the main body; Appendix E contains more baselines and full details.

[4] Path-256 requires longer sequences but has relatively shorter paths than Path-X, so it is easier to obtain a higher accuracy.

Runtime. Figure 3 (left) reports the runtime in milliseconds of the forward + backward pass of FlashAttention and block-sparse FlashAttention compared to the baselines in exact, approximate, and sparse attention (exact numbers in Appendix E). Runtime grows quadratically with sequence length, but FlashAttention runs significantly faster than **exact attention** baselines, up to 3× faster than the PyTorch implementation. The runtimes of many approximate/sparse attention mechanisms grow linearly with sequence length, but FlashAttention still runs faster than approximate and sparse attention for short sequences due to fewer memory accesses. The **approximate attention** runtimes begin to cross over with FlashAttention at sequences between 512 and 1024. On the other hand, block-sparse FlashAttention is faster than all implementations of exact, sparse, and approximate attention that we know of, across all sequence lengths.

Memory Footprint. Figure 3 (right) shows the memory footprint of FlashAttention and block-sparse FlashAttention compared to various exact, approximate, and sparse attention baselines. FlashAttention and block-sparse FlashAttention have the same memory footprint, which grows linearly with sequence length. FlashAttention is up to 20× more memory efficient than **exact attention** baselines, and is more memory-efficient than the **approximate attention** baselines. All other algorithms except for Linformer run out of memory on an A100 GPU before 64K, and FlashAttention is still 2× more efficient than Linformer.

5 Limitations and Future Directions

We discuss limitations of our approach and future directions. Related work is given in Appendix A.

Compiling to CUDA. Our current approach to building IO-aware implementations of attention requires writing a new CUDA kernel for each new attention implementation. This requires writing the attention algorithm in a considerably lower-level language than PyTorch, and requires significant engineering effort. Implementations may also not be transferrable across GPU architectures. These limitations suggest the need for a method that supports writing attention algorithms in a high-level language (e.g., PyTorch), and compiling to IO-aware implementations in CUDA—similar to efforts such as Halide in image processing [70].

IO-Aware Deep Learning. We believe that the IO-aware approach can extend beyond attention. Attention is the most memory-intensive computation in Transformers, but every layer in a deep network touches GPU HBM. We hope our work inspires IO-aware implementations of additional modules. We discuss these potential extensions in Appendix D.

Multi-GPU IO-Aware Methods. Our IO-aware implementation of attention is optimal within constants for computing attention on a single GPU. However, the attention computation may be parallelizable across multiple GPUs [72]. Using multiple GPUs adds an additional layer to IO analysis—accounting for data transfer between GPUs. We hope our work inspires future work in this direction.

Acknowledgments

Our implementation uses Apex's FMHA code (`https://github.com/NVIDIA/apex/tree/master/apex/contrib/csrc/fmha`) as a starting point. We thank Young-Jun Ko for the in-depth explanation of his FMHA implementation and for his thoughtful answers to our questions about CUDA. We thank Sabri Eyuboglu, Megan Leszczynski, Laurel Orr, Yuhuai Wu, Beidi Chen, and Xun Huang for their constructive feedback and suggestions on early drafts of the paper. We thank Markus Rabe and Charles Staats for helpful discussion of their attention algorithm.

We gratefully acknowledge the support of NIH under No. U54EB020405 (Mobilize), NSF under Nos. CCF1763315 (Beyond Sparsity), CCF1563078 (Volume to Velocity), and 1937301 (RTML); ARL under No. W911NF-21-2-0251 (Interactive Human-AI Teaming); ONR under No. N000141712266 (Unifying Weak Supervision); ONR N00014-20-1-2480: Understanding and Applying Non-Euclidean Geometry in Machine Learning; N000142012275 (NEPTUNE); NXP, Xilinx, LETI-CEA, Intel, IBM, Microsoft, NEC, Toshiba, TSMC, ARM, Hitachi, BASF, Accenture, Ericsson, Qualcomm, Analog Devices, Google Cloud, Salesforce, Total, the HAI-GCP & HAI-Azure Cloud Credits for Research program, the Stanford Data Science Initiative (SDSI), Department of Defense (DoD) through the National Defense Science and Engineering Graduate Fellowship (NDSEG) Program, and members of the Stanford DAWN project: Facebook, Google, and VMWare. The U.S. Government is authorized to reproduce and distribute reprints for Governmental purposes

A Related Work

IO-Aware Runtime Optimization. The broad concept of optimizing for reading and writing to fast/slow memory has a long history in computer science and has been known by many names. We draw the most direct connection to the literature of analyzing I/O complexity in this work [1], but concepts of memory hierarchies are fundamental and has appeared in many forms, from the working set model [21], to data locality [86], to the Roofline model of arithmetic intensity [85], to analyses of scalability [59], to standard textbook treatments of computer architecture [40]. We hope that this work encourages the community to adopt these ideas in more parts of the deep learning stack.

Efficient ML Models with Structured Matrices. Matrix multiply is the core computational bottleneck of most machine learning models. To reduce the computational complexity, there have been numerous approaches to learn over a more efficient set of matrices. These matrices are called *structured matrices*, which have subquadratic ($o(n^2)$ for dimension $n \times n$) number of parameters and runtime. Most common examples of structured matrices are sparse and low-rank matrices, along with fast transforms commonly encountered in signal processing (Fourier, Chebyshev, sine/cosine, orthogonal polynomials). There have been several more general classes of structured matrices proposed in machine learning: Toeplitz-like [78], low-displacement rank [49], quasi-separable [25]). The butterfly pattern we use for our block-sparse attention is motivated by the fact that butterfly matrices [15, 64] and their products have been shown to be able to express any structured matrices with almost optimal runtime and number of parameters [16, 20]. However, even though structured matrices are efficient in theory, they have not seen wide adoption since it is hard to translate their efficiency to wall-clock speedup since dense unconstrained matrix multiply has very optimize implementation, a phenomenon known as the hardware lottery [41]. Extensions of butterfly matrices [17, 18] aimed to make butterfly matrices more hardware-friendly.

Sparse Training. Our block-sparse FlashAttention can be seen as a step towards making sparse model training more efficient. Sparse models have seen success in compressing models for inference (pruning) by sparsifying the weight matrices [23, 38, 39, 55, 76]. For model training, the lottery tickets hypothesis [28, 29, 30] suggests that there are a set of small sub-networks derived from a larger dense network that performs as well as the original dense network. Our block-sparse FlashAttention can also be seen as a fixed lottery ticket in the context of attention: we fix the sparsity pattern to be the butterfly pattern through training, and observe that it performs almost as well as the (dense) FlashAttention on the Long-range Arena tasks.

Efficient Transformer. Transformer-based models have become the most widely-used architecture in natural language processing [22] and computer vision [24, 91]. However, one of their computational bottlenecks is that their time and memory scales quadratic in the sequence length. There are numerous approaches to overcome this bottleneck, including approximation with hashing (i.e., sparse) such as Reformer [51] and Smyrf [19] and with low-rank approximation such as Performer [12, 54]. One can even combine sparse and low-rank approximation for better accuracy (e.g., Longformer [3], BigBird [92], Scatterbrain [9], Long-short transformer [94], Combiner [73]). Other approaches include compressing along the sequence dimension to attend to multiple tokens at once [52, 57, 79, 89]. One can also attend over the states from previous sequences to help lengthen the context (e.g., Transformer-XL [14] and Compressive Transformer [69]). We recommend the survey [81] for more details.

There are several lines of work on developing other modules instead of attention to model longer context. HiPPO [35] and its extensions, most notably S4 [31, 36, 37] projects the history on a polynomial basis, allowing accurate reconstruction of the history through state-space models. They combine the strengths of CNNs (efficient training), RNNs (efficient inference), and continuous models (robust to change in sampling rates). LambdaNetworks [2], AFT [93] and FLASH [42] are other attempts at replacing attention in the context of image classification and language modeling.

B Algorithm Details

We first derive the forward and backward passes of attention and show that they can be computed in a memory-efficient manner (requiring extra memory linear instead of quadratic in the sequence length). Though they reduce the amount of extra memory required, naively they still incur quadratic HBM accesses, resulting in slower execution speed. We describe the FlashAttention algorithm to implement both the forward

and the backward passes on GPUs that reduces HBM accesses, leading to both faster runtime and smaller memory footprint.

B.1 Memory-efficient forward pass

The main challenge in making attention memory-efficient is the softmax that couples the columns of $\mathbf{K}$ (and columns of $\mathbf{V}$). Our approach is to compute the softmax normalization constant separately to decouple the columns. This technique [60] has been used in the literature [51, 66] to show that attention computation does not need quadratic *extra* memory (though the number of HBM accesses is still quadratic, resulting in slow run-time).

For simplicity, we omit here the max-shifting step during softmax. The full algorithm in Appendix B.3 contains all the steps.

Recall that given input sequences $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$, we want to compute the attention output $\mathbf{O} \in \mathbb{R}^{N \times d}$:

$$\mathbf{S} = \mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{N \times N}, \quad \mathbf{P} = \mathrm{softmax}(\mathbf{S}) \in \mathbb{R}^{N \times N}, \quad \mathbf{O} = \mathbf{P}\mathbf{V} \in \mathbb{R}^{N \times d}.$$

We have that $S_{ij} = q_i^T k_j$ where q_i and k_j are the i-th and j-th columns of $\mathbf{Q}$ and $\mathbf{K}$ respectively. Define the normalization constants of softmax:

$$L_i = \sum_j e^{q_i^T k_j}. \tag{1}$$

Let v_j be the j-th column of $\mathbf{V}$, then the i-th columns of the output is

$$o_i = P_{i:}\mathbf{V} = \sum_j P_{ij} v_j = \sum_j \frac{e^{q_i^T k_j}}{L_i} v_j. \tag{2}$$

We see that once L_i is computed, we can compute o_i without extra memory by repeatedly summing $\frac{e^{q_i^T k_j}}{L_i} v_j$. Therefore the forward pass can be computed with $O(n)$ extra memory:

1. Compute L_i for all i according to Eq. (1), which takes $O(n)$ extra memory.
2. Compute o_i for all i according to Eq. (2), which takes $O(d)$ extra memory.

B.2 Memory-efficient backward pass

We derive the backward pass of attention and show that it can also be computed with linear memory. Rabe and Staats [66] suggests that the backward pass can be done without quadratic extra memory by applying gradient checkpointing to the memory-efficient forward pass. We instead derive the backward pass explicitly and show how it can be computed in a memory-efficient manner.

Suppose that there is a scalar loss function ϕ, and let the output gradient be $\mathbf{dO} \in \mathbb{R}^{n \times d}$ (where $\mathbf{dO}$ denotes $\frac{\partial \phi}{\partial \mathbf{O}}$). We want to compute the input gradients $\mathbf{dQ}, \mathbf{dK}, \mathbf{dV} \in \mathbb{R}^{n \times d}$ (where $\mathbf{dQ}, \mathbf{dK}, \mathbf{dV}$ denote $\frac{\partial \phi}{\partial \mathbf{Q}}, \frac{\partial \phi}{\partial \mathbf{K}}, \frac{\partial \phi}{\partial \mathbf{V}}$ respectively).

The gradient $\mathbf{dV}$ is easy to see. Applying reverse-mode autodiff by hand (aka the chain rule), we obtain (in matrix notation) $\mathbf{dV} = \mathbf{P}^T \mathbf{dO}$. Thus:

$$dv_j = \sum_i P_{ij} do_i = \sum_i \frac{e^{q_i^T k_j}}{L_i} do_i. \tag{3}$$

Since we already computed L_i, dv_j can be computed without extra memory by repeated summing.

The gradients $\mathbf{dQ}$ and $\mathbf{dK}$ are a little more complicated. We go through the gradients $\mathbf{dP}$ and $\mathbf{dS}$ first. From Eq. (2), we have that $\mathbf{dP} = \mathbf{dO}\mathbf{V}^T$, and so:

$$dP_{ij} = do_i^T v_j.$$

Recall that $P_{i:} = \mathrm{softmax}(S_{i:})$. Using the fact that the Jacobian of $y = \mathrm{softmax}(x)$ is $\mathrm{diag}(y) - yy^T$, we have that

$$dS_{i:} = (\mathrm{diag}(P_{i:}) - P_{i:}P_{i:}^T) dP_{i:} = P_{i:} \circ dP_{i:} - (P_{i:}^T dP_{i:}) P_{i:},$$

where $\circ$ denotes pointwise multiplication.

Define

$$D_i = P_{i:}^T dP_{i:} = \sum_j \frac{e^{q_i^T k_j}}{L_i} do_i^T v_j = do_i^T \sum_j \frac{e^{q_i^\top k_j}}{L_i} v_j = do_i^T o_i, \tag{4}$$

then

$$dS_{i:} = P_{i:} \circ dP_{i:} - D_i P_{i:}.$$

Hence

$$dS_{ij} = P_{ij} dP_{ij} - D_i P_{ij} = P_{ij}(dP_{ij} - D_i).$$

Now we can get the gradients $\mathbf{dQ}$ and $\mathbf{dK}$. Recall that $S_{ij} = q_i^T k_j$, so

$$dq_i = \sum_j dS_{ij} k_j = \sum_j P_{ij}(dP_{ij} - D_i) k_j = \sum_j \frac{e^{q_i^T k_j}}{L_i}(do_i^T v_j - D_i) k_j. \tag{5}$$

Similarly,

$$dk_j = \sum_i dS_{ij} q_i = \sum_i P_{ij}(dP_{ij} - D_i) q_i = \sum_i \frac{e^{q_i^T k_j}}{L_i}(do_i^T v_j - D_i) q_i. \tag{6}$$

Therefore the backward pass can also be computed with $O(n)$ extra memory:

1. Compute dv_j for all j according to Eq. (3), which takes $O(d)$ extra memory.
2. Compute D_i for all i according to Eq. (4), which takes $O(n)$ extra memory.
3. Compute dq_i for all i according to Eq. (5), which takes $O(d)$ extra memory.
4. Compute dk_j for all j according to Eq. (6), which takes $O(d)$ extra memory.

B.3 FlashAttention: Forward Pass

We describe the full details of FlashAttention forward pass. Given input sequences $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$, we want to compute the attention output $\mathbf{O} \in \mathbb{R}^{N \times d}$:

$$\mathbf{S} = \tau \mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{N \times N}, \quad \mathbf{S}^{\text{masked}} = \text{MASK}(S) \in \mathbb{R}^{N \times N}, \quad \mathbf{P} = \text{softmax}(\mathbf{S}^{\text{masked}}) \in \mathbb{R}^{N \times N},$$
$$\mathbf{P}^{\text{dropped}} = \text{dropout}(\mathbf{P}, p_{\text{drop}}), \quad \mathbf{O} = \mathbf{P}^{\text{dropped}} \mathbf{V} \in \mathbb{R}^{N \times d},$$

where $\tau \in \mathbb{R}$ is some softmax scaling (typically $\frac{1}{\sqrt{d}}$), MASK is some masking function that sets some entries of the input to $-\infty$ and keep other entries the same (e.g., key padding mask when sequences in the batch don't have the same lengths and are padded), and $\text{dropout}(x, p)$ applies dropout to x elementwise (i.e., output $\frac{x}{1-p}$ with probability $1-p$ and output 0 with probability p for each element x).

The full algorithm is in Algorithm 2. We save the output $\mathbf{O}$, the softmax statistics ℓ and m, and the pseudo-random number generator state $\mathcal{R}$ for the backward pass.

Algorithm 2 FlashAttention Forward Pass

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, on-chip SRAM of size M, softmax scaling constant $\tau \in \mathbb{R}$, masking function MASK, dropout probability p_{drop}.

Initialize the pseudo-random number generator state $\mathcal{R}$ and save to HBM.
Set block sizes $B_c = \lceil \frac{M}{4d} \rceil$, $B_r = \min\left(\lceil \frac{M}{4d} \rceil, d\right)$.
Initialize $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}, \ell = (0)_N \in \mathbb{R}^N, m = (-\infty)_N \in \mathbb{R}^N$ in HBM.
Divide $\mathbf{Q}$ into $T_r = \left\lceil \frac{N}{B_r} \right\rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ in to $T_c = \left\lceil \frac{N}{B_c} \right\rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_i, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_i, \dots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \dots, m_{T_r}$ of size B_r each.
for $1 \le j \le T_c$ **do**
  Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
  for $1 \le i \le T_r$ **do**
    Load $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
    On chip, compute $\mathbf{S}_{ij} = \tau \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
    On chip, compute $\mathbf{S}_{ij}^{\text{masked}} = \text{MASK}(\mathbf{S}_{ij})$.
    On chip, compute $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}^{\text{masked}}) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij}^{\text{masked}} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.
    On chip, compute $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}$, $\ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$.
    On chip, compute $\tilde{\mathbf{P}}_{ij}^{\text{dropped}} = \text{dropout}(\tilde{\mathbf{P}}_{ij}, p_{\text{drop}})$.
    Write $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1}(\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij}^{\text{dropped}} \mathbf{V}_j)$ to HBM.
    Write $\ell_i \leftarrow \ell_i^{\text{new}}$, $m_i \leftarrow m_i^{\text{new}}$ to HBM.
  end for
end for
Return $\mathbf{O}, \ell, m, \mathcal{R}$.

B.4 FlashAttention: Backward Pass

We describe the full details of FlashAttention backward pass. Given input sequences $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$, the output $\mathbf{O} \in \mathbb{R}^{N \times d}$, and the output gradient $\mathbf{dO}$, we want to compute the input gradients $\mathbf{dQ}, \mathbf{dK}, \mathbf{dV} \in \mathbb{R}^{N \times d}$.

We first describe the standard attention backward pass in Algorithm 3 for completeness.

Algorithm 3 Standard Attention Backward Pass

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{dO} \in \mathbb{R}^{N \times d}$, $\mathbf{P} \in \mathbb{R}^{N \times N}$ in HBM.

Load $\mathbf{P}, \mathbf{dO}$ by blocks from HBM, compute $\mathbf{dV} = \mathbf{P}^\top \mathbf{dO} \in \mathbb{R}^{N \times d}$, write $\mathbf{dV}$ to HBM.
Load $\mathbf{dO}, \mathbf{V}$ by blocks from HBM, compute $\mathbf{dP} = \mathbf{dO}\mathbf{V}^\top \in \mathbb{R}^{N \times N}$, write $\mathbf{dP}$ to HBM.
Read $\mathbf{P}, \mathbf{dP}$ from HBM, compute $\mathbf{dS} \in \mathbb{R}^{N \times N}$ where $dS_{ij} = P_{ij}(dP_{ij} - \sum_l P_{il} dP_{il})$, write $\mathbf{dS}$ to HBM.
Load $\mathbf{dS}$ and $\mathbf{K}$ by blocks from HBM, compute $\mathbf{dQ} = \mathbf{dS}\mathbf{K}$, write $\mathbf{dQ}$ to HBM.
Load $\mathbf{dS}$ and $\mathbf{Q}$ by blocks from HBM, compute $\mathbf{dK} = \mathbf{dS}^\top \mathbf{Q}$, write $\mathbf{dK}$ to HBM.
Return $\mathbf{dQ}, \mathbf{dK}, \mathbf{dV}$.

We now make two observations about FlashAttention backward pass:

1. We do not need to store the dropout mask of size $O(N^2)$ from the forward pass. Instead, we can save the pseudo-random number generator states from the forward pass and re-generate the dropout mask in the backward pass. This allows us to only use $O(N)$ extra memory.

2. When computing the softmax gradient, we use Eq. (4) to compute $D_i = P_{i:}^\top dP_{i:}$ without reducing over $P_{i:}$ and $dP_{i:}$ of size N (they might not fit into SRAM). Instead we can rewrite $D_i = do_i^\top o_i$ and compute the dot product between vectors of size d.

The full FlashAttention backward pass algorithm is in Algorithm 4. Conceptually it is just a block version of the derivation in Appendix B.2.

Algorithm 4 FlashAttention Backward Pass

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{O}, \mathbf{dO} \in \mathbb{R}^{N \times d}$ in HBM, vectors $\ell, m \in \mathbb{R}^N$ in HBM, on-chip SRAM of size M, softmax scaling constant $\tau \in \mathbb{R}$, masking function MASK, dropout probability p_{drop}, pseudo-random number generator state $\mathcal{R}$ from the forward pass.

Set the pseudo-random number generator state to $\mathcal{R}$.
Set block sizes $B_c = \lceil \frac{M}{4d} \rceil$, $B_r = \min\left(\lceil \frac{M}{4d} \rceil, d\right)$.
Divide $\mathbf{Q}$ into $T_r = \left\lceil \frac{N}{B_r} \right\rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ in to $T_c = \left\lceil \frac{N}{B_c} \right\rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_i, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide $\mathbf{dO}$ into T_r blocks $\mathbf{dO}_i, \dots, \mathbf{dO}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_i, \dots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \dots, m_{T_r}$ of size B_r each.
Initialize $\mathbf{dQ} = (0)_{N \times d}$ in HBM and divide it into T_r blocks $\mathbf{dQ}_1, \dots, \mathbf{dQ}_{T_r}$ of size $B_r \times d$ each. Initialize $\mathbf{dK} = (0)_{N \times d}, \mathbf{dV} = (0)_{N \times d}$ in HBM and divide $\mathbf{dK}, \mathbf{dV}$ in to T_c blocks $\mathbf{dK}_1, \dots, \mathbf{dK}_{T_c}$ and $\mathbf{dV}_1, \dots, \mathbf{dV}_{T_c}$, of size $B_c \times d$ each.
for $1 \le j \le T_c$ **do**
  Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
  Initialize $\mathbf{d\tilde{K}}_j = (0)_{B_c \times d}, \mathbf{d\tilde{V}}_j = (0)_{B_c \times d}$ on SRAM.
  for $1 \le i \le T_r$ **do**
    Load $\mathbf{Q}_i, \mathbf{O}_i, \mathbf{dO}_i, \mathbf{dQ}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
    On chip, compute $\mathbf{S}_{ij} = \tau \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
    On chip, compute $\mathbf{S}_{ij}^{\text{masked}} = \text{MASK}(\mathbf{S}_{ij})$.
    On chip, compute $\mathbf{P}_{ij} = \text{diag}(l_i)^{-1} \exp(\mathbf{S}_{ij}^{\text{masked}} - m_i) \in \mathbb{R}^{B_r \times B_c}$.
    On chip, compute dropout mask $\mathbf{Z}_{ij} \in \mathbb{R}^{B_r \times B_c}$ where each entry has value $\frac{1}{1-p_{\text{drop}}}$ with probability $1 - p_{\text{drop}}$ and value 0 with probability p_{drop}.
    On chip, compute $\mathbf{P}_{ij}^{\text{dropped}} = \mathbf{P}_{ij} \circ \mathbf{Z}_{ij}$ (pointwise multiply).
    On chip, compute $\mathbf{d\tilde{V}}_j \leftarrow \mathbf{d\tilde{V}}_j + (\mathbf{P}_{ij}^{\text{dropped}})^\top \mathbf{dO}_i \in \mathbb{R}^{B_c \times d}$.
    On chip, compute $\mathbf{dP}_{ij}^{\text{dropped}} = \mathbf{dO}_i \mathbf{V}_j^\top \in \mathbb{R}^{B_r \times B_c}$.
    On chip, compute $\mathbf{dP}_{ij} = \mathbf{dP}_{ij}^{\text{dropped}} \circ \mathbf{Z}_{ij}$ (pointwise multiply).
    On chip, compute $D_i = \text{rowsum}(\mathbf{dO}_i \circ \mathbf{O}_i) \in \mathbb{R}^{B_r}$.
    On chip, compute $\mathbf{dS}_{ij} = \mathbf{P}_{ij} \circ (\mathbf{dP}_{ij} - D_i) \in \mathbb{R}^{B_r \times B_c}$.
    Write $\mathbf{dQ}_i \leftarrow \mathbf{dQ}_i + \tau \mathbf{dS}_{ij} \mathbf{K}_j \in \mathbb{R}^{B_r \times d}$ to HBM.
    On chip, compute $\mathbf{d\tilde{K}}_j \leftarrow \mathbf{d\tilde{K}}_j + \tau \mathbf{dS}_{ij}^\top \mathbf{Q}_i \in \mathbb{R}^{B_c \times d}$.
  end for
  Write $\mathbf{dK}_j \leftarrow \mathbf{d\tilde{K}}_j, \mathbf{dV}_j \leftarrow \mathbf{d\tilde{V}}_j$ to HBM.
end for
Return $\mathbf{dQ}, \mathbf{dK}, \mathbf{dV}$.

We see that similar to the forward pass, the backward pass performs $O(N^2)$ FLOPs and only requires $O(N)$ extra memory beyond inputs, output, output gradient, and input gradients.

We analyze the IO-complexity of the backward pass, similar to the forward pass (Theorem 2).

Theorem 5. *Let N be the sequence length, d be the head dimension, and M be size of SRAM with $d \le M \le Nd$. Standard attention (Algorithm 0) backward pass requires $\Theta(Nd + N^2)$ HBM accesses, while* FlashAttention *backward pass (Algorithm 4) requires $\Theta(N^2 d^2 M^{-1})$ HBM accesses.*

The proof is in Appendix C.

B.5 Comparison with Rabe and Staats [66]

We describe here some similarities and differences between our FlashAttention algorithm and the algorithm of Rabe and Staats [66].

Conceptually, both FlashAttention and Rabe and Staats [66] operate on blocks of the attention matrix using the well-established technique of tiling (or softmax scaling) [51, 60]. To reduce the memory footprint, both methods avoid storing the large attention matrix in the forward pass and recompute it in the backward pass.

The first major difference is that Rabe and Staats [66] focuses on the reducing the total memory footprint (maximum amount of GPU memory required) while FlashAttention focuses on reducing memory accesses (the number of memory reads/writes). As mentioned in Section 2, the amount of memory access is the primary determining factor of runtime. Reducing memory accesses also necessarily reduces the total amount of memory required (e.g., if an operation incurs A memory accesses, then its total memory requirement is at most A). As a result, FlashAttention is faster than standard attention (2-4×) while Rabe and Staats [66] is around the same speed or slightly slower than standard attention. In terms of total memory required, both methods offer substantial memory saving.

The second difference between the two methods is the way information is summarized from each block to pass to the next block. Rabe and Staats [66] summarizes each block with its temporary output along with the softmax normalization statistics. At the end of the forward pass, the temporary outputs of all the blocks are combined using the statistics to produce the final output. FlashAttention instead incrementally updates the output (Algorithm 1 line 12) after processing each block, so only one copy of the output is needed (instead of K copies for K blocks). This means that FlashAttention has smaller total memory requirement compared to Rabe and Staats [66].

The final major difference is the way the backward pass is computed. Rabe and Staats [66] uses gradient checkpointing to recompute the attention matrix and the temporary output of each block. FlashAttention instead simplifies the backward pass analytically (Appendices B.2 and B.4). It only recomputes the attention matrix and does not recompute the temporary output of each block. This reduces the memory requirement for the backward pass and yields speedup.

C Proofs

Proof of Theorem 1. We first count the number of FLOPs and extra memory required.

The dominating FLOPs are from matrix multiplication. In the inner loop, (Algorithm 1 line 9), we compute $\mathbf{Q}_i \mathbf{K}_j^\top \in \mathbb{R}^{B_r \times B_c}$ for $\mathbf{Q}_i \in \mathbb{R}^{B_r \times d}$ and $\mathbf{K}_j \in \mathbb{R}^{B_c \times d}$, which takes $O(B_r B_c d)$ FLOPs. We also compute (Algorithm 1 line 12) $\tilde{\mathbf{P}}_{ij} \mathbf{V}_j \in \mathbb{R}^{B_r \times d}$ for $\tilde{\mathbf{P}}_{ij} \in \mathbb{R}^{B_r \times B_c}$ and $\mathbf{V}_j \in \mathbb{R}^{B_c \times d}$, which takes $O(B_r B_c d)$ FLOPs. We execute the inner loops $T_c T_r = \left\lceil \frac{N}{B_c} \right\rceil \left\lceil \frac{N}{B_r} \right\rceil$ times. Therefore the total number of FLOPs is

$$O\left(\frac{N^2}{B_c B_r} B_r B_c d \right) = O(N^2 d).$$

In terms of extra memory required, we see that we need $O(N)$ memory to store the statistics (ℓ, m).

We now prove the algorithm's correctness by induction on j for $0 \le j \le T_c$. Let $\mathbf{K}_{:j} \in \mathbb{R}^{jB_c \times d}$ be the first jB_c rows of $\mathbf{K}$, and similarly $\mathbf{V}_{:j} \in \mathbb{R}^{jB_c \times d}$ the the first jB_c rows of $\mathbf{V}$. Let $\mathbf{S}_{:,:j} = \mathbf{Q}\mathbf{K}_{:j}^\top \in \mathbb{R}^{N \times jB_c}$, and $\mathbf{P}_{:,:j} = \mathrm{softmax}(\mathbf{S}_{:,:j}) \in \mathbb{R}^{N \times jB_c}$ (softmax applied row-wise). Let $m^j, \ell^{(j)}, \mathbf{O}^{(j)}$ be the values of $m, \ell, \mathbf{O}$ in HBM after the j-th iteration of the outer loop (Algorithm 1 line 5). (Note that these values of $m, \ell, \mathbf{O}$ are updated after each iteration of the outer loop.) We want to show that after the j-th iteration of the outer loop, we have computed in HBM:

$$m^{(j)} = \mathrm{rowmax}(\mathbf{S}_{:,:j}) \in \mathbb{R}^N, \quad \ell^{(j)} = \mathrm{rowsum}(\exp(\mathbf{S}_{:,:j} - m^{(j)})) \in \mathbb{R}^N, \quad \mathbf{O}^{(j)} = \mathbf{P}_{:,:j} \mathbf{V}_{:j} \in \mathbb{R}^{N \times d}.$$

Based on our initialization (Algorithm 1 line 2), this claim is true for $j = 0$ (i.e., before the any iteration of the outer loop is executed). Suppose that the claim holds for some $j = 0, \ldots, T_c - 1$. We want to show that the claim also holds for $j + 1$. Indeed, when we update the statistics in the inner loop (Algorithm 1 line 10)

on the $(j+1)$-th iteration of the outer loop, we update $m^{(j+1)} = \max(m^{(j)}, \tilde{m})$ where $\tilde{m} \in \mathbb{R}^N$ is the row-max of $\mathbf{S}_{:,j:j+1}$, the slice of $\mathbf{S}$ from column jB_c to column $(j+1)B_c - 1$. This implies that

$$m^{(j+1)} = \text{rowmax}(\mathbf{S}_{:,:j+1}) \in \mathbb{R}^N.$$

Similarly, we update

$$\ell^{(j+1)} = e^{m^{(j)} - m^{(j+1)}} \ell^{(j)} + e^{\tilde{m} - m^{(j+1)}} \tilde{\ell},$$

where $\tilde{\ell} = \text{rowsum}(\exp(\mathbf{S}_{:,j:j+1} - \tilde{m})) \in \mathbb{R}^N$. By the same algebraic manipulation in Section 3.1, we obtain:

$$\ell^{(j+1)} = \text{rowsum}(\exp(\mathbf{S}_{:,:j+1} - m^{(j+1)})) \in \mathbb{R}^N.$$

Let $\mathbf{V}_{j:j+1}$ be the slice of $\mathbf{V}$ from column jB_c to column $(j+1)B_c - 1$, we also update:

$$\begin{aligned}
\mathbf{O}^{(j+1)} &= \text{diag}(\ell^{(j+1)})^{-1}(\text{diag}(\ell^{(j)})e^{m^{(j)} - m^{(j+1)}}\mathbf{O}^{(j)} + e^{\tilde{m} - m^{(j+1)}} \exp(\mathbf{S}_{j:j+1} - \tilde{m})\mathbf{V}_{j:j+1}) \\
&= \text{diag}(\ell^{(j+1)})^{-1}(\text{diag}(\ell^{(j)})e^{m^{(j)} - m^{(j+1)}}\mathbf{P}_{:,:j}\mathbf{V}_{:j} + e^{-m^{(j+1)}} \exp(\mathbf{S}_{j:j+1})\mathbf{V}_{j:j+1}) \\
&= \text{diag}(\ell^{(j+1)})^{-1}(\text{diag}(\ell^{(j)})e^{m^{(j)} - m^{(j+1)}}\text{diag}(\ell^{(j)})\exp(\mathbf{S}_{:,:j} - m^{(j)})\mathbf{V}_{:j} + e^{-m^{(j+1)}} \exp(\mathbf{S}_{j:j+1})\mathbf{V}_{j:j+1}) \\
&= \text{diag}(\ell^{(j+1)})^{-1}(e^{-m^{(j+1)}} \exp(\mathbf{S}_{:,:j})\mathbf{V}_{:j} + e^{-m^{(j+1)}} \exp(\mathbf{S}_{j:j+1})\mathbf{V}_{j:j+1}) \\
&= \text{diag}(\ell^{(j+1)})^{-1}(\exp(\mathbf{S}_{:,:j} - m^{(j+1)})\mathbf{V}_{:j} + \exp(\mathbf{S}_{j:j+1} - m^{(j+1)})\mathbf{V}_{j:j+1}) \\
&= \text{diag}(\ell^{(j+1)})^{-1}\left(\exp\left(\begin{bmatrix}\mathbf{S}_{:,:j} & \mathbf{S}_{j:j+1}\end{bmatrix} - m^{(j+1)}\right)\right)\begin{bmatrix}\mathbf{V}_{:j} \\ \mathbf{V}_{j:j+1}\end{bmatrix} \\
&= \text{softmax}(\mathbf{S}_{:j+1})\mathbf{V}_{:j+1}.
\end{aligned}$$

We then see that the claim is also true for $j+1$. By induction, the claim is true for all $j = 0, \ldots, T_c$.

When $j = T_c$, we conclude that the final value of $\mathbf{O}$ in HBM is $\text{softmax}(\mathbf{S})\mathbf{V} = \text{softmax}(\mathbf{QK}^\top)\mathbf{V}$.

□

Proof of Theorem 2. We first analyze the IO complexity of standard attention implementation. The inputs $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ reside in HBM, and the at the end of the algorithm the output $\mathbf{O} \in \mathbb{R}^{N \times d}$ is written to HBM.

In the first step of computing the matrix multiply $\mathbf{S} = \mathbf{QK}^\top$, the inputs $\mathbf{Q}, \mathbf{K}$ are read from HBM and the output $\mathbf{S} \in \mathbb{R}^{N \times N}$ is written to HBM (Algorithm 0 line 1). This incurs $\Theta(Nd + N^2)$ HBM accesses.

In the second step of computing $\mathbf{P} = \text{softmax}(\mathbf{S})$, the input $\mathbf{S}$ is read from HBM and the output $\mathbf{P}$ is written to HBM (Algorithm 0 line 2). This incurs $\Theta(N^2)$ HBM accesses.

In the last step of computing $\mathbf{O} = \mathbf{PV}$, the inputs $\mathbf{P}, \mathbf{V}$ are read from global memory and the output $\mathbf{O}$ is written to HBM (Algorithm 0 line 3). This incurs $\Theta(Nd + N^2)$ HBM accesses.

Overall, standard attention implementation requires $\Theta(Nd + N^2)$ global memory accesses.

We now analyze the IO complexity of streaming attention.

Following Algorithm 1, we see that each element of $\mathbf{K}$ and $\mathbf{V}$ is loaded from HBM once (Algorithm 1 line 6). We make T_c passes over $\mathbf{Q}$ and $\mathbf{O}$, each pass loading all of $\mathbf{Q}$ and all of $\mathbf{O}$ to HBM (Algorithm 1 line 8). Therefore the number of HBM accesses is $\Theta\left(Nd + NdT_c\right) = \Theta(NdT_c)$.

We derive the conditions on the block sizes B_c and B_r. We need the blocks $\mathbf{K}_j$ and $\mathbf{V}_j$ of size $B_c \times d$ to fit into on-chip memory, which translates to:

$$B_c d = O(M) \Leftrightarrow B_c = O\left(\frac{M}{d}\right).$$

Similarly, we need the blocks $\mathbf{Q}_i, \mathbf{O}_i$ of size $B_r \times d$ to fit into on-chip memory, which translates to:

$$B_r d = O(M) \Leftrightarrow B_r = O\left(\frac{M}{d}\right).$$

Finally, we need the block $\mathbf{S}_{ij}$ of size $B_r \times B_c$ to fit into on-chip memory, which translates to:

$$B_r B_c = O(M).$$

We therefore set:

$$B_c = \Theta\left(\frac{M}{d}\right), \qquad B_r = \Theta\left(\min\left(\frac{M}{d}, \frac{M}{B_c}\right)\right) = \Theta\left(\min\left(\frac{M}{d}, d\right)\right).$$

We then have:

$$T_c = \frac{N}{B_c} = \Theta\left(\frac{Nd}{M}\right).$$

As a result, the number of HBM accesses is:

$$\Theta\left(NdT_c\right) = \Theta\left(\frac{N^2d^2}{M}\right).$$

□

Proof of Proposition 3. For contradiction, suppose that there exists an algorithm that computes exact attention where the number for HBM access for all $M \in [d, Nd]$ is

$$o\left(\frac{N^2d^2}{M}\right).$$

In the regime of $M = \Theta(Nd)$, this results in the number of HBM accesses:

$$o\left(\frac{N^2d^2}{Nd}\right) = o(Nd).$$

However, the input to attention (matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V}$) and the output $\mathbf{O}$ have size Nd and they start out being in HBM, so if the algorithm computes exact attention it must incur at least $\Omega(Nd)$ HBM accesses. This is a contradiction. □

Proof of Theorem 5. The IO complexity of the attention backward is very similar to the IO complexity of the attention forward (Theorem 2). Here we provide a sketch of the proof.

We first analyze the IO complexity of standard attention backward pass. The inputs $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{dO} \in \mathbb{R}^{N \times d}$ reside in HBM, and the at the end of the algorithm the outputs $\mathbf{dQ}, \mathbf{dK}, \mathbf{dV} \in \mathbb{R}^{N \times d}$ are written to HBM.

At each step of the standard attention backward pass, one needs to load inputs of size Nd or N^2 from HBM, and needs to write the outputs of size N^2 or Nd to HBM. This incurs $\Theta(Nd + N^2)$ HBM accesses.

We now analyze the IO complexity of FlashAttention backward pass.

Similar to Theorem 2, we see that each element of $\mathbf{K}$ and $\mathbf{V}$ is loaded from HBM once. Each element of $\mathbf{dK}$ and $\mathbf{dV}$ is only written to HBM once. We make T_c passes over $\mathbf{Q}, \mathbf{O}, \mathbf{dO}$, each pass loading all of $\mathbf{Q}, \mathbf{O}, \mathbf{dO}$ to HBM. We also make T_c passes over $\mathbf{dQ}$, each pass reading/writing all of $\mathbf{dQ}$ from/to HBM. Therefore the number of HBM accesses is $\Theta\left(Nd + NdT_c\right) = \Theta(NdT_c)$.

As in the proof of Theorem 2, the constraints on the block sizes are that:

$$B_c = \Theta\left(\frac{M}{d}\right), \qquad B_r = \Theta\left(\min\left(\frac{M}{d}, d\right)\right).$$

We then have:

$$T_c = \frac{N}{B_c} = \Theta\left(\frac{Nd}{M}\right).$$

As a result, the number of HBM accesses is:

$$\Theta\left(NdT_c\right) = \Theta\left(\frac{N^2d^2}{M}\right).$$

□

Algorithm 5 Block-Sparse FlashAttention Forward Pass

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, on-chip SRAM of size M, softmax scaling constant $\tau \in \mathbb{R}$, masking function MASK, dropout probability p_{drop}, block sizes $B_c = \lceil \frac{M}{4d} \rceil$, $B_r = \min\left(\lceil \frac{M}{4d} \rceil, d\right)$, block sparsity mask $M \in \{0,1\}^{N/B_r \times N/B_c}$..

Initialize the pseudo-random number generator state $\mathcal{R}$ and save to HBM.
Initialize $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}, \ell = (0)_N \in \mathbb{R}^N, m = (-\infty)_N \in \mathbb{R}^N$ in HBM.
Divide $\mathbf{Q}$ into $T_r = \left\lceil \frac{N}{B_r} \right\rceil$ blocks $\mathbf{Q}_1, \dots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ in to $T_c = \left\lceil \frac{N}{B_c} \right\rceil$ blocks $\mathbf{K}_1, \dots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \dots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_i, \dots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_i, \dots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \dots, m_{T_r}$ of size B_r each.
for $1 \le j \le T_c$ **do**
  Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
  for $1 \le i \le T_r$ **do**
    if $M_{ij} \neq 0$ **then**
      Load $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
      On chip, compute $\mathbf{S}_{ij} = \tau \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
      On chip, compute $\mathbf{S}_{ij}^{\text{masked}} = \text{MASK}(\mathbf{S}_{ij})$.
      On chip, compute $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}^{\text{masked}}) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij}^{\text{masked}} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.
      On chip, compute $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}$, $\ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$.
      On chip, compute $\tilde{\mathbf{P}}_{ij}^{\text{dropped}} = \text{dropout}(\tilde{\mathbf{P}}_{ij}, p_{\text{drop}})$.
      Write $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1}(\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij}^{\text{dropped}} \mathbf{V}_j)$ to HBM.
      Write $\ell_i \leftarrow \ell_i^{\text{new}}$, $m_i \leftarrow m_i^{\text{new}}$ to HBM.
    end if
  end for
end for
Return $\mathbf{O}, \ell, m, \mathcal{R}$.

D Extension Details

D.1 Block-sparse FlashAttention

We describe the full block-sparse FlashAttention algorithm in Algorithm 5. The algorithm is identical to Algorithm 2, except that we skip zero blocks.

We prove the IO-complexity of block-sparse FlashAttention.

Proof of Proposition 4. The proof is very similar to the proof of Theorem 2. For the block-sparse case, notice that we only need to load blocks corresponding to nonzero blocks. As a result, the number of HBM accesses are scaled by s, the fraction of nonzero blocks in the block-sparsity mask. However, for small values of s, we would still need to write the result $\mathbf{O} \in \mathbb{R}^{N \times d}$. Therefore the number of HBM accesses is

$$\Theta\left(Nd + \frac{N^2 d^2}{M} s\right).$$

□

D.2 Potential Extensions

We discuss here a few potential extensions of the IO-aware approach to speed up deep learning training.

Multi-GPU Attention. Large language models are trained on hundreds or thousands of GPUs, and one typically splits the attention computation between 4-8 GPUs on the same node [77]. This introduces another level of memory hierarchy: beside GPU SRAM and GPU HBM, we also have the HBM of other

GPUs. For very long sequences, the different GPUs on the same node can cooperate to compute attention by taking into account the asymmetry of different levels of memory hierarchy.

Sparse MLP layers. Typical dense MLP layers are compute-bound and not memory-bound. To improve their efficiency, MLP layers with sparse weight matrices can be used [17]. However, many sparse MLP layers are instead memory-bound, and their speedup is often not proportional to the sparsity. We believe that an IO-aware implementation can alleviate this issue and realize the benefits of sparsity. We are excited about future work in this direction, to reduce the computational requirement of large models and improve their wall-block runtime.

Kernel machine learning. Our approach in FlashAttention relies on the fact that the $N \times N$ attention matrix is a function of a low-rank matrix $\mathbf{Q}\mathbf{K}^\top$ (of rank $d \ll N$). As a result, we can repeatedly load the inputs $\mathbf{Q}, \mathbf{K}$ and recompute the block of the attention matrix that we need, significantly reducing HBM access. As similar scenario happens in kernel machine learning: each element K_{ij} of the $N \times N$ kernel matrix $\mathbf{K}$ is a function of two vectors of size $d \ll N$, as it measures the similarity between two datapoints x_i and x_j. The KeOps library [8, 26] is a successful example of how reducing memory reads/writes can speed up kernel operations. We hope that this will motivate kernel methods that focus more on reducing IOs instead of just FLOPs.

E Full Experimental Results

E.1 BERT

We train BERT-large following the training procedure and hyperparameters of the reference MLPerf 1.1 implementation. In particular, we use the LAMB optimizer with learning rate 3.75e-3, with batch size 448, trained for at most 7100 steps. The training is stopped once the validation accuracy (for masked language modeling) reaches the target 72.0%, and the wall-clock run-time is measured. We train with FP16 precision using Apex AMP (with O2 optimization level).

We compare our results with the reported training speed from Nvidia that was submitted to MLPerf 1.1 (Table 1).

We use the same train / validation data split provided by MLPerf 1.1 reference implementation. In particular, we evaluate on the same 10000 validation examples as the baseline from Nvidia.

We train the model on 8×A100-80GB GPUs. Each training run takes between 16 and 19 minutes, and we average the results of 10 runs.

E.2 GPT-2

We use the standard implementations of GPT-2 [67] from Huggingface `transformers` library and from Nvidia's Megatron-LM repo. We follow the training recipe of the Megatron-LM repo.

We use an effective batch size of 512, and use gradient accumulation to fit into available GPU memory. We use the AdamW optimizer, with learning rate 6e-4 for GPT-2 small and 1.5e-4 for GPT-2 medium, and weight decay of 0.1. All models are trained with the same hyperparameters for 400K steps. We run all implementations with mixed-precision training (PyTorch AMP).

We use the Openwebtext dataset, with the GPT-2 BPE tokenizer. We randomly select 0.5% of the dataset as the validation set, with the rest being used as training set. This random selection of validation set is done once, and all models are evaluated on the same validation set.

We train the model on 8×A100-40GB GPUs, and we measure the wall-clock training time. Training GPT-2 small takes between 2.7-9.5 days, and training GPT-2 medium takes between 6.9-21.0 days (Table 2).

In Fig. 4, we plot of the validation perplexity throughout training of GPT-2 small/medium, using either HuggingFace implementation or our FlashAttention implementation. We see that FlashAttention behaves the same as the baseline implementation and the validation perplexity curves of the two implementations almost lie on top of each other.

Long Document Classification. For MIMIC-III and ECtHR, we follow the hyperparameters of Dai et al. [13].

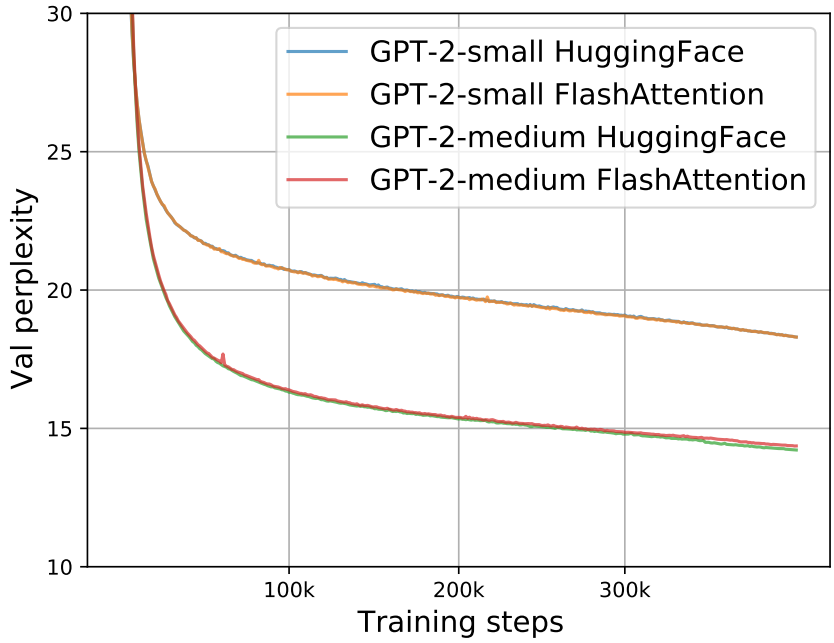


Figure 4: Validation perplexity of GPT-2 small/medium using two implementations. We confirm that FlashAttention yields the same validation curves as the baseline implementation from HuggingFace.

E.3 LRA details

We follow the hyperparameters from the Long-range arena paper [80], the Long-range arena repo (`https://github.com/google-research/long-range-arena`), and the Nyströmformer reproduction [90]. To be generous to the baseline methods, if we are unable to reproduce the performance of any baseline for any of the five tasks, we report the better performance from Tay et al. [80] or Xiong et al. [90] for that baseline on that task.

After hyperparameter tuning, almost all of the attention methods achieve similar accuracy on all of the five LRA tasks.

We run all methods with mixed-precision training, except for Performer (not stable with mixed precision) and Local Attention (implementation does not support FP16).

To calculate the overall wallclock-time speedup, we take the geometric mean of the wallclock-time speedup of each of the five tasks.

Path-X For Path-X and Path-256, we follow the hyperparameters from the PathFinder-32 experiments from the long-range arena paper[80]. For both, we first pretrain a model on Path-64. We take the checkpoint after 200 epochs, upsample its positional embedding (we duplicate the positional embeddings gridwise in space), and fine-tune it on the downstream task for 200 epochs with one epoch of linear warmup, and cosine decay of the learning rate. For Path-X, we take the best performing checkpoint (according to val accuracy), and additionally fine-tune it for 200 epochs with the same warmup and learning rate (this adds roughly 4 points of accuracy to FlashAttention for Path-X, but the model starts overfitting afterwards).

E.4 Comparison with Apex FMHA

We compare our method/implementation with Apex FMHA (`https://github.com/NVIDIA/apex/tree/master/apex/contrib/csrc/fmha`).

When we started this project, Apex FMHA was the fastest implementation of attention (that we knew of), tailored for short sequences of length at most 512. In fact, almost all MLPerf submissions for BERT training benchmark running on Nvidia GPUs use FMHA for their model code, as of MLPerf 1.1 [58]. Since

Table 7: Runtime (ms) of FlashAttention compared to FMHA by sequence length, with masking and dropout, measured on an A100-SXM4-40GB GPU. Batch size 64, 16 heads, head dimension 64 (i.e., BERT-large size).

Attention Method	128	256	512
Apex FMHA forward	0.10	0.29	1.14
FlashAttention forward	**0.08**	**0.22**	**0.81**
Apex FMHA backward	**0.17**	**0.52**	**1.81**
FlashAttention backward	0.20	0.53	2.00
Apex FMHA forward + backward	**0.27**	0.81	2.95
FlashAttention forward + backward	0.28	**0.75**	**2.81**

FMHA targets BERT models, it only supports head dimension 64, and only runs on A100 GPUs. FMHA fuses the attention computation dropout(softmax(mask($\mathbf{QK}^\top$)))$\mathbf{V}$ into one CUDA kernel. In the forward pass, it stores the attention matrix softmax(mask($\mathbf{QK}^T$)) to HBM to be used in gradient computation. As a result, it does not offer substantial memory saving (though for shorter sequences memory footprint is often not a primary concern).

We use FMHA code as a starting point, and apply two well-established techniques (tiling and recomputation) to deal with long sequences and to save memory as mentioned in Section 3. As a result, we can support much longer sequences (e.g., up to length 64K). We also support more head dimensions (16, 32, 64, 128) and broader GPU types (all Turing and Ampere GPUs at the time of writing).

In Table 7, we compare the performance of FlashAttention and Apex FMHA for short sequences (as FMHA only supports sequence length at most 512). Generally FlashAttention is slightly faster than FMHA in the forward pass and slightly slower than FMHA in the backward pass. This is because we do not store the attention matrix in the forward pass and recompute it in the backward pass. Compared to FMHA, the overall runtime of FlashAttention is about 4% slower for sequence length 128, 8% faster for sequence length 256, and 5% faster for sequence length 512.

E.5 Speedup On Different Hardware and Configurations

Speedup varies between different types of GPU types and generations depending on HBM bandwidth and SRAM size. In this section, we profile FlashAttention speedup on different GPUs and configurations.

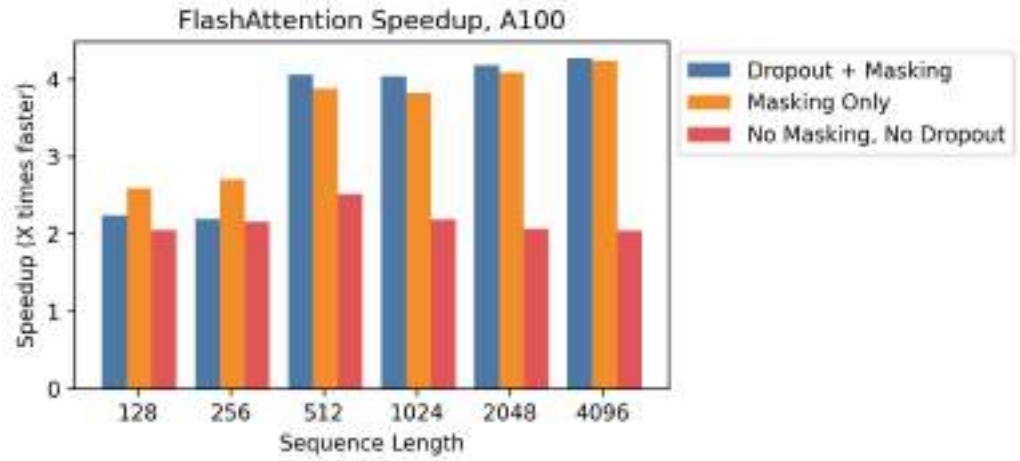


Figure 5: Speedup over standard PyTorch attention at different sequence lengths, on A100.

A100 Figure 5 shows speedup on an A100 GPU with batch size 8, head dimension 64, and 12 attention heads, across different sequence lengths. We generally see 2-4× speedup, and we see more speedup when using dropout and masking due to kernel fusion.

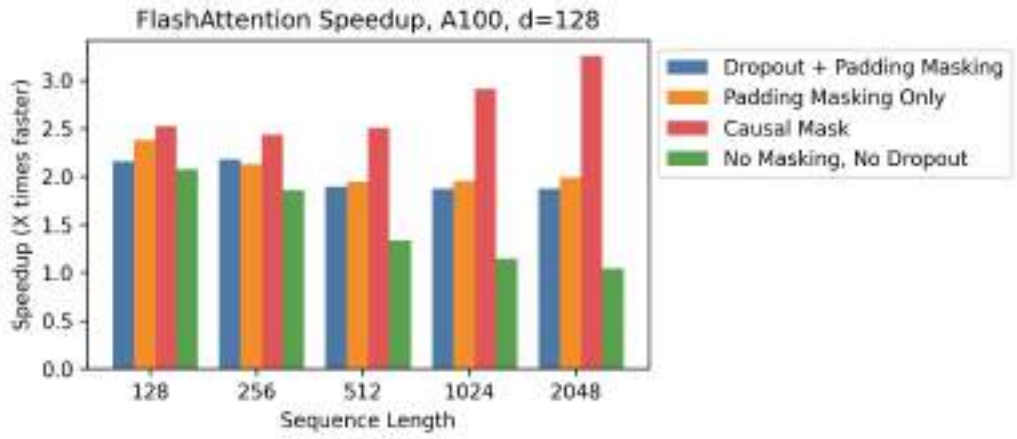


Figure 6: Speedup over standard PyTorch attention at different sequence lengths, on A100, with head dimension 128.

A100, Head Dimension 128 Speedup also changes when we increase the head dimension. Each block requires more memory, so we need to use smaller block sizes to fit into SRAM. Figure 6 shows speedup with head dimension 128 on an A100 (batch size 16, 12 heads). We see less speedup overall—but we can still see significant speedup (up to 3×) with a causal mask, where half the blocks are masked out.

Figure 7: Speedup over standard PyTorch attention at different sequence lengths, on RTX 3090.

RTX 3090 Figure 7 shows speedup on an RTX 3090 GPU. Here, we use batch size 12 with 12 attention heads. We observe slightly higher speedups on the RTX 3090 (between 2.5-4.5×), since the memory bandwidth on an RTX 3090 is lower than on an A100 (roughly 900 GB/s vs. 1.5 TB/s).

T4 Figure 8 shows speedup on a T4 GPU. T4 SRAM is smaller than A100, so we need to make the block sizes smaller in FlashAttention. As a result, we observe less speedup on T4, which matches the IO complexity analysis in Section 3.2. T4 GPUs are commonly used for inference, so we also report speedup on the forward pass only.

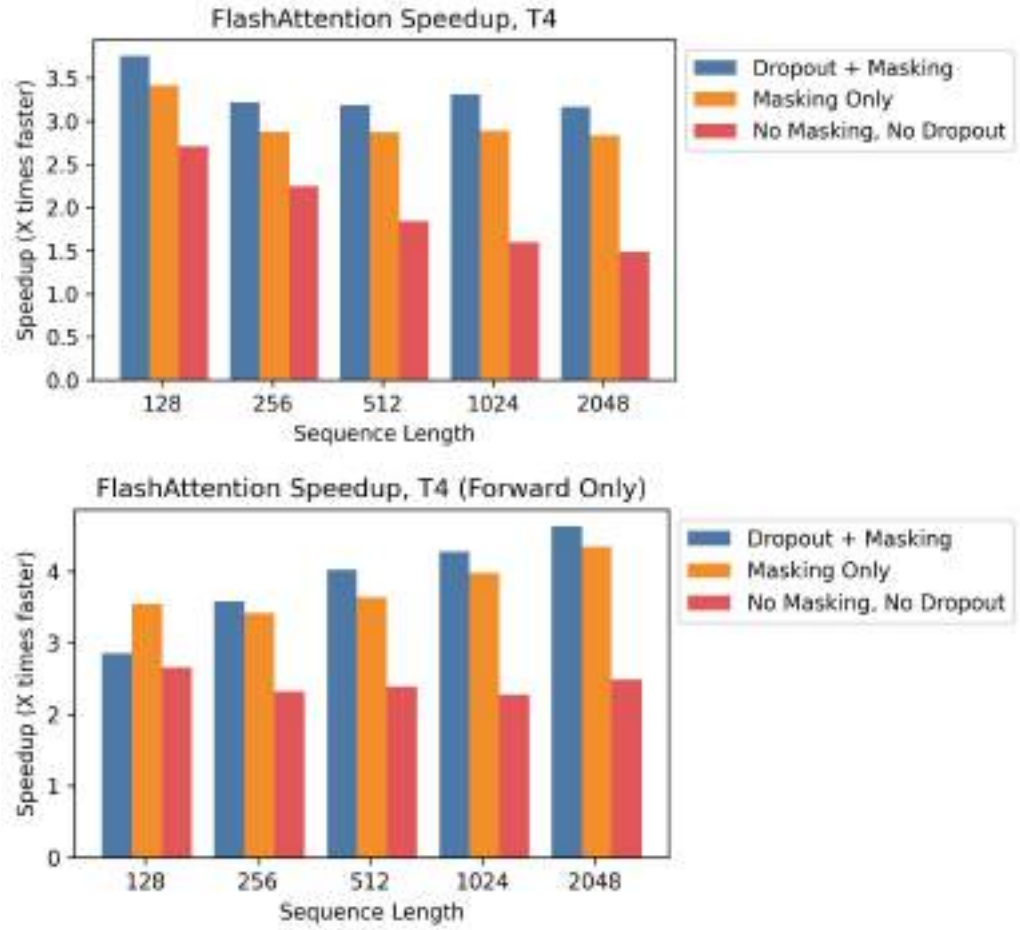


Figure 8: Speedup over standard PyTorch attention at different sequence lengths, on T4. **Top:** Combined forward pass + backward pass. **Bottom:** Forward pass only.

E.6 Full Benchmarking Results

We report the full benchmarking results and experimental details on A100.

Baselines We compare against reference implementations for exact attention from PyTorch/HuggingFace and Megatron, approximate attention, and sparse attention. For approximate attention, we compare against reference implementations of Reformer [51], Local Attention [68], Linformer Attention [84], Smyrf [19], and LongShortFormer (LSFormer) [94]. For sparse attention, we compare against reference implementations of Block-Sparse Attention form OpenAI [11], Longformer[3], and BigBird Attention [92]. For the approximate and sparse attention, we use a compression ratio of 1/8, or a compressed sequence length of 256, whichever is smaller.

Setup We measure runtime and memory usage of the attention computation with 8 heads of dimension 64, and batch size 16 on a machine with one A100 GPU with 40 GB of GPU HBM. We vary sequence length in our experiments. We compute attention on random vectors for **Q**, **K**, and **V** (we do not measure the projection from the hidden layer). For dropout, we use dropout 0.1; for masking, we use a padding mask with uniformly-random mask lengths between the total sequence length and the total sequence length minus 20. To measure runtime, we take the average of 100 measurements of the attention call. We only measure memory footprint once, since it does not vary between runs.

Table 8: Pointers to results tables.

Dropout	Masking	Pass	Table
Yes	Yes	Forward	Table 9
Yes	Yes	Backward	Table 10
Yes	Yes	Combined	Table 11
No	Yes	Forward	Table 12
No	Yes	Backward	Table 13
No	Yes	Combined	Table 14
Yes	No	Forward	Table 15
Yes	No	Backward	Table 16
Yes	No	Combined	Table 17
No	No	Forward	Table 18
No	No	Backward	Table 19
No	No	Combined	Table 20
No	No	Memory Usage (Combined)	Table 21

Table 9: Forward pass runtime (ms) of various exact/approximate/sparse attention mechanisms by sequence length, **with dropout and masking**. Best in **bold**, second best underlined.

Attention Method	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch Attention	0.36	0.34	0.78	2.54	9.33	36.33	-	-	-	-
Megatron	0.40	0.40	1.10	3.65	16.19	-	-	-	-	-
Reformer	2.03	3.15	5.67	11.02	22.59	46.14	97.38	212.13	-	-
Local Attention	0.83	0.86	1.01	2.20	7.13	14.32	28.60	57.79	117.67	-
Linformer	0.67	0.52	0.69	0.71	1.65	3.18	6.15	12.16	24.17	52.39
Smyrf	2.27	2.34	3.91	7.44	14.71	29.22	58.27	116.41	-	-
LSformer	1.18	1.27	1.34	3.38	11.40	22.55	44.95	89.76	179.66	-
Block Sparse	1.12	1.11	2.13	2.77	6.95	20.91	-	-	-	-
Longformer	1.22	1.14	1.08	1.95	5.72	12.98	-	-	-	-
BigBird	1.13	1.12	1.12	1.77	6.03	13.68	-	-	-	-
FlashAttention	**0.04**	0.06	0.21	0.82	2.85	10.41	41.74	167.19	670.76	2682.35
Block-Sparse FlashAttention	0.06	**0.06**	**0.06**	**0.12**	**0.44**	**0.86**	**1.70**	**3.29**	**6.55**	**13.34**

We report timing results on the forward pass, backward pass, and combined forward + backward pass. We measure each method with and without dropout, masking, or both—except for Block Sparse, Longformer, and BigBird. These methods did not successfully run the backward pass with masking due to a bug in external libraries, so we measured them without masking to be generous. We use FP16 for all measurements, except for Local Attention, whose implementation only supports FP32.

For each baseline, we increase sequence length until it runs out of memory on the GPU, except for the following exceptions: The Megatron implementation does not support sequence lengths longer than 2048. Block-Sparse (OpenAI) does not support sequence lengths longer than 4096. Longformer and BigBird do not support sequence lengths longer than 8092.

We measure memory usage on the combined forward + backward pass, without dropout or masking.

Results Table 8 summarizes all the experimental configurations and contains pointers to the results tables.

Table 10: Backward pass runtime (ms) of various exact/approximate/sparse attention mechanisms by sequence length, **with dropout and masking**. Best in **bold**, second best underlined.

Attention Method	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch Attention	0.37	0.49	1.66	5.81	22.32	87.67	-	-	-	-
Megatron	0.35	0.32	0.77	2.42	8.43	-	-	-	-	-
Reformer	2.37	4.59	8.91	17.68	35.13	70.05	140.01	-	-	-
Local Attention	0.55	0.62	1.49	4.03	13.78	27.61	55.20	110.27	221.40	-
Linformer	0.89	0.80	0.81	0.93	2.48	4.75	9.29	18.27	36.53	-
Smyrf	1.41	2.83	5.43	10.72	21.25	42.31	84.48	168.95	-	-
LSformer	1.75	1.76	3.01	7.50	20.07	39.08	76.39	150.82	-	-
Block Sparse	1.29	1.28	2.18	3.04	7.27	21.16	-	-	-	-
Longformer	1.27	1.31	1.29	2.04	5.24	10.74	25.95	-	-	-
BigBird	1.33	1.28	1.32	1.81	5.55	11.44	27.45	-	-	-
FlashAttention	**0.30**	**0.26**	0.68	2.02	6.84	26.89	105.70	418.96	1666.89	6660.44
Block-Sparse FlashAttention	**0.30**	0.27	**0.29**	**0.59**	**1.50**	**2.94**	**5.82**	**11.85**	**23.98**	**47.61**

Table 11: Forward pass + backward pass runtime (ms) of various exact/approximate/sparse attention mechanisms by sequence length, **with dropout and masking**. Best in **bold**, second best underlined.

Attention Method	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch Attention	0.84	0.86	2.35	8.29	31.75	124.19	-	-	-	-
Megatron	0.87	0.89	1.33	4.21	16.50	-	-	-	-	-
Reformer	4.30	7.76	14.60	28.74	57.79	116.34	237.57	-	-	-
Local Attention	1.40	1.60	2.06	6.06	20.94	42.01	84.08	168.48	339.45	-
Linformer	1.57	1.49	1.55	1.60	4.19	8.04	15.71	30.92	61.47	-
Smyrf	3.41	5.08	9.35	18.18	36.03	71.68	143.04	285.87	-	-
LSformer	3.08	3.10	4.26	10.90	31.59	61.72	121.51	241.18	-	-
Block Sparse	2.54	2.52	3.71	5.44	13.29	39.19	-	-	-	-
Longformer	2.47	2.49	2.51	3.10	10.39	22.49	60.44	-	-	-
BigBird	2.51	2.49	2.52	3.40	10.97	23.89	63.28	-	-	-
FlashAttention	**0.43**	**0.41**	0.95	2.55	9.56	37.49	147.75	586.61	2339.11	9341.30
Block-Sparse FlashAttention	0.44	0.44	**0.45**	**0.89**	**1.95**	**4.12**	**7.64**	**16.60**	**32.73**	**64.11**

Table 12: Forward pass runtime (ms) of various exact/approximate/sparse attention mechanisms by sequence length, **with masking**. Best in **bold**, second best underlined.

Attention Method	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch Attention	0.30	0.30	0.63	1.93	7.08	27.45	112.90	-	-	-
Megatron	0.45	0.41	0.43	1.52	5.80	-	-	-	-	-
Reformer	1.87	3.00	5.37	10.43	21.40	43.83	92.80	203.24	-	-
Local Attention	0.70	0.81	1.02	2.09	6.64	13.34	26.77	54.02	110.11	-
Linformer	0.63	0.50	0.67	0.65	1.36	2.60	5.04	9.92	19.69	43.47
Smyrf	2.38	2.32	3.76	7.16	14.14	28.09	55.98	111.73	-	-
LSformer	1.22	1.29	1.44	3.28	10.99	21.72	43.29	86.32	172.76	-
Block Sparse	0.96	1.04	1.66	2.16	5.41	16.15	-	-	-	-
Longformer	0.99	0.98	0.99	1.56	4.79	11.07	32.98	-	-	-
BigBird	0.96	1.02	1.02	1.48	5.05	11.59	34.16	-	-	-
FlashAttention	**0.03**	**0.04**	0.17	0.68	2.28	8.40	33.55	134.14	537.50	2150.88
Block-Sparse FlashAttention	0.05	**0.04**	**0.05**	**0.11**	**0.35**	**0.68**	**1.33**	**2.54**	**5.34**	**10.73**

Table 13: Backward pass runtime (ms) of various exact/approximate/sparse attention mechanisms by sequence length, **with masking**. Best in **bold**, second best underlined.

Attention Method	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch Attention	0.44	0.46	1.53	5.33	20.34	79.87	-	-	-	-
Megatron	0.29	0.31	0.65	1.95	6.49	-	-	-	-	-
Reformer	2.31	4.47	8.68	17.20	34.14	68.09	136.02	-	-	-
Local Attention	0.51	0.62	1.30	3.81	13.33	26.72	53.41	106.82	214.15	-
Linformer	0.76	0.81	0.94	0.87	2.24	4.25	8.35	16.38	32.67	72.11
Smyrf	1.34	2.77	5.30	10.46	20.73	41.27	82.41	164.86	-	-
LSformer	1.66	1.61	3.09	7.42	19.68	38.35	74.92	147.86	-	-
Block Sparse	1.24	1.25	2.04	2.91	6.78	19.67	-	-	-	-
Longformer	1.27	1.23	1.24	1.85	4.99	10.21	24.89	-	-	-
BigBird	1.43	1.50	1.44	1.69	5.25	10.86	26.26	-	-	-
FlashAttention	**0.21**	**0.22**	0.62	1.84	5.77	22.25	86.21	338.91	1343.91	5361.09
Block-Sparse FlashAttention	0.22	0.22	**0.26**	**0.57**	**1.55**	**3.13**	**5.98**	**12.21**	**23.49**	**47.85**

Table 14: Forward pass + backward pass runtime (ms) of various exact/approximate/sparse attention mechanisms by sequence length, **with masking**. Best in **bold**, second best underlined.

Attention Method	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch Attention	0.80	0.81	2.08	7.23	27.51	107.58	-	-	-	-
Megatron	0.81	0.83	1.09	3.36	12.39	-	-	-	-	-
Reformer	4.16	7.46	14.06	27.68	55.66	112.15	229.37	-	-	-
Local Attention	1.39	1.68	2.08	5.83	20.04	40.16	80.44	161.35	325.11	-
Linformer	1.51	1.42	1.56	1.67	3.67	6.99	13.63	26.77	53.36	117.56
Smyrf	3.38	4.93	9.07	17.66	34.94	69.55	138.72	277.41	-	-
LSformer	3.08	3.10	4.26	10.90	31.59	61.72	121.51	241.18	-	-
Block Sparse	2.39	2.40	3.31	5.02	12.25	35.94	-	-	-	-
Longformer	2.36	2.34	2.38	2.94	9.83	21.35	58.12	-	-	-
BigBird	2.35	2.35	2.37	3.25	10.36	22.57	60.63	-	-	-
FlashAttention	**0.32**	**0.30**	0.83	2.37	7.95	30.77	119.98	473.65	1883.43	7513.01
Block-Sparse FlashAttention	0.34	0.34	**0.36**	**0.69**	**1.85**	**3.89**	**7.16**	**14.85**	**30.46**	**60.03**

Table 15: Forward pass runtime (ms) of various exact/approximate/sparse attention mechanisms by sequence length, **with dropout**. Best in **bold**, second best underlined.

Attention Method	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch Attention	0.26	0.24	0.57	1.80	6.56	25.34	-	-	-	-
Megatron	0.27	0.27	0.56	1.88	6.56	-	-	-	-	-
Reformer	1.83	2.96	5.31	10.33	21.19	43.42	91.96	201.34	-	-
Local Attention	0.51	0.60	0.78	2.01	6.23	12.52	25.07	50.50	102.18	-
Linformer	0.47	0.37	0.49	**0.52**	1.37	2.65	5.12	10.13	20.25	44.16
Smyrf	2.12	2.01	3.15	5.97	11.83	23.36	46.48	92.72	-	-
LSformer	1.28	1.33	1.51	3.39	11.40	22.54	44.96	89.85	179.73	-
Block Sparse	1.03	1.00	1.72	2.39	5.96	17.88	-	-	-	-
Longformer	1.02	1.03	1.03	1.73	5.10	11.63	34.22	-	-	-
BigBird	0.99	1.03	1.01	1.58	5.36	12.27	35.56	-	-	-
FlashAttention	**0.10**	**0.10**	**0.22**	0.83	2.81	10.38	41.63	167.01	668.74	2678.11
Block-Sparse FlashAttention	0.54	0.51	0.68	0.61	**0.67**	**1.10**	**1.89**	**3.71**	**7.18**	**14.41**

Table 16: Backward pass runtime (ms) of various exact/approximate/sparse attention mechanisms by sequence length, **with dropout**. Best in **bold**, second best underlined.

Attention Method	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch Attention	0.44	0.35	0.90	2.94	10.77	41.67	-	-	-	-
Megatron	0.28	0.33	0.92	2.94	10.80	-	-	-	-	-
Reformer	2.24	4.34	8.39	16.62	33.02	65.77	131.52	-	-	-
Local Attention	0.51	0.58	1.41	3.71	12.96	25.98	51.94	103.72	207.78	-
Linformer	0.84	0.74	0.79	0.85	2.28	4.37	8.66	17.02	33.78	-
Smyrf	1.27	2.56	4.90	9.66	19.16	38.13	76.17	152.39	-	-
LSformer	1.67	1.77	3.03	7.52	20.10	39.13	76.35	150.83	-	-
Block Sparse	1.27	1.36	2.15	3.04	7.27	21.18	-	-	-	-
Longformer	1.28	1.34	1.38	1.98	5.24	10.74	25.95	-	-	-
BigBird	1.48	1.47	1.50	1.81	5.57	11.38	27.43	-	-	-
FlashAttention	**0.15**	0.18	0.58	1.86	6.50	26.21	104.27	416.10	1661.92	6643.01
Block-Sparse FlashAttention	0.17	**0.17**	**0.17**	**0.40**	**1.10**	**2.04**	**4.43**	**9.33**	**18.28**	**37.31**

Table 17: Forward pass + backward pass runtime (ms) of various exact/approximate/sparse attention mechanisms by sequence length, **with dropout**. Best in **bold**, second best underlined.

Attention Method	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch Attention	0.66	0.67	1.43	4.82	17.47	67.29	-	-	-	-
Megatron	0.88	0.90	1.49	4.73	17.41	-	-	-	-	-
Reformer	4.06	7.28	13.68	26.98	54.27	109.39	223.80	-	-	-
Local Attention	1.09	1.40	1.99	5.61	19.23	38.62	77.30	154.63	311.12	-
Linformer	1.31	1.21	1.30	1.39	3.73	7.15	14.05	27.69	55.00	-
Smyrf	3.00	4.37	8.05	15.66	31.04	61.64	123.04	245.65	-	-
LSformer	3.07	3.17	4.31	10.89	31.54	61.78	121.56	240.94	-	-
Block Sparse	2.54	2.52	3.71	5.44	13.29	39.19	-	-	-	-
Longformer	2.47	2.49	2.51	3.10	10.39	22.49	60.44	-	-	-
BigBird	2.51	2.49	2.52	3.40	10.97	23.89	63.28	-	-	-
FlashAttention	**0.35**	**0.36**	**0.80**	2.52	9.16	36.70	146.13	583.45	2332.01	9323.63
Block-Sparse FlashAttention	0.91	0.83	0.94	**0.92**	**1.83**	**3.50**	**7.02**	**13.56**	**26.71**	**53.92**

Table 18: Forward pass runtime (ms) of various exact/approximate/sparse attention mechanisms by sequence length. Best in **bold**, second best underlined.

Attention Method	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch Attention	0.21	0.22	0.43	1.27	4.32	16.47	67.77	-	-	-
Megatron	0.24	0.26	0.42	1.33	4.28	-	-	-	-	-
Reformer	1.77	2.82	5.01	9.74	20.03	41.11	87.39	192.40	-	-
Local Attention	0.48	0.57	0.80	1.90	5.76	11.56	23.13	46.65	94.74	-
Linformer	0.46	0.36	0.45	**0.50**	1.09	2.09	4.01	7.90	15.70	35.40
Smyrf	1.94	1.96	3.01	5.69	11.26	22.23	44.21	88.22	-	-
LSformer	1.21	1.34	1.34	3.31	11.01	21.71	43.27	86.32	172.85	-
Block Sparse	0.96	1.04	1.66	2.16	5.41	16.15	-	-	-	-
Longformer	0.99	0.98	0.99	1.56	4.79	11.07	32.98	-	-	-
BigBird	0.96	1.02	1.02	1.48	5.05	11.59	34.16	-	-	-
FlashAttention	**0.08**	**0.09**	**0.18**	0.68	2.40	8.42	33.54	134.03	535.95	2147.05
Block-Sparse FlashAttention	0.56	0.52	0.63	0.65	**0.61**	**0.96**	**1.69**	**3.02**	**5.69**	**11.77**

Table 19: Backward pass runtime (ms) of various exact/approximate/sparse attention mechanisms by sequence length. Best in **bold**, second best underlined.

Attention Method	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch Attention	0.26	0.29	0.78	2.44	8.82	33.87	-	-	-	-
Megatron	0.29	0.30	0.80	2.59	8.86	-	-	-	-	-
Reformer	2.18	4.21	8.14	16.12	32.02	63.84	127.60	-	-	-
Local Attention	0.51	0.64	1.28	3.60	12.52	25.08	50.22	100.23	200.66	-
Linformer	0.69	0.76	0.69	0.80	2.04	3.88	7.67	15.04	30.11	63.15
Smyrf	1.24	2.49	4.77	9.42	18.65	37.12	74.15	148.35	-	-
LSformer	1.68	1.61	3.02	7.40	19.72	38.27	74.89	147.99	-	-
Block Sparse	1.24	1.25	2.04	2.91	6.78	19.67	-	-	-	-
Longformer	1.27	1.23	1.24	1.85	4.99	10.21	24.89	-	-	-
BigBird	1.43	1.50	1.44	1.69	5.25	10.86	26.26	-	-	-
FlashAttention	**0.11**	0.16	0.52	1.62	5.45	21.57	84.75	336.00	1338.56	5343.19
Block-Sparse FlashAttention	0.11	**0.12**	**0.16**	**0.38**	**1.20**	**2.34**	**4.69**	**9.10**	**18.74**	**37.04**

Table 20: Forward pass + backward pass runtime (ms) of various exact/approximate/sparse attention mechanisms by sequence length. Best in **bold**, second best underlined.

Attention Method	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch Attention	0.67	0.70	1.18	3.67	13.22	50.44	-	-	-	-
Megatron	0.74	0.65	1.23	3.80	13.21	-	-	-	-	-
Reformer	3.93	7.01	13.15	25.89	52.09	105.00	215.13	-	-	-
Local Attention	1.09	1.27	1.99	5.38	18.32	36.77	73.67	147.29	296.35	-
Linformer	1.31	1.25	1.30	1.29	3.20	6.10	11.93	23.39	46.72	100.52
Smyrf	2.98	4.23	7.78	15.12	29.96	59.45	118.60	237.02	-	-
LSformer	3.03	3.05	4.26	10.70	30.77	60.15	118.33	234.94	-	-
Block Sparse	2.39	2.40	3.31	5.02	12.25	35.94	-	-	-	-
Longformer	2.36	2.34	2.38	2.94	9.83	21.35	58.12	-	-	-
BigBird	2.35	2.35	2.37	3.25	10.36	22.57	60.63	-	-	-
FlashAttention	**0.31**	**0.31**	**0.73**	2.29	7.64	30.09	118.50	470.51	1876.08	7492.85
Block-Sparse FlashAttention	0.74	0.77	0.82	**0.88**	**1.71**	**3.21**	**6.56**	**12.60**	**24.93**	**50.39**

Table 21: Memory usage (MB) of various exact/approximate/sparse attention mechanisms by sequence length. Best in **bold**, second best underlined.

Attention Method	128	256	512	1024	2048	4096	8192	16384	32768	65536
PyTorch Attention	36	104	336	1184	4416	17024	-	-	-	-
Megatron	36	104	336	1184	4416	-	-	-	-	-
Reformer	377	754	1508	3016	6033	12067	24134	-	-	-
Local Attention	53	110	232	592	1696	3392	6784	13568	27136	-
Linformer	25	52	114	287	832	1652	3292	6572	13132	26252
Smyrf	217	434	868	1737	3474	6947	13894	27788	-	-
LSformer	72	152	333	796	2540	5068	10125	20240	-	-
Block Sparse	33	82	228	408	910	2401	-	-	-	-
Longformer	30	61	124	277	681	1370	2748	-	-	-
BigBird	33	66	131	294	708	1431	2872	-	-	-
FlashAttention	**22**	**44**	**104**	**209**	**418**	**836**	**1672**	**3344**	**6688**	**13376**
Block-Sparse FlashAttention	22	44	104	209	418	836	1672	3344	6690	13384

Attention Is All You Need

Ashish Vaswani[*]
Google Brain
avaswani@google.com

Noam Shazeer[*]
Google Brain
noam@google.com

Niki Parmar[*]
Google Research
nikip@google.com

Jakob Uszkoreit[*]
Google Research
usz@google.com

Llion Jones[*]
Google Research
llion@google.com

Aidan N. Gomez[*] [†]
University of Toronto
aidan@cs.toronto.edu

Łukasz Kaiser[*]
Google Brain
lukaszkaiser@google.com

Illia Polosukhin[*] [‡]
illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

[*]Equal contribution. Listing order is random. Jakob proposed replacing RNNs with self-attention and started the effort to evaluate this idea. Ashish, with Illia, designed and implemented the first Transformer models and has been crucially involved in every aspect of this work. Noam proposed scaled dot-product attention, multi-head attention and the parameter-free position representation and became the other person involved in nearly every detail. Niki designed, implemented, tuned and evaluated countless model variants in our original codebase and tensor2tensor. Llion also experimented with novel model variants, was responsible for our initial codebase, and efficient inference and visualizations. Lukasz and Aidan spent countless long days designing various parts of and implementing tensor2tensor, replacing our earlier codebase, greatly improving results and massively accelerating our research.

[†]Work performed while at Google Brain.

[‡]Work performed while at Google Research.

31st Conference on Neural Information Processing Systems (NIPS 2017), Long Beach, CA, USA.

1 Introduction

Recurrent neural networks, long short-term memory [13] and gated recurrent [7] neural networks in particular, have been firmly established as state of the art approaches in sequence modeling and transduction problems such as language modeling and machine translation [35, 2, 5]. Numerous efforts have since continued to push the boundaries of recurrent language models and encoder-decoder architectures [38, 24, 15].

Recurrent models typically factor computation along the symbol positions of the input and output sequences. Aligning the positions to steps in computation time, they generate a sequence of hidden states h_t, as a function of the previous hidden state h_{t-1} and the input for position t. This inherently sequential nature precludes parallelization within training examples, which becomes critical at longer sequence lengths, as memory constraints limit batching across examples. Recent work has achieved significant improvements in computational efficiency through factorization tricks [21] and conditional computation [32], while also improving model performance in case of the latter. The fundamental constraint of sequential computation, however, remains.

Attention mechanisms have become an integral part of compelling sequence modeling and transduction models in various tasks, allowing modeling of dependencies without regard to their distance in the input or output sequences [2, 19]. In all but a few cases [27], however, such attention mechanisms are used in conjunction with a recurrent network.

In this work we propose the Transformer, a model architecture eschewing recurrence and instead relying entirely on an attention mechanism to draw global dependencies between input and output. The Transformer allows for significantly more parallelization and can reach a new state of the art in translation quality after being trained for as little as twelve hours on eight P100 GPUs.

2 Background

The goal of reducing sequential computation also forms the foundation of the Extended Neural GPU [16], ByteNet [18] and ConvS2S [9], all of which use convolutional neural networks as basic building block, computing hidden representations in parallel for all input and output positions. In these models, the number of operations required to relate signals from two arbitrary input or output positions grows in the distance between positions, linearly for ConvS2S and logarithmically for ByteNet. This makes it more difficult to learn dependencies between distant positions [12]. In the Transformer this is reduced to a constant number of operations, albeit at the cost of reduced effective resolution due to averaging attention-weighted positions, an effect we counteract with Multi-Head Attention as described in section 3.2.

Self-attention, sometimes called intra-attention is an attention mechanism relating different positions of a single sequence in order to compute a representation of the sequence. Self-attention has been used successfully in a variety of tasks including reading comprehension, abstractive summarization, textual entailment and learning task-independent sentence representations [4, 27, 28, 22].

End-to-end memory networks are based on a recurrent attention mechanism instead of sequence-aligned recurrence and have been shown to perform well on simple-language question answering and language modeling tasks [34].

To the best of our knowledge, however, the Transformer is the first transduction model relying entirely on self-attention to compute representations of its input and output without using sequence-aligned RNNs or convolution. In the following sections, we will describe the Transformer, motivate self-attention and discuss its advantages over models such as [17, 18] and [9].

3 Model Architecture

Most competitive neural sequence transduction models have an encoder-decoder structure [5, 2, 35]. Here, the encoder maps an input sequence of symbol representations $(x_1, ..., x_n)$ to a sequence of continuous representations $\mathbf{z} = (z_1, ..., z_n)$. Given $\mathbf{z}$, the decoder then generates an output sequence $(y_1, ..., y_m)$ of symbols one element at a time. At each step the model is auto-regressive [10], consuming the previously generated symbols as additional input when generating the next.

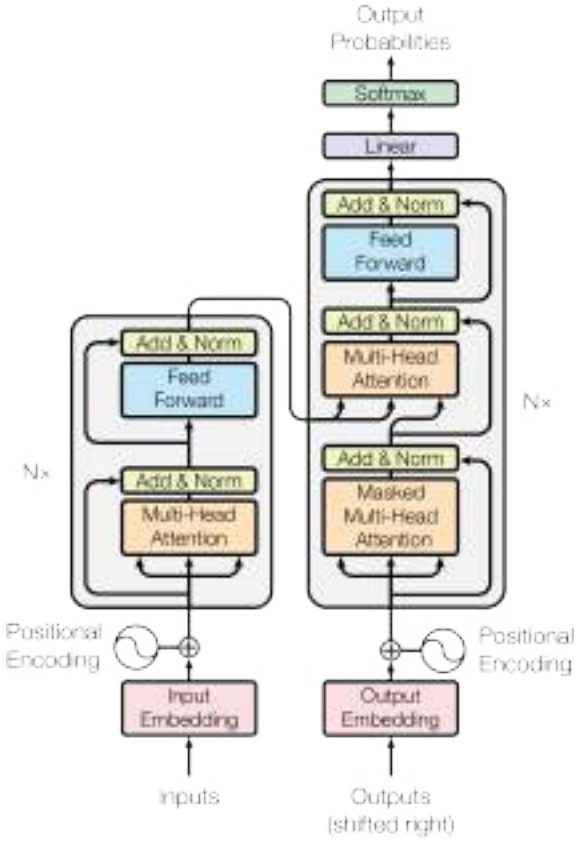


Figure 1: The Transformer - model architecture.

The Transformer follows this overall architecture using stacked self-attention and point-wise, fully connected layers for both the encoder and decoder, shown in the left and right halves of Figure 1, respectively.

3.1 Encoder and Decoder Stacks

Encoder: The encoder is composed of a stack of $N = 6$ identical layers. Each layer has two sub-layers. The first is a multi-head self-attention mechanism, and the second is a simple, position-wise fully connected feed-forward network. We employ a residual connection [11] around each of the two sub-layers, followed by layer normalization [1]. That is, the output of each sub-layer is $\mathrm{LayerNorm}(x + \mathrm{Sublayer}(x))$, where $\mathrm{Sublayer}(x)$ is the function implemented by the sub-layer itself. To facilitate these residual connections, all sub-layers in the model, as well as the embedding layers, produce outputs of dimension $d_{\text{model}} = 512$.

Decoder: The decoder is also composed of a stack of $N = 6$ identical layers. In addition to the two sub-layers in each encoder layer, the decoder inserts a third sub-layer, which performs multi-head attention over the output of the encoder stack. Similar to the encoder, we employ residual connections around each of the sub-layers, followed by layer normalization. We also modify the self-attention sub-layer in the decoder stack to prevent positions from attending to subsequent positions. This masking, combined with fact that the output embeddings are offset by one position, ensures that the predictions for position i can depend only on the known outputs at positions less than i.

3.2 Attention

An attention function can be described as mapping a query and a set of key-value pairs to an output, where the query, keys, values, and output are all vectors. The output is computed as a weighted sum

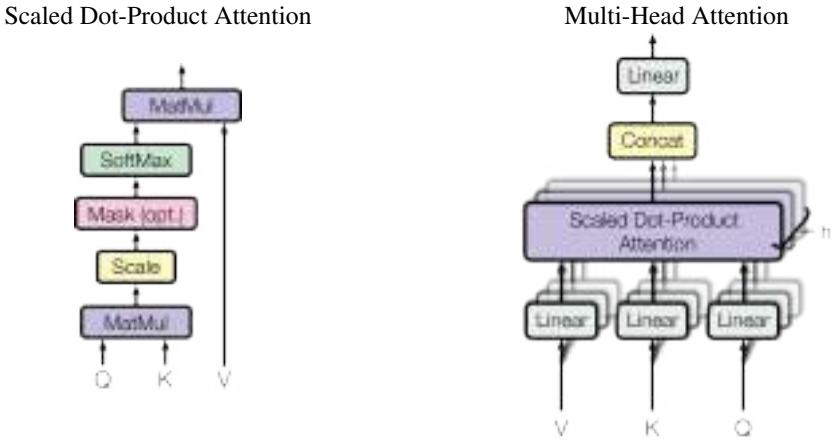


Figure 2: (left) Scaled Dot-Product Attention. (right) Multi-Head Attention consists of several attention layers running in parallel.

of the values, where the weight assigned to each value is computed by a compatibility function of the query with the corresponding key.

3.2.1 Scaled Dot-Product Attention

We call our particular attention "Scaled Dot-Product Attention" (Figure 2). The input consists of queries and keys of dimension d_k, and values of dimension d_v. We compute the dot products of the query with all keys, divide each by $\sqrt{d_k}$, and apply a softmax function to obtain the weights on the values.

In practice, we compute the attention function on a set of queries simultaneously, packed together into a matrix Q. The keys and values are also packed together into matrices K and V. We compute the matrix of outputs as:

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V \tag{1}$$

The two most commonly used attention functions are additive attention [2], and dot-product (multiplicative) attention. Dot-product attention is identical to our algorithm, except for the scaling factor of $\frac{1}{\sqrt{d_k}}$. Additive attention computes the compatibility function using a feed-forward network with a single hidden layer. While the two are similar in theoretical complexity, dot-product attention is much faster and more space-efficient in practice, since it can be implemented using highly optimized matrix multiplication code.

While for small values of d_k the two mechanisms perform similarly, additive attention outperforms dot product attention without scaling for larger values of d_k [3]. We suspect that for large values of d_k, the dot products grow large in magnitude, pushing the softmax function into regions where it has extremely small gradients [4]. To counteract this effect, we scale the dot products by $\frac{1}{\sqrt{d_k}}$.

3.2.2 Multi-Head Attention

Instead of performing a single attention function with d_{model}-dimensional keys, values and queries, we found it beneficial to linearly project the queries, keys and values h times with different, learned linear projections to d_k, d_k and d_v dimensions, respectively. On each of these projected versions of queries, keys and values we then perform the attention function in parallel, yielding d_v-dimensional

[4] To illustrate why the dot products get large, assume that the components of q and k are independent random variables with mean 0 and variance 1. Then their dot product, $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$, has mean 0 and variance d_k.

output values. These are concatenated and once again projected, resulting in the final values, as depicted in Figure 2.

Multi-head attention allows the model to jointly attend to information from different representation subspaces at different positions. With a single attention head, averaging inhibits this.

$$\begin{aligned}\text{MultiHead}(Q, K, V) &= \text{Concat}(\text{head}_1, ..., \text{head}_\text{h})W^O \\ \text{where head}_\text{i} &= \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)\end{aligned}$$

Where the projections are parameter matrices $W_i^Q \in \mathbb{R}^{d_\text{model} \times d_k}$, $W_i^K \in \mathbb{R}^{d_\text{model} \times d_k}$, $W_i^V \in \mathbb{R}^{d_\text{model} \times d_v}$ and $W^O \in \mathbb{R}^{hd_v \times d_\text{model}}$.

In this work we employ $h = 8$ parallel attention layers, or heads. For each of these we use $d_k = d_v = d_\text{model}/h = 64$. Due to the reduced dimension of each head, the total computational cost is similar to that of single-head attention with full dimensionality.

3.2.3 Applications of Attention in our Model

The Transformer uses multi-head attention in three different ways:

- In "encoder-decoder attention" layers, the queries come from the previous decoder layer, and the memory keys and values come from the output of the encoder. This allows every position in the decoder to attend over all positions in the input sequence. This mimics the typical encoder-decoder attention mechanisms in sequence-to-sequence models such as [38, 2, 9].
- The encoder contains self-attention layers. In a self-attention layer all of the keys, values and queries come from the same place, in this case, the output of the previous layer in the encoder. Each position in the encoder can attend to all positions in the previous layer of the encoder.
- Similarly, self-attention layers in the decoder allow each position in the decoder to attend to all positions in the decoder up to and including that position. We need to prevent leftward information flow in the decoder to preserve the auto-regressive property. We implement this inside of scaled dot-product attention by masking out (setting to $-\infty$) all values in the input of the softmax which correspond to illegal connections. See Figure 2.

3.3 Position-wise Feed-Forward Networks

In addition to attention sub-layers, each of the layers in our encoder and decoder contains a fully connected feed-forward network, which is applied to each position separately and identically. This consists of two linear transformations with a ReLU activation in between.

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2 \tag{2}$$

While the linear transformations are the same across different positions, they use different parameters from layer to layer. Another way of describing this is as two convolutions with kernel size 1. The dimensionality of input and output is $d_\text{model} = 512$, and the inner-layer has dimensionality $d_{ff} = 2048$.

3.4 Embeddings and Softmax

Similarly to other sequence transduction models, we use learned embeddings to convert the input tokens and output tokens to vectors of dimension d_model. We also use the usual learned linear transformation and softmax function to convert the decoder output to predicted next-token probabilities. In our model, we share the same weight matrix between the two embedding layers and the pre-softmax linear transformation, similar to [30]. In the embedding layers, we multiply those weights by $\sqrt{d_\text{model}}$.

Table 1: Maximum path lengths, per-layer complexity and minimum number of sequential operations for different layer types. n is the sequence length, d is the representation dimension, k is the kernel size of convolutions and r the size of the neighborhood in restricted self-attention.

Layer Type	Complexity per Layer	Sequential Operations	Maximum Path Length
Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(log_k(n))$
Self-Attention (restricted)	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

3.5 Positional Encoding

Since our model contains no recurrence and no convolution, in order for the model to make use of the order of the sequence, we must inject some information about the relative or absolute position of the tokens in the sequence. To this end, we add "positional encodings" to the input embeddings at the bottoms of the encoder and decoder stacks. The positional encodings have the same dimension d_{model} as the embeddings, so that the two can be summed. There are many choices of positional encodings, learned and fixed [9].

In this work, we use sine and cosine functions of different frequencies:

$$PE_{(pos,2i)} = sin(pos/10000^{2i/d_{\text{model}}})$$
$$PE_{(pos,2i+1)} = cos(pos/10000^{2i/d_{\text{model}}})$$

where pos is the position and i is the dimension. That is, each dimension of the positional encoding corresponds to a sinusoid. The wavelengths form a geometric progression from 2π to $10000 \cdot 2\pi$. We chose this function because we hypothesized it would allow the model to easily learn to attend by relative positions, since for any fixed offset k, PE_{pos+k} can be represented as a linear function of PE_{pos}.

We also experimented with using learned positional embeddings [9] instead, and found that the two versions produced nearly identical results (see Table 3 row (E)). We chose the sinusoidal version because it may allow the model to extrapolate to sequence lengths longer than the ones encountered during training.

4 Why Self-Attention

In this section we compare various aspects of self-attention layers to the recurrent and convolutional layers commonly used for mapping one variable-length sequence of symbol representations $(x_1, ..., x_n)$ to another sequence of equal length $(z_1, ..., z_n)$, with $x_i, z_i \in \mathbb{R}^d$, such as a hidden layer in a typical sequence transduction encoder or decoder. Motivating our use of self-attention we consider three desiderata.

One is the total computational complexity per layer. Another is the amount of computation that can be parallelized, as measured by the minimum number of sequential operations required.

The third is the path length between long-range dependencies in the network. Learning long-range dependencies is a key challenge in many sequence transduction tasks. One key factor affecting the ability to learn such dependencies is the length of the paths forward and backward signals have to traverse in the network. The shorter these paths between any combination of positions in the input and output sequences, the easier it is to learn long-range dependencies [12]. Hence we also compare the maximum path length between any two input and output positions in networks composed of the different layer types.

As noted in Table 1, a self-attention layer connects all positions with a constant number of sequentially executed operations, whereas a recurrent layer requires $O(n)$ sequential operations. In terms of computational complexity, self-attention layers are faster than recurrent layers when the sequence

length n is smaller than the representation dimensionality d, which is most often the case with sentence representations used by state-of-the-art models in machine translations, such as word-piece [38] and byte-pair [31] representations. To improve computational performance for tasks involving very long sequences, self-attention could be restricted to considering only a neighborhood of size r in the input sequence centered around the respective output position. This would increase the maximum path length to $O(n/r)$. We plan to investigate this approach further in future work.

A single convolutional layer with kernel width $k < n$ does not connect all pairs of input and output positions. Doing so requires a stack of $O(n/k)$ convolutional layers in the case of contiguous kernels, or $O(log_k(n))$ in the case of dilated convolutions [18], increasing the length of the longest paths between any two positions in the network. Convolutional layers are generally more expensive than recurrent layers, by a factor of k. Separable convolutions [6], however, decrease the complexity considerably, to $O(k \cdot n \cdot d + n \cdot d^2)$. Even with $k = n$, however, the complexity of a separable convolution is equal to the combination of a self-attention layer and a point-wise feed-forward layer, the approach we take in our model.

As side benefit, self-attention could yield more interpretable models. We inspect attention distributions from our models and present and discuss examples in the appendix. Not only do individual attention heads clearly learn to perform different tasks, many appear to exhibit behavior related to the syntactic and semantic structure of the sentences.

5 Training

This section describes the training regime for our models.

5.1 Training Data and Batching

We trained on the standard WMT 2014 English-German dataset consisting of about 4.5 million sentence pairs. Sentences were encoded using byte-pair encoding [3], which has a shared source-target vocabulary of about 37000 tokens. For English-French, we used the significantly larger WMT 2014 English-French dataset consisting of 36M sentences and split tokens into a 32000 word-piece vocabulary [38]. Sentence pairs were batched together by approximate sequence length. Each training batch contained a set of sentence pairs containing approximately 25000 source tokens and 25000 target tokens.

5.2 Hardware and Schedule

We trained our models on one machine with 8 NVIDIA P100 GPUs. For our base models using the hyperparameters described throughout the paper, each training step took about 0.4 seconds. We trained the base models for a total of 100,000 steps or 12 hours. For our big models,(described on the bottom line of table 3), step time was 1.0 seconds. The big models were trained for 300,000 steps (3.5 days).

5.3 Optimizer

We used the Adam optimizer [20] with $\beta_1 = 0.9$, $\beta_2 = 0.98$ and $\epsilon = 10^{-9}$. We varied the learning rate over the course of training, according to the formula:

$$lrate = d_{\text{model}}^{-0.5} \cdot \min(step\_num^{-0.5}, step\_num \cdot warmup\_steps^{-1.5}) \quad (3)$$

This corresponds to increasing the learning rate linearly for the first $warmup\_steps$ training steps, and decreasing it thereafter proportionally to the inverse square root of the step number. We used $warmup\_steps = 4000$.

5.4 Regularization

We employ three types of regularization during training:

Table 2: The Transformer achieves better BLEU scores than previous state-of-the-art models on the English-to-German and English-to-French newstest2014 tests at a fraction of the training cost.

Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [9]	26.36	**41.29**	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	**$3.3 \cdot 10^{18}$**	
Transformer (big)	**28.4**	**41.8**	$2.3 \cdot 10^{19}$	

Residual Dropout We apply dropout [33] to the output of each sub-layer, before it is added to the sub-layer input and normalized. In addition, we apply dropout to the sums of the embeddings and the positional encodings in both the encoder and decoder stacks. For the base model, we use a rate of $P_{drop} = 0.1$.

Label Smoothing During training, we employed label smoothing of value $\epsilon_{ls} = 0.1$ [36]. This hurts perplexity, as the model learns to be more unsure, but improves accuracy and BLEU score.

6 Results

6.1 Machine Translation

On the WMT 2014 English-to-German translation task, the big transformer model (Transformer (big) in Table 2) outperforms the best previously reported models (including ensembles) by more than 2.0 BLEU, establishing a new state-of-the-art BLEU score of 28.4. The configuration of this model is listed in the bottom line of Table 3. Training took 3.5 days on 8 P100 GPUs. Even our base model surpasses all previously published models and ensembles, at a fraction of the training cost of any of the competitive models.

On the WMT 2014 English-to-French translation task, our big model achieves a BLEU score of 41.0, outperforming all of the previously published single models, at less than $1/4$ the training cost of the previous state-of-the-art model. The Transformer (big) model trained for English-to-French used dropout rate $P_{drop} = 0.1$, instead of 0.3.

For the base models, we used a single model obtained by averaging the last 5 checkpoints, which were written at 10-minute intervals. For the big models, we averaged the last 20 checkpoints. We used beam search with a beam size of 4 and length penalty $\alpha = 0.6$ [38]. These hyperparameters were chosen after experimentation on the development set. We set the maximum output length during inference to input length + 50, but terminate early when possible [38].

Table 2 summarizes our results and compares our translation quality and training costs to other model architectures from the literature. We estimate the number of floating point operations used to train a model by multiplying the training time, the number of GPUs used, and an estimate of the sustained single-precision floating-point capacity of each GPU [5].

6.2 Model Variations

To evaluate the importance of different components of the Transformer, we varied our base model in different ways, measuring the change in performance on English-to-German translation on the

[5] We used values of 2.8, 3.7, 6.0 and 9.5 TFLOPS for K80, K40, M40 and P100, respectively.

Table 3: Variations on the Transformer architecture. Unlisted values are identical to those of the base model. All metrics are on the English-to-German translation development set, newstest2013. Listed perplexities are per-wordpiece, according to our byte-pair encoding, and should not be compared to per-word perplexities.

	N	d_{model}	d_{ff}	h	d_k	d_v	P_{drop}	ϵ_{ls}	train steps	PPL (dev)	BLEU (dev)	params $\times 10^6$
base	6	512	2048	8	64	64	0.1	0.1	100K	4.92	25.8	65
(A)				1	512	512				5.29	24.9	
				4	128	128				5.00	25.5	
				16	32	32				4.91	25.8	
				32	16	16				5.01	25.4	
(B)					16					5.16	25.1	58
					32					5.01	25.4	60
(C)	2									6.11	23.7	36
	4									5.19	25.3	50
	8									4.88	25.5	80
		256			32	32				5.75	24.5	28
		1024			128	128				4.66	26.0	168
			1024							5.12	25.4	53
			4096							4.75	26.2	90
(D)							0.0			5.77	24.6	
							0.2			4.95	25.5	
								0.0		4.67	25.3	
								0.2		5.47	25.7	
(E)	positional embedding instead of sinusoids									4.92	25.7	
big	6	1024	4096	16			0.3		300K	**4.33**	**26.4**	213

development set, newstest2013. We used beam search as described in the previous section, but no checkpoint averaging. We present these results in Table 3.

In Table 3 rows (A), we vary the number of attention heads and the attention key and value dimensions, keeping the amount of computation constant, as described in Section 3.2.2. While single-head attention is 0.9 BLEU worse than the best setting, quality also drops off with too many heads.

In Table 3 rows (B), we observe that reducing the attention key size d_k hurts model quality. This suggests that determining compatibility is not easy and that a more sophisticated compatibility function than dot product may be beneficial. We further observe in rows (C) and (D) that, as expected, bigger models are better, and dropout is very helpful in avoiding over-fitting. In row (E) we replace our sinusoidal positional encoding with learned positional embeddings [9], and observe nearly identical results to the base model.

6.3 English Constituency Parsing

To evaluate if the Transformer can generalize to other tasks we performed experiments on English constituency parsing. This task presents specific challenges: the output is subject to strong structural constraints and is significantly longer than the input. Furthermore, RNN sequence-to-sequence models have not been able to attain state-of-the-art results in small-data regimes [37].

We trained a 4-layer transformer with $d_{model} = 1024$ on the Wall Street Journal (WSJ) portion of the Penn Treebank [25], about 40K training sentences. We also trained it in a semi-supervised setting, using the larger high-confidence and BerkleyParser corpora from with approximately 17M sentences [37]. We used a vocabulary of 16K tokens for the WSJ only setting and a vocabulary of 32K tokens for the semi-supervised setting.

We performed only a small number of experiments to select the dropout, both attention and residual (section 5.4), learning rates and beam size on the Section 22 development set, all other parameters remained unchanged from the English-to-German base translation model. During inference, we

Table 4: The Transformer generalizes well to English constituency parsing (Results are on Section 23 of WSJ)

Parser	Training	WSJ 23 F1
Vinyals & Kaiser el al. (2014) [37]	WSJ only, discriminative	88.3
Petrov et al. (2006) [29]	WSJ only, discriminative	90.4
Zhu et al. (2013) [40]	WSJ only, discriminative	90.4
Dyer et al. (2016) [8]	WSJ only, discriminative	91.7
Transformer (4 layers)	WSJ only, discriminative	91.3
Zhu et al. (2013) [40]	semi-supervised	91.3
Huang & Harper (2009) [14]	semi-supervised	91.3
McClosky et al. (2006) [26]	semi-supervised	92.1
Vinyals & Kaiser el al. (2014) [37]	semi-supervised	92.1
Transformer (4 layers)	semi-supervised	92.7
Luong et al. (2015) [23]	multi-task	93.0
Dyer et al. (2016) [8]	generative	93.3

increased the maximum output length to input length + 300. We used a beam size of 21 and $\alpha = 0.3$ for both WSJ only and the semi-supervised setting.

Our results in Table 4 show that despite the lack of task-specific tuning our model performs surprisingly well, yielding better results than all previously reported models with the exception of the Recurrent Neural Network Grammar [8].

In contrast to RNN sequence-to-sequence models [37], the Transformer outperforms the Berkeley-Parser [29] even when training only on the WSJ training set of 40K sentences.

7 Conclusion

In this work, we presented the Transformer, the first sequence transduction model based entirely on attention, replacing the recurrent layers most commonly used in encoder-decoder architectures with multi-headed self-attention.

For translation tasks, the Transformer can be trained significantly faster than architectures based on recurrent or convolutional layers. On both WMT 2014 English-to-German and WMT 2014 English-to-French translation tasks, we achieve a new state of the art. In the former task our best model outperforms even all previously reported ensembles.

We are excited about the future of attention-based models and plan to apply them to other tasks. We plan to extend the Transformer to problems involving input and output modalities other than text and to investigate local, restricted attention mechanisms to efficiently handle large inputs and outputs such as images, audio and video. Making generation less sequential is another research goals of ours.

The code we used to train and evaluate our models is available at `https://github.com/tensorflow/tensor2tensor`.

Acknowledgements We are grateful to Nal Kalchbrenner and Stephan Gouws for their fruitful comments, corrections and inspiration.

References

[1] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.

[2] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. *CoRR*, abs/1409.0473, 2014.

[3] Denny Britz, Anna Goldie, Minh-Thang Luong, and Quoc V. Le. Massive exploration of neural machine translation architectures. *CoRR*, abs/1703.03906, 2017.

[4] Jianpeng Cheng, Li Dong, and Mirella Lapata. Long short-term memory-networks for machine reading. *arXiv preprint arXiv:1601.06733*, 2016.

Attention Visualizations

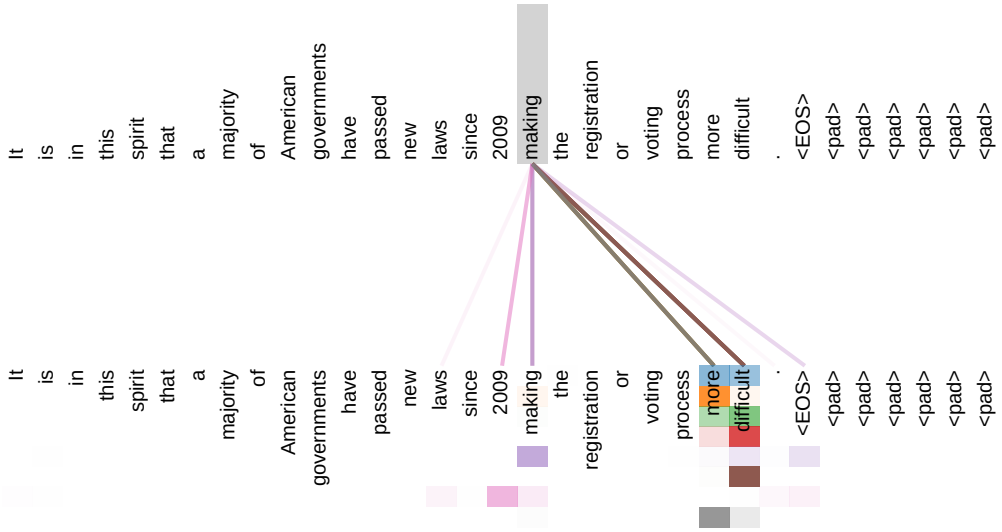


Figure 3: An example of the attention mechanism following long-distance dependencies in the encoder self-attention in layer 5 of 6. Many of the attention heads attend to a distant dependency of the verb ‘making’, completing the phrase ‘making...more difficult’. Attentions here shown only for the word ‘making’. Different colors represent different heads. Best viewed in color.

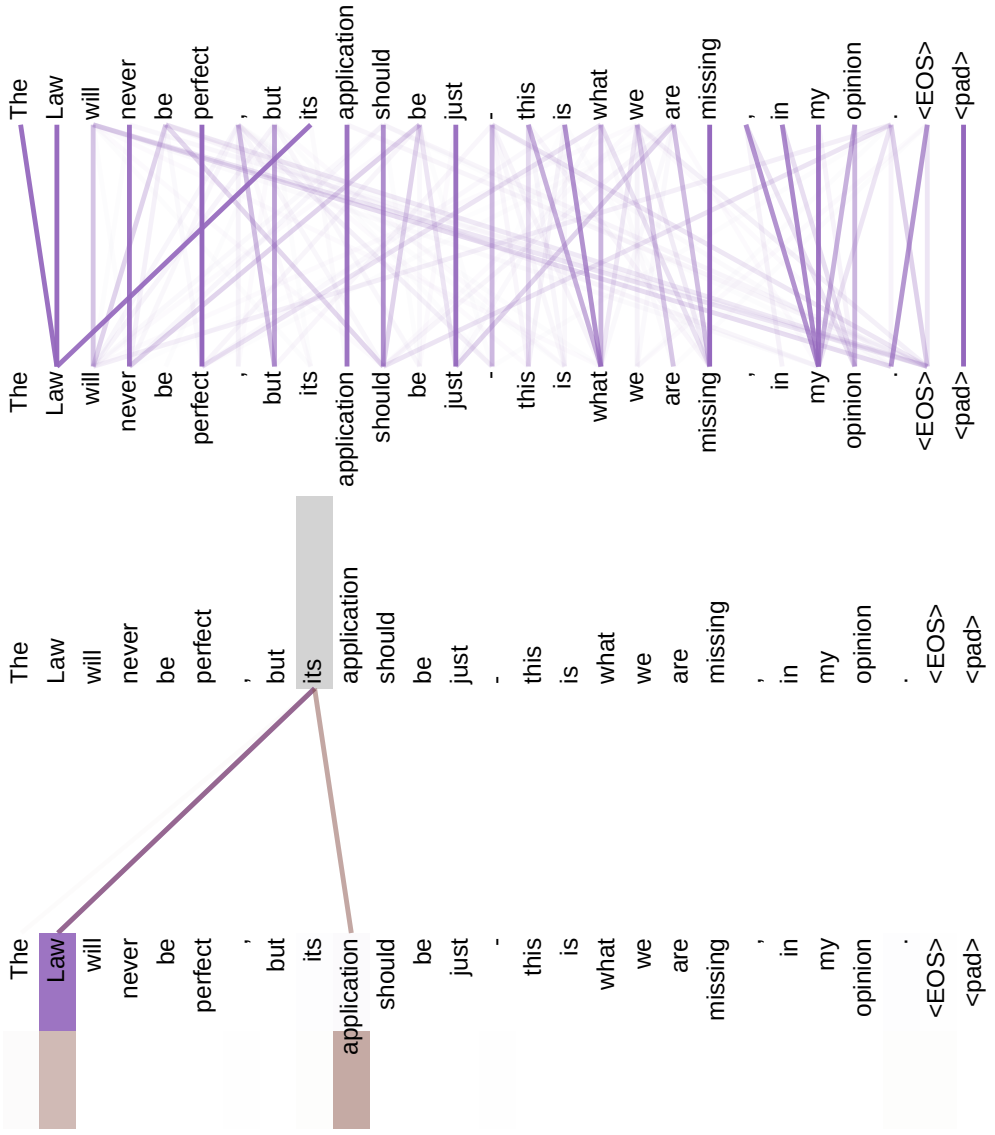


Figure 4: Two attention heads, also in layer 5 of 6, apparently involved in anaphora resolution. Top: Full attentions for head 5. Bottom: Isolated attentions from just the word 'its' for attention heads 5 and 6. Note that the attentions are very sharp for this word.

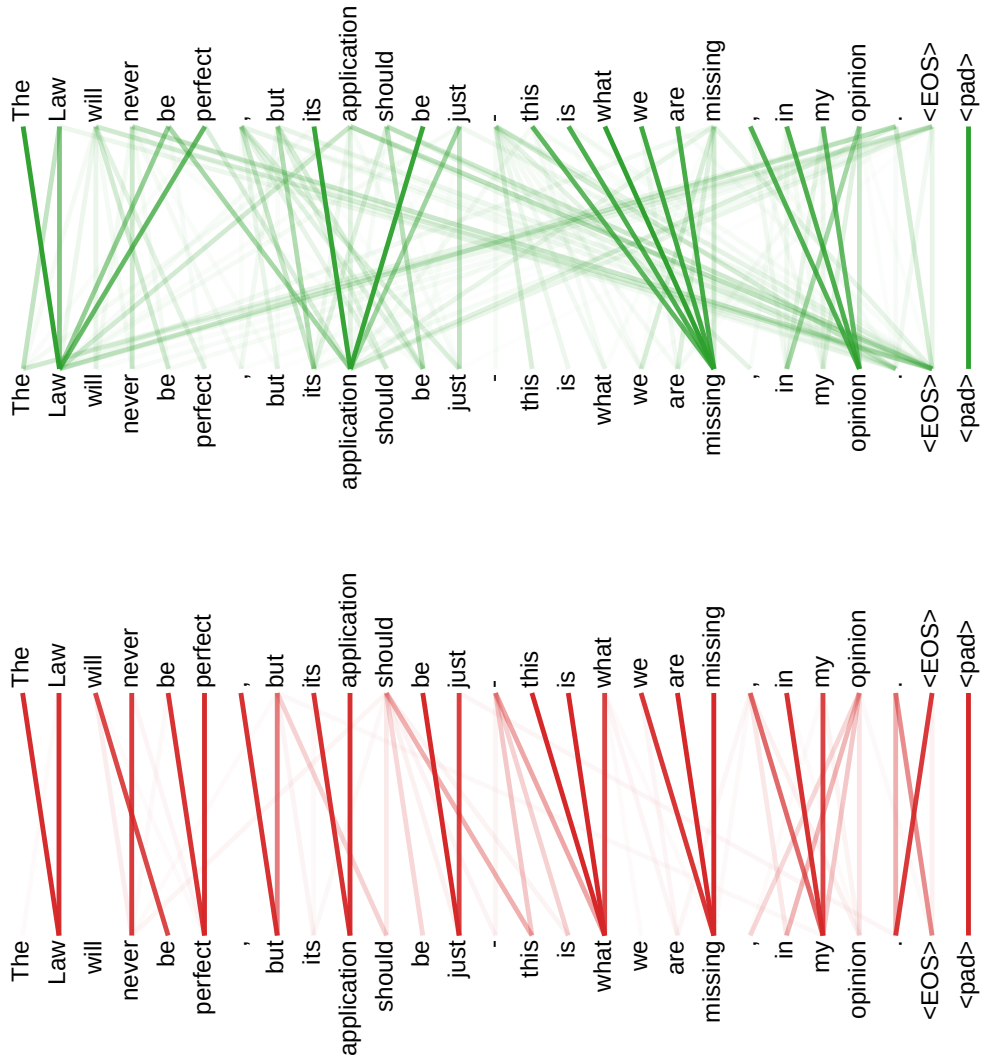


Figure 5: Many of the attention heads exhibit behaviour that seems related to the structure of the sentence. We give two such examples above, from two different heads from the encoder self-attention at layer 5 of 6. The heads clearly learned to perform different tasks.